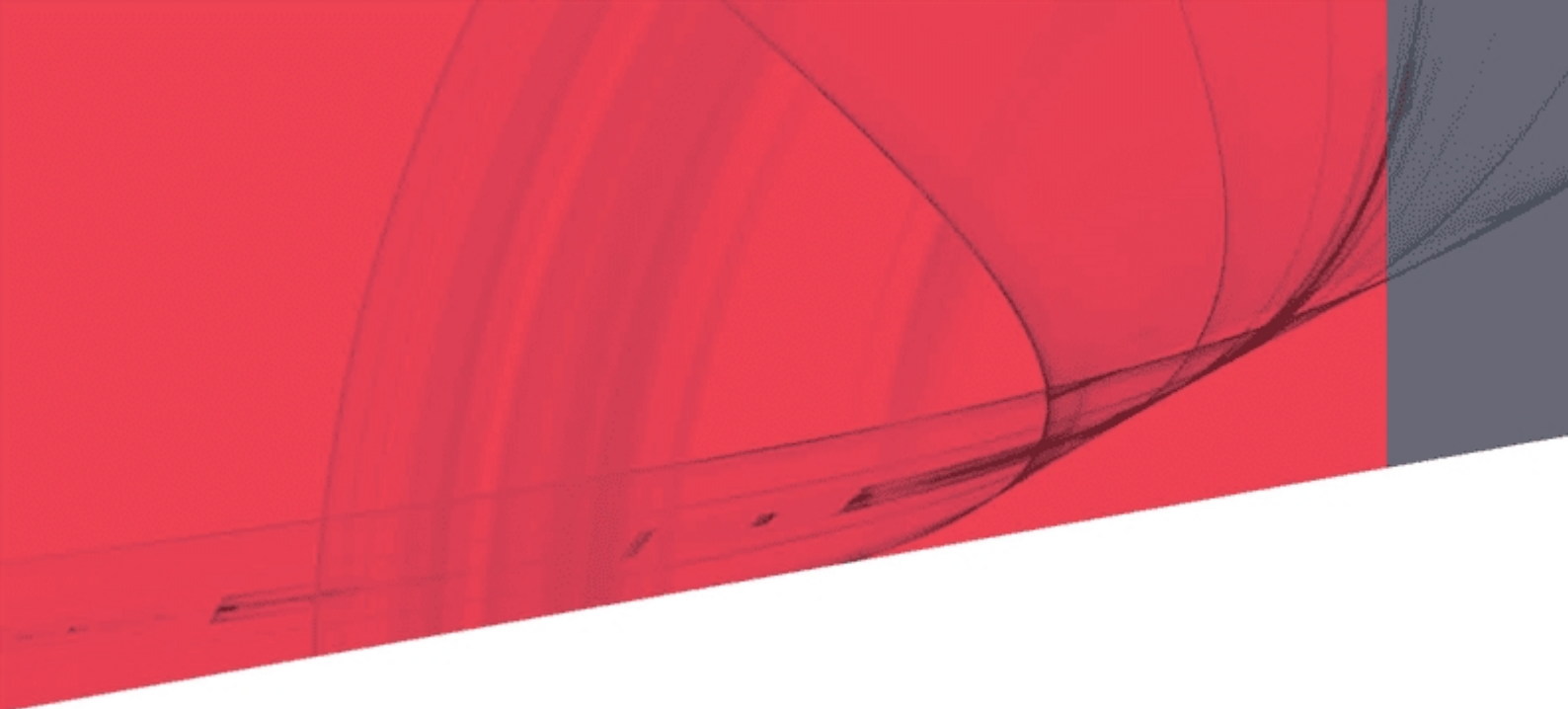




ACSPL+

Programmer's Guide

February 2023

Document Revision: 3.13.01

ACSPL+

Release Date: February 2023

COPYRIGHT

© ACS Motion Control Ltd., 2023. All rights reserved.

Changes are periodically made to the information in this document. Changes are published as release notes and later incorporated into revisions of this document.

No part of this document may be reproduced in any form without prior written permission from ACS Motion Control.

TRADEMARKS

Windows and Intellisense are trademarks of Microsoft Corporation.

EtherCAT® is registered trademark and patented technology, licensed by Beckhoff Automation GmbH, Germany.

Any other companies and product names mentioned herein may be the trademarks of their respective owners.

PATENTS

Israel Patent No. 235022

US Patent Application No. 14/532,023

Europe Patent application No.15187586.1

Japan Patent Application No.: 2015-193179

Chinese Patent Application No.: 201510639732.X

Taiwan(R.O.C.) Patent Application No. 104132118

Korean Patent Application No. 10-2015-0137612

www.acsmotioncontrol.com

support@acsmotioncontrol.com

sales@acsmotioncontrol.com

NOTICE

The information in this document is deemed to be correct at the time of publishing. ACS Motion Control reserves the right to change specifications without notice. ACS Motion Control is not responsible for incidental, consequential, or special damages of any kind in connection with using this document.

Revision History

Date	Revision	Description
February 2023	3.13.01	Cross-reference to C&V MATRIX information Corrections to MATRIX documentation NURBS, SPATH correction
November 2022	3.13	New Release, correct struct type variable definition
September 2022	3.12.03	Update ASSIGNPOUTS tables and links Correct BSEG parameter order
July 2022	3.12.02	Single-source XSEG Description String support
June 2022	3.12.01	Added /e switch description to PEG_I
May 2022	3.12	New Release
December 2021	3.11.01.02	Note minimum version supporting feature
December 2021	3.11.01.01	Document /q switch for XSEG, PTP, other motion commands Local Coordinates function explanations
November 2021	3.11.01	PEG & MARK Improvements, Addressing Analog I/O correction
June 2021	3.10.01	Add XXMsa table to ASSIGNPOUTS % and ++ operators
September 2020	3.02	Remove reference to obsolete SPiiPlus PCI device
July 2020	3.01	Structures and Functions
June 2020	3.00	Enhanced input shaping
July 2018	2.60	Added XRMSD, XRMSM, XRMSTD, and XRMSTM Added gantry mode commutation support Updated the following BSEG motions commands: BSEG, ARC1, ARC2, Line

Date	Revision	Description
December 2017	2.50	Added BSEG
June 2017	2.40	Updated #ETHERCAT, COEWRITE, COEREAD
October 2016	2.30.01	Changed switch to /f for end_velocity argument in XSEG
August 2016	2.30	Updated for SPiiPlus ADK Suite v2.30 Updated Position Event Generation (PEG) and MARK section Removed ASSIGNFINS section Removed PLCFLG
November 2015	2.29.10	XSEG updated for new features.
August 2015	2.29.04	XSEG updated for new features.
March 2015	2.29.01	XSEG updated for new features.

Conventions Used in this Guide

Text Formats

Format	Description
Bold	Names of GUI objects or commands
BOLD + UPPERCASE	ACSPL+ variables and commands
<code>Monospace + grey background</code>	Code example
<i>Italic</i>	Names of other documents
Blue	Hyperlink
[]	In commands indicates optional item(s)
	In commands indicates either/or items

Flagged Text

	Note - includes additional information or programming tips.
	Caution - describes a condition that may result in damage to equipment.
	Warning - describes a condition that may result in serious bodily injury or death.
	Model - highlights a specification, procedure, condition, or statement that depends on the product model
	Advanced - indicates a topic for advanced users.

Related Documents

Documents listed in the following table provide additional information related to this document.

The most updated version of the documents can be downloaded by authorized users from www.acsmotioncontrol.com/downloads.

Online versions for all ACS software manuals are available to authorized users at [ACS Motion Control Knowledge Center](#).

Document	Description
<i>SPiiPlus Command & Variable Reference Guide</i>	Describes all of the variables and commands available in the ACSPL+ programming language.
<i>SPiiPlus MMI Application Studio User Guide</i>	Explains how to use the SPiiPlus MMI Application Studio and associated monitoring tools.
<i>SPiiPlus C Library Reference Programmer Guide</i>	C++ and Visual Basic® libraries for host PC applications. This guide is applicable for all the SPiiPlus motion control products.
<i>SPiiPlus COM Library Programmer's Guides</i>	COM Methods, Properties, and Events for Communication with the Controller.
<i>SPiiPlus .NET Library Programmer's Guide</i>	.NET Methods, Properties, and Events for Communication with the Controller.
<i>SPiiPlus Utilities User Guide</i>	A guide for using the SPiiPlus User Mode Driver (UMD) for setting up communication with the SPiiPlus motion controller
<i>SPiiPlusNT-DC Hardware Guide</i>	Technical description of the SPiiPlusNT-DC product line.
<i>PDMNT Product Guide</i>	Technical description of the PDMNT Universal Drive Module
<i>SDMNT Product Guide</i>	Technical description of the SDMNT Universal Drive Module
<i>UDMNT Installation Guide</i>	Technical description of the UDMNT Universal Drive Module
<i>MC4U Product Guide</i>	Technical description of the MC4U Control Module integrated motion control product line.
<i>HSSI Expansion Modules Guide</i>	High-Speed Synchronous Serial Interface (HSSI) for expanded I/O, distributed axes, and nonstandard devices.

Document	Description
<i>PEG and MARK Operations</i>	Provides details on using the PEG commands in SPiiPlus systems.

Table of Contents

1. Introduction	28
1.1 ACSPL+ Programming Language	28
2. SPiiPlus Architecture	29
2.1 Hardware Structure	29
2.1.1 Firmware	30
2.1.2 Controller Cycle and Servo Tick	30
2.1.3 Realtime and Background Tasks	30
2.2 User Application	32
2.2.1 Firmware, User Application and Tools	32
2.2.2 User Application Components	32
2.2.3 User Applications Categories	33
2.2.4 SPiiPlus MMI Application Studio	33
2.2.5 File Extensions Used in SPiiPlus MMI Application Studio	35
2.3 Programming Resources	35
2.3.1 Commands	35
2.3.2 Program Buffers	36
2.3.2.1 ACSPL+ Buffers Compilation Behavior	37
2.3.3 Declaration Buffer (D-Buffer)	37
2.3.3.1 Defining Global Objects in D-Buffer	37
2.3.3.2 D-Buffer Default Contents	37
2.3.4 Command Execution	38
2.3.4.1 Terminal Commands	38
2.3.4.2 ACSPL+ Commands	38
2.3.5 ACSPL+ Standard Variables	39
2.3.6 User-Defined Variables	39
2.3.7 Nonvolatile Memory and Power Up Process	39
2.4 Executing ACSPL+ Programs	40
2.4.1 Program Buffers	40
2.4.2 Execution of a Single Program	40
2.4.3 Concurrent Execution	41
2.4.4 Immediate Execution	41
2.4.5 Immediate Execution vs. Stored Program	42

2.4.6	Autoroutine Execution	42
2.4.7	Synchronization and Mutual Exclusion	42
2.4.7.1	Mutual Exclusion	42
2.4.7.2	Synchronization	43
2.4.7.3	Execution Rate	44
3.	ACSPL+ Overview	45
3.1	ACSPL+ Syntax	45
3.1.1	Commands, Lines, and Command Aggregates	45
3.1.2	BLOCK..END Control Structure	45
3.1.3	ACSPL+ Reserved Words	46
3.1.4	Names: Variable and Label	46
3.1.5	Case Sensitivity	46
3.1.6	Axis Designations	46
3.1.7	Comments	47
3.2	Variables	47
3.2.1	Variable Name	47
3.2.2	Variable Class: ACSPL+ or User	47
3.2.3	Variable Scope	48
3.2.3.1	Global Variable Scope	48
3.2.3.2	Local Variable Scope	48
3.2.4	Variable Lifetime	49
3.2.5	Variable Accessibility	50
3.2.6	Variable Type: Integer and Real	50
3.2.7	Matrix Type	50
3.2.7.1	Matrix Initialization in Compilation Time	51
3.2.7.2	Matrix Operations Performance	51
3.2.7.3	Matrix Applications	52
3.2.7.4	Remarks	52
3.2.8	String Type	52
3.2.8.1	String Concatenation	53
3.2.8.2	String Comparison	53
3.2.8.3	String Standard Variables	54
3.2.9	Variable Size	54
3.2.10	Variable Value	55

3.3 Variable Declaration	55
3.3.1 Declaration of Global Variables	55
3.3.2 Persistent Global Variables	56
3.4 Arrays and Indexing	56
3.4.1 Scalars and Arrays	56
3.4.2 ACSPL+ Array Variables	57
3.4.3 Explicit Indexing	58
3.4.4 Postfix Indexing of Standard Arrays	58
3.4.5 Axis Indexing	59
3.4.6 User-Defined Axis Names	59
3.4.6.1 Axis Name as Symbolic Constant	59
3.4.6.2 Axis Name in Indexing	60
3.4.6.3 Axis Specification in Commands	60
3.4.7 Array Processing Functions	61
3.5 ACSPL+ Composite Data Structures	61
3.5.1 Introduction	61
3.5.2 STRUCT Type Definition	61
3.5.3 Fields	61
3.5.4 STRUCT Type Variable Definition and Use	62
3.5.4.1 Variable definition	62
3.5.4.2 Variable initialization	63
3.5.4.3 Expressions Using STRUCTs	63
3.5.5 STRUCT Functions	63
3.5.5.1 Function Definition	63
3.5.5.2 Out-of-scope Function Implementation	63
3.5.5.3 STRUCT Function Parameters	64
3.5.5.4 Function Variables Definition	64
3.6 Using Variables	64
3.6.1 Querying Variables	64
3.6.2 Variables as Operands in Expressions	64
3.6.3 Variables as Arguments in Command or Function	65
3.6.4 Variables in ACSPL+ Terminal Commands	65
3.6.5 Accessing Variables by Tags	66
3.6.5.1 Variable Tags	66

3.6.5.2 GETVAR and SETVAR Functions	66
3.7 ACSPL+ Functions	67
3.8 ACSPL + User-Defined Functions with Parameters	67
3.8.1 Introduction	67
3.8.2 Function definition	67
3.8.2.1 Function Signature	67
3.8.2.2 FASTCALL Functions	68
3.8.2.3 Function visibility scope	68
3.8.2.4 Function Code	70
3.8.3 Function Parameters	70
3.8.3.1 Value Types	70
3.8.3.2 Reference types	70
3.8.3.3 Composite Data Type Parameters	71
3.8.3.4 Array Type Parameters	71
3.8.4 Variable Definition	73
3.8.5 Return value	74
3.8.6 DISP command	74
3.8.7 Stack Memory	74
3.8.8 Function Execution Time	75
3.8.9 FILL function	75
3.9 Expressions	75
3.9.1 General	75
3.9.2 Calculation Order	76
3.9.3 Expression Type	76
3.9.4 Operands	77
3.9.4.1 Arithmetical Operators	78
3.9.4.2 Compare Operators	79
3.9.4.3 Bitwise and Logical Operators	79
3.9.4.4 Unary Operators	80
3.9.4.5 Bit Selection Operator (Dot)	81
3.9.5 Character Constants	81
3.10 ACSPL+ Commands	81
3.10.1 Assignment Command	81
3.10.1.1 ACSPL+ Variable Assignment	82

3.10.1.2 User Variable Assignment	82
3.10.1.3 Bit Assignment	83
3.10.1.4 Type Conversion	84
3.10.2 Synchronization Commands	84
3.10.2.1 WAIT Command	84
3.10.2.2 TILL Command	85
3.10.3 Autoroutines	86
3.10.3.1 ON Command	86
3.10.3.2 Autoroutine Body and Execution	87
3.10.3.3 Autoroutine and the Host Buffer Interactions	87
3.10.3.4 Examples	88
3.10.4 Program Management Commands	89
3.10.4.1 START Command	90
3.10.4.2 STOP and STOPALL Commands	90
3.10.4.3 PAUSE and RESUME Commands	91
3.10.4.4 ENABLEON and DISABLEON Commands	92
4. ACSPL+ Motion Programming	94
4.1 Axis/Motor Management Commands	94
4.1.1 ENABLE & DISABLE Commands	94
4.1.2 COMMUT Command	95
4.1.3 KILL and KILLALL Commands	97
4.1.4 FCLEAR Command	99
4.1.5 SET Command	99
4.1.6 GROUP, SPLIT & SPLITALL Commands	101
4.1.7 GO Command	103
4.1.8 HALT Command	104
4.1.9 BREAK Command	104
4.1.10 IMM Command	105
4.2 Point-to-Point Motion	106
4.2.1 PTP	106
4.2.2 SPTP Command	108
4.2.3 Move and Settle Time Calculation	109
4.2.4 MPTP, POINT, MPOINT, and ENDS Commands	112
4.2.4.1 MPTP Command	113

4.2.4.2 POINT Command	114
4.2.4.3 MPOINT Command	115
4.2.5 The GRTIME Variable	117
4.2.6 Modulo Axis	118
4.2.7 Motion Boost	119
4.2.7.1 20 kHz Motion	119
4.2.7.2 BPTP	121
4.2.7.3 BPTPCalc	124
4.3 JOG Command	125
4.4 TRACK Command	126
4.5 Segmented Motion	130
4.5.1 Understanding Slaved Segmented Motion	130
4.5.2 MSEG, LINE, ARC1, ARC2, STOPPER Commands	131
4.5.3 PROJECTION Command	134
4.5.4 Arguments as Expression	135
4.5.5 STOPPER Command	136
4.5.6 Cyclic Motion	137
4.5.7 Slaved Motion at Extreme Points	137
4.6 Extended Segmented Motion (XSEG)	138
4.6.1 Corner Processing	138
4.6.1.1 Corner Detection	138
4.6.1.2 Supported Options	138
4.6.2 Automatic Corner Processing	140
4.6.3 Enhanced automatic corner and curvature discontinuity points processing (switch/y)	140
4.6.4 Velocity Control and Look-ahead Algorithm	141
4.6.5 Corners and Curvature Discontinuity Points	142
4.6.6 Dynamic Velocity Profile Generation	143
4.6.7 Controller Usage Considerations	144
4.6.8 Approaches for Adding Segments	144
4.6.8.1 Adding Segments In Advance	144
4.6.8.2 Adding Segments Only When Requested	145
4.6.9 Secondary Axes	145
4.6.9.1 Secondary Axes Functionality	145

4.6.9.2 Using Secondary Axes in XSEG	146
4.6.9.3 Secondary Axes in ARC1, ARC2, LINE	146
4.6.10 Supported Motion Parameters	146
4.6.11 Non Supported Motion Parameters	147
4.6.12 XSEG...ENDS	147
4.6.13 ARC1 with XSEG	153
4.6.14 ARC2	156
4.6.15 LINE	157
4.6.16 Using ARC1, ARC2 and LINE Switches	159
4.6.17 Examples	161
4.6.17.1 Simple Two Axes Example	161
4.6.17.2 Simple Four Axes Example	163
4.6.18 New AST Bits	164
4.6.19 The IMM Command	165
4.6.20 XSEGAMIN and XSEGAMAX	166
4.6.21 XSEGRMIN and XSEGRMAX	167
4.7 Blended Segmented Motion	167
4.7.1 Motion definition through time intervals	167
4.7.2 Motion blending	168
4.7.2.1 Motion blending in special cases	170
4.7.3 ACSPL+ interface	170
4.7.3.1 Supported motion parameters	170
4.7.3.2 Non-supported motion parameters	171
4.7.3.3 BSEG...ENDS	171
4.7.3.4 LINE	172
4.7.3.5 ARC1	173
4.7.3.6 ARC2	174
4.7.3.7 Suffixes with commands LINE, ARC1, ARC2	175
4.7.3.8 HALT execution	175
4.8 Two axes example	176
4.9 Master/Slave Motion	177
4.9.1 MASTER Command	177
4.9.2 SLAVE Command	177
4.9.2.1 Synchronization	178

4.9.2.2 Velocity Lock vs. Position Lock	179
4.9.2.3 Stalled Motion	179
4.10 PATH Command	180
4.11 Spline Motion	181
4.11.1 Spline Motion Theory	181
4.11.2 Main Definitions	181
4.11.3 PVSPLINE Command	182
4.11.3.1 POINT Command	183
4.11.3.2 MPOINT Command	185
4.11.4 Spline Motion Variables	185
4.12 Multi-Axis Point-to-Point Motions With Smooth Transitions (SMOVE)	187
4.12.1 SMOVE	187
4.13 SmoothPath	188
4.13.1 SmoothPath ACSPL+	188
4.13.1.1 SPATH	188
4.13.1.2 SEGMENT	191
4.13.2 SmoothPath G-code	192
4.14 NURBS	192
4.14.1 NURBS Command	192
4.14.2 NPOINT Command	194
4.15 Open-Loop Operation (Torque Control)	196
4.16 Step Velocity Profile (Non-Zero Minimal Velocity)	198
4.16.1 The NVEL Variable	198
4.16.2 Special NVEL Cases	199
4.16.2.1 Specified Velocity Less Than NVEL	199
4.16.2.2 Multi-Axis Motion	199
4.16.2.3 NVEL and Non-Default Connection	199
4.17 Local Coordinate System	200
4.17.1 CSCREATE	200
4.17.2 CSDESTROY	201
4.17.3 LCS Examples	202
5. Inputs and Outputs	206
5.1 Digital Inputs and Outputs	206
5.1.1 Addressing Digital I/Os	206

5.1.2 Querying Digital I/Os	207
5.1.3 Assigning Outputs	208
5.1.4 Digital I/O in Conditional Commands	208
5.1.5 PLC Implementation	209
5.1.6 Digital I/O in Autoroutines	210
5.1.7 Using HSSI I/O Extension	210
5.1.8 SPI Communication	210
5.1.8.1 ACSPL+ Commands and Variables for SPI	210
5.2 Analog Inputs and Outputs	213
5.2.1 Addressing Analog I/Os	214
5.2.2 Assigning Analog Outputs	214
6. Fault Handling	216
6.1 Safety Control	216
6.1.1 Types of Malfunctions	216
6.1.2 How the Controller Detects Malfunctions	217
6.1.3 Faults	217
6.1.3.1 The FAULT Variable	217
6.1.3.2 The S_FAULT Variable	217
6.1.4 Controller Response	217
6.2 Safety Control Summaries	218
6.2.1 Summary of Faults and Default Responses	218
6.2.2 Summary of Safety Inputs	226
6.2.3 Summary of Safety-Related Variables	226
6.2.4 Integrity Control	228
6.2.4.1 Integrity Violation Fault	229
6.2.4.2 Integrity Report Command	229
6.2.5 Report of Realtime Usage Command	231
6.2.6 Application Protection	231
6.2.6.1 Protected Features	231
6.2.6.2 Enhanced Application Protection	232
6.2.6.3 CFG Variable	232
6.2.6.4 Protection of Variables	232
6.2.6.5 Protection of ACSPL+ Programs	233
6.2.6.6 Privileged Buffer	233

6.2.6.7 Communication Shutdown	234
6.2.7 Report Safety Configuration	234
6.3 Working with Faults	235
6.3.1 Addressing the Fault Bits	235
6.3.2 Querying Faults	237
6.3.3 Using the Fault Bits in if, while, till Commands	238
6.3.4 Creating Fault-Processing Autoroutines	238
6.3.5 Disabling Fault Processing	240
6.3.6 Defining the Active Level of Safety Input	241
6.3.7 Fault Processing Modes	242
6.4 Network Faults	243
6.4.1 Axis Network-Related Faults	243
6.4.2 Initialization Failure	244
6.4.3 Network Failure During Operation	244
6.4.4 SP Software Failure	244
6.5 Detailed Description of Faults	245
6.5.1 Limit Switches: #LL, #RL	245
6.5.2 Network Fault: #NT	246
6.5.3 Software Limit Switches: #SLL, #SRL	247
6.5.4 Non-Critical Position Error: #PE	248
6.5.5 Critical Position Error: #CPE	250
6.5.6 Encoder Error: #ENC, #ENC2	251
6.5.7 Encoder Not Connected: #ENCNC, #ENC2NC	252
6.5.8 Drive Alarm: #DRIVE	253
6.5.9 Motor Overheat: #HOT	254
6.5.10 Velocity Limit: #VL	254
6.5.11 Acceleration Limit: #AL	255
6.5.12 Current Limit: #CL	256
6.5.13 Servo Processor Alarm: #SP	257
6.5.14 HSSI Not Connected: #HSSINC	257
6.5.15 Emergency Stop: #ES	258
6.5.16 Program Error: #PROG	258
6.5.17 Memory Overflow: #MEM	260
6.5.18 Time Overuse: #TIME	261

6.5.19 Servo Interrupt: #INT	262
6.5.20 Component Failure Faults: #FAILURE	263
6.5.20.1 Safety Variables	263
6.5.20.2 Component Failure Fault Handling in ACSPL+	263
6.6 Detailed Description of Safety Controls	264
6.6.1 Examining Fault Conditions - Flow Chart	264
6.6.2 Examining Motor Fault Conditions	265
6.6.3 Examining System Fault Conditions	266
6.7 Extended Fault Configuration	267
7. Connection to the Plant	269
7.1 General Diagram	269
7.2 User-Defined Units	270
7.3 Direct and Feedback Transform	271
7.4 Index and Mark Values	272
7.5 Safety Inputs	273
7.6 Digital Inputs/Outputs Repetitive	274
7.7 Analog Inputs/Outputs	274
7.8 High-Speed Synchronous Serial Interface	275
8. Advanced Features	276
8.1 Data Collection	276
8.1.1 DC Command	276
8.1.2 SPDC - High-Speed Data Collection	278
8.1.3 ACSPL+ Variables Involved in Data Collection	278
8.1.4 Understanding System Data Collection	280
8.1.5 Axis Data Collection	281
8.1.6 STOPDC Command	283
8.2 PEG Overview	283
8.2.1 PEG Modes and Operation	284
8.2.1.1 Random PEG Commands	286
8.2.1.2 PEG Commands	294
8.2.1.3 ACSPL Plus Functions and Definitions	307
8.3 Sin-Cos Encoder Multiplier Configuration	309
8.3.1 Sin-Cos Encoder Multiplier	309
8.3.1.1 Technical Data	309

8.3.1.2 Configuring the Sin-Cos Multiplier	310
8.4 Interrupts	310
8.4.1 Hardware Interrupts	310
8.4.2 Software Interrupts	311
8.4.3 Software Interrupt Tags	312
8.4.4 Interrupt Configuration Variables	313
8.4.4.1 IENA Variable	313
8.4.4.2 ISENA Variable	314
8.5 Dynamic Braking	315
8.6 Constant Current Mode	315
8.7 Hall Sensor Commutation	316
8.8 Communicating with the SPiiPlus C Library	317
8.8.1 Remote Connection	317
8.8.2 Callbacks in all Communication Channels	318
8.8.2.1 Timing	318
8.8.2.2 Software Interrupts	318
8.8.2.3 Hardware Interrupts	319
8.8.3 TCP/IP Port Assignment for Remote Connection	319
8.8.3.1 TCP/IP Port Assignmnet	319
8.8.3.2 Disabling Remote UMD Connections	320
8.8.3.3 UMD Log Types	320
8.8.3.4 Unloading the UMD from Memory	322
8.9 Communicating with Non-ACS Devices	322
8.9.1 Channel Configuration Report	322
8.9.2 Assigning COM Channel for Special Input	323
8.9.3 Setting Communication Parameters	324
8.9.4 Parity, Stop Bit	325
8.9.5 INP Function	325
8.9.6 String Handling Commands	326
8.9.6.1 DISP Command	326
8.9.6.2 SEND Command	329
8.9.6.3 Differences between Query Commands and the DISP/SEND Commands	330
8.9.6.4 STR Function	330
8.9.6.5 DSTR Function	331

8.10	TRIGGER	332
8.11	Dynamic TCP/IP Addressing	334
8.11.1	TCP/IP Variable	334
8.11.2	Using GETCONF/SETCONF to Access TCP/IP Address	334
8.11.3	Addressing Scenarios	335
8.12	Non-Default Connections	336
8.12.1	ROFFS Variable	336
8.12.2	DAPOS Variable	337
8.12.3	CONNECT Command	337
8.12.4	DEPENDS Command	340
8.12.5	MATCH Function	341
8.13	Input Shaping	342
8.13.1	INSHAPEON	342
8.13.2	INSHAPEOFF	343
8.13.3	AST	344
8.13.4	APOSFILT	347
8.13.5	Using the Convolve Web Site	347
8.13.6	Data Entry Dialog	347
8.14	DRA Algorithm	356
8.15	BI-Quad Filter	359
8.16	Feedback Routing	362
8.17	Voltage Mode	363
9.	Mechanical Brake	364
9.1	Enable/Disable/Brake	364
9.2	Mechanical Brake On Startup	365
10.	Generic EtherCAT Master	370
10.1	Stack Behavior	370
10.2	Interface Description	370
10.2.1	ACSPL+ Variables	370
10.2.1.1	ECST - EtherCAT State	370
10.2.2	ECERR	371
10.2.3	#ETHERCAT	371
10.3	EtherCAT Functions	372
10.3.1	Mapping Functions	372

10.3.1.1 ECIN	372
10.3.1.2 ECOUT	373
10.3.1.3 ECUNMAP	373
10.3.1.4 ECCLRREG	374
10.3.1.5 ECGETREG	374
10.3.1.6 ECGETSLAVES	376
10.3.1.7 ECUNMAPIN	376
10.3.1.8 ECUNMAPOUT	377
10.3.2 CoE Functions	377
10.3.2.1 COEWRITE	377
10.3.2.2 COEREAD	378
11. Stepper Motor Servo Tuning	380
11.1 Stepper Closed-loop Position Correction	380
11.1.1 Stepper Closed Loop Firmware Based	380
11.1.2 Stepper Closed Loop Servo Processor	380
12. Errors & Diagnostics	381
12.1 Error Codes	381
12.1.1 Error Code Ranges	381
12.2 Error Indication	383
12.2.1 Errors in Received Commands	383
12.2.2 Errors in ACSPL+ Programs	383
12.2.3 Motion Termination Codes	383
12.2.4 Motion Termination and Motor Disable Codes	384
12.2.5 Getting Extended Drive Fault Status	385
13. Application Examples	386
13.1 Encoder Error Compensation with Constant Step	386
13.2 Encoder Error Compensation with Arbitrary Step	386
13.3 Backlash Compensation	387
13.4 Compensation of Encoder Error and Backlash	387
13.5 Cam Motion	388
13.6 Joystick	388
13.7 XYZ Rotation	388
Appendix A. PEG And MARK Mapping Tables	391
A.1 ASSIGNPEG Mapping	391

A.2 ASSIGNPOUTS Mapping	398
A.3 ASSIGNMARK Mapping	418

List Of Figures

Figure 2-1. SPiiPlus Controller Hardware Structure	29
Figure 2-2. Multiple SPs Connected via EtherCAT	30
Figure 2-3. The Internal Structure of the Controller Cycle	31
Figure 2-4. User Application Block Diagram	32
Figure 2-5. SPiiPlus MMI Application Studio Main Screen	34
Figure 2-6. Communication Terminal Window	36
Figure 4-1. GRTIME Behavior in PTP or TRACK Motion	118
Figure 4-2. Corner Processing - Exact Path Option	139
Figure 4-3. Corner Processing - Permitted Deviation, Permitted Radius and Corner Rounding Options	139
Figure 4-4. Third-Order Velocity Profile	141
Figure 4-5. Primary Axes Motion	163
Figure 4-6. Secondary Axes Motion	164
Figure 4-7. Example of motion with segment commands	176
Figure 6-1. The Use of Limit Switches	245
Figure 6-2. Use of Variables in a Typical Motion Profile	249
Figure 6-3. 32-bit Error Data Number	264
Figure 6-4. Fault Examination Flow Chart	265
Figure 7-1. SPiiPlus-Plant Connections and Related Parameters	269
Figure 8-1. Incremental PEG Generation Process	285
Figure 8-2. Incremental PEG Pulses Examples	285
Figure 8-3. Random PEG Generation Process	287
Figure 8-4. Random PEG Pulses	287
Figure 8-5. Time-Based PEG Pulses Example	291
Figure 8-6. Simultaneous Connection for Remote Support	318
Figure 8-7. UMD Log Settings - Dump on Request	321
Figure 8-8. UMD Log Settings - Continuous	322
Figure 8-9. Data Entry Dialog	349
Figure 8-10. Screen at the Conclusion of Calculation	350
Figure 8-11. Window Accessed by Download	351
Figure 8-12. Insensitivity Curve Illustration	353
Figure 8-13. Insensitivity Curve without Robust	355

Figure 8-14. Example 1 of Using DRA	357
Figure 8-15. Example 2 of using DRA (zoomed)	357
Figure 8-16. Example of Velocity Error	359
Figure 8-17. Bi-Quad Configured as a Notch Filter	360
Figure 8-18. Bi-Quad Configured as a 2nd Order Lead Filter	361
Figure 8-19. Bi-Quad Configured as a 2nd Order Lag Filter	361
Figure 8-20. Bi-Quad Configured as a 2nd Order Low Pass Filter	362
Figure 11-1. XYZ Rotation	389

List of Tables

Table 2-1. SPiiPlus MMI Application Studio Extensions	35
Table 3-1. Example of the possible index formats.	60
Table 3-2. Mathematical Operators	76
Table 4-1. Motor Modes	196
Table 6-1. Types of Malfuntions	216
Table 6-2. Faults and the Controller's Default Response	219
Table 6-3. Safety Inputs	226
Table 6-4. Safety Related Variables	227
Table 8-1. Typical PEG Engine Loading Times on Products with Support for Fast Loading of Random PEG Arrays	288
Table 8-2. Typical PEG Engine Loading Times on Products without Support for Fast Loading of Random PEG Arrays	289
Table 8-3. Typical Times to Load PEG Engines for the products that support fast loading of Random PEG arrays	292
Table 8-4. Typical Times to Load PEG Engines for the products that do not support fast loading of Random PEG arrays	293
Table 8-5. PEG Output Signal Configuration	304
Table 8-6. Variables and Functions for Hall Support	316
Table 8-7. Hardware Interrupt Callback Conditions	319
Table 8-8. String Format Type	327
Table 8-9. Channel Number Argument	330
Table 8-10. Channel Designation for TRIGGER	333
Table 8-11. AST Bit Descriptions	344
Table 8-12. Digital Outputs for Brake Control in SPiiPlus	365
Table 8-13. Digital Outputs for Brake Control in MC4U	365
Table 9-1. ECST Bits	370
Table 9-2. EtherCAT Error Codes	371
Table 9-3. Supported Error Counter Registers	375
Table 10-1. SPiiPlus Error Code Ranges	381
Table A-1. Mapping PEG Engines to Encoders (Servo Processor 0) for SPiiPlusNT/DC-LT/HP/LD	391
Table A-2. Mapping PEG Engines to Encoders (Servo Processor 1) for SPiiPlusNT/DC-LT/HP/LD	391

Table A-3. Mapping PEG Engines to Encoders (Servo Processor 0) for SPiiPlus CMnt/CMhv/CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa/UDMhv/UDMnt/UDMpa/UDMpm/UDMpc/UDMcb	392
Table A-4. Mapping PEG Engines to Encoders (Servo Processor 0) for UDMlc/UDlIt/UDlhp/UDMmc/PDIcl	393
Table A-5. Mapping PEG Engines to Encoders (Servo Processor 0) for NPMpm/NPMpc-	393
Table A-6. General Purpose Outputs Assignment for Use as PEG Pulse Outputs (Servo Processor 0) for SPiiPlusNT/DC-LT/HP/LD	394
Table A-7. General Purpose Outputs Assignment for Use as PEG Pulse Outputs (Servo Processor 1) for SPiiPlusNT/DC-LT/HP/LD	395
Table A-8. General Purpose Outputs Assignment for Use as PEG Pulse Outputs (Servo Processor 0) for SPiiPlus CMnt/UDMpm/CMhv/UDMhv-	395
Table A-9. General Purpose Outputs Assignment for Use as PEG Pulse Outputs (Servo Processor 0) for UDMnt/UDMpa/UDMcb	396
Table A-10. Engine to Encoder Assignment for IDMxx, ECMxx, and UDMxx	397
Table A-11. Mapping of Engine Outputs to Physical Outputs (Servo Processor 0) for SPiiPlusNT/DC-LT/HP/LD	398
Table A-12. SPiiPlusNT/DC-LT/HP/LD Mapping of Engine Outputs to Physical Outputs (Servo Processor 1)	399
Table A-13. Mapping of Engine Outputs to Physical Outputs (Servo Processor 0) for CMnt/UDMpm/UDMpc/CMhv/UDMhv	400
Table A-14. Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, OUT 0-4) for CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa	401
Table A-15. Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, OUT_5-9) for CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa	402
Table A-16. Mapping of Engine Outputs to Physical Outputs (Servo Processor 0) for UDMnt/UDMpa/UDMcb	403
Table A-17. Mapping of Engine Outputs to Physical Outputs (Servo Processor 0) for UDMlc/UDMmc/UDlIt/UDlhp/PDIcl	404
Table A-18. NPMpm/NPMpc Mapping of Engine Outputs to Physical Outputs (Servo Processor 0)	404
Table A-19. IDMsm/ECMsm/UDMsm Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, Outputs 0-5)	406
Table A-20. IDMsm/ECMsm/UDMsm Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, Outputs 6-11)	408
Table A-21. ECMsa/IDMsa/UDMsa Mapping of Engine Outputs to Physical Outputs (Servo Processor 0)	410
Table A-22. ECMma / IDMma / UDMma Mapping of Engine Outputs to Physical Outputs (Servo Processor 0)	411

Table A-23. ECMdx / IDMdx / UDMdx Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, Outputs 0-5)	414
Table A-24. ECMdx / IDMdx / UDMdx Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, Outputs 6-11)	416
Table A-25. Mark-1 Inputs to Encoders Mapping for SPiiPlusNT/DC-LT/HP/LD	419
Table A-26. Mark-2 Inputs to Encoders Mapping for SPiiPlusNT/DC-LT/HP/LD	421
Table A-27. Mark-1 Inputs to Encoders Mapping for with SPiiPlus CMnt/UDMpm-x/UDMpc/CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa/CMhv/UDMhv	422
Table A-28. Mark-2 Inputs to Encoders Mapping for with SPiiPlus CMnt/UDMpm/UDMpc/CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa/CMhv/UDMhv	423
Table A-29. IDMxx/ECMxx/UDMsm/UDMsa/UDMma Encoder Mapping	424

1. Introduction

This guide provides a general overview for programming the SPiiPlus™ motion controller products using the ACSPL+ programming language.



The term “controller” is used in this guide whenever information applies for both controllers and control modules. If information applies to only one of these product groups, the group is stated explicitly.

1.1 ACSPL+ Programming Language

ACSPL+ is a powerful programming language developed specifically for SPiiPlus motion controllers. ACSPL+ incorporates many advanced features, including: powerful programming elements such as arithmetical and logical expressions, user-defined variables with local and global scope, user-defined one- and two-dimensional arrays.

SPiiPlus ACSPL+ enables:

- > Execution of up to 32 programs simultaneously
- > Program isolation - each program resides in a separate buffer
- > Rich set of motion types, providing a large degree of versatility
- > Advanced implementation of master-slave motion
- > Axis-independent programming
- > On-condition autoroutines

Complete details of all ACSPL+ commands and variables are given in the *SPiiPlus Command & Variable Reference Guide*.

ACSPL+ libraries are provided for host programming in other high level languages. The library for C, C++, and Visual Basic are described in the *SPiiPlus C Library Reference*. Routines for synchronizing communication between the host program and the SPiiPlus motion controller are given in *SPiiPlus COM Library*. Routines using .NET are described in the *SPiiPlus SPiiPlus .NET Library Programmer's Guide*.

2. SPiiPlus Architecture

This chapter provides an overview the SPiiPlus architecture.

2.1 Hardware Structure

The following diagram shows the principal parts of the SPiiPlus controller hardware:

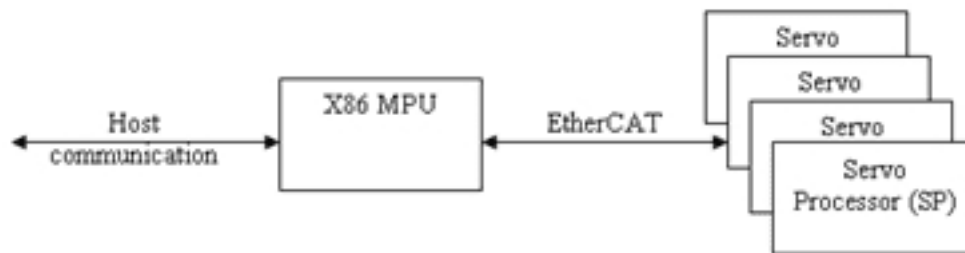


Figure 2-1. SPiiPlus Controller Hardware Structure

The Motion Processing Unit (MPU), which executes most of the controller tasks, is a powerful x86 processor. The MPU is an EtherCAT bus master and the Servo Processors (SP) are EtherCAT slaves. One master can control several slaves.

The master MPU runs compiled ACSPL+ programs and generates motion profiles to the SPs. Its principle tasks are:

- > Communication with the SPs
- > Motion profile generation (calculation of **APOS**)
- > Calculation of Reference Position (**RPOS**)
- > Safety control
- > Data collection
- > Position Event Generation (**PEG**)
- > Processing of Index and Mark inputs
- > Execution of ACSPL+ programs
- > Communicating to Serial Link or Ethernet
- > Execution of Immediate commands received from the Host
- > Housekeeping

The MPU is equipped with a Flash (nonvolatile) memory that retains the stored data after the power to the controller is turned off.

The SP executes the realtime tasks, such as implementation of the realtime control algorithms. Each SP controls four axes. The SP includes all the necessary peripherals that are needed for a high performance axis control.

A single MPU module can manage several units over the EtherCAT bus thus expanding the number of controlled axes. For example,



Figure 2-2. Multiple SPs Connected via EtherCAT



The part of the system that is connected to the rest of the system via the EtherCAT bus is called a **UNIT**.

2.1.1 Firmware

The firmware consists of a set of files factory-written in the flash memory.

You cannot erase or modify the firmware; however, you are able to update the firmware version with a special tool that is part of the SPiiPlus MMI that is supplied with the controller.

The firmware files include the following:

- > MPU program.
- > Default values of the controller's configuration variables.

2.1.2 Controller Cycle and Servo Tick

The firmware operates in a rigid realtime framework. The two principal parts of the firmware, the MPU program and the SP programs are realtime programs operating in strict synchronism.

The SP interrupt has a fixed interval period (called a "servo tick") of 50 μ sec (20kHz). Most SP tasks are executed each servo tick. Therefore, the SP executes the servo control algorithm at a constant rate of 20kHz irrespective of the number of axes and other factors.

The 20kHz SP clock is divided by a factor of 20, generating a 1kHz clock for MPU interrupts. The new clock, the controller cycle, is 1000 μ sec.

2.1.3 Realtime and Background Tasks

MPU program tasks are divided into two categories:

- > Realtime tasks
The realtime tasks are executed in strict synchronism with the MPU interrupt. Each controller cycle, a required part of each realtime task is executed. The overall time of all realtime tasks executed in one controller cycle must be less than the one controller cycle.

The following MPU tasks are the realtime tasks that are executed each controller cycle:

- > Communication with the SPs
- > Motion profile generation (calculation of **APOS**)
- > Calculation of Reference Position (**RPOS**)
- > Safety control
- > Data collection
- > Position Event Generation (**PEG**)

- > Processing of Index and Mark inputs
- > Execution of ACSPL+ programs
- > Background tasks

Background tasks are not synchronous with the MPU interrupt. Execution of a background task may overlap two or more controller cycles.

The following MPU tasks are the background tasks and are asynchronous to the controller cycle:

- > Communicating to Serial Link or Ethernet
- > Execution of Immediate commands received from the Host
- > Housekeeping

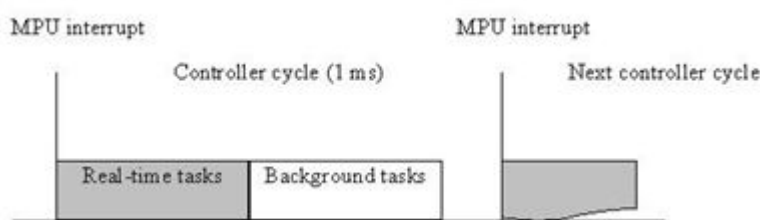


Figure 2-3. The Internal Structure of the Controller Cycle

The MPU interrupt invokes the realtime tasks. When all realtime tasks required in the current cycle are completed, the controller starts executing the background tasks. If the background tasks complete before the next MPU interrupt occurs, the controller remains idle for the rest of the cycle.

The exact time of the realtime and background tasks in each controller cycle depends on many factors and cannot be precisely specified. The following paragraphs explain different situations in controller cycle utilization.

If the background task execution does not finish in one controller cycle, the background execution is interrupted and continues after execution of the realtime tasks during the next cycle. Therefore, background tasks may overlap into the next MPU interrupt without causing a problem. However, overflow of realtime tasks into the next MPU interrupt is abnormal, and may cause problems with program execution. When this occurs, the controller latches the **TIME OVERUSE** fault. This fault has no default response in the controller, but your application can monitor the fault and define a proper response.



You can monitor if the usage of the controller cycle is close to critical. The SPiiPlus MMI Application Studio Communication Terminal command: **#U** when entered, reports the usage as a ratio of realtime tasks execution time and the controller cycle. A maximum value of 90% is considered dangerous. If the usage limit is reached, you have to modify your application.

2.2 User Application

This section provides a look at the elements that go into the construction of a user SPiiPlus application.

2.2.1 Firmware, User Application and Tools

The firmware is a program that is stored in the controller's nonvolatile (flash) memory, that defines the basic functionality of the controller. Among these functions are the preparing, storing and executing your applications.

Your application tailors the controller to your specific controlled plant. The controller can control various plants with different number of axes, mechanical construction, timing requirements, etc. Your application specifies the exact control and monitoring actions that must be executed in different conditions, including the exact sequences of motions, activation of outputs, response to inputs and interactions with the operator.

SPiiPlus Tools are Windows-based programs that provide you with support in different stages of the application such as initial set up and tuning, ACSPL+ application development, host application interaction with the controller, and manual control.

2.2.2 User Application Components

The following diagram shows the parts of a user application in the gray blocks and the relevant parts of firmware in the white blocks:

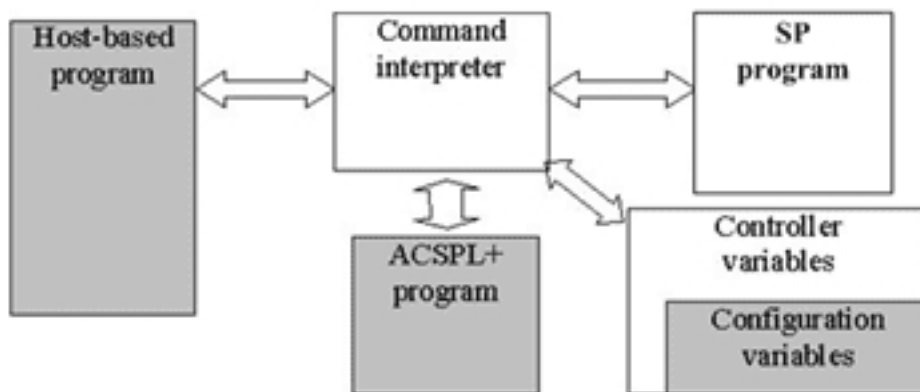


Figure 2-4. User Application Block Diagram

- > **Host-based program:**
A program written in C, C++ or any other programming language that runs on the host computer and communicates with the controller. The host-base program uses any communication channel provided by the controller; serial link, Ethernet, FIFO, dual port ram. The program issues commands to, and reads data from, the controller. The program can provide front-end user interfaces, motion sequencing, high-level decision-making and other application specific functions. This part of user application can be absent if the controller works stand-alone without connection to the host. The design of host-based programs is not the primary subject of this guide. For Windows programming, refer to the *SPiiPlus C Library Reference Guide*.
- > **ACSPL+ program:**
A sequence of ACSPL+ commands can be downloaded to the controller as an ACSPL+

program. There are 10 buffers for ACSPL+ programs. An ACSPL+ program is executed inside the controller with strict timing and with no communication delay.

ACSPL+ programs are almost always present in user applications. Occasionally, the ACSPL+ programs are absent and the host commands all controller actions.

- > Configuration variables:
The firmware includes a set of predefined variables that can be used by ACSPL+ programs and by Immediate commands. The configuration variables are included in this set. The values of the configuration variables are defined by the user to tailor the controller operation to a specific task and plant control. For example, **ACC** defines the acceleration that is used for motion generation. The **SAFINI** variable defines the polarity of the input safety signals.
The configuration variables must always be present in user applications.
- > SP programs:
The firmware includes SP (Servo Processor) real time control programs as a standard part of the controller.

2.2.3 User Applications Categories

You can use different strategies to build an application, including:

- > Stand-alone application:
No physical link to a host. No host-based program. ACSPL+ programs are stored in the controller flash memory. The ACSPL+ programs start running after power-up and implement all application functions.
- > Host-driven application:
No ACSPL+ programs. The host issues all commands to be executed by the controller. This approach is applicable only for non-time-critical applications.
- > Hybrid application:
A host-based program plus one or more ACSPL+ programs. Most user applications fall into this category.

2.2.4 SPiiPlus MMI Application Studio

SPiiPlus MMI Application Studio is a multipurpose user interface with the controller that provides the user with the means to fully control and monitor the performance of the motion controller.

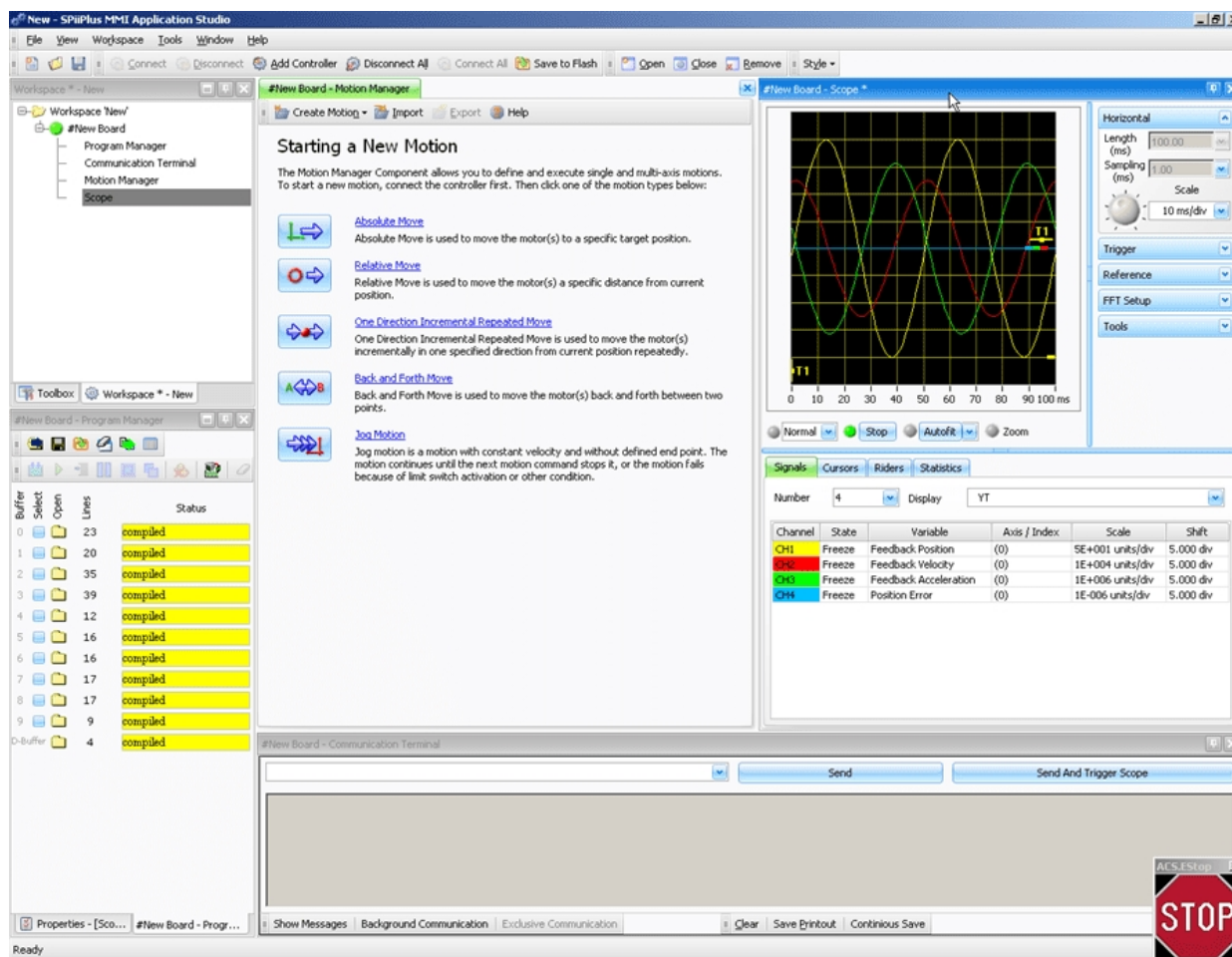


Figure 2-5. SPiiPlus MMI Application Studio Main Screen

The main features of the SPiiPlus MMI Application Studio are, amongst others:

- > Program Manager
Used for entering user-written programs into the motion controller's buffers.
- > Motion Manager
Used for completely defining the motion for all axes in the system.
- > Communication Terminal
Used for entering commands directly into the controller.
- > Scope
A digital oscilloscope providing a realtime graphic display of the motion.
- > Variables Manager and Watch
Enables the user to set watch windows for the values of critical variables.

For complete details see *SPiiPlus MMI Application Studio Guide*.

2.2.5 File Extensions Used in SPiiPlus MMI Application Studio

Several file formats are used to store data and programs used with SPiiPlus MMI Application Studio. The file formats are:

Table 2-1. SPiiPlus MMI Application Studio Extensions

File Extension	Content	Associated SPiiPlus MMI Application Studio Component
.acsw	Workspace configuration	Workspace
.awd	Drive database	Adjuster Wizard
.awf	Feedback database	Adjuster Wizard
.awm	Motor database	Adjuster Wizard
.frf	FRF data	FRF Analyzer
.log	MMI log files	All components that generate logs
.par	Controller parameters	Configuration Wizard
.prg	ACSPL+ Program	Program Manager
.rtf	Print to file	All components that enable printouts
.sgn	Scope data	Scope
.spi	Application (includes controller parameters + adjustment parameters + ACSPL+ program + SP files)	Upgrade and Recovery Wizard

2.3 Programming Resources

The controller-based parts of the user application operate in the environment created by the firmware. The environment includes a set of resources that the user application can use. This section provides a short description of the available resources.

2.3.1 Commands

The controller supports a rich set of commands which are divided into two types of command sets:

- > **Terminal Commands**
Terminal commands are those that are sent directly to the controller. They are entered through the SPiiPlus MMI Application Studio Communication Terminal. The general structure of the Communication Terminal windows is:

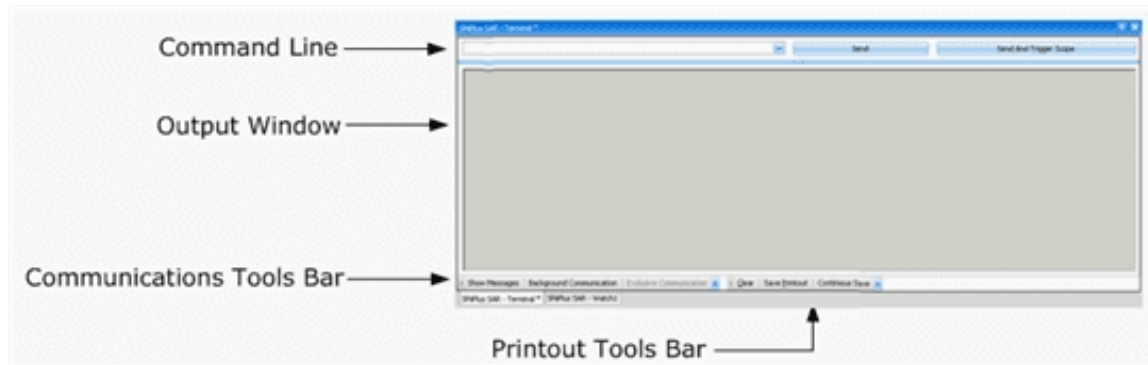


Figure 2-6. Communication Terminal Window

The Communication Terminal window is described in the *SPiiPlus MMI Application Studio Guide*.

As soon as the command is received through one of the communication channels it is executed. Each Terminal command starts with ? (query command) or # (program management command), for example:

```
?FPOS    Display the current position of all motors.
#0L      List the complete program in buffer 0.
```

A Terminal command cannot be stored in a buffer. Once a Terminal command is received via any communication channel, the controller executes it immediately or rejects the command if it cannot be executed.

> **Buffered Commands**

The controller stores a sequence of commands in a buffer and executes them as a program. ACSPL+ commands can either be executed immediately or can be stored in a buffer. Examples of ACSPL+ commands that are stored in a buffer:

```
ENABLE 0      Enable axis 0
Var = 5*Var2   Assign the result of expression to variable Var
WAIT 50       Delay the program for 50 msec
PTP 0,3000     Move the 0 motor to point 3000
```

2.3.2 Program Buffers

The controller provides up to 64 buffers for storing ACSPL+ programs (depending on controller configuration). The controller defines the size of each buffer according to the required size of the program. For all practical purposes, you can consider the size of each buffer to be unlimited.

A program stored in a buffer can be edited, compiled and executed independently of the programs in other buffers. For example, a program in buffer 0 may be running while you edit the program in buffer 1.

Programs stored in different buffers can be executed concurrently. Each buffer defines an execution thread connected to this buffer. When you activate a program in a buffer, the program is executed in a separate thread. Therefore, up to 64 ACSPL+ programs (depending on controller configuration) can be executed concurrently.

A buffer also provides isolation between the programs. All labels and local variables defined in a program are isolated in their buffer and are inaccessible from any other buffer. For example, two programs can contain identical user-defined labels, but the controller considers each label as belonging only to the buffer in which it is contained, and relates to all references to the label appropriately.

2.3.2.1 ACSPL+ Buffers Compilation Behavior

Until Version 2.40, when compiling all ACSPL+ buffers together (either in the SPiiPlus MMI Application Studio, by using `CompileBuffer()` function with `ACSC_NONE` parameter, or by issuing `##C` command), the D-Buffer was compiled last and in most of the cases was not compiled at all.

Starting from Version 2.40, the D-Buffer is compiled first and therefore all global variables that are defined in D-Buffer, are set to 0 as part of the compilation process. These global variables must later be set to their desired values.

2.3.3 Declaration Buffer (D-Buffer)

The D-Buffer is an additional special buffer that provides a place for the definitions of axis names and global variables.



Executing programs and autoroutines is not supported in D-Buffer.

2.3.3.1 Defining Global Objects in D-Buffer

Axes and global variables defined in D-Buffer are not required to be defined in other buffers before use. However, such redefinition is not an error, given all attributes of the definitions are identical.

The **#SAVE** and **#SAVEPROG** commands store the D-Buffer in the flash along with other buffers.



The values are also stored using the SPiiPlus MMI Application Studio Program Manager Buffer Editor.

At start-up, the controller loads and compiles the D-Buffer before any other buffers.



After any change in the D-Buffer, all other buffers should be recompiled.

2.3.3.2 D-Buffer Default Contents

The default contents of D-Buffer differ from other buffers. The other buffers are initially empty by default; however, the D-Buffer contains a set of definitions that provides compatibility with previous firmware versions. The default contents of the D-Buffer are:

```
axisdef X=0,Y=1,Z=2,T=3,A=4,B=5,C=6,D=7
axisdef x=0,y=1,z=2,t=3,a=4,b=5,c=6,d=7
global int
I(100),I0,I1,I2,I3,I4,I5,I6,I7,I8,I9,I90,I91,I92,I93,I94,I95,I96,I97,I98,I99
global real
V(100),V0,V1,V2,V3,V4,V5,V6,V7,V8,V9,V90,V91,V92,V93,V94,V95,V96,V97,V98,V99
```

This provides you with the means to define names for the axes in your system (axisdef). The two 100 element arrays, one an integer and one a real, are, however, for internal use.

The **#RESET** command restores the default content in the D-Buffer.

2.3.4 Command Execution

2.3.4.1 Terminal Commands

Once a Terminal command is received from the SPiiPlus MMI Application Studio Communication Terminal, the controller executes it immediately or rejects the command if it cannot be executed.

The controller executes Terminal commands as a background task. One Terminal command cannot be interrupted by another. The controller finishes execution of a command, sends a reply, and only then continues to the next command. Therefore, a host-based process that sends commands is guaranteed to receive the replies in the same order as the commands were sent.

Typically, execution of a Terminal command takes 1-2 controller cycles. The commands that supply or request a great amount of data, such as query of a large array, may require a longer processing time.

Processing time can also be affected by a high MPU usage. The realtime tasks always have the greatest priority. If the usage (percentage of the realtime tasks in the controller cycle) reaches 90% or more, the response time of the controller deteriorates. If an application requires the fastest response to Terminal commands, you must keep the usage below 50%.

2.3.4.2 ACSPL+ Commands

There are two methods for executing ACSPL+ commands:

- > Execute immediately - via the SPiiPlus MMI Application Studio Communication Terminal.
- > Store a sequence of commands in a buffer and then execute the sequence as an ACSPL+ program.

If the prompt is: (colon), no program buffer is open for editing, and an ACSPL+ command, transmitted to the controller through the Communication Terminal, is executed immediately.

Immediate execution of ACSPL+ commands is a background task. Therefore, the processing time can be affected by a high MPU usage. The realtime tasks always have the highest priority. If the usage (percentage of the realtime tasks in the controller cycle) reaches 90% or more, the response time of the controller deteriorates. If an application requires the fastest response to Immediate commands, you must keep the usage below 50%.

Executing an ACSPL+ program from a buffer is different because the controller executes a buffered ACSPL+ program as a realtime task. Up to 32 buffered ACSPL+ programs can be executed simultaneously, and in parallel.

2.3.5 ACSPL+ Standard Variables

ACSPL+ standard variables are a set of predefined variables provided by the controller. You can use ACSPL+ standard variables in any command, either immediate or buffered, without having to declare them.

The ACSPL+ standard variables are divided into two categories:

- > State variables - Examples are **FPOS** (Feedback Position), which reports the immediate position of a motor and **MST** (Motor State), which indicates the motor status, including whether it is enabled and whether is involved in motion.
- > Configuration variables - The values of the configuration variables tailor the controller to a specific control object. Examples are **ACC** (Acceleration), which specifies a tolerable acceleration of a motor and **SLLIMIT** (Software Left Limit), which specifies a lower limit for the area of motion, etc.

ACSPL+ variables are mentioned throughout this guide where they relate to a particular element or feature of ACSPL+.

2.3.6 User-Defined Variables

In addition to the set of ACSPL+ standard variables, you can declare variables with user-defined names as required in the application.

A user-defined variable can be declared as scalar, one-dimensional array (vector) or two-dimensional array (matrix).



A user array can contain up to 100,000 elements.

A user-defined variable can be declared as local or global. Local variables are accessible only within the buffer that the declaration resides in. Global variables are common to all buffers and can be accessed from any buffer.

2.3.7 Nonvolatile Memory and Power Up Process

The on-board nonvolatile (flash) memory retains information while power is off.

The flash memory stores the following data:

- > Firmware as well as SP programs
- > User application

A new controller is supplied with only the firmware and SP programs stored in the nonvolatile (flash) memory. You cannot erase or modify the firmware or the SP programs. You can, however, store ACSPL+ programs and configuration variable values in the controller's flash memory.

During the power-up process the controller loads the firmware, the ACSPL+ programs, the configuration variables and the SP programs from the flash memory. If you did not store any component of configuration variables, the controller loads the default firmware component.

2.4 Executing ACSPL+ Programs

2.4.1 Program Buffers

The controller provides up to 64 program buffers.

Each buffer provides:

- > Separate storage for each ACSPL+ program
- > An isolated environment for program editing/execution
- > Separate thread for concurrent execution
- > Independent autoroutine execution

Each buffer is managed independently of the other buffers. For example, you can edit a program in one buffer while the program in another buffer is executing.

All labels and local variables in a program are local in the encapsulating buffer. Programs in other buffers do not have access to these label and variables. Even if two programs in different buffers both define a local variable with the same name, the variables are considered as two different variables, each in its corresponding buffer.

A program in a buffer can be executed independently of any other program. The program executed in a buffer does not affect the program executed in other buffers, unless you have provided for synchronization through the global variables or common resources.

If a program in a buffer includes one or more autoroutines, the buffer manages the autoroutines independently of other buffers. If an autoroutine condition is satisfied, only the program executed in the enclosing buffer is interrupted. No other buffer is affected.

The time allotted for processing non-executable lines, e.g., comment, a new line, a label, etc., is controlled by the **S_FLAGS.1** bit. If the bit is 0 (default), the non-executable line will be skipped during execution. If the bit is 1, the line will be allotted a controller cycle. executed as before taking a standard time for execution.

The bit affects the program compilation; therefore, if the bit is changed, the results will be visible only after a program is recompiled.

2.4.2 Execution of a Single Program

Assume a compiled ACSPL+ program containing no errors is stored in buffer 0, and you issue the following command in the SPiiPlus MMI Application Studio Communication Terminal:

```
#0X      Execute the program in buffer 0
```

The controller starts execution from the first line.

The controller executes the program according to this simple model:

- > One program line is executed each controller cycle. If a line contains several ACSPL+ commands, all them are executed in one controller cycle. See also [Execution Rate](#).
- > If a program is linear, meaning that it contains no program flow commands, like goto, and no autoroutines, the program lines are executed sequentially: the first line in the first controller cycle, the second - in the second controller cycle, and so on. A program flow

command redirects execution and defines another line to be executed in the next controller cycle.

- > A number of commands can delay program execution. For example, the command:

```
WAIT 50
```

- > will execute for 50 milliseconds instead of one controller cycle. The command:

```
TILL ^MST(0) .#MOVE
```

provides a delay of execution time until the 0 axis motion ends.

Commands that use the controller resources, like motion commands, also can be delayed if the resource is busy. After a command that caused a delay has finished, the controller continues executing one line per one cycle.

2.4.3 Concurrent Execution

Assume the programs in buffers 0, 1, 2 are linear and include no commands that can delay execution. After starting simultaneously, the programs execute in a simple and predictable way: one line of each running program per one controller cycle.

In the first controller cycle the controller executes line number 1 from buffer 0, line number 1 from buffer 1 and line number 1 from buffer 2. In the second controller cycle the controller executes line number 2 from buffer 0, line number 2 from buffer 1 and line number 2 from buffer 2, in progression.

If a program contains a program flow command or a command that delays execution, this strict synchronization vanishes, but the general principle remains the same: one line of each running program per each controller cycle.

The rule does not depend on the number of concurrent programs. If all 64 programs run concurrently, the controller executes 64 ACSPL+ lines in each controller cycle - one from each running program.

Delay in one executed program has no affect on other executed programs. Each buffer provides an isolated thread for program execution.

Within a controller cycle, the order of executing the program lines follows the buffer numbers: first, a line from buffer 0 is executed, then a line from buffer 1, and so on.

2.4.4 Immediate Execution

What happens if while one or more programs are running at the same time and the controller receives a SPiiPlus MMI Application Studio Communication Terminal ACSPL+ command through a communication channel?

The controller executes the Communication Terminal command in an additional thread not connected with any buffer. Therefore, the controller provides up to 64 + 1 threads for ACSPL+ execution. Up to 64 threads are assigned to the program buffers and one thread is reserved for immediate execution.

All rules of the execution model also apply to immediate execution. For example, if several commands are combined in one line for immediate execution, all of them are executed in one controller cycle in parallel with the lines from each running program.

Within a controller cycle, the immediate line is executed after all lines from the running programs. Therefore, in one cycle the controller executes a line from buffer 0, then a line from buffers 1, 2, up to 63, and then executes a Communication Terminal command line (if any has been received).

2.4.5 Immediate Execution vs. Stored Program

The controller provides three options for executing ACSPL+ commands:

- > Execute a command immediately
 - > Store a sequence of commands in a buffer and then execute the sequence as an ACSPL+ program
 - > Execution in a dynamic buffer
- If the controller prompt is: *No program buffer is open for editing*, and an ACSPL+ command, transmitted to the controller through any communication channel, is executed immediately. If the prompt contains a line number like 2:00001>, a program buffer is open for editing, and an ACSPL+ command, transmitted to the controller through any communication channel, is stored in the open buffer. ACSPL+ commands stored in a buffer constitute an ACSPL+ program.

2.4.6 Autoroutine Execution

An autoroutine is a part of ACSPL+ program and can be placed in any program buffer. Each ACSPL+ program can contain from zero to any number of autoroutines. An autoroutine placed in a buffer shares the local variables and the same thread for execution with other autoroutines in the same buffer and with the rest of ACSPL+ program.

If a buffer contains one or more autoroutines, the execution model described above is slightly modified. In each controller cycle, the controller examines the conditions of all autoroutines in the buffer, before executing the next line in a buffer. If a condition is satisfied, the controller does not execute the next line but executes the first line of the body of the corresponding autoroutine. Therefore, the autoroutine interrupts the program executed in the same buffer. An autoroutine in one buffer has no affect on the program or autoroutines in another buffer.

Autoroutines provide an interrupt-like response to external or internal events. For more information about autoroutines see [Autoroutines](#).

2.4.7 Synchronization and Mutual Exclusion

Though the controller provides independent execution of concurrent programs, applications can often require that their component programs be synchronized at certain points. The controller provides a simple and flexible approach to solving synchronization problems. In addition to an ACSPL+ line serving as a unit for concurrent execution, the same ACSPL+ line serves as a unit of mutual exclusion. Namely, execution of an ACSPL+ line cannot be interrupted by a concurrent program or by an autoroutine.

Therefore, an ACSPL+ line provides an atomic unit of execution. Given that a single ACSPL+ line can contain any number of ACSPL+ commands, various synchronization tasks can be resolved using global variables.

2.4.7.1 Mutual Exclusion

Mutual exclusion is the most frequently used synchronization task. As the execution of an ACSPL+ line is atomic, mutual exclusion of the lines in concurrent programs is automatic and does not

require any intervention from the user. If a critical section requiring mutual exclusion comprises only a few commands, then you simply place these commands in one line.

However, if a critical section is long, or combining it in one line is undesirable for any reason, another solution must be found. The following construction implements a simple semaphore, which is ON (one) while the program is inside the critical section, and is OFF (zero) otherwise.

<code>global int Mutex</code>	Variable Mutex implements semaphore
<code>.</code>	Any program actions
<code>TILL ^Mutex; Mutex = 1</code>	Enter critical section
<code>.</code>	Critical section
<code>Mutex = 0</code>	Exit critical section

The Enter and Exit lines enclose the critical section. If the program contains several critical sections, each section must be enclosed with the same Enter and Exit lines as shown in the example above.

All programs that require mutual exclusion must include the same declaration of the Mutex variable and the same embracing of each critical section.

This construction guarantees that only one of the concurrent programs may be inside a critical section. If the second program tries to enter the critical section, the command **TILL ^MUTEX** delays the program execution until the first program zeroes **MUTEX** on exit from critical section.

It should be noted that the solution is based on automatic one-line mutual exclusion. Therefore, the two commands:

`TILL ^Mutex; Mutex = 1`

must be in one line. If they are placed in two sequential lines, they cannot provide mutual exclusion.

2.4.7.2 Synchronization

Assume two programs that run mostly asynchronously must execute certain commands at the same time. There is a point in each program that whichever program comes to the point first, it must wait for the second program to come to its synchronization point. Then the next lines in both programs will be executed in the next controller cycle.

The problem is solved by the following construction:

<code>global int Sem</code>	The SEM variable implements a general semaphore
<code>.</code>	Asynchronous part of the program
<code>Sem = Sem+1; till Sem = 2; Sem = 0</code>	Synchronization point
<code>.</code>	The line will be executed synchronously

The same definition of the **SEM** variable and the same line of synchronization point must be in the second program.

Whichever program comes to its synchronization point first, the command **TILL SEM = 2** provides waiting for the second program. The assignment **SEM = 0** provides reuse of the construction if necessary.

The solution can be also be extended to three or more concurrent programs.

2.4.7.3 Execution Rate

In the execution model described above, the controller executes one line from each running program per each controller cycle. The execution rate in each buffer is the same.

You can modify the execution model by specifying how many lines in each buffer the controller must execute in one controller cycle.

ACSPL+ variables **PRATE** and **ONRATE** (see *SPiiPlus Command & Variable Reference Guide*) contain one element per each program buffer. Each element of **PRATE** specifies how many lines in the corresponding buffer the controller must execute in one controller cycle, except the case if an autoroutine is executed. Each element of **ONRATE** specifies the same when an autoroutine is executed.

For both variables, the default value is 1 (one line per each controller cycle). The user can increase the value up to 10 (ten lines per each controller cycle).

In a typical case the user increases **PRATE** and **ONRATE** in one buffer, providing the higher execution rate in this buffer. The program in this buffer runs faster, than the programs in either buffers, as if the buffer has a higher priority.

You should, however, exercise caution when simultaneously increasing the execution rate in buffers. Increasing the execution rate increases the usage of the controller realtime cycle and may cause a Time Overuse fault. Use the **#U** Immediate command (see *SPiiPlus Command & Variable Reference Guide*) to monitor the present usage. If maximum usage value approaches 90%, the application places an excessive load on the controller. Decrease execution rates, simplify your application, or use a more powerful model of the controller, to solve the problem.

3. ACSPL+ Overview

SPiiPlus enables running up to 64 separate ACSPL+ programs. The programs are stored in what are referred to as "buffers". The programs are entered via the SPiiPlus MMI Application Studio Program Manager (see *SPiiPlus MMI Application Studio User Guide* for details).

This chapter provides a general overview of ACSPL+ programming.

For complete details of the ACSPL+ command set and variables refer to the *SPiiPlus Command & Variable Reference Guide*.

3.1 ACSPL+ Syntax

3.1.1 Commands, Lines, and Command Aggregates

A command is the smallest executable unit of ACSPL+.

One program line may contain one or several commands. If a line contains several commands, the commands must be separated by a semicolon (;). A semicolon after the last (or single) command in a line, however, is not required.

Examples:

The following is an example of a program line that contains one assignment command:

```
V0 = V1 + 2*(V2 - V3)
```

The following is an example of a program line that contains two assignment commands:

```
V0 = V1 + 2*(V2 - V3); V4 = 0
```

White spaces (spaces and tabs) may be inserted arbitrarily inside or between commands. However, white spaces must not be inserted within a keyword or variable name.

A command aggregate consists of several commands. It starts with a specific command and terminates with the **END** command. For example, a loop structure starts with the loop command followed by an arbitrary number of commands and terminates with the **END** command signalling the completion of the loop. The commands of a structure may reside in one or more program lines.

3.1.2 BLOCK..END Control Structure

The commands specified within the **BLOCK..END** structure are executed in one controller cycle.

The structure has the following syntax:

```
BLOCK command_list END
```

The structure provides an alternative to specifying **command_list** commands in one line. The commands within the structure can be specified in several lines. However, the controller executes all commands in one controller cycle, as if they were written in one line.

The following limitations are applied to the commands within the structure:

Commands and functions that may cause delay (**WAIT**, **TILL**, **GETSP**, **WHILE**, **LOOP**, etc.) provide delays even if they are used within the **BLOCK..END** structure.

3.1.3 ACSPL+ Reserved Words

All ACSPL+ function and variable names (see the *SPiiPlus Command & Variable Reference Guide*) are reserved words that have specific meanings. They cannot be used as user-defined names in a program.

3.1.4 Names: Variable and Label

A name is a sequence of alphanumeric characters used to denote one of the following:

- > Variable (ACSPL+ or user-defined)
- > Label

The first character of a name must be a letter.

The names of ACSPL+ variables are predefined and cannot be changed. For a full list of ACSPL+ variables see the *SPiiPlus Command & Variable Reference Guide*.

There is also an ACSPL+ label: **AUTOEXEC**. If this label is used in a program, then execution will start at this point when the controller is powered up.

You can declare user variables and user labels as required in the application. The number of user-defined names and the length of each name are essentially unlimited. However, you may not declare variable names that coincide with the following:

- > Keywords (**IF**, **WAIT**, **SIN**, etc.)
- > Names of ACSPL+ variables (**FPOS**, **MST**, etc.)
- > **AUTOEXEC** (ACSPL+ label)
- > Postfix- indexed form of an ACSPL+ variable

3.1.5 Case Sensitivity

All keywords are case-insensitive, e.g., the words **if**, **IF**, **If** or **iF** all have identical meaning in a program.

Variable and label names, however, are case-sensitive, e.g., the names **FPOS** and **Fpos** designate two different variables.

3.1.6 Axis Designations

Many ACSPL+ commands take one or more axes as arguments. This applies particularly for motion commands (see [ACSPL+ Motion Programming](#)).

Axes can be designated as a single digit, like **0** (indicating the 0 axis), or as a list of values enclosed within parentheses, for example, **(0, 2, 4)**. The values can also be represented by variables.

Some commands support the keyword: **ALL**, to designate all the axes supported by the controller.

The following examples are equivalent:

AXISDEF XY=0	!Define axis variable
ENABLE XY	!Enable axis 0
PTP XY, 100, 200	
ENABLE (0,1)	!Axes designated as list of values.
PTP (0,1), 100, 200	

```

int first_axis, second_axis      !Variable declarations.
first_axis = 0                   !Variable assigned value.
second_axis = 1                  !Variable assigned value.
ENABLE (first_axis, second_axis) !Axes designated as list of variables.
PTP (first_axis, second_axis), 100, 200

```

3.1.7 Comments

A comment is text in a program that the controller stores along with the program but ignores while executing the program. Comments are normally used to annotate a program.

A comment starts with an exclamation mark (!). An exclamation mark encountered in a line designates all subsequent characters in the line as part of a comment. If the exclamation sign is the first character in a line, the whole line is a comment.

! This entire line is a comment.

V0 = V1!This comment starts from the exclamation mark.

3.2 Variables

Variables have the following attributes:

- > Name
- > Class (standard or user variable)
- > Scope (global or local)
- > Lifetime
- > Accessibility (read-write, read-only, protected)
- > Type (integer, real, or matrix)
- > Size
- > Value



The controller has a large set of ACSPL+ variables with predefined names. You cannot change these names. These ACSPL+ variables can be used in ACSPL+ commands without an explicit declaration.

3.2.1 Variable Name

Variable names follow the general syntax rules given in [Names: Variable and Label](#).

User-defined variable names must be specified by explicit declaration. For example:

```

int Var1, Var2      !Declare local variables Var1 and Var2
global real RealVar !Declare global variable RealVar

```

3.2.2 Variable Class: ACSPL+ or User

ACSPL+ variables are predefined in the controller. ACSPL+ variables can be used in ACSPL+ commands without an explicit declaration.

ACSPL+ variables are mentioned throughout this guide where they relate to a particular element or feature of ACSPL+. For details of the ACSPL+ variables see the *SPIIPlus Command & Variable Reference Guide*.

User-defined variables are defined by explicit declarations. A declaration can appear as:

- > A part of an ACSPL+ program. The declared variable becomes available when the program is inserted into one of the program buffers and is compiled. Any attempt to query a variable in a buffer window that has not been compiled will result in an error.
- > An immediate ACSPL+ command. Only global variables can be used as a Terminal command. The declared variable becomes available immediately after the controller executes the command.

3.2.3 Variable Scope

Variables have either global or local scope.

3.2.3.1 Global Variable Scope

A variable with global scope can be used in any program buffer and also in Terminal commands.

All ACSPL+ variables have global scope. A user-defined variable can be declared as either global or local.

Declaration of user global variable starts with the keyword **GLOBAL**. For example:

```
global real GlobVar      !Declare the GLOBVAR variable as a global variable
```

This declaration may appear in several program buffers. However, in distinction to local variables, all these declarations are considered to be the same variable.

Using a ACSPL+ variable in a program does not require explicit declaration of the variable. However, a global user variable must be declared before it can be used in a program. Terminal commands can use any global variable without explicit declaration.

3.2.3.2 Local Variable Scope

A variable with local scope can be used only in the program buffer where it is declared. Only user variables can be local.

By default, if a declaration does not contain the keyword **GLOBAL**, the variable is declared as local. However, it is recommended that you include the keyword **LOCAL** for clarity. For example, the declaration

```
real LocVar
```

is the same as

```
local real LocVar
```

Both of them define **LOCVAR** as a local variable.

A local variable is valid only within the buffer where it is declared. If a local variable with the same name is declared in another buffer, the controller considers them as two different variables.

The name of a local variable may also be the same as the name of a global variable. The controller considers them as different. The program where the local variable is declared has access to the local variable only.

3.2.4 Variable Lifetime

All ACSPL+ variables are valid as long as the controller firmware is active; from power-up to power-down.

A user local variable is valid as long as the program containing it is compiled. Therefore, a local variable becomes active when the program containing it is inserted into one of the buffers and is successfully compiled. After compilation, the variable is assigned its value and is available for the **#V** (list of variables) SPiiPlus MMI Application Studio **COMMUNICATION TERMINAL** command and queries.

Local variables are erased when the program they belong to returns to non-compiled state. Programs may enter a non-compiled state as a result of:

- > Explicit **#SR** command (Stop and Reset)
- > Editing or inserting another program in the buffer

When a program terminates normally, it remains in a compiled state and local variables therefore remain valid. In addition, if a controller-executed program causes an error, the controller terminates the program, but the buffer remains in a compiled state. Any user variable in the buffer therefore remains valid.

A user global variable declared in one or more ACSPL+ programs is valid as long as at least one of the programs that contains it is compiled. After compilation, the global variable is assigned its value and is available for the **#V** (list of variables) Terminal command and queries.

A user global variable disappears when the last program that it is contained in returns to a non-compiled state. The conditions when a program may return to non-compiled state are discussed above.

A special case of user global variable is the persistent global variable. A global variable is defined as persistent when the variable declaration is not a part of any program but is issued as an immediate ACSPL+ command.

Assume, that you execute the following commands via the **COMMUNICATION TERMINAL** window:

```
global real PersistentVar      !Declaration
?PersistentVar                 !Query
0                               !Initial value
```

The controller immediately accepts the declaration and creates the persistent global variable: **PERSISTENTVAR**. The variable is now valid and can be queried as illustrated by the query command, **?PERSISTENTVAR**, that follows.

The lifetime of a persistent global variable is not connected with any program. A persistent variable survives any change in the program buffers and may be deleted only by explicitly calling the **#VGV** (Clear Global Variables) Terminal command. Note that this command does not release the memory allocated for the global variables.

3.2.5 Variable Accessibility

All user variables have read-write access and are not protected. Any ACSPL+ command located in the scope of a variable can use or assign the variable's value at any time.

An ACSPL+ variable can belong to any of three access classes:

- > read-write
Read-write ACSPL+ variables such as user variables can be used or assigned at any time. Examples of read-write ACSPL+ variables are **VEL** (Velocity) and **IST** (Index State).
- > read-only
Many ACSPL+ variables are read-only. These variables cannot be assigned. The read-only variables provide readouts of the controller state. Examples include **RPOS** (Reference Position), **FPOS** (Feedback Position), **MST** (Motor Status).
- > protected
Protected ACSPL+ variables define the configuration of the controller, and are therefore called configuration variables. The values of configuration variables can be read at any time. Assignment to configuration variables is allowed only in the Configuration Mode. Examples of configuration variables are **ENTIME** (Enable Time), and **XACC** (Maximal Acceleration).

In most applications, you set the values of configuration variables during the application development process by using the special tools provided for this purpose in the Protection Wizard of the SPiiPlus MMI Application Studio (see the *SPiiPlus MMI Application Studio User Guide*).

3.2.6 Variable Type: Integer and Real

ACSPL+ supports integer and real variables for scalar values. See [Matrix Type](#)

Each ACSPL+ variable has a predefined type. For a user variable the type is specified in the declaration.

The controller provides automatic conversion from integer to real and from real to integer if required. Therefore you are not restricted in using variables of both types in ACSPL+ commands.

For example in the following fragment:

```

int Var1      !Declare Var1 integer variable
real Var2     !Declare Var2 real variable
Var1 = Var2   !Controller automatically converts real to integer
Var2 = Var1   !Controller automatically converts integer to real

```

The types differ by internal presentation and behavior in arithmetical operation.

- > An integer value occupies 4 bytes, 32 bits, numbered from 0 to 31. The sign is located in bit 31, bit 0 is the least significant bit of the mantissa.
- > A real value occupies 8 bytes, 52 bits of mantissa and 12 bits of exponent. The format corresponds to the standard double format of PC.

3.2.7 Matrix Type

ACSPL+ supports a **MATRIX** type starting with version 3.10.

A **MATRIX** is defined as 2-dimensional **REAL** array.

Syntax

- > MATRIX A(N)(N) – A is square matrix of size NxN where N is a positive constant.
- > MATRIX B(N)(M) – B is NxM matrix having N rows and M columns, where N and M are positive constants.

Example

```
MATRIX A(2)(2) !Defines a square matrix of 2x2 size
```

3.2.7.1 Matrix Initialization in Compilation Time

1. Regular Initialization (similar to 2D arrays initialization):
MATRIX A(M)(N) can be initialized by a 2D array (array of M rows, each of N size)
 - a. Preconditions:
 - i. Number of rows of initialization values = matrix first dimension
 - ii. Number of columns of initialization values = matrix second dimension
 - b. Error conditions:
 - i. If preconditions are violated, then a compilation error will occur.
 - c. Example:

```
!///Compilation-time initializations///!  
MATRIX A(2)(2)=( (1,2), (3,4) ) !Regular initialization of a 2x2 matrix
```

2. Default Initialization:
A matrix which not initialized by the user on definition is automatically initialized with zeros.

Example:

```
!///Compile-time initializations///!  
MATRIX C(2)(2) !Default initialization, filled by zeros
```

For details about MATRIX type functions and operators see Matrix Functions in the *ACSPL+ Commands & Variables Reference Guide*.

3.2.7.2 Matrix Operations Performance

Matrix related operations can be time consuming, especially matrix inverse operation. A matrix size threshold is defined to distinguish between running such operations in real-time or not. By default, this value is set to 36. Any matrix whose order (rows * columns) exceeds that threshold is considered time consuming, and any operation performed on this matrix will not run in real-time, and may take 2 or more MPU cycles, depending on the matrix's order.

SETCONF(325)

The matrix calculation threshold can be changed by the user by using **SETCONF(325, 0, value)** command.

1. Key = 325

2. Index = 0
3. Value [0 - 400]
 - a. 0 – No matrix related operations will be performed in real-time
 - b. 400 – Operations on matrices of size up to 400 (e.g., 20x20) will be performed in real-time. Operations on matrices of size bigger than 400 are not allowed to run in real-time due to critical impact on the MPU usage value.

3.2.7.3 Matrix Applications

Matrices are fully supported in applications in the same way as 2D arrays are supported.

Save Matrix to Flash

Any global variable of **MATRIX** type can be saved to flash as "User Array", producing a file on the controller's flash.

Import/Export Matrix

1. Export

Any global variable of **MATRIX** type can be exported to a file on computer.
2. Import

Any 2D array can be imported from a file saved on computer to a global variable of **MATRIX** type, assuming both the array and the matrix have the same size (columns and rows).

3.2.7.4 Remarks

The **MATRIX** type is not supported for the following uses:

- > ACSPL+ custom functions' parameters.
- > ACSPL+ struct fields.

3.2.8 String Type

ACSPL+ supports a "**STRING**" primitive. ACSPL+ **STRING** types are declared with a constant buffer length. All writing and modification of the string must be within the boundary of that buffer length

- > Supports variables of type **STRING** and **STRING ARRAY** (one-dimensional)
 - > May be in a local, global, or static buffer
 - > May be a function variable
 - > May be a **STRUCT** field
 - > May be up to 254 characters long
- > Function parameters may be
 - > **STRING** of explicitly specified size or unspecified size
 - > Reference to a **STRING** variable
 - > **STRING** array of specified or unspecified length

Example

```

VOID myFunc(String s1(10), String REF s2, String strArr());

! define two variables that can contain 8 charecters
! and an array of two cells, each can contain 8 charecters
String str1(8) = "Take", str2(8) = "on", strArray(8)(2) = ("me", "on")
DISP str1," ", str2," ", strArray(0)! Take on me
myFunc(str1, str2, strArray);
DISP str2," ", strArray(0)," ",strArray(1)! I'll be gone
STOP

VOID myFunc(String s1(10), String REF s2, String strArr())
{
    Disp s1," ", strArr(0)," ", strArr(1) ! Take me on
    s2 = "i'll";           !change original string
    strArr(0) = "be";      !change original array
    strArr(1) = "gone";
    RET
}

```

3.2.8.1 String Concatenation

The plus sign ("+") operator supports concatenation of strings, returning a result which is also a string.

Example

```

String s1(12) = "Hello"
s1= s1 + " World"
DISP s1

```

3.2.8.2 String Comparison

Operators support string comparison: "=" returns TRUE if the strings are equal and "<>" returns TRUE if strings are not equal.

Example of Equals Operator

```

String s1(8) = "abcde"
if (s1 = "abcde") DISP "EQUAL" ELSE DISP "NOT-EQUAL"
END

```

Example of Not Equals Operator

```

String s1(8) = "abcde"
if (s1 <> "abcde") DISP "EQUAL" ELSE DISP "NOT-EQUAL"
END

```

3.2.8.3 String Standard Variables

3.2.8.3.1 SN

Description

SN is a string type variable that contains the controller serial number.

Example

```
DISP SN
```

3.2.8.3.2 PN

Description

PN is a string variable containing the controller part number.

Syntax

```
DISP PN
```

3.2.8.3.3 VR

Description

VR String type variable that contains the controller FW version.

Syntax

```
DISP VR
```

3.2.9 Variable Size

A variable can be a scalar, one-dimensional or two-dimensional array. A one-dimensional array is referred to as a vector, and a two-dimensional array is referred to as a matrix.

The size of each ACSPL+ variable is predefined in the controller. For example, the **S_FAULT** (System Faults) variable is a scalar variable, and the **FAULT** (Motor Faults) variable is an array of 8 elements.

Many ACSPL+ variables are sized according to the number of axes, with one element per each axis. For example, each element of the **FAULT** array displays faults for one axis.

The size of user variables is defined in the variable declaration. For example:

```
int ScalarVar          !Declare the ScalarVar variable as a scalar variable
global real Ar1(100)    !Declare Ar1 as a global array of 100 real numbers
int Ar2(10)(200)        !Declare Ar2 as a 10x200 matrix of integers
```



The maximum size of a user array is 100,000 elements.

3.2.10 Variable Value

Value is the only variable attribute that can change during a variable lifetime. In most applications, a value is changed explicitly, as a result of assignment.

Read-only variables change their values implicitly. For example, the controller updates the elements of variable **FPOS** (Feedback Position) each controller cycle, so that each element displays the actual position of the corresponding motor.

3.3 Variable Declaration

ACSPL+ variables are predefined in the controller and do not require explicit declaration; however, each user variable must be declared in the program where it is used.

Syntax:

[scope-specification][type-specification] name [size-specification]

Where:

- > **[scope-specification]** is one of:
 - > local
 - > global
- > **[type-specification]** is one of:
 - > int
 - > real
- > **name** - the name of the variable
- > **[size-specification]** - required when defining an array, it can be one of:
 - > **(integer)** - One dimensional array
 - > **(integer) (integer)** - Two dimensional array

Any of the parts of either the scope-specification or the type-specification can be omitted, but not both. If scope-specification is omitted, the **local** specification is implied. If type-specification is omitted, the **int** specification is implied.

If no size-specification is specified, the variable is taken as scalar.

3.3.1 Declaration of Global Variables

A global variable is common for all programs in all program buffers.

To have access to a global variable, a program must declare it. Without explicit declaration a global variable is invalid in a program. Therefore declaration of a global variable may appear in several programs, but all these declarations refer to the same variable.

Declarations of a global variable in different programs must be identical. Declaring a global variable in two buffers with the same name, but different type or size, causes a compile error.

A global variable becomes active once the first program containing it is compiled in one of the program buffers. The variable remains valid as long as at least one of the programs containing it remains compiled.

Local and global variables are initialized to zero during compilation.

Examples of the global variable declarations:

<code>global int Var1</code>	Var1 is declared as a global integer variable
<code>global real Var2, Var3</code>	Var2 and Var3 are declared as global real variables
<code>global int Var4, Var5(2)(100)</code>	Var4 is declared as a global integer variable and Var5 is declared as a two-dimensional array of integers

3.3.2 Persistent Global Variables

A persistent global variable is a global variable that is declared by Terminal command. Terminal execution of ACSPL+ commands is described in [Immediate Execution](#). A command executed immediately is not included in a program or stored in a buffer.

A global command executed immediately declares a persistent global variable, whose lifetime is not dependent on any buffer. A persistent variable becomes active immediately when the command executes. The variable remains valid throughout any buffer manipulations. The only way to erase a persistent variable is by executing the **#VGV** (Vanish Global Variables) command. The command erases all persistent global variables that are not referred in any compiled buffer. Note that this command does not release the memory allocated for the global variables.

A program in any buffer can access a persistent global variable. Like a regular global variable, a persistent variable must be re-declared in each buffer where it has to be used. The declaration in a buffer does not affect the persistency of the variable. However, once declared in an Terminal command, the variable remains persistent irrespective of all re-declarations. A local variable, on the other hand, cannot be declared by an Terminal command. For example, assuming that you are communicating with the controller through the SPiPlus MMI Application Studio **COMMUNICATION TERMINAL** window and want to start data collection without preparing a special ACSPL+ program. The following dialog occurs:

<code>global Data(2)(1000)</code>	!Declare persistent global array of size 2x1000
<code>DC Data,1000,1,FPOS(0),PE(0)</code>	!Collect Feedback Position and Position Error of 0 axis motor, 1000 samples, 1 millisecond sampling period
<code>?S_ST</code>	!Query the System State variable
<code>3 ON Data Collection (#DC)</code>	Response to query: data collection is in progress.
<code>?S_ST</code>	!Query the System State variable
<code>3 OFF Data Collection</code>	Response to query: data collection is off.
<code>?Data</code>	!Query the Data array

3.4 Arrays and Indexing

3.4.1 Scalars and Arrays

A variable can be either scalar or array. A scalar variable contains a single value. An array contains a set of values. The arrays are subdivided into one-dimensional arrays and two-dimensional arrays or matrices.

Before an array is used, its size must be declared. The size of a one-dimensional array is the number of elements in the array. The size of a two-dimensional array is determined by multiplying each dimension, for example 2x2000 (for a total of 4000 elements).

Access to the value of a scalar variable is provided by its name. For example:

<code>int Scalar1</code>	Declare SCALAR1 as a local integer
<code>Scalar1 = 4</code>	Assign 4 to SCALAR1

DISP Scalar1	Display a value of SCALAR1
DISP Ar2(1)	Display value of column 1 of the AR2 matrix

A typical use of an array requires access to a specific element of the array. To specify an element of an array the array name must be followed by index specification. For example:

<code>int Ar1(100),Ar2(3)(100)</code>	Declare local integer AR1 as a vector of size 100 and local integer AR2 as a matrix of size 3x100
<code>Ar1(4) = 3000</code>	Assign 3000 to element 4 of AR1
<code>Ar2(0)(99) = 20</code>	Assign 20 to the element of AR2(0,99)
<code>DISP Ar2(0)(99)</code>	Display the element of AR2(0,99)

Indexing of arrays starts from zero. In the example above the first element of **AR1** has index 0, the last element has index 99.

For information on saving user arrays in the nonvolatile memory, see [Nonvolatile Memory and Power Up Process](#).

3.4.2 ACSPL+ Array Variables

ACSPL+ variables include both scalar and one-dimensional arrays. For example, the variable **S_ST** (System State) is scalar, while the variable **FPOS** (Feedback Position) is a one-dimension array of size 8.

Many ACSPL+ arrays are sized according to the axis number (eight elements). Such arrays contain one element per each controlled axis or motor. For example, each element of read-only variable **FPOS** reads a feedback position of the corresponding motor. **FPOS(0)** provides the position of the 0 motor, **FPOS(1)** provides the position of 1 motor, and so on.

Other ACSPL+ variables are related to the program buffers and therefore are sized according to the number of buffers. For example, each element of **PRATE** specifies a program rate of the corresponding buffer.

DIM1 and **DIM2** are reserved words used to retrieve an array's dimension properties. The property also applies for scalar variables. In the case of a single-dimension array or scalar variable, the **DIM2** property will return 0. In the case of a scalar variable **DIM1** will return 1.

For example:

```
int len, var
real array (2)(3), array2(5)
len = array.DIM1 ! yields 2
len = array.DIM2 ! yields 3
len = array2.DIM1 ! yields 5
len = array2.DIM2 ! yields 0
len = var.DIM1 ! yields 1
len = var.DIM2! yields 0
```

3.4.3 Explicit Indexing

Explicit indexing is applicable to both standard and user arrays. To access a specific element of an array, the array name must be followed by one or two indexes. Each index must be enclosed in parenthesis. A one-dimensional array requires one index and two-dimensional array requires two indexes.

The following is the syntax of index specification:

- > (expression) - One dimensional array
- > **(expression) (expression)** - Two dimensional array

For example:

```
real Ar1(100),Ar2(2)(8)  Declare AR1 as a local real vector of size 100
                           and AR2 as a local real matrix of size 2x8
int J                    Ar1(0) = 0 Assign 0 to the first element of AR1
J = 0                    Assign 0 to J
loop 99                  Repeat 99 times
Ar1(J+1) = Ar1(J) + 1    Fill Ar1 with a sequence of numbers 0-99
J = J + 1                Increment
end                       End of loop
J = 0                    Assign 0 to J
loop 8                   Repeat for each axis
Ar1(0)(J) = FPOS(J)      Store current FPOS in AR1(0,J)
Ar1(1)(J) = PE(J)        Store current PE in AR1(1,J)
J = J + 1                Increment
end                       End of loop
```

3.4.4 Postfix Indexing of Standard Arrays

Postfix indexing consists of a number appended to a standard array name without parenthesis. For example, **FPOS0** is equivalent to **FPOS(0)**; both read the feedback position of the 0 motor. **ERRI1** is equivalent to **ERRI(1)** that specifies a tolerable position error for the 1 motor.

Postfix indexing is applicable to any standard array. However, user variables cannot accept postfix indexing.

Postfix indexing is convenient if an element of an ACSPL+ array is used in an application as a separate variable. For example, elements of ACSPL+ arrays **V** and **I** are often used as counters and temporary variables. Postfix indexes allow using the elements as separate variables such as **V0**, **V22**, **I99**.

Arrays **IN** and **OUT** (Digital Inputs and Digital Outputs) are also often used with postfix indexing. The typical access to a digital input looks like:

IN0.4

where the first number after name **IN** is a postfix index and selects a group of 32 inputs. The number after the period is a bit specifier used to select one of the 32 bits (in this case bit #4).

Explicit indexing can be either constant or expression, while postfix indexing is always constant.

Postfix indexing and explicit indexing can be used interchangeably in one program. However, it is recommended that you select one style and use it throughout the application. If an application

requires non-constant indexing of axis-related variables, neither axis-like or post-fix indexing can be used.

3.4.5 Axis Indexing

Each controller axis has an index. The index is an integer number from 0 to *Number_Of_Axes-1*.



Number_Of_Axes is defined by the controller model and cannot be changed, for example, if there are 3 supported axes in a specific controller specification, 0 designates the first axis, 1 the next, and 2 the last.

The index of an axis is fixed, for example, **INDEX 2** always refers to a specific physical axis. For example:

VEL(1) – stores the velocity of the axis designated as 1.

ACC(5) – stores the acceleration of the axis designated as 5.

3.4.6 User-Defined Axis Names

You can assign names to the axes, and once assigned, the names are used as aliases throughout your program. You do this through the command: **AXISDEF**.

Axis names follow the general rules for ACSPL+ names: a valid name can be any sequence of letters and digits, but must start with a letter. However, some limitations on the axis names are recommended as noted in this discussion.

The command can be used to define one or more axis names, for example, the command:

```
AXISDEF X=23, W12=9, T9=0
```

defines the name **X** for axis 23, **W12** for axis 9, and **T9** for axis 0.

AXISDEF can be repeated many times as you like to define all required names; however, the following restrictions apply:

- > Only one name can be defined for the same axis
- > The names must be unique, i.e., you cannot define two axes with the same name
- > The name must not conflict with any other name of variable, label, keyword, etc.

A compilation error occurs if one of the above restrictions is not satisfied.

Axis names must be defined either in D-Buffer - see [Declaration Buffer \(D-Buffer\)](#), or in a program, where it is used. In any case, the axis definition has global scope; therefore the definition of the same axis in a different program must be identical (similar rules apply to global variables). Axis defined in D-Buffer can be used in any other buffer without re-definition.

3.4.6.1 Axis Name as Symbolic Constant

The axis name can be used in expressions as a symbolic constant. For example, given a program that includes declaration:

```
AXISDEF Q=3
```

the following command

```
VEL(Q)=1000;
```

assigns 1000 to the required velocity of axis 3.

3.4.6.2 Axis Name in Indexing

In axis-related ACSPL+ standard array variables that contain *Number_Of_Axes* components, one component per each controller axis, where the Index of the array ranges from 0 to Number_Of_Axes-1. User axis names can be used for indexing, not only for explicit indexing, but also for prefix and postfix indexing, for example: given the program includes declaration:

```
AXISDEF Q=3, X1=12, X2=13
```

Table 3-1. Example of the possible index formats.

Explicit Indexing	Postfix Indexing
VEL(3) or VEL(Q)	VEL3
ACC(12) or ACC(X1)	ACC12
SLVKI(13) or SLVKI(X2)	SLVKI13

3.4.6.3 Axis Specification in Commands

Another use of user-defined axes are arguments specifying a set of axes in ACSPL+ commands (like **ENABLE**, **KILL**, **PTP**, etc.) which are considered as integer arrays. You can declare an array, assign its elements, and then use it as a predefined axis group. For example, assume the application frequently uses axes 1, 12, and 15 as a group. The program may contain the following commands:

```
int AxisGroup(3)
AxisGroup(0)=0; AxisGroup(1)=1; AxisGroup(2)=2;
ENABLE AxisGroup(0)
ENABLE AxisGroup(1)
ENABLE AxisGroup(2)
PTP AxisGroup,1000,1500,1200
HALT AxisGroup(0)
HALT AxisGroup(1)
HALT AxisGroup(2)
DISABLE AxisGroup(0)
DISABLE AxisGroup(1)
DISABLE AxisGroup(2)
stop
```


Other formats of axis specification are also supported (actually, they are considered as special forms of array specification):

1. Axis expression, like (0, 1, 2), (Ax1, Ax2, Ax3, Ax4).
2. The keyword: **ALL** that specifies all available axes.

3.4.7 Array Processing Functions

ACSPL+ provides the following functions for processing arrays:

- > **MIN** – finds the minimum value in an array or any section of it
- > **MINI** – finds the minimum value in an array or any section of it and returns its index
- > **MAX** – finds the maximum value in an array or any section of it
- > **MAXI** – finds the minimum value in an array or any section of it and returns its index
- > **AVG** – finds the average value in an array or any section of it
- > **FILL** – fills an array or section of it with the specified value

These functions are detailed in the *SPiiPlus Command & Variable Reference Guide*.

3.5 ACSPL+ Composite Data Structures

3.5.1 Introduction

A composite data type is a data type that is defined by the user and is comprised of multiple primitive (**INT/REAL**) and other composite data types known as fields.

Composite data types are used to describe object with multiple characteristics; the different fields of the data type relate to the different properties of the object.

Composite data types are the basis for object oriented programming.

From Version 3.00 and onward ACSPL+ will support **STRUCTs**.

3.5.2 **STRUCT** Type Definition

STRUCTs can only be defined in the D-BUFFER.

A **STRUCT** is defined by using the reserved word "**STRUCT**", followed by the name of the data type and an opening curly bracket.

Example

```
STRUCT MyStruct{  
  !Fields and functions  
}
```

3.5.3 Fields

In between the brackets will appear the fields of the **STRUCT**.

Fields can be of several types:

- > **INT**
- > **REAL**
- > **INT ARRAY**

- > **REAL** ARRAY
- > **STRUCT**s (recursion of **STRUCT** types is not supported)

All fields must have unique names.

All fields must be defined before functions are defined.

```
D-Buffer:
STRUCT MyStruct{
  int field1;
  real field2
  real arr(10)
}
```

3.5.4 **STRUCT** Type Variable Definition and Use

3.5.4.1 Variable definition

To declare an instance of a **STRUCT** type variable the user specifies the name of the data type and the name of the variable

Fields of type **INT/REAL** will be initialized to 0 and arrays will not be initialized.

STRUCT fields are accessed by use of a period (".") after the variable name, followed by the field name.

Setting a field value follows the same syntax.

Global variables of **STRUCT** type can only be defined in the D-Buffer.

```
Buffer 0:
MyStruct var;
var.field1 = 6
var.arr(4) = 9.3
```

Complex **STRUCT** data type example

```
D-Buffer:
STRUCT InternalStruct{
  int field1;
  real field2
  real arr(10)
}

STRUCT ExternalStruct{
  InternalStruct internal;
}

Buffer 0:
ExternalStruct var;
var.internal.field1 = 6
var.internal.arr(4) = 9.3
```

3.5.4.2 Variable initialization

Local variables (non-global) are initialized in at run time.

- > Primitive fields are initialized to zero
- > Arrays are not initialized at all
- > **STRUCT** fields (fields that are other **STRUCT**s) are also initialized accordingly.

Global **STRUCT**s (defined in the D-Buffer) are initialized only once – during compilation.

3.5.4.3 Expressions Using **STRUCT**s

A **STRUCT** can be passed to a function that receives the same data type. Similarly, a **STRUCT** field can be passed as a parameter.

Accessing an interior **STRUCT** (a **STRUCT** inside a **STRUCT**) uses the same syntax as value assignment.

3.5.5 **STRUCT** Functions

3.5.5.1 Function Definition

It is also possible to define behavior for a composite data type. For ACSPL+ **STRUCT**s this behavior is defined using functions.

A function may be defined inside the scope of the **STRUCT**, between the curly braces. Alternatively, a function may be declared inside the **STRUCT** with the function signature and defined outside the **STRUCT** definition.

Like any other function it has a return type, name and parameters. Unlike other functions, a **STRUCT** function can refer to the fields of the data type in which it was defined.

Unlike C/C++ and other programming languages, ACSPL+ **STRUCT**s have only default constructors that are automatically executed when the **STRUCT** is created.

3.5.5.2 Out-of-scope Function Implementation

A function can be declared in the scope of the struct and implemented outside.

The following example implements such a function in the D-Buffer.

```
DBUFFER:
struct MyStruct{
    int field1;
    real field2
    real arr(10)

    real getFieldsTotal();
}
real MyStruct::getFieldsTotal(){
    RET field1+ field2;
}
```

This feature is meant to allow the user to separate struct definition (signature) from its implementation, making the struct easier to read and understand.

3.5.5.3 **STRUCT** Function Parameters

STRUCT functions can accept variables of the same types that regular functions accept.

If a variable name used was previously used hiding type semantics is used.

A **STRUCT** function can also accept a **STRUCT** of the same data type in which it is defined, supporting implementation of comparison and cloning type functions.

3.5.5.4 Function Variables Definition

A **STRUCT** function can define the same variables as a regular function.

3.6 Using Variables

3.6.1 Querying Variables

The value of any variable can be displayed in response to a query command. See *SPiiPlus Command & Variable Reference Guide* for a description of query commands.

Examples:

?FPOS	Query feedback positions of all motors
1003 4001 233000	1 0 -1 0 1
?FPOS0	Query feedback positions of the 0 axis motors
1003	
?FPOS0,FPOS2	Query feedback positions of the 0 and 2 axes motors
1003	
233000	
?GlobVar	Query global user variable GlobVar
23.35	
?0:LocVar	Query local user variable LOCVAR from buffer 0
100	
?0:LocVar,5:LocVar	Query two local user variables from different buffers
100	
233.7	

3.6.2 Variables as Operands in Expressions

Variables can be used as operands in expressions.

An array, as a unit, cannot be an operand in an expression. Only the array elements can be used. Therefore the array name must be fully indexed to provide access to a specific element.

In addition, a bit specifier can be added to an integer variable to provide access to a specific bit (as shown in example).

An expression is calculated each time that a command that includes the expression is executed. During calculation of the expression each variable is substituted with its current value.

Examples:

RPOS(0) - FPOS(0)	An arithmetical expression which produces the position error of the X motor.
RPOS(0) < FPOS(0)	A logical expression which produces: > 0 - if the 0 axis position error is positive

	> 1 - if the 0 axis position error is negative
2*(VEL(3) - LocVar)	An expression that combines standard and user variables.
(INO.3 INO.4)&^INO.6	A logical expression that uses bit specifiers to access specific digital inputs.
V0	An expression that contains only one variable with no operations.

3.6.3 Variables as Arguments in Command or Function

Variables can be used as arguments in commands and functions.

Arguments used to specify commands and functions have specific requirements. In a typical example, an argument is required to be an expression. In this case a single variable can be used as a simplest case of expression. Use of variables in expressions is discussed in [Variables as Operands in Expressions](#) above.

A number of commands and functions require a variable or an array as one of their arguments. For example, the first argument of the **DC** command (Data Collection - see [DC Command](#)) is an array that accumulates the collected data. Other examples are statistical functions, such as **MAX**, **MIN**, and **AVG**, that process the whole array specified as an argument. Unlike their use in expressions, an array name without indexes specifies the array as a whole.

For example, the following fragment collects data 1000 times on the **FPOS** for the 0 axis at intervals of **V0**:

```
dc Data,1000,V0,FPOS(0)
```

Where:

1. The first argument of the **DC** command is required to be an array, in this case it is the user array, **DATA**, that is specified without indexes, as a whole.
2. The second argument is an expression that defines the number of samples to be collected, in this case it is a simple expression: the constant 1000.
3. The third argument is an expression that defines the sampling period. The **V0** variable is a simple expression. Using a variable instead of an integer provides changing the sampling period, based on **V0**, from one execution of this data collection to another.
4. The fourth argument must be a variable or an array element. The values of the variable (in this case **V0**) will be collected in the array. The syntax of **DC** requires that the fourth element be a variable or an array element. Neither a general expression nor an array without indexes can be specified. An array without indexes neither can be specified. **FPOS(0)** addresses element 0 of the array **FPOS** that corresponds to the 0 axis feedback position.

3.6.4 Variables in ACSPL+ Terminal Commands

The SPiPlus MMI Application Studio **COMMUNICATION TERMINAL** execution of the ACSPL+ commands is described in [Immediate Execution](#). A command immediately executed is not included in a program or stored in a buffer.

Use of variables in immediate ACSPL+ commands is limited to ACSPL+ and global variables. Local variables cannot be referenced in immediate ACSPL+ commands.

3.6.5 Accessing Variables by Tags

3.6.5.1 Variable Tags

The controller supports integer numerical tags for all standard ACSPL+ variables. You are also able to define tags for user-defined variables. The ACSPL+ program is able to access a variable by a *tag* as described in the next section.

- > **ACSPL+ Standard Variable Tags**
The tags of standard ACSPL+ variables are fixed in all firmware versions, and you cannot change the tag of a standard variable. For the tag of any given ACSPL+ variable see *SPiiPlus Command & Variable Reference Guide*.
- > **User-Defined Variable Tags**
A user-defined global variable can be declared with a tag.

The extended syntax of a variable declaration is:

global {int|real} tag tag_number variable_name [,variable_name, variable_name,...]

Where:

tag_number	A positive integer to be associated with the variable(s).
variable_name	A unique variable name, or list of names.

Only global variables can be declared with a tag. The following conditions also apply:

- > The tag is not mandatory in the variable declaration. However, if the tag is not declared, the variable cannot be accessed by tag.
- > The value of **TAG_NUMBER** must be greater than 1000. Values below 1000 are reserved for the standard ACSPL+ variables.
- > The **TAG_NUMBER** value must be unique in the application.
- > If more than one **VARIABLE_NAME** is included, when the program is compiled, the controller builds a sequence of tag numbers: the specified **TAG_NUMBER** is attached to the first variable in the list, **TAG_NUMBER+1** value is attached to the second variable, **TAG_NUMBER+2** to the third, and so on.

3.6.5.2 GETVAR and SETVAR Functions

Description:

The **GETVAR** function serves for retrieving the value of a given variable. The **SETVAR** function serves for setting the value of a given variable.

Syntax:

real GETVAR(tag_number[, index1, index2])

SETVAR(value, tag_number[, index1, index2])

Arguments:

tag_number	A positive integer associated with the variable.
index1, index2	Integer indexes. The indexes must be omitted if the variable is scalar. The second index must be omitted if the variable is one-dimensional array.

Comments:

The **GETVAR** function reads the current value of the variable and returns it as a real value. The **SETVAR** function assigns the specified **VALUE** to the variable designated by **TAG_NUMBER**.

Though the **VALUE** argument and the return value are defined as real, the functions can be used for integer variables as well. The controller implements all necessary transforms automatically.

The functions provide read/write access to all standard ACSPL+ variables and to those user-defined variables declared with **TAG**.

3.7 ACSPL+ Functions

For a complete description of all ACSPL+ functions see the *SPiiPlus Command & Variable Reference Guide*.

3.8 ACSPL + User-Defined Functions with Parameters

3.8.1 Introduction

Until version 3.13.01, the need to execute repeated sections of code was met by using subroutines, implemented by the use of labels. This method had several shortcomings that are solved by functions.

Functions allow the user to :

- > Pass parameters
- > Return a result value
- > Implement safe parallel execution of code
- > Declare local variables

Functions are sequences of code that can be executed.

3.8.2 Function definition

3.8.2.1 Function Signature

A function is defined by several characteristics:

- > Function return type – the type of value returned by the function. ACSPL+ functions may return the following types:

INT – integer return type

REAL – fraction return type

VOID – no value return

- > Function name – the unique name of the function, the function cannot share the same name of a variable / other function that is defined in the same scope of visibility.

- > Function parameters list – a list of type-name separated by comma (',').

Functions may be declared in the scope of a struct

Example:

```
REAL VECTOR_CALC (REAL X_COORD, REAL Y_COORD, REAL V_ANGLE)
```

3.8.2.2 FASTCALL Functions

A **FASTCALL** function is a function that executes in a single cycle

EXAMPLE

```
REAL FASTCALL SAVESTATE (INT AXIS_INDEX)
```

If a function is defined as **FASTCALL** in function declaration, it must also be declared as **FASTCALL** in the function implementation. In other words, function signature conformity is a requirement.

FASTCALL functions do not allow the following commands as they all add delays:

- > **DISP**
- > **WAIT**
- > **TILL**

Calls to any of these functions will produce a compiler error.

Loops inside **FASTCALL** functions complete all iterations in a single cycle.



An infinite loop in a **FASTCALL** function will execute indefinitely, as the **FASTCALL** definition prevents the controller from moving on to the next buffer. In this situation the controller will be completely frozen and will require a reset. Use loops inside **FASTCALL** functions carefully.



Other commands, such as **GETSP**, can also cause the function to hold the processor past the controller cycle time (**CTIME**) limit; it is the user's responsibility to use these functions correctly.



Calls to other functions are not allowed inside a **FASTCALL** function

3.8.2.3 Function visibility scope

A function can be defined as a local (for the buffer) function or a global (all buffers) function.

3.8.2.3.1 Local function

If a function is defined in a regular buffer, the function will only be visible inside that buffer.

Since a function cannot be used before its signature is available, it is necessary to declare the function before use.

3.8.2.3.2 Global function

A function defined in the D-Buffer is available from any buffer (including the communication terminal).

Functions that are defined in D-Buffer do not require declaration to be used in other buffers.

3.8.2.3.3 Function Declaration

A function defined in a local buffer can be declared anywhere before it is used, but the best practice is to declare all functions in the top of the buffer.

A function declaration is just the function signature. Names specified in the function declaration could later be changed in the function definition. Return-type, name of the function and function parameter types in the definition must conform exactly to the function declaration.

EXAMPLES

1. Local function – the function is defined, and used only in buffer 0:

```
BUFFER 0:
!This is the function declaration
void myFunc(int param);
!The use of the function is only possible
!after the function has been declared
myFunc(25)
STOP
!This is the function definition
void myFunc(int param){
DISP "param = ", param
param = 3
RET
}
```

2. Global function

```
D-BUFFER:
!This is the function definition
void myFunc(int param){
DISP "param = ", param
param = 3
RET
}
BUFFER 0:
!The use of the function does not
!require declaration since the function
!was defined in D_Buffer
myFunc(25)
STOP
```

3.8.2.4 Function Code

In the function definition, after the function signature, the function code is written. The function code starts with an opening curly bracket '{', and ends with '}' (as in C/Java/C#, etc.).

All functions must be defined after a **STOP** command or the close of another function.

If the function code is reached not with a function code, but rather by running the buffer without a stop command before the function definition – a runtime error will be thrown. Therefore, all functions must be defined after the **STOP** command, and if a label is defined before a function definition, it must return or stop.

Example

```

REAL VECTOR_CALC(REAL X_COORD, REAL Y_COORD, REAL V_ANGLE) {
  Real Aux
  Aux = X_COORD * SIN(V_ANGLE)
  DISP Aux
  IF Aux > 5 RET 3.0 END !if the condition is not met, an error is thrown
}

```

3.8.3 Function Parameters

A function can define several types of parameters.

3.8.3.1 Value Types

A value type is a parameter that is passed by value. The expression stated in the function call is evaluated and the function is sent to the function execution.

Value types are: **INT** / **REAL** data types.

Example – param is passed by value

```

void myFunc(int param);
INT myVar = 16
! The value calculated by the expression is passed to the function
myFunc(myVar * 2)
STOP
void myFunc(int param){
  DISP "param = ", param
  param = 3
  RET
}

```

3.8.3.2 Reference types

A reference parameter type is a parameter that is passed by reference. When the function is called the variable is passed. Any change to the parameter will affect the variable passed to the function.

A reference parameter can only accept a variable (user defined) as the actual parameter.

The reference parameter type is marked by the reserved word "**REF**". It can be of **INT** or **REAL** data type.

Example – param is passed by reference

```
void myFunc(REAL REF param);
REAL myVar = 16.5
myFunc(myVar ) ! myVar is passed to the function (and not its value)
STOP
void myFunc(REAL REF param){
  DISP "param = ", param
  param = 3.2 ! This line changes the value of myVar
  RET
}
```

3.8.3.3 Composite Data Type Parameters

A function can receive composite data types (composite data structures). Composite data types are passed by reference and any change to the parameter inside the function will change the variable that was passed.

3.8.3.4 Array Type Parameters

Functions can accept arrays as parameters. When defining an array parameter the user can choose between arrays with a single dimension or two dimensions.

All arrays have reference type semantics and any change to the array parameter will change the array in the calling context.

3.8.3.4.1 Single Dimension Array

The user will only specify the data type (**INT/ REAL**) and leave the brackets empty (no size should be specified). Such a parameter can also accept two dimensional arrays and use them as a single dimension, although such usage must be implemented with great care.

Single dimension array parameters are not size validated at compilation (unlike regular arrays in ACSPL+) and can accept any array of the same data-type (single / two dimensional).

If the user attempt to use an illegal index a runtime exception will be thrown.

Example – accept an array and change it

```
void printAndChangeArray(int arr(), int Size)
int myArray(3) = (66,77,88)
DISP myArray(0) !Print 66
printAndChangeArray(myArray, 3)
DISP myArray(0) !Print 88
STOP
!The function accepts an array and array size and puts the highest
!value in the first cell, if no positive value - put 0
void printAndChangeArray(int arr(),int Size){
  if(Size = 0) RET end
  INT i = 0
  INT maxNum = 0
  loop Size
```

```

DISP "arr(%d) = %d " ,i, arr(i)
if (arr(i) > maxNum) maxNum = arr(i) end
i = i+1
end
arr(0) = maxNum
RET
}

```

Example – single dimension array parameter accepts a two dimensional array, and uses it as a single dimension array

```

void printAndChangeArray(int arr(), int Size)
int myArray(3) = (66,77,88)
int array2D(2)(2) = ( (3,4), (5,6) )
DISP array2D(0)(0) !Print 3
printAndChangeArray(array2D, 4)
DISP array2D(0)(0) !Print 6
STOP
!the function accepts an array and array size and puts the highest
!value in the first cell, if no positive value - put 0
void printAndChangeArray(int arr(),int Size){
if(Size = 0) RET end
INT i = 0
INT maxNum = 0
loop Size
DISP "arr(%d) = %d " ,i, arr(i)
if (arr(i) > maxNum) maxNum = arr(i) end
i = i+1
end
arr(0) = maxNum
RET
}

```

3.8.3.4.2 Two Dimensional Array

The user will specify only the data type (**INT/ REAL**) and the size of the second dimension (no size should be specified for first dimension). Such a parameter can only accept two dimensional arrays with the same second dimension size.

Validation of the second dimension is done in compile time if the value is known.

If the user attempts to use an illegal index, a runtime exception will be thrown.

Example – accept a two dimensional array

```

void printAndChangeArray(int arr()(3), int Size)
int myArray(2)(3) = ((33,44,55), (66,77,88))
DISP myArray(0)(0)
printAndChangeArray(myArray, 2)
DISP myArray(0)(0)

```

```

STOP
!the function accepts an array and array size and puts the highest
!value in the first cell, if no positive value - put 0
void printAndChangeArray(int arr() (3),int Dim1){
INT row = 0
INT col = 0
INT maxNum = 0
loop Dim1
col = 0
loop 3
DISP "arr(%d) (%d) = %d " ,row, col, arr(row) (col)
if (arr(row) (col) > maxNum) maxNum = arr(row) (col) end
col = col+1
end
row = row +1
end
arr(0) (0) = maxNum
RET
}

```

3.8.4 Variable Definition

A function can define variables. Variables defined inside a function only exist from the moment they are defined until the function returns.

Initialization of variables follows the syntax of regular ACSPL+ variable Initialization. Since function variables exist only at runtime it is possible to initialize parameters according to runtime values.

Like ACSPL+ variables that are not explicitly initialized, local variables in functions will be initialized to the default value of 0.

Arrays are not initialized to default values and must be explicitly initialized.

A function has access to the variables defined outside the function (local/ global/ constants). If a variable is defined by the same name as an outside variable, hiding semantics will be used and any mention of the shared name will refer to the local function variable.

A function can define variables of the following types:

- > **INT**
- > **REAL**
- > **INT/REAL** arrays

```

Example - uninitialized array
void printAxisPos(int Axis);
int var1 = 3

printAxisPos(var1)
STOP
void printAxisPos(int Axis){
real arr(5)

```

```

DISP arr(3) ! Unknown value - arrays must be explicitly initialized
int var1 ! Hides the original var1
var1 = Axis - 1
DISP RPOS(var1)
ret
}

```

Example – initialization to runtime values

```

void SomeFunc(int Axis);
SomeFunc (5)
STOP
void SomeFunc (int Axis){
! Initialize parameters according to runtime value
real arr(3) = ( RPOS(Axis), RVEL(Axis), RACC(Axis))
int var1 = PEXL(5)
DISP "line = %d, RPOS = %.3f", var1, arr(0)
ret
}

```

3.8.5 Return value

A function must end with a **RET** statement, a function execution that does not, will throw an exception.

Functions that are not defined as **VOID** function must return a value as well. The expression after the **RET** command will be evaluated and converted to the desired type if necessary.

If function execution ends without a return statement, an error will be thrown.

3.8.6 DISP command

It is illegal to call a function as a part of a DISP Statement.

```

DISP f1(33) ! function call is not allowed

```

3.8.7 Stack Memory

Functions use the stack memory that was allocated to the buffer where they are executed.

Every function call reduces the amount of available memory.

The amount of memory occupied by a function call is decided by:

- > Return type
- > Parameters
- > Variables in the function
- > Additional (8-12) bytes for software proposes

Example:

```
real func(real depth, real ref Axis){
  int arr1(40)
  real coaf = 8.8
  RET 8.0
}
```

This function requires:

- > REAL return type - 8 bytes
- > Real parameter (8 bytes) + ref parameter (4 bytes) = 12 bytes
- > Int array of size 40 (160 bytes) + real variable (8 bytes) = 168 bytes
- > 8 additional bytes

Total = 196 bytes

So if this were a recursive function it would only be able to get to a depth of about 80 calls before the memory would run out.

When there is not enough memory to make additional calls, an error will be thrown.

3.8.8 Function Execution Time

A function should execute in the same number of cycles as the same code content written in the buffer.

A FASTCALL function should execute in a single cycle.

3.8.9 FILL function

FILL function is an ACSPL+ function designed to fill an array of a given size with a predefined value. The **FILL** function will work with arrays defined as function parameters or variables.

3.9 Expressions

3.9.1 General

An expression is a sequence of operators and operands that specify a calculation.

Expressions serve as building blocks for many ACSPL+ commands. For example, assignment commands include expressions to the right of the equal sign:

$V0 = V1 + 2 * (V2 - V3)$

When the controller executes a command that includes an expression, the expression is calculated and the result is used as required by the command.

Complexity of expression ranges from the simplest expressions that include only one constant or variable name, to extended formulae containing multiple operators and functions with several levels of brackets. For example:

I99 = 5	5 is a simple expression
I98 = V0	V0 is a simple expression
I97 = ((I1-1)*sin(I2)+2)/5	Complex expression

3.9.2 Calculation Order

In a complex expression, the following are the factors, listed in priority order, which determine the order of calculation:

1. Brackets in the expression
2. Operator precedence
3. Order of operators in the expression (left-to-right order)

If the brackets do not unambiguously define the calculation order, then the operator precedence is taken into account, and if the calculation order is still ambiguous then the left-to-right calculation order is applied.

The operator precedence, summarizing them in order of precedence from highest to lowest is:

Table 3-2. Mathematical Operators

Operator	Operation
.(dot)	Bit selection
++	Increment
- ~ ^	Unary minus, Inversion, Logical not
* / %	Multiplication, Division, Modulo
+ -	Addition, Subtraction
<< >>	Left shift, Right shift
= <> < > <= >=	Compare
& ~	Logical and bitwise AND, OR, XOR

If several operators appear on the same line or in a group, they have equal precedence.

3.9.3 Expression Type

The controller uses two types of numerical values;

- > Integer values that are 4-bytes (32-bits) long and range from -2147483648 to 2147483647
- > Real values that are 8-bytes long, 53-bits of mantissa and 11-bits of exponent with a range of $\pm 10^{-1023}$

Expression calculations produce either integer or real values.

Within the group of expressions that produce integer values, there is a subset, called logical expressions, that consists of expressions that produce only two values: 0 or 1.

According to the type of result, an expression is defined as integer, real or logical. The controller provides automatic type conversion of the result so that expression type required does not restrict expression use. For example:

$V0 = 0.01 * V1$

In this assignment command, the integer variable **V0** is to the left of the equals sign, while a real expression is to the right of the equal sign. The controller automatically uses rounding type conversion of the result obtained on the right side, so that the real value can be converted to an integer variable and stored.

The types of operands and operators used in the expression define the expression type. Each operator has an associated rule that defines the type of result according to the types of operands. The rules are summarized in the following tables.



Integer (0,1) is specified for the operators that produce only two values: 0 or 1 (logical result).

3.9.4 Operands

An operator requires one or two operands.

- > An operator that requires one operand is called a unary operator. A unary operator is placed before its operand, with the exception of the increment operator.
- > An operator that requires two operands is called a binary operator. A binary operator is placed between its operands.

Unary operators:

Operator	Type of Operand	
	integer	real
- (unary minus)	integer	real
~ (inversion)	integer	integer
^ (logical not)	integer (0,1)	integer (0,1)
++ (increment)	integer	integer

Binary operators:

Operator	Type of Operand			
	integer-integer	integer-real	real-integer	real-real
+ (addition)	integer	real	real	real
- (subtraction)	integer	real	real	real
* (multiplication)	integer	real	real	real
/ (division)	real	real	real	real

Operator	Type of Operand			
%(modulo)	real	real	real	real
& (and), (or), ~ (xor)	integer	integer	integer	integer
<< (left shift) >> (right shift)	integer	integer	undefined	undefined
=, <>, <, >, <=, >= (compare)	integer (0,1)	integer (0,1)	integer (0,1)	integer (0,1)
. (bit selection)	integer (0,1)	integer (0,1)	integer (0,1)	integer (0,1)

Each operand can be either integer or real. If the operator requires, the controller automatically converts the operand to the required type.

Operands can be one of the following:

- > Constant
A constant can consist of any integer or real number. Integer constants can be presented in decimal, hexadecimal and binary notation. Real constants can include a decimal point and/or exponent.
- > Symbolic constant
Symbolic constants are predefined in the controller. Each symbolic constant presents an integer number.
- > Scalar variable name or Array name with indexing
A variable name is a name of any standard or user-defined variable. If a user-defined name is used, it must be declared before being used in the program. If the variable presents an array, the name must be fully indexed to specify one element of the array.
- > Function call or Expression
Any ACSPL+ function can be used in an expression. Using expression as an operand of other expression provides unlimited variety of expressions. In many cases expression used as operand must be enclosed in brackets. For example:

(V0+5)*7	Expression V0+5 is the left operand of multiplication
(V0+5)*(I88+3)	Both operands of multiplication are expressions
V1*3 + I2*6	Operands of addition are expressions V1*3 and I2*6

The following sections provide further elaborations on the operands and operators.

3.9.4.1 Arithmetical Operators

Arithmetical operators provide four arithmetical operations. The arithmetical operators are:

- > + (addition)
- > - (subtraction)

- > * (multiplication)
- > / (division)
- > % (modulo)

Addition, subtraction and multiplication calculate an integer result if both operands are integers, and a real result if at least one operand is real.

Division and modulo always calculate real results, for example:

real Var	Declare real variable VAR
Var = 5/4	Assigns 1.25 to variable VAR .

3.9.4.2 Compare Operators

Compare operators are:

- > = (equal to)
- > <> (not equal to)
- > > (greater than)
- > >= (greater than or equal to)
- > < (less than)
- > <= (less than or equal to).

Compare operators work with any combination of integer and real operands. Compare operator results are always the integers 0 or 1. A positive result of a comparison provides value 1, while a negative result provides value 0.

Compare operators are typically used in a variety of contexts, for example:

if V0 > 5 ...

while I99 <> 0 ...

>on IN0.5 = 1 ...

Because they always produce an integer result, compare operators can be used in arithmetical calculations, for example:

V1 = (V0 = 5) * V4

The expression in parenthesis, **V0 = 5**, results in 1 if **V0** is equal to 5 and results in 0 if **V0** is not equal to 5. Therefore, this command assigns **V1** with the current value of **V4** if **V0** equals 5, and assigns **V1** with 0 if **V0** has any other value.

3.9.4.3 Bitwise and Logical Operators

Bitwise and logical operators are:

- > & (and)
- > | (or)
- > ~ (xor - exclusive or)

The result of a bitwise operator is always an integer. If an operand of a bitwise operator is real, the controller automatically converts it to an integer before the operation.

Bitwise means that the operation is executed separately on each bit of the operand. Each integer operand is considered as a set of 32-bits. Bit 0 of the left operand is combined with bit 0 of the right operand to produce bit 0 of the result.

The following example illustrates the AND operator. Both operands and the result are considered as sets of 32-bits.

First operand	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	1	0
Second operand	0	0	1	1	1	0	0	0	0	1	1	1	1	1	1	1	1	0	0	0	0	0	1	1	1	1	0	0	
Result	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0

If both operands are logical, i.e., have only values of 0 or 1, the result is also logical. In this case the operators can be treated as logical AND, OR and XOR. For example, the following condition:

$(V0 = 5) \& (I30 = 0)$

is satisfied only if **V0** is 5 and **I30** is 0 simultaneously.

3.9.4.4 Unary Operators

The unary operators are applied only to the operand. The unary operators are:

- > - (unary minus) – Negates its operand, either integer or real. The type of result follows the type of operand. For example:

$V0 = -V1$	V0 is assigned with the negative value of V1
------------	--

- > $\sim$ (inversion) – Provides bitwise inversion of its operand. If the operand is real, the controller provides automatic conversion to integer before the operation. The result is always an integer.
- > $\wedge$ (logical not) – Accepts either an integer or real operand and calculates an integer result. The result is 1 if the operand equals to 0, and is 0 if the operand is non-zero.
- > $++$ (incrementation) - Increments the value of an integer, an integer function parameter, or an integer type structure field by 1.

The following two examples illustrate the difference between the $\sim$ and $\wedge$ operations:

X	0	0	1	1	1	1	0	0	0	0	0	1	1	1	1	1	1	1	0	0	0	0	1	1	1	1	1	0	0
$\sim X$	1	1	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	0	1	1	1	1	0	0	0	0	0	1	1
$\wedge X$	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

X	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
$\sim X$	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
$\wedge X$	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0



Inversion is a bitwise operator, while the logical not applies to the entire value.

3.9.4.5 Bit Selection Operator (Dot)

Bit selection operator (dot) extracts one bit from an integer number. The result is always an integer and can yield only two values: 0 or 1.

The operands can be integer or real. The controller converts a real operand to integer before the operation.

The left operand supplies a number from which a bit will be extracted. The number is treated as a set of 32 bits numbered from 0 to 31. Bit 0 is the least significant bit.

The right operand supplies the ordinal number of the bit to be extracted. The value of the right operand must be in the range from 0 to 31.

Typical use of the bit selection is for the flag variables, such as **MST**, which are treated as a collection of one-bit flags. For example, the following command

```
till ^MST(0).#MOVE
```

provides waiting for the end of the 0 axis motion. The symbolic constant **#MOVE** provides selection of bit 5 of the **MST** (Motor State) variable. The fifth bit of **MST** reflects the state of the motion: it is 1 while the axis is in motion, and is 0 while the axis is idle.

3.9.5 Character Constants

An ASCII character enclosed in single quotation marks is interpreted as integer constant and can be used in assignment and expressions as a normal integer constant. For example:

```
int Char
Char = 'A'
```

assigns value 65 (or 0x41, which is the numerical equivalent of A) to the variable: **CHAR**.

3.10 ACSPL+ Commands

The complete ACSPL+ command set is detailed in the *SPiiPlus Command & Variable Reference Guide*. This section addresses certain ACSPL+ commands with expanded information on their use.

3.10.1 Assignment Command

The Assignment (=) command is used to give a variable a value.

Syntax:

left-term = expression

left-term can be:

- > Standard or user variable
- > An element of a ACSPL+ or user array
- > One bit of integer variable or integer array element

Assigning to an ACSPL+ variable is limited by the following rules:

- > Assignment to read-only variable (for example, **FPOS**) is prohibited
- > Assignment to a protected variable (for example, **ERRI**) is allowed in only in the Configuration mode.

expression can be of integer or real type. By using different operators and parenthesis, an unlimited number of expressions can be constructed.

After assignment, the previous value of the variable is replaced by the new value.

The controller executes assignment commands in the following order:

1. Calculate **expression**
2. Convert the type of calculated value to the type of **left-term** (if the types differ)
3. Assign the result to **left-term**

The following sections explain assignment for specific types of **left-term**.

3.10.1.1 ACSPL+ Variable Assignment

ACSPL+ variables can be used in the left side with the following restrictions:

- > The variable must not be read-only. Using read-only ACSPL+ variable in the left side causes a compile-time error.
- > If it is a protected variable, protection is checked when the command is executed. In protected mode, the assignment fails, producing a run-time error (3077).

Examples:

The following command tries to assign a read-only variable, and will cause a compilation error:

```
FPOS(0) = 0
```

The following command assigns the protected variable **FMASK**.

```
FMASK(0) = 0
```

is a legal command; however, when the command executes, if the controller is in the Protected mode, the assignment fails and produces a run-time error.

- > If the ACSPL+ variable is scalar, no indexing is required.
- > If a ACSPL+ variable is an array, explicit or implicit indexing is required. For indexing of ACSPL+ variables see [Explicit Indexing](#) through [Postfix Indexing of Standard Arrays](#).

Examples:

VEL(0) = 1000	Assign 1000 to 0 axis default velocity - explicit indexing
VEL0 = 1000	The same as above - postfix indexing
Var1 = FPOS(0)	Assign to user variable
Var2(0)(5) = 200	Assign to element of user array
OUT0.5 = 1	Assign to digital output 5

3.10.1.2 User Variable Assignment

User local and global variables must be declared before they can be used in an assignment command.

- > Explicit indexing only is allowed for user array variables.

- > If a user variable is scalar, no indexing is required.
- > If a user variable is one-dimensional array, it requires one index. Two-dimensional arrays require two indexes.

Examples:

<pre>global int Int_Scalar local real Real_Array1(20), Real_Array2(10)(10) Array2(10)(10) Int_Scalar = 5 Real_Array1(2) = 5*Int_Scalar Real_Array2(Int_Scalar)(Int_Scalar+2) = 1000</pre>	INT_SCALAR is declared as a global integer variable REAL_ARRAY1 is a local real array of 20 elements. REAL_ARRAY2 is a local real array of size 10x10. Assign 5 to INT_SCALAR Calculate INT_SCALAR multiplied by 5 and assign the result to the third element of REAL_ARRAY1 Assign 1000 to the element of REAL_ARRAY2 with first index equal to INT_SCALAR and the second index equal to INT_SCALAR+2
---	---

3.10.1.3 Bit Assignment

You use a bit specifier, added to an integer variable or integer array element, to provide assignment to a single bit. The syntax of bit specifier is

VAR.bit_specifier = expression

The **EXPRESSION** must calculate to an integer number that provides an ordinal number. Typically the **EXPRESSION** is simply a constant that specifies the number. However, an arbitrary expression can be specified that provides calculation of the bit value in run time.

In the controller, an integer number is presented by 32 bits. The bits are numbered from 0 to 31. The least significant bit is bit 0; therefore, **BIT_SPECIFIER** must be an integer number in the range 0-31.

A bit can have only two possible values: 0 (false) or 1 (true), while the **EXPRESSION** result, which defines the bit value, can be any value. Assignments convert the value as follows:

- > If the value is zero, the bit is set to zero
- > If the value is non-zero, the bit is set to one

Although bit assignments are applicable to any integer variable or array element, they are mainly used for changing flag variables and output bits.

Examples:

<pre>OUT0.13 = 1 IST(0).#IND = 0 FMASK(0).#DRIVE = 1</pre>	Set output 13 to one. Reset index flag of 0 axis. Enable Drive Fault exception. The command is allowed only in the Configuration mode.
--	--

3.10.1.4 Type Conversion

Left-side terms and right-side expressions may be of different types: integer or real in any combination. (The special case of Bit assignment is handled differently, as is described in [Bit Assignment](#)).

If the types differ, the type of calculated right-side expression is automatically converted to the type of left-side term. There are two possible conversions:

- > Integer to real; conversion is exact.
- > Real to integer; conversion is not always exact. A real number is rounded to the closest integer.

3.10.2 Synchronization Commands

Synchronization commands provide delay in program execution for a specified number of milliseconds or until a specified condition is satisfied. The following synchronization commands are available:

- > WAIT
- > TILL

3.10.2.1 WAIT Command

Description:

The **WAIT** command delays program execution for a specified number of milliseconds.

Syntax:

WAIT *expression*

Comments:

The **WAIT** command executes in the following order:

1. Calculate expression. The result is the required delay time in milliseconds.
2. Delay the program for the calculated amount of time.

Typically the expression is specified as a constant that provides a constant delay time. However, in some cases a variable delay time may be needed as shown in the example.

If the wait command is located in a separate line, the total execution time of this line is the delay time plus the standard one line execution time as defined by the **PRATE** variable, which defines the program execution rate (see *SPIIPlus Command & Variable Reference Guide* for details on the **PRATE** variable).

Example:

The following example is a program that tests the **WAIT** command:

```
V0 = 0                      Assign 0 to V0
loop 100
V1 = TIME
WAIT V0
DISP TIME - V1
V0 = V0 + 1
```


END	End loop
Stop	End of program

If the controller cycle is 1 millisecond and **PRATE** is 1, the program displays a list of numbers from 2 to 101. The first number, 2, corresponds to the standard execution time of two lines, because the first time the additional delay provided by the **WAIT** command is zero. Each loop executed adds one to the requested delay, therefore the displayed time grows correspondingly.

3.10.2.2 TILL Command

Description:

The **TILL** command delays program execution until a specified expression produces a non-zero (true) result.

Syntax:

TILL *expression* [, *timeout*]

Comments:

The optional **TIMEOUT** argument specifies a timeout for the **TILL** command in milliseconds.

The **TILL** command executes in the following order:

1. Calculate expression.
2. If the expression result is non-zero, go to the next command.
3. If the expression result is zero, wait one controller cycle and repeat the **TILL** command execution.

Examples:

The following fragment demonstrates a typical use of till command that provides waiting for a specific state before the execution of the next command:

PTP 0, 2000	Start positioning of the 0 axis to absolute point 2000
TILL ^AST(0).#MOVE	Wait until the 0 axis motion finishes

The bit: **AST(0).#MOVE** is raised as long as the 0 axis is involved in a motion. Inversion of the bit (**^AST(0).#MOVE**), causing the bit to become non-zero, occurs when the motion ends for any reason. Therefore the above **TILL** command provides a delay of execution of the next command until the motion is over.

In the following example, the program starts a data collection and then a motion. The feedback position is sampled with a period of 1 millisecond and stored in the data array. After the data collection finishes, the data array contains a transient process of ptp motion. Synchronous data collection used in the example displays its state in the **AST(1).#DCit** which is raised as long as the data collection is in progress. The collected data can be safely used only after the data collection process has terminated. Here, the **TILL** command validates that both the motion and the data collection are over:

global real Data(1000)	Declare global real array Data of 1000 elements
------------------------	---

DC/s 1, Data, 1000, 1, FPOS(1)	Start data collection of FPOS(1) to array DATA , 1000 samples, 1ms period
PTP 1, 2000	Start positioning of the 1 axis to absolute point 2000
TILL ^AST(1).#MOVE & ^AST(1).#DC	Wait until both the 1 axis motion and the data collection finish

The following example provides the 3 axis motion in negative direction until a general purpose input becomes active and then terminates the motion:

JOG 3, -	Start jog motion of the 3 axis in negative direction
TILL IN0.5	Wait until input 5 is activated
HALT 3	Terminate the 3 axis motion

In the following example a general purpose output must be activated 25 millisecond before the motion end. The ACSPL+ **GRTIME** variable (for details on the **GRTIME** variable, see [The GRTIME Variable](#)) contains the estimated time that remains to the motion end.

PTP 0, 10000	Start positioning of the 0 axis to absolute point 10000
TILL GRTIME(0) <= 25; OUT0.4 = 1	Activate output 4 25 milliseconds before the motion ends.

The output activation, **OUT0.4 = 1**, is placed in the same line as the **TILL** command in order to avoid one controller cycle delay between program lines.

3.10.3 Autoroutines

The technique of autoroutines is similar to hardware interrupts. In distinction to routines that must be explicitly executed (by way of the **CALL** command), the autoroutine is automatically executed when a specific condition is satisfied. The routine interrupts the currently executing program, executes the commands specified in the autoroutine body, and then returns control to the interrupted program.

3.10.3.1 ON Command

Description:

The **ON** command flags the routine as an autoroutine and specifies the condition upon which the execution of the routine is based.

Syntax:

ON *expression*

Comments:

The value of **EXPRESSION** defines the condition. The condition is considered true if the expression calculates to a non-zero result. A zero result corresponds to a false condition, and the routine is not executed.

The controller never executes the **ON** command directly.



If the program execution flow hits an **ON** command, the controller asserts a run time error and aborts the program. Therefore you must either end the program before the **ON** command, or use an unconditional **GOTO** command to skip over the routine.

Instead of direct execution, the controller registers an autoroutine when the program containing the routine is compiled. Then the controller calculates the expression each controller cycle in parallel with executing ACSPL+ programs. If the expression calculates to a non-zero value, the controller interrupts the ACSPL+ program being executed in the same buffer where the autoroutine is located, and transfers the execution point to the autoroutine. If no ACSPL+ program is executed in the buffer, the controller does not interrupt any program and simply starts the autoroutine execution.

The controller implements edge-detection in autoroutine condition verification. If a condition becomes true, the controller activates the autoroutine only once. If the condition remains true afterwards, the controller does not activate the autoroutine again. The condition must become false and then become true again in order to activate the autoroutine again.

3.10.3.2 Autoroutine Body and Execution

The autoroutine body is a sequence of ACSPL+ commands that starts from the command following the autoroutine header. The body continues until it reaches a **RET** command.

The **RET** command terminates the autoroutine execution and transfers execution control back to the interrupted program. If no program was interrupted, the **RET** command simply terminates the program.

As explained above, an autoroutine interrupts the program in the host buffer, but is executed in parallel with programs that are executed in other buffers.

You can specify different execution rates (number of lines executed per one controller cycle) for regular programs and for autoroutines in the same buffer. The ACSPL+ **PRATE** array contains elements for each program buffer and specifies the execution rate for regular programs. The ACSPL+ **ONRATE** array specifies the execution rate for autoroutines.

For example, if you have configured the controller so that **PRATE(2)** is one, but **ONRATE(2)** is four, the program in buffer 2 will be executed one line per one controller cycle, and any autoroutine specified in buffer 2 that interrupts the program will be executed four lines per one controller cycle. When the **RET** command that terminates the autoroutine is executed, the controller switches back to the rate of one line per one cycle.

3.10.3.3 Autoroutine and the Host Buffer Interactions

An autoroutine can reside in any program buffer. The controller examines the conditions each controller cycles for all compiled autoroutines in all buffers.

There are, however, specific autoroutine-host buffer interactions:

- > The buffer's local variables can be used in the autoroutine condition as well as in the autoroutine body only in an autoroutine defined in the host buffer. However, all ACSPL+ and user global variables can be used in any buffer.
- > When the condition is satisfied, the autoroutine interrupts only the program executed in the host buffer. Programs that are concurrently executing in other buffers continue

executing in parallel with the autoroutine. When activated, the autoroutine prevents activation of other autoroutines in the same buffer. A program that is executed in any other buffer can be interrupted by an autoroutine specified in its own host buffer.

The following approaches are available to you for defining a set of autoroutines and assigning them to one or more buffers:

- > A specific autoroutine occupies a separate buffer with no other program or autoroutine in the buffer. Activating and executing the autoroutine has no direct affect on other programs or autoroutines. This approach is the most suitable for an autoroutine that takes a long time to execute, because a large autoroutine that shares a buffer with another program or autoroutines would prevent the activity of other programs or autoroutines during its execution.
- > Several autoroutines are specified in one buffer with no regular program in the same buffer. In this case the activation of an autoroutine does not interrupt any program, all programs executed in the other buffers continue executing concurrently. An activated autoroutine prevents the activation of another autoroutine in the same buffer until its termination.
- > One or more autoroutines are specified in a buffer along with a regular program. In this case the activation of the autoroutine interrupts the program execution. This approach is the most suitable if the program and the autoroutine are closely related and must use the same local variables. For example, the autoroutine processes the failure conditions for the program, and must interrupt the program if a failure occurs.

3.10.3.4 Examples

The following fragment demonstrates a typical use of autoroutine for processing the controller faults. The autoroutine provides an error message when the Drive Alarm of 0 axis occurs:

```
ON FAULT(0).#DRIVE      Activate autoroutine when bit FAULT(0).#DRIVE
                        changes from 0 to 1
DISP "X Drive Alarm"    Display an error message
RET                    End of autoroutine
```

The following autoroutine responds when either the Left Limit or Right Limit are activated:

```
ON FAULT(1).#LL | FAULT(1).#RL      Activate autoroutine when the right
                                     or left limit bit is activated on
                                     the 1 axis.
DISP "1 axis Limit Switch activated" Display an error message
RET                                End of autoroutine
```

The following example assumes that an extra ventilator must be activated when the motor overheat input signal is activated for the axis 2. The ventilator is controlled by the output bit: **OUT0.4**. The ventilator must be disabled when the signal returns to inactive state.

```
ON FAULT(2).#HOT      Activate autoroutine when bit FAULT(2).#HOT changes
                       from 0 to 1.
OUT0.4 = 1            Set output 4 to 1
```

```

RET                                End of autoroutine.
ON FAULT(2).#HOT                  Activate autoroutine when bit FAULT(2).#HOT changes
                                from 1 to 0.
OUT0.4 = 0                        Set output 4 to 0
RET                                End of autoroutine.

```

All bits, not only faults, can be used in autoroutine conditions. Assuming that output **OUT0.6** (of the 0 axis) is connected to a LED indicator, the following autoroutines signals the motion state bit to activate the indicator, and deactivate it when the 0 axis is no longer in motion:

```

ON MST(0).#MOVE                  When the MST(0).#MOVE bit changes from 0 to 1
                                (signaling that the X axis is moving)
OUT0.6 = 1                      Set output 6 to 1 (turns on the LED)
RET                              End of autoroutine.
ON ^MST(0)                      #MOVE When the MST(0).#MOVE bit changes from 1 to 0
                                (signaling that the X axis is no longer moving)
OUT0.6 = 0                      Set output 6 to 0 (turns off the LED)
RET                              End of autoroutine

```

The condition of an autoroutine can be any type of expression, not only bit verification. The following autoroutine provides an alarm message if a fault occurs in the controller:

```

ON S_FAULT                      When a fault occurs
DISP "Something happened"       Display an error message
RET                             End of autoroutine

```

The above autoroutine displays the alarm message only on the first fault. If one fault bit is already raised, and another fault occurs, the second fault does not cause the alarm message.

The ACSPL+ **MERR** (motor error) array can be used for motor failure processing. While a motor is enabled, the corresponding element of **MERR** is zero. If a motor is disabled, the element stores the reason why the motor was disabled. Codes greater than or equal to 5010 correspond to fault conditions. The following autoroutine displays a message when the controller disables the 0 axis due to any fault.

```

ON MERR(0) >= 5010              When the 0 axis motor is disabled
DISP "Motor 0 was disabled. Error code: ", MERR(0)  Display a message stating that the motor
                                                    was disabled, and the error code of the fault
RET                              End of autoroutine

```

The ACSPL+ **AERR** array can be used to detect abnormal motion termination.

The ACSPL+ **MERR** and **AERR** variables expose only those faults that cause motor disable or abnormal motion termination.

3.10.4 Program Management Commands

Program management commands are used for controlling the execution of a program. As any other command, a program management command can be either executed immediately as an Terminal Command, or stored in a buffer. Using program management commands within a program provides the ability to create a master program that manages execution of other programs.

3.10.4.1 START Command

Description:

The **START** command activates program execution in a buffer.

Syntax:

START *buffer_number, label_name*

Comments:

The command specifies a target buffer (**BUFFER_NUMBER**) that contains the program that must be activated. The **BUFFER_NUMBER** argument can be a constant or expression that calculates to integer number. The specified or calculated buffer number must fall into the range 0 to 9. If the number is out of range, error 3052 is generated.

The **LABEL_NAME** argument is a label in the program ([Names: Variable and Label](#)). Execution starts from that label.

If the **START** command is executed from a program, the specified **BUFFER_NUMBER** must be different from the buffer that contains the current program because a program cannot start itself. It will be aborted, generating error 3044.

The **START** command executes successfully if the target buffer is loaded with a program, compiled, but not running. Otherwise, the **START** command causes a run-time error and aborts the current program.

The program activated by the **START** command executes concurrently with the program containing the **START** command, and other active programs.

Examples:

The following fragment starts the program in buffer 2 from label **PSTART**:

```
START 2, Pstart           Start executing buffer #2 at the line labeled PSTART.
```

The following Terminal command displays change in buffer state after the **START** command was executed:

```
?2                               Querying status of buffer #2.
Buffer 2: 192 lines, running in line 153      Response to query.
```

3.10.4.2 STOP and STOPALL Commands

Description:

The **STOP** command terminates program execution in a buffer. The **STOPALL** command terminates all currently executing programs except the program that issued the command.

Syntax:

STOP [*buffer_number*]

STOPALL

Comments:

BUFFER_NUMBER is the buffer designator (an integer between 0 and one less than the total number of Program Buffers in the system)

The **STOP** command without **BUFFER_NUMBER** affects the currently executing program in the buffer and is the normal method of program termination.

The **STOP** command with **BUFFER_NUMBER** terminates a program in the specified buffer. A master program that manages the whole application can use this command in order to terminate a certain activity.

The **STOPALL** command executed by a program terminates all other concurrently executed programs, but the program itself continue executing.

After termination by the **STOP** or **STOPALL** command, a program remains in the compiled state. Therefore, if the program contains autoroutines, the autoroutines can be activated after the program termination whenever its condition is satisfied.

Examples:

The following command terminates the current program:

STOP

The following command terminates the program only if the 0 axis is disabled:

```
IF ^MST(0).#ENABLED STOP; END
```

The following command terminates the program executed in buffer 3:

```
STOP 3
```

The following command executed in buffer 0 terminates the programs currently executed in all buffers except buffer 0:

```
STOPALL
```

The following Terminal command displays change in buffer state after executing the **STOP** command:

?3	Querying status of buffer #3
Buffer 3: 35 lines, compiled, not running	Response to query.

3.10.4.3 PAUSE and RESUME Commands**Description:**

The **PAUSE** command suspends program execution in a buffer. The **RESUME** command resumes execution of a suspended program.

Syntax:

PAUSE *buffer_number*

RESUME *buffer_number*

Comments:

The **PAUSE** command suspends the program executed in the specified buffer (**BUFFER_NUMBER**). If no program is executed in the buffer, the command has no effect.

The **RESUME** command resumes execution of the program suspended in the specified buffer. If the program was not suspended, the command has no effect.

Examples:

The following command suspends the program currently executed buffer 0:

```
pause 0
```

The following Terminal command displays change in buffer state after executing the pause command:

```
?0                                     Querying status of buffer #0.
Buffer 0: 97 lines, suspended in line 69   Response to query.
```

The following command resumes program execution in buffer 0:

```
resume 0
```

The following Terminal command displays change in buffer state after executing the resume command:

```
?0                                     Querying status of buffer #0.
Buffer 0: 97 lines, running in line 83     Response to query.
```

3.10.4.4 ENABLEON and DISABLEON Commands

Description:

The **ENABLEON** command enables the activation of an autoroutine in a buffer. The **DISABLEON** command disables the autoroutine activation in a buffer.

Syntax:

ENABLEON *buffer_number*

DISABLEON *buffer_number*

Comments:

The commands alter the **NOAUTO** bit in the ACSPL+ **PFLAGS** variable that controls autoroutine activation (see *SPiiPlus Command & Variable Reference Guide* for details on the **PFLAGS** variable).

If the bit is reset, the controller starts verifying the condition of an autoroutine and can activate the autoroutine as soon as the buffer is compiled. Setting the bit prevents the autoroutine activation even if the buffer is compiled and the condition is true.

Examples:

The following dialog shows the effect of the commands on the buffer state:

```
DISABLEON 0                               Disabling autoroutines in
                                           buffer #0
```



```
?0
Buffer 0: 97 lines, compiled, not running, autoroutines disabled
ENABLEON 0

?0
Buffer 0: 97 lines, compiled, not running
```

```
Querying status of buffer #0.
Response to query
Enabling autoroutines in
buffer #0
Querying status of buffer #0.
Response to query.
```

4. ACSPL+ Motion Programming

This chapter provides practical details for using ACSPL+ to program motion. It covers the specific commands for programming motion. It should be used in conjunction with the *SPiiPlus Command & Variable Reference Guide*.

4.1 Axis/Motor Management Commands

Axis/Motor Management commands comprise various operations that change the state of the motors and the axes, establish relations between the motors and the axes, and manage executed motion.

4.1.1 ENABLE & DISABLE Commands

The **ENABLE** command activates one or more motors and drives. After the **ENABLE** command, the motor starts following the reference and physical motion is available.

The **DISABLE** command shuts off one or more drives and motors. After the disable command the motor cannot follow the reference and remains idle.

Syntax:

ENABLE *axis_specification*

DISABLE *axis_specification* [, *reason*]

In simple cases **AXIS_SPECIFICATION** is a single axis like 0 or 13, or a string consisting of axis enclosed in parentheses and separated by commas, for example: (0, 2, 13), or the keyword **ALL** (specifying all available non-dummy axes).

The **ENABLE** and **DISABLE** commands affect all specified axes.

The optional second parameter of the **DISABLE** command (**REASON**) must be an integer constant or expression and specifies a reason why the motor was disabled. If the parameter is specified, its value is stored in the **MERR** variable. If the parameter is omitted, **MERR** stores zero after the disable operation.

A reason stored in the **MERR** variable is cleared by the **FCLEAR** ([FCLEAR Command](#)) or **ENABLE** command.

As long as the motor is enabled, the controller provides the following:

- > Holds output **ENA** (enable drive) in active state.
- > Calculates **PE** (non-critical position error).
- > Performs closed loop control (for servo motors).
- > Examines conditions of **PE** and other faults and raises the corresponding fault bits if required.

The following variables/bits can modify execution of the enable command:

ENTIME	Defines the time (or maximum time) of ENABLE execution
MFLAGS.#ENMOD	Defines the mode of ENABLE execution

FMASK.#DRIVE	Defines if the drive alarm fault is processed
SAFINI.#DRIVE	Defines an active level of the drive alarm safety signal

If the **MFLAGS.#ENMOD** bit is 1, the **ENTIME** value defines the time of enable execution. In executing the **ENABLE** command, an ACSPL+ program always waits for **ENTIME** milliseconds. If then the drive alarm fault is zero, the **ENABLE** command is considered successful; otherwise the **ENABLE** command fails.

If the **MFLAGS.#ENMOD** bit is 0, the **ENTIME** value defines the maximum time allotted for **ENABLE** execution. Executing **ENABLE**, an ACSPL+ program monitors the drive alarm input signal. As soon as the drive alarm becomes inactive, the **ENABLE** command finishes execution with success. If the drive alarm signal does not change to inactive state within **ENTIME** milliseconds, the **ENABLE** command fails.

Examples:

```
ENABLE 0          Enable axis 0
ENABLE (2,3)      Enable axes 2 and 3
DISABLE 2,5011    Disable axis 2, store 5011 as a disable reason. Code 5011
                  corresponds to left limit error, therefore the 2 axis
                  motor will be reported as disabled due to fault involving
                  left limit.
DISABLE (2,3)     Disable motors of axes 0 and 3
```

4.1.2 COMMUT Command

The **COMMUT** command performs autocommutation and may be used when the following conditions hold true:

- > The motor is DC brushless (AC servo)
- > The motor is enabled
- > The motor is idle (not moving)



Versions 2.60 and later support **COMMUT** command in GANTRY mode. Commutation of the primary axis will automatically trigger commutation of the secondary axis.

The **COMMUT** command is used in autocommutation-based startup programs.

The command will operate properly only after the SPiiPlus MMI Adjuster has been used to:

- > Perform initial commutation adjustment
- > Adjust the motor properly
- > Save the adjustment parameters to the controller's flash memory.

Commutation using the SPiiPlus MMI Adjuster is described in depth in the *SPiiPlus MMI Application Studio User Guide*.

Motor movement during commutation very much depends on the motor drive settings. The **COMMUT** command will not operate properly if the **SLPKP** variable is set to zero, or the integrator is very low.

Syntax:

COMMUT *axis, [excitation_current,] [settle_time,] [slope_time][gantry_commut_delay]*

Arguments

axis	Specifies the motor to start commutation.
excitation_ current	Specifies the motor current used during autocommutation. The current is specified in a percentage of the maximal value. The controller restricts the actual current value by the XCURI value. The argument can be omitted in which case the default value is $0.98 \cdot \mathbf{XRMS}$. You may wish to specify a greater value if the axis static friction is high, or a lower value if the axis static friction is low.
settle_time	Specifies the settling time in the autocommutation process in milliseconds. The argument can be omitted in which case the default value is 500 milliseconds. You may wish to specify a greater value in case of low-bandwidth or slow damping systems.
slope_time	Specifies the time that the excitation current rises from zero to the desired value. The argument can be omitted in which case the default value is 0 providing an immediate build-up of the excitation current. Slope time is required only in special cases and it is usually recommended to omit this argument in which case the excitation current is built instantly.
gantry_ commut_delay	Optional – can be used only in Gantry mode. It defines the delay time in milliseconds after the commutation of the primary axis is completed and before the commutation of the complimentary axis begins. The default value is 500 msec.

The **COMMUT** command executes the autocommutation algorithm three times for verification and elimination of unstable equilibrium. The approximate execution time of the command is therefore $3 \cdot (\mathbf{SETTLE_TIME} + \mathbf{SLOPE_TIME})$.

It should be noted that:

- > In air bearing systems a lower **EXCITATION_CURRENT** may be required.
- > In high friction systems a higher **EXCITATION_CURRENT** value is required.

The **EXCITATION_CURRENT** should be the same as that which you determined in the initial commutation adjustment process.

The **SETTLE_TIME** parameter determines the settling time for the autocommutation process initiated by the **COMMUT** command. The entire autocommutation process lasts approximately three times longer, since the command executes the algorithm three times for verification.



In low-bandwidth systems (high inertia, etc.) a higher value may be required.

The **SETTLING_TIME** should be the same as that you have determined in the initial commutation adjustment process.

4.1.3 KILL and KILLALL Commands

The **KILL** command causes one or more motors to terminate motion using a second-order deceleration profile. The deceleration value is specified by the **KDEC** variable (see *SPIIPlus Command & Variable Reference Guide*).

The **KILLALL** command provides kill operation for all motors.

The commands have the following syntax:

KILL *axis_specification* [, *cause*]

KILLALL [, *cause*]

In simple cases **AXIS_SPECIFICATION** is a single axis like 0 or 13, or a string consisting of axis enclosed in parentheses and separated by commas, for example: (0, 2, 13), or the keyword **ALL** (specifying all available non-dummy axes).

The optional **CAUSE** argument, specifying a cause why the motor was killed, must be an integer constant or expression that results in an integer. If the parameter is specified, its value is stored in the **MERR** variable. If the parameter is omitted, the **MERR** stores zero after the kill operation.

If several sequential kill operations specify different causes for the same motor, only the first **CAUSE** will be stored in **MERR** and all subsequent causes will be ignored.

A **CAUSE** stored in the **MERR** variable is cleared by the **FCLEAR** (see [FCLEAR Command](#)) or **ENABLE** command.

Each motor specified in a kill operation decelerates individually using its **KDEC** deceleration value.

Any motion related to the killed motors is terminated. If a motion involves several motors and only some of the motors are specified in a kill command, all other motors decelerate synchronously using a third-order profile and the **DEC** deceleration value (same behavior as with the **HALT** command, [HALT Command](#)).

The following examples illustrate **KILL** execution under different conditions. (Some of the examples involve a default connection. This is a condition where a motor depends only on the corresponding axis and the difference between motor and axis can be ignored. For more information about default and non-default connections, see [CONNECT Command](#) and [DEPENDS Command](#).)

- > **KILL** command with motor idle
Assume, none of the currently executed motions involves the 2 axis motor.
The command **KILL 2** does not affect the motor in any way.
The command **KILL 2,6100** does not affect the motor, but stores code 6100 (user-defined cause) in the **MERR(2)** variable. The code is stored only if at this moment the variable is zero, otherwise the command does not overwrite the previously stored code and the cause specified in the command is ignored.

> **KILL command with single-axis motion, default connection**

Assume, axis 1 executes motion **PTP/V 1,6000,20000**.

Once **KILL 1** is executed, the motor starts decelerating from its instant velocity using constant deceleration value specified by the **KDEC(1)** variable. Deceleration time is given by:
 $V_1 / KDEC(1)$

where V_1 is the instant velocity at the moment of the **KILL** command. V_1 is not necessarily 20000 as specified in the motion command, it can be lower if the **KILL** command is executed in acceleration or deceleration phases of the motion.

Typically, the motor finishes the **KILL** process and stops before it reaches the final motion point of 6000. However, if **KDEC(1) < DEC(1)** (not recommended in most applications), the motor can overrun the final point.

The motion is considered to continue execution as long as the kill process is executed. Bit **AST(1).#MOVE** remains 1 while the motor is decelerating and drops to 0 once the motor reaches zero reference velocity. Bit **MST(1).#MOVE** also remains 1 while the motor is decelerating but drops to 0 only when the motor reference velocity is zero and the motor position error **PE(1)** remains less than **TARGRAD(1)** for more than **SETTLE(1)** milliseconds.

> **KILL command with several single-axis motions, default connection**

Assume, each of the axes 0, 2, 4 executes independent single-axis motion.

The command **KILL (0,2,4),6088** is equivalent to **KILL 0,6088; KILL 2,6088; KILL 4,6088** and acts on each motion independently. Each motor uses its own component of **KDEC** and the time of the **KILL** process is different for the motors.

The reason for the **KILL, 6088** (user-defined code), is stored in **MERR(0)** (for the 0 axis), **MERR(2)**(for the 2 axis) and **MERR(4)** (for the 4 axis). However, if at the moment one of these variables contains a non-zero value, the value is not overwritten and the previously stored cause is retained.

The command **KILL 4** (again with default connection) kills the axis 4 motor and terminates the axis 4 motion, but does not affect motors and motions of axes 0 and 2.

> **KILL command with multi-axis motion, default connection**

Assume, motion **MPTP (0,1,4)** is executed.

The command **KILL 1** causes the axis 1 motor to start decelerating from its instant velocity using the constant deceleration value specified by the **KDEC(1)** variable. The deceleration continues until the motor reaches zero velocity.

The behavior of 0 and 4 axes is different. Once the **KILL 1** is executed, the motion starts a third-order deceleration process just as if a **HALT** command was executed. The 0 and 1 axes continue moving in the common motion. The vector deceleration of the motion is **DEC(0)** and the vector jerk is **JERK(0)**.

If command **KILL (1,4)** is executed, the **KILL** process applies to 1 and 4 axes motors. Each motor decelerates independently from its instant velocity to zero using the constant decelerations **KDEC(1)** and **KDEC(4)**. At the same time the 0 axis motor decelerates using the third-order profile and the **DEC(0)** deceleration value, just as if a **HALT** command was executed.

If command **KILL (0,1,4)** is executed, each motor decelerates independently from its instant velocity to zero using the constant decelerations **KDEC(0)**, **KDEC(1)** and **KDEC(4)**.

In all cases bits **AST.#MOVE** and **MST.#MOVE** of axes 0,1, and 4 remain 1 as long as any of the motor continues decelerating. Once all motors reach zero velocity, bits **AST(0).#MOVE**, **AST(1).#MOVE** and **AST(4).#MOVE** drop to zero. Bit **MST(0).#MOVE** drops to zero as soon as

position error **PE(0)** remains less than **TARGRAD(0)** for more than **SETTLE(0)** milliseconds. So do bits **MST(1).#MOVE** and **MST(4).#MOVE** (for the 1 and 4 axes respectively).

> **KILL command with non-default connection**

If a motor is in non-default connection but depends only on the corresponding axis the effect of the **KILL** command is similar to the case of default connection. For example, if the connection was specified as

```
CONNECT RPOS (0) = 0.5*APOS (0) *APOS (0)
DEPENDS 0,0
```

(in this case the **DEPENDS** command is not necessary), the **KILL 0** command starts the same kill process on the 0 axis motor and the halt process on the motion that involves the 0 axis. All above considerations about the idle motor, single-axis motion and multi-axis motion remain the same.

The result is a little different if a motor depends on another axis or on several axes, for example:

```
CONNECT RPOS (2) = APOS (0) + APOS (2) - APOS (4)
DEPENDS 2, (0,2,4)
```

(in this case the **DEPENDS** command is required). The difference is that the **KILL 2** command applies the halt operation to all executed motions involving any of the axes 0, 2, or 4. Correspondingly, bits **AST(2).#MOVE** and **MST(2).#MOVE** remain 1 as long as any of these motions continues its termination process.

A **KILLALL** command always terminates all executed motions and therefore makes no difference between the default and non-default connection.

4.1.4 FCLEAR Command

The **FCLEAR** command clears the current faults and the result of the previous fault stored in the **MERR** variable.

The command syntax is:

FCLEAR [axis_specification]

In simple cases **AXIS_SPECIFICATION** is single axis like 0 or 13, a string consisting of axis enclosed in parentheses and separated by commas, for example: (0, 2, 13), or keyword **ALL** for all axes.

If **AXIS_SPECIFICATION** is omitted, the command clears the system faults. If **AXIS_SPECIFICATION** is specified, the command clears the **FAULT** and **MERR** components for the specified axes.

However, if a reason for a fault is still active, the controller will set the fault again immediately after the **FCLEAR** command.

If one of the cleared faults is an encoder error, the command also resets the feedback position to zero.

4.1.5 SET Command

The **SET** command determines a current value of the feedback, reference or master position. The command syntax is:

SET axis_VAR=expression

Only the following variables can be specified in **AXES_VAR**:

FPOS	Feedback Position
F2POS	Secondary Feedback Position
RPOS	Reference Position
APOS	Axis Reference Position

Although the **SET** command resembles the **ASSIGNMENT** command, execution of the **SET** command is different from **ASSIGNMENT**. The **SET** command induces a complex operation in the controller instead of a simple assignment.

Regardless of the left-side variable, execution of the **SET** command starts with calculation of **EXPRESSION**. The result of the calculation provides the right-side value. Then the execution depends on the variable specified on the left side.

The following are examples of the use of **SET**.

> SET RPOS and SET FPOS

The **SET** command that contains **RPOS** or **FPOS**, shifts the origin of an axis. For example, command

```
SET FPOS(0) = 0
```

places the origin of the 0 axis to the point where the motor is located this moment.

FPOS and **RPOS** provide a reference and a feedback value for a motor. If a control loop works properly, **FPOS** follows **RPOS** with small or zero error.

If the error is zero, both **SET FPOS** and **SET RPOS** provide the same result: both **FPOS** and **RPOS** become equal to the right-side value. This is not a simple assignment, and the command adjusts the controller offsets so that the periodic calculation of **FPOS** and **RPOS** will provide the required results.

If the error is non-zero, the result of **SET FPOS** and **SET RPOS** may differ slightly. Consider the following example:

```
?RPOS(0), FPOS(0)      Query RPOS and FPOS for the 0 axis
6000                   The RPOS and FPOS differ by 2 counts due to, for
                        instance, the bias in the amplifier

6002
SET RPOS(0) = 0        Set RPOS to 0 for the 0 axis
?RPOS(0), FPOS(0)      Query RPOS and FPOS for the 0 axis
0                      RPOS is set to exact zero
2                      FPOS is set to 2 in the current point in order to
                        retain the offset between RPOS and FPOS

SET FPOS(0) = 0        Set FPOS to 0 for the 0 axis
?FPOS(0), RPOS(0)      Query RPOS and FPOS for the 0 axis
0                      FPOS is set to zero in the current point
```



```
-2      RPOS is set to -2 in order to retain the offset
      between RPOS and FPOS
```

In both **SET** commands no physical motion occurs. The 0 axis remains in the same position, only the internal offsets in the controller are adjusted to shift the origin as required.

Even if a motor is idle, several identical **SET FPOS** commands may place the origin at slightly different points due to the jitter in feedback.

If a motor is flagged by the Default Connection bit (**MFLAGS.#DEFCON**), the **RPOS** and **APOS** variables are conjugate. Therefore, any command that changes **RPOS**, also changes the corresponding **APOS** to the same value.

> SET F2POS

The command **SET F2POS** shifts the origin of the secondary axis feedback. For example, command

```
SET F2POS (0) = 0
```

places the origin of the 0 axis secondary feedback to the point where the motor is currently located.

As a result of the command execution, **F2POS** becomes equal to the right-side value. This is not a simple assignment, as the command adjusts the controller offsets so that the periodic calculation of **F2POS** will provide the required result (the specified value in the current point).

Even if a motor is idle, several identical **SET F2POS** commands may place the origin in slightly different points due to the jitter in feedback.

> SET APOS

If a motor is flagged by the Default Connection bit (**MFLAGS.#DEFCON**), variables **RPOS** and **APOS** are conjugate, and always keep the same value. In this case, the **SET APOS** command is identical to the **SET RPOS** command for the same axis.

For non-default connection a motor and the corresponding axis are separated. Variables **RPOS** and **APOS** may have different values. In this case, command **SET APOS** shifts the origin of the axis but has no effect on the origin of the motor. The controller adjusts offsets so that the command causes no jerk in the motor.



In the case of non-default connection the controller adjusts offsets only for the motors that depend on the specified axis. Therefore, the depends command is significant in a connection specification. If dependence is specified incorrectly, one or more motors can jump once **SET APOS=...** is executed.

4.1.6 GROUP, SPLIT & SPLITALL Commands

The **GROUP**, **SPLIT** and **SPLITALL** commands manage grouping the axes in coordinate systems for multi-axis motion.

- > The **GROUP** command creates a coordinate system for multi-axis motion.

For most applications there is no need for the **GROUP** commands in that the controller automatically creates and splits groups. Mainly you would include the command in order to keep a check that you have included all of the relevant axes in the subsequent motion commands. If you include a motion command that does not relate to all of the axes in the group (without a previous **SPLIT** command), the controller issues an error.

- > The **SPLIT** command breaks down an axis group previously created with a **GROUP** command.
- > The **SPLITALL** command breaks down all axis groups previously created with a **GROUP** command.

Syntax:

GROUP *axes_specification*

SPLIT *axes_specification*

SPLITALL

The format of the **AXES_SPECIFICATION** is a list of the axes separated by commas and enclosed in parentheses, for example (0,1,2,4). After power-up, each controller axis is a single axis, no axis group exists. One-axis motion does not require any axis group. One-axis motion can be activated immediately after power-up, assuming that the motor is enabled. Several one-axis motions can be activated in parallel, and do not require any axis group definition.

An axis can belong to only one group at a time. If the application requires restructuring the axes, it must split the existing group and only then create the new one.

For example, the command:

```
GROUP (0,2,3)
```

creates an axis group that includes axes 0, 2 and 3.

The first axis in the **AXES_SPECIFICATION** (0 in the above command) is the leading axis. The motion parameters of the leading axis become the default motion parameters for the group. For example, for the above (0,2,3) group, the values of ACSPL+ variables **VEL(0)**, **ACC(0)**, **DEC(0)**, **JERK(0)**, **KDEC(0)** become the default values of velocity, acceleration, deceleration, jerk and kill deceleration for all motions executed in this group. If, for example, a group was defined as (3,2,0), the 3 axis is leading and the values of 3 will be used as the default motion parameters for 0 and 2.

In all other aspects the leading axis has no preference, and the order of axis letters in group definition is not important. The motion commands referencing a group are not required to specify all axes of the group or in the same order. However, an axis can belong to only one group, so all specified axes must belong to the same group. A motion command that references axes from different groups will fail.

The **SPLIT** command must specify the same axes as the **GROUP** command that created the group. After the **SPLIT** command the group no longer exists.



If the **SPLIT** command specifying an axis that is currently in motion is executed within the buffer, the buffer execution is suspended until the motion is completed. However, if the **SPLIT** command is sent from the host or as a Terminal command, it returns error 3087: "Command cannot be executed while the axis is in motion".

The **SPLITALL** command breaks up all existing groups. An ACSPL+ program that starts in an unknown environment (not just after power-up) can execute the **SPLITALL** command in order to ensure that no axes are grouped.

4.1.7 GO Command

The **GO** command starts a motion that was created using the **/W** switch (see [SLAVE Command](#)). A motion that has been created without this switch starts automatically after creation and does not require the **GO** command.

Syntax:

GO *axes_specification*

In simple cases **AXES_SPECIFICATION** is a single axis like 0 or 13, or a string consisting of axis enclosed in parentheses and separated by commas, for example: (0, 2, 13), or the keyword: **ALL** (specifying all of the axes).

There following possibilities are available:

> Starting single-axis motion

A **GO** command specifies one axis that is not included in any group. The command starts the last created motion for the same axis. If the motion was not created, or has been started before, the command has no effect. For example:

```
PTP/w 0, 1000    Create the motion, but do not start it
GO 0            Start the motion
```

> Starting common motion

A **GO** command specifies a leading axis in a group. The command starts the last created motion for the same axis group. If the motion was not created, or has been started before, the command has no effect. For example:

```
PTP/w 0, 1000    Create the motion, but do not start it
TILL IN0.1       Wait until input 1 is activated
GO 0            Start the motion
```

> Synchronous start of several motions

A **GO** command can specify several axes. Each axis in the specification must be either a single axis not included in any group or a leading axis in a group. The command synchronously starts the last created motions for all specified axes and groups. If any of referenced motions was not created, or has been started before, the command does not affect this axis/group but does affect all other specified axes/groups. For example:

```
PTP/w (0,1), 1000,1000   Create the motion, but do not start it
PTP/w 2, 8000           Create the motion, but do not start it
GO (0,1)                Start both motions synchronously
```

4.1.8 HALT Command

The **HALT** command terminates a motion using a deceleration profile. The deceleration value is specified by the **DEC** variable (see *SPiPlus Command & Variable Reference Guide*).

Syntax:

HALT *axes_specification*

In simple cases **AXES_SPECIFICATION** is a single axis like 0 or 13, or a string consisting of axis enclosed in parentheses and separated by commas, for example: (0, 2, 13), or the keyword: **ALL** for all axes.

The following possibilities are supported:

- > **Terminating single-axis motion**
A **HALT** command specifies one axis that is not included in any group. The command terminates the currently executed motion for the same axis. If no motion is executed, the command has no effect.
- > **Terminating common motion**
A **HALT** command specifies a leading axis in a group. The command terminates the currently executed motion in the same axis group. If no motion is executed, the command has no effect.
- > **Terminating several motions**
A **HALT** command specifies several axes. Each axis in the specification must be either a single axis not included in any group or a leading axis in a group. The command terminates currently executed motions in all specified axes and groups.
If any of referenced axes are idle, the command does not affect this axis/group but does affect all other specified axes/groups.

4.1.9 BREAK Command

The **BREAK** command provides premature termination of a motion with smooth transition to the next motion. The command executes differently in the following two cases:

1. The next motion already waits in the motion queue. The **BREAK** command terminates the current motion and starts the next motion immediately. The controller provides smooth velocity profile of transition from motion to motion.
2. There is no next motion in the motion queue. The **BREAK** command has no immediate effect. The current motion continues until the next motion appears in the motion queue. At that moment the controller breaks the current motion and starts the next as described above. If the current motion finishes before the next motion comes to the queue, the command has no effect.

Syntax:

BREAK *axes_specification*

In simple cases **AXES_SPECIFICATION** is a single axis like 0 or 13, a string consisting of axis enclosed in parentheses and separated by commas, for example: (0, 2, 13), or the keyword: **ALL** (specifies all available non-dummy axes).



The **BREAK** command is not supported in path or master-slave motion.

The following possibilities exist:

> **Terminating single-axis motion**

A **BREAK** command specifies one axis that is not included in any group. The command terminates the currently executed motion for the same axis. If no motion is executed, the command has no effect.

> **Terminating multi-axis motion**

A **BREAK** command specifies a leading axis in a group. The command terminates the currently executed motion in the axis group. If no motion is executed, the command has no effect.



In multi-axis motion, smooth vector velocity profiles do not always assure smooth motion of the coordinates. The user application must provide nearly tangent motion trajectories in the junction point to avoid jumps in coordinate velocity, which may cause damage to equipment.

> **Terminating several motions**

A **BREAK** command specifies several axes. Each axis in the specification must be either a single axis not included in any group or a leading axis in a group. The command terminates currently executed motions in all specified axes and groups. If any of referenced axes are idle, the command does not affect this axis/group but does affect all other specified axes/groups.

4.1.10 IMM Command

The **IMM** command provides on-the-fly changes of the motion parameters: velocity, acceleration, deceleration, jerk, and kill deceleration.

Syntax:

IMM motion_var=command

Only the following variables can be specified as **MOTION_VAR**:

VEL	Velocity
ACC	Acceleration
DEC	Deceleration
JERK	Jerk

KDEC

Kill deceleration

Although the **IMM** command resembles the **ASSIGNMENT** command, execution of the **IMM** command differs from normal assignment to the same variables.

As in conventional assignment, execution of the **IMM** command starts from calculation of the right-side expression. The calculated right-side value is assigned to the left-side variable. Execution of flat assignment finishes at this point.

The difference between the conventional **ASSIGNMENT** and the **IMM** commands becomes apparent when the command executes while a motion is in progress. The **ASSIGNMENT** command does not affect the motion in progress or any motion that was already created and is waiting in a motion queue. Only the motions created after the **ASSIGNMENT** command is executed will use the motion parameters changed by the command. The **IMM** command, on the other hand, not only changes the specified variable, but also affects the motion in progress and all motions waiting in the corresponding motion queue. To change a motion on-the-fly, the **IMM** command must change a variable of the axis that is a single axis of the motion or a leading axis if the motion is in axis group.

4.2 Point-to-Point Motion

This section covers the commands and relevant ACSPL+ standard variables for Point-to-Point (PTP) motion.

4.2.1 PTP

Description

PTP (point-to-point) generates motion for the specified axis or axes to a specified destination point.

When **PTP** specifies a single axis, the motion profile is calculated according to VEL, ACC, DEC, JERK values of the axis.

In group motion, when **PTP** specifies multiple axes, the group motion profile is based on the leading axis' VEL, ACC, DEC, JERK motion values, unless **PTP/m** is used.

PTP terminates due to:

- > Interruption by any new motion command
- > Any fault activation that disables the drive or kills the motion
- > User termination by **HALT**, **KILL**, or **BREAK**.

Syntax

PTP[switches]*axis_list, destination-point[,value for v, value for f, motor_movement_delay]*

Arguments

axis_list	Single axis or axis group, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.
destination point	Final destination.
value for v	Optional argument for user-defined velocity.

value for f	Optional argument for user-defined velocity at a destination point
motor_movement_delay	(Optional, used only with /q switch) Delay, in milliseconds, before motor motion actually starts.

Switches

/e	Wait for motion termination before executing next command.
/f	Specify non-zero velocity at each destination point (or points) in a series of PTP motions
/m	Use the motion profile values of the axis group as a whole, rather than those of the leading axis, without exceeding any of the defined axes motion VEL , ACC , DEC , JERK values.
/r	The destination point is relative to the start point.
/v	Use the specified velocity instead of the default velocity (VEL).
/w	Create the motion, but do not start until GO.
/z	Interpret entered coordinates according to the Local Coordinate System.
/q	Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The maximum delay is 100 controller cycles: 100ms for CTIME=1ms or 20ms for CTIME=0.2ms. Allows delaying actual motor movement start (RPOS) for the specified delay, while motion profile generation (APOS) starts immediately.

Comments

- > Axes destination points, and relative velocity in the **PTP** command can also be an expression.
- > **PTP** can be used for executing point-to-point motion for a group of axes. For example, **PTP (0, 1, 2)** creates motion for axes 0, 1, and 2 as a group.
- > In single axis **PTP** motion, the final velocity (indicated by the **/f** switch) may be negative. In this case, the motion terminates at the specified coordinate with a velocity opposite to the motion direction.
- > In single axis **PTP** motion, if the motion is in the negative direction, **GVEL** can be negative.

Related ACSPL+ Commands

MPTP...ENDS, POINT

COM Library Methods and .NET Library Methods

ToPoint

C Library Functions

acsc_ToPoint

Examples

Example 1:

```
PTP/v 1, 2000, 500      !PTP axis 1 to point 2000 with velocity 500
```

Example 2:

```
PTP/rw (0,1), 1000, 2000 !PTP axes 0 and 1 where the 1 target point is
1000
                                !and the 0 target point is 2000. The target points
                                !are relative to the start point. Motion will not
                                !commence until GO command is issued.
```

Example 3:

```
ENABLE 0          !Enables axis 0
VEL(0)=10000      !Sets the default axis 0 velocity to 10000
SET RPOS(0)=0     !Sets the current axis 0 position to 0
PTP/rf 0, 2000, 1000 !Initiates a relative axis 0 motion of 2000 with end
                    !velocity of 1000 at the destination point
PTP 0, 4000       !Initiates an absolute axis 0 motion to 4000
STOP              !Ends program
```

4.2.2 SPTP Command

Description

The **SPTP** command is a multi-axis Point-to-Point motion defining motion to a specific target point, using syntax similar to the **PTP** command. Unlike **PTP**, **SPTP** uses a 4th order motion profile.

If the axis is moving when the command is issued, the controller creates the motion and inserts it into the axis motion queue. The motion waits in the queue until all motions before it finish, and only then starts.

The **SPTP** command can act on either a single axis or an axis group.

SPTP responds to **KILL** /**HALT** commands according to the same rules as other motion commands.

Syntax

```
SPTP[/switches] axis_list, target_point[, velocity]
```

Arguments

axis_list	Motion axes designation.
target_point	Target point coordinates
velocity	Optional argument for user-defined velocity

Switches

/e	Wait for motion termination before executing next command
/v	Use the specified velocity instead of the default velocity (VEL).
/w	Create the motion, but do not start until the GO command is issued
/z	Move relative to Local Coordinate System

Comments

SPTP looks like the **PTP** command. Unlike **PTP**, **SPTP** uses a 4th order motion profile.

The **SPTP** command can take either a single axis or an axis group as a parameter.

Example

```
SPTP (X,Y,Z) 100, 100, 300
```

4.2.3 Move and Settle Time Calculation

ACSPL+ supports measurement of time elapsed for point-to-point motion. Motion is defined as complete when the axis enters a predefined radius of the target point and stays there for a predefined period of time. Three settle radiuses may be defined using the variables **TARGRADA**, **TARGRADB**, and **TARGRADC**. The variables **SETTLEA**, **SETTLEB**, and **SETTLEC** define the period during which the axis must be within the corresponding **TARGRAD** distance of the motion end-point to meet the criterion for settling. In terms of ACSPL+ variables, this criterion means that $(\text{abs}(\text{TPOS-FPOS}) < \text{TARGRADA/B/C})$ for a period of **SETTLEA/B/C**

When the move and settle feature is activated by the SETCONF(318) to either single mode (value 1) or auto mode (value 2), the elapsed time of the motion up to entry to the **TARGRAD<x>** radius may be retrieved from the corresponding variables **MSTIMEA**, **MSTIMEB**, and **MSTIMEC**.

Example Code

```
! User units - mm
!Example 1 - Motor is settled to 1 micron window
! Window A
TARGRADA(0) = 1e-3 !Set threshold to 1um
SETTLEA(0) = 100 ! Stay in window for 100ms till assigning value to
MSTIMEA

! Example 2 - Motor is settled to 75 micron window - in this case the
motor is considered settled before the motion is completed
! Window B
TARGRADB(0) = 80e-3 !Set threshold to 80um
SETTLEB(0) = 150 ! Stay in window for 150ms till assigning value to
MSTIMEB

! Window C
```

```
TARGRADC(0) = 1e-3 !Set threshold to 1um
SETTLEEC(0) = 5 ! Stay in window for 5ms till assigning value to MSTIMEC

setconf(318, 0, 1)

!Example 3
!setconf(318, 0, 2) !Uncomment for running example 3

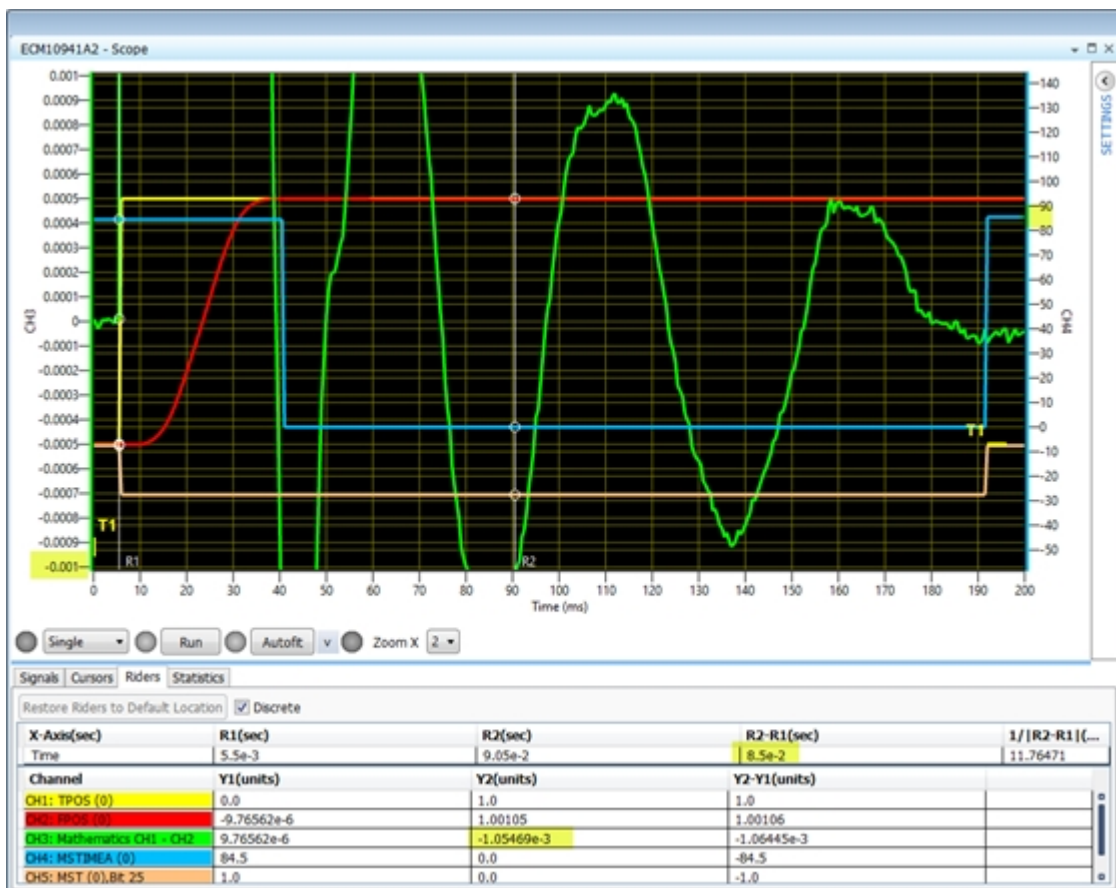
enable(0)
PTP/e(0), 1

wait 1000
PTP/e(0), 0

stop
```

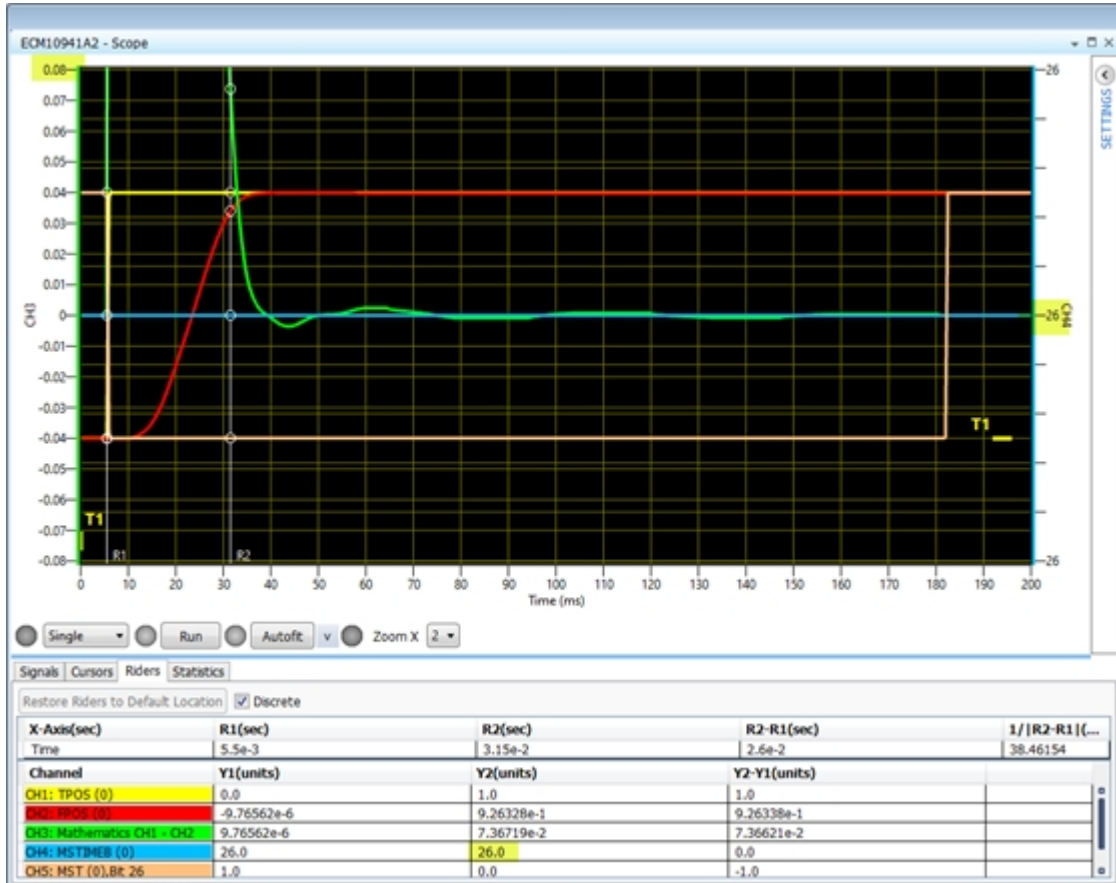
Example 1 Results

The motor is considered settled 85ms after the motion started. **MSTIMEA** is updated 100ms after the motor is within 1 μ m from target position **abs(TPOS-FPOS)** **MST.25** is set once this condition is satisfied.



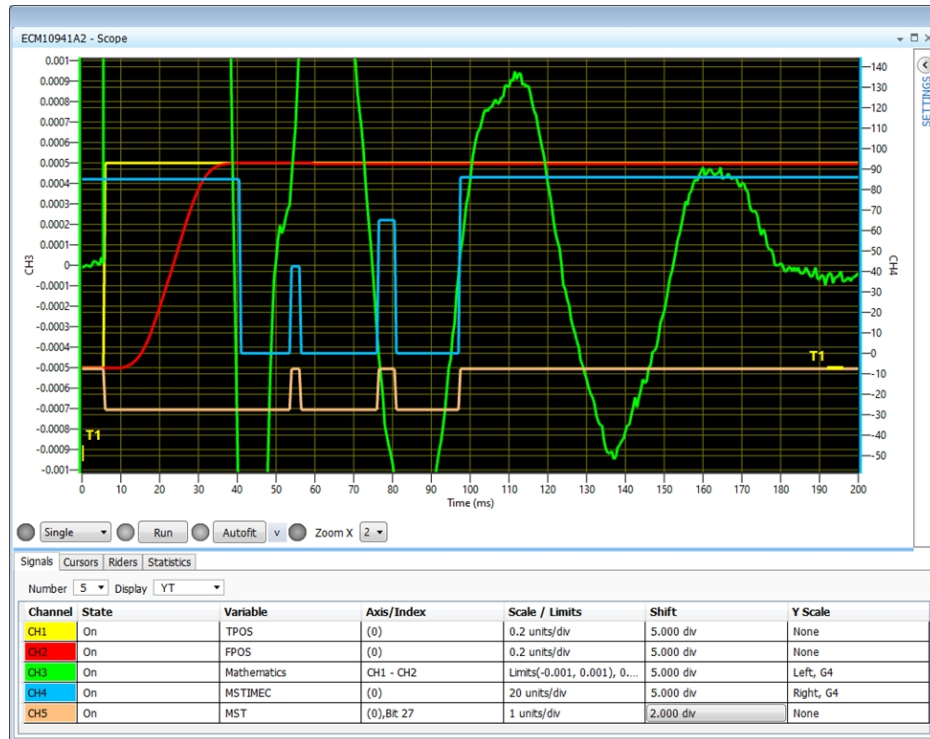
Example 2 Results

The motor is considered settled 26ms after the motion started. In this case, the motor is considered settled before the motion profile is completed. **MSTIMEB** variable is updated 150ms after the motor is within the 75 μ m from target position **abs(TPOS-FPOS)** **MST.26** is set once this condition is satisfied.



Example 3 Results

The value of **MSTIMEC** is updated each time the **abs(TPOS-FPOS)** is within **TARGRADC** for a period of **SETTLEC**. **MST.27** bit is cleared each time the condition above is not satisfied anymore, meaning that the value in **MSTIMEC** is no longer valid. Only after the condition is satisfied again, the bit is set.



4.2.4 MPTP, POINT, MPOINT, and ENDS Commands

These commands are used for programming a sequence of multi-point motion.

Syntax:

MPTP [/switch] axis_designators [,dwell_time][, motor_motion_delay]

POINT axis_designators, coordinate [,coordinate] [,velocity]

MPOINT axis_designators, point_matrix, number_of_points

ENDS

Where **SWITCH** can be one or a combination of:

w	Create the motion, but do not start until the GO command has been issued.
v	Use the velocity specified in the command instead of the default velocity.
r	Treat points as relative.
c	Use the point sequence as a cyclic array: after positioning to the last point do positioning to the first point and continue.
z	Interpret entered coordinates according to the Local Coordinate System.

q

Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The maximum delay is 100 controller cycles: 100ms for CTIME=1ms or 20ms for CTIME=0.2ms. Allows delaying actual motor movement start (**RPOS**) for the specified delay, while motion profile generation (**APOS**) starts immediately.

4.2.4.1 MPTP Command

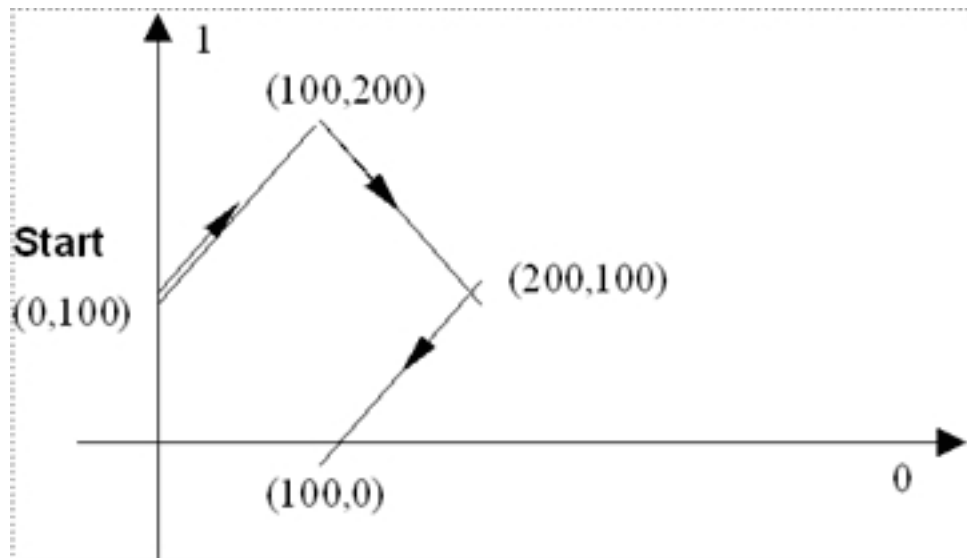
Use the **MPTP** command to specify axis and dwell time:

```
MPTP 0, 1000
```

This command creates a multi-point motion of the 0 axis and specifies a dwell time of 1000 msec at each point. If dwell is not required, **DWELL_TIME** may be omitted. The **MPTP** command itself does not specify any point, so the created motion starts only after the first point is specified. The points of motion are specified by the **POINT** or **MPOINT** commands that follow the **MPTP** command.

Consider the following program fragment:

The **MPTP** command creates the multipoint motion. However, the motion does not start until a point is defined. After the first **POINT** command the motion starts if all involved axes are idle (not involved in some previous motion), or waits until a motion that is in progress ends, and then starts. The four **POINT** commands specify the following sequence:



The controller performs sequential positioning to each point. The **ENDS** command informs the controller that no more points will be specified for the current motion. The motion cannot finish until the **ENDS** command executes. If the **ENDS** command is omitted, the motion will stop at the last point of the sequence and wait for the next point. No transition to the next motion in the queue will occur until the **ENDS** command executes.

Normally, multi-point motion starts with the first **POINT** command, and the next **POINT** command executes while the motion is already in progress. However, sometimes you may need to delay starting the motion until all points are defined. You use the **W** switch to prevent the motion from starting until a **GO** command executes. The motion created by the command:

```
MPTP/w 0, 1000
```

will not start until a **GO 0** command is issued.

Adding the **R** switch to the **MPTP** command causes all points to be treated as relative. The first point is relative to the position when the motion starts, the second point is relative to the first, and so on. The previous example, using the **MPTP/R** command, will look like this:

PTP (0,1), 0, 100	Create PTP motion to the first point (this serves as the reference point).
MPTP/r (0,1)	Create multipoint motion in group (0,1) with no dwell time.
POINT (0,1), 100, 100	Add point.
POINT (0,1), 100, -100	Add point.
POINT (0,1), -100, -100	Add point.
ENDS (0,1)	End the point sequence.

The **MPTP** command uses the default velocity **VEL** for positioning to each point. The **V** switch allows using a specific velocity for each positioning. The desired velocity must be specified in the **POINT** command after the point coordinates. The previous example is modified for using different velocities:

MPTP/v (0,1)	Create multipoint motion in group (0,1) with no dwell time.
POINT (0,1), 0, 100, 30000	Move to first point at velocity 30000.
POINT (0,1), 100, 200, 10000	Move to second point at velocity 10000.
POINT (0,1), 200, 100, 5000	Move to third point at velocity 5000.
POINT (0,1), 100, 0, 10000	Move to fourth point at velocity 10000.
ENDS (0,1)	End the point sequence.

Several switches can be appended to one command. For example, the command:

```
MPTP/rv 0, 1000
```

creates a multi-point motion with dwell time of 1000msec. The points will be specified by relative coordinates, and velocity will be specified for each point.

4.2.4.2 POINT Command

The **POINT** command adds a destination point to multi-point or arbitrary motion paths.

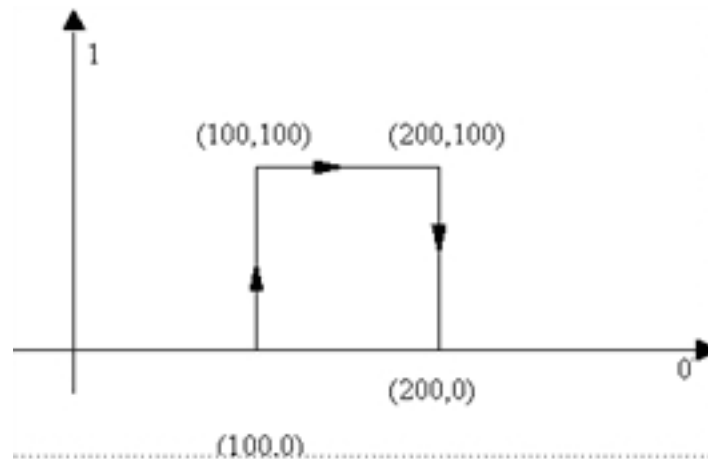
The **POINT** command does not require a specific value for all axes involved in a multi-point motion. If an axis is not specified in a **POINT** command, the axis remains idle and retains the previous value. Similarly, if a multi-point motion is created with the **V** switch, the velocity argument in a **POINT** command can be omitted, and the velocity of the previous segment will be used. If velocity is omitted for the first point, the default velocity **VEL** will be used.

Consider the following example:

MPTP/v (0,1)	Create multipoint motion in group (0,1) with no dwell time.
--------------	---

<code>POINT (0,1), 100, 0, 30000</code>	Move to first point at velocity 30000.
<code>POINT 1, 100</code>	Move to second point at velocity 30000.
<code>POINT 0, 200</code>	Move to third point at velocity 30000.
<code>POINT 1, 0, 10000</code>	Move to fourth point at velocity 10000.
<code>ENDS (0,1)</code>	End the point sequence.

The four **POINT** commands specify the following sequence:



4.2.4.3 MPOINT Command

The **MPOINT** command adds an array of points to either multi-point or arbitrary path motion.

The arguments of the **MPOINT** command are:

- > **AXIS_DESIGNATORS** – must specify the same axes in the same order as in the axes-specification of the corresponding mptp or path command.
- > **POINT_MATRIX** – name of declared two dimensional array.
- > **NUMBER_OF_POINTS** – specifies how many points are added to the motion by the command.

Before the **MPOINT** command can be executed, an array must be declared and filled with the point coordinates. Each row of the array contains coordinates of one point.



POINT_MATRIX must be a two-dimensional array, each column of which containing the specification of one point. The matrix must contain at least **NUMBER_OF_POINTS** columns. If the matrix contains more columns, the extra columns are ignored.

If the corresponding motion command is **MPOINT** without the **V** switch or **PATH** without the **T** switch (see [PATH Command](#)), a column of the matrix must contain the coordinates of the point. Therefore, if the **AXIS_DESIGNATORS** includes M axes, the matrix must contain exactly M rows.

If the corresponding motion command is **MPTP/V**, the matrix must contain M+1 rows. An additional value in each column specifies the desired velocity for transition from the previous to the current point. The velocity is specified in position units per second.

If the corresponding motion command is **PATH/T**, the matrix must contain M+1 rows. An additional value in each column specifies the time interval between the previous and the current point. The time is specified in milliseconds.

The following example illustrates how the **MPOINT** command can be used for adding points on-the-fly. The example also shows a simple approach to synchronization between the host computer that calculates the points and the controller that executes the motion.

The host computer calculates the desired points and transmits the coordinates via the Ethernet link. The motion involves 6 axes. Therefore, each point specification contains 6 real numbers.

Because transmitting each point separately would be very ineffective in the Ethernet, the host calculates the desired points in advance and transmits them in batches of 50 points. The controller executes the motion. As soon as the controller is ready to accept the next batch of points and the host is ready to send the batch, the next transmission occurs, and so on. The pace of the host and the controller may be very different. However, the host is assumed fast enough to calculate the next 50 points before the controller has moved through the previous 50 points.

The controller executes the following program:

<code>real Points(50)(6)</code>	Declare an array of 50 points for each of six axes. The host will write the coordinates to the array.
<code>int Sync</code>	Declare synchronization variable.
<code>MPTP (0,1,2,4,5)</code>	Create multi-point motion for axes 0, 1, 2, 3, 4 and 5.
<code>WHILE Sync >= 0</code>	Continue until the host writes negative number to Sync.
<code>TILL Sync</code>	Wait until the points are received. Once the host has filled the Points array, it writes the Sync variable with a number of points written to the Points array.
<code>IF Sync > 0</code>	Sync < 0 indicates that the host has finished the point generation. Add points from the Points matrix. The controller informs the host that the next batch is expected. At this moment the motion through the accepted points has not finished, but the controller is ready to receive more points.
<code>MPOINT (0,1,2,4,5), Points, Sync</code>	
<code>Sync = 0</code>	End if.
<code>END</code>	End while.
<code>END</code>	End mptp.
<code>ENDS (0,1,2,4,5)</code>	End program.
<code>STOP</code>	

The program running on the host in pseudo-code looks like this:

```
double HPoints(50)(6);
int N, HSync, NBuf;
HANDLE Com;
```



```
open communication, start program in buffer NBuf of the controller;
while (Continue)
calculate N (<= 50) points in array HPoints;
acsc_WriteReal(Com, NBuf, "Points", 0, N-1, 0, 6, HPoints, 1000) ;
acsc_WriteInteger(Com, NBuf, "Sync", -1, -1, -1, -1, &N, 0);
do
acsc_ReadInteger(Com, NBuf, "Sync", -1, -1, -1, -1, &HSync, 0);
while HSync;
reset Continue to zero if all points have been calculated;
end;
N = -1
acsc_WriteReal(Com, NBuf, "Points", 0, N-1, 0, 6, HPoints, 0) ;
```

Synchronization between the host and the controller is provided by the **SYNC** variable. When the host has finished transmitting the next batch of points to the controller, it writes to **SYNC** the number of points in the batch. The controller waits for non-zero **SYNC** and then adds the points to the motion. When the controller has added the points, it writes zero to **SYNC**, which signals to the host to transmit the next batch of points.

When the host comes to the end, it writes -1 to **SYNC** to indicate the end of the motion.

4.2.5 The **GRTIME** Variable

The ACSPL+ **GRTIME** variable is an 8 element real, read-only array. **GRTIME** displays the time in milliseconds remaining until the end of motion. The **GRTIME** value is valid in **PTP** and **TRACK** motion. For other motion types, the **GRTIME** value is not valid. **GRTIME** behavior is as follows:

- > Each element of **GRTIME** refers to one axis. In a multi-axis motion, only the **GRTIME** element of the leading axis is updated; the elements of other involved axes are zero.
- > If an axis is idle, its **GRTIME** element is zero.
- > At the beginning of motion, **GRTIME** is not valid and is assigned a large value.
- > In Firmware versions previous to Version 4.50, **GRTIME** was invalid during motion phases 1 and 2. In Firmware Version 4.50, the invalid period is shorter, but its exact duration is not guaranteed. Normally, the period of invalid **GRTIME** is from one to five milliseconds. In the worst case, the period may span phases 1 and 2.

This illustrates the **GRTIME** behavior in **PTP** or **TRACK** motion.

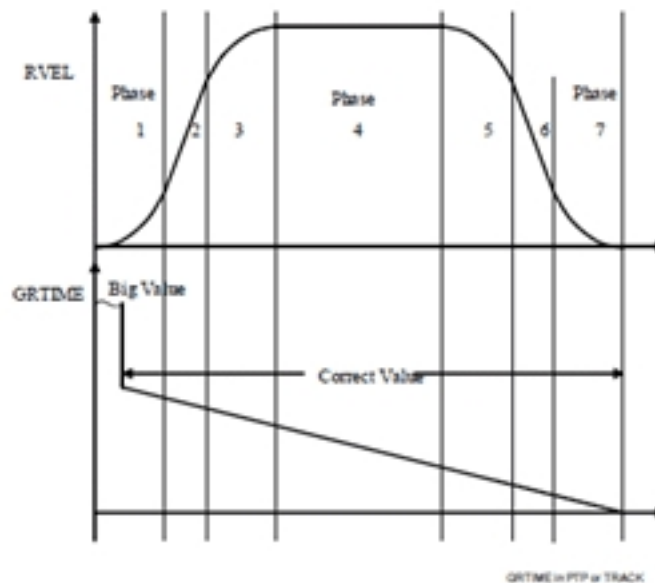


Figure 4-1. GRTIME Behavior in PTP or TRACK Motion

4.2.6 Modulo Axis

Bit 29 (**#MODULO**) of the **MFLAGS** variable specifies modulo axis. If the bit is one, the axis feedback changes between the specified minimal and maximal positions. The options are:

- > The ACSPL+ **SLPMIN** variable specifies the lower limit of modulo axis.
- > The ACSPL+ **SLPMAX** variable specifies the upper limit of modulo axis.

The reference position **RPOS** of the modulo axis changes in the range from **SLPMIN** to **SLPMAX** inclusively.

Physically, the motion of the modulo axis is not limited, but each time when the **RPOS** comes out from range **SLPMIN..SLPMAX**, the controller brings **RPOS** into the range by changing the internal offset **EOFFS**. Note the following conditions:

- > If the axis goes down and crosses the **SLPMIN** value, the controller adds value **SLPMAX-SLPMIN** to **EOFFS**. Assume the axis comes down to value **SLPMIN-D**. Correcting **EOFFS**, the controller brings **RPOS** to value **SLPMAX-D**.
- > If the axis goes up and crosses **SLPMAX** value, the controller subtracts value **SLPMAX-SLPMIN** from **EOFFS**. Assume the axis comes up to value **SLPMAX+D**. Correcting **EOFFS**, the controller brings **RPOS** to value **SLPMIN+D**.
- > Changing **EOFFS** immediately affects also feedback position **FPOS**. However, there is a slight difference between **FPOS** and **RPOS** behavior. **RPOS** always remains within the **SLPMIN..SLPMAX** interval. As **FPOS** differs from **RPOS** by position error, the corresponding **FPOS** may occur beyond the interval.

In the case of a default connection, the modulo operation also affects the **APOS** value (**APOS=RPOS**).



SLPMIN and **SLPMAX** variables can be changed only when the motor is disabled.

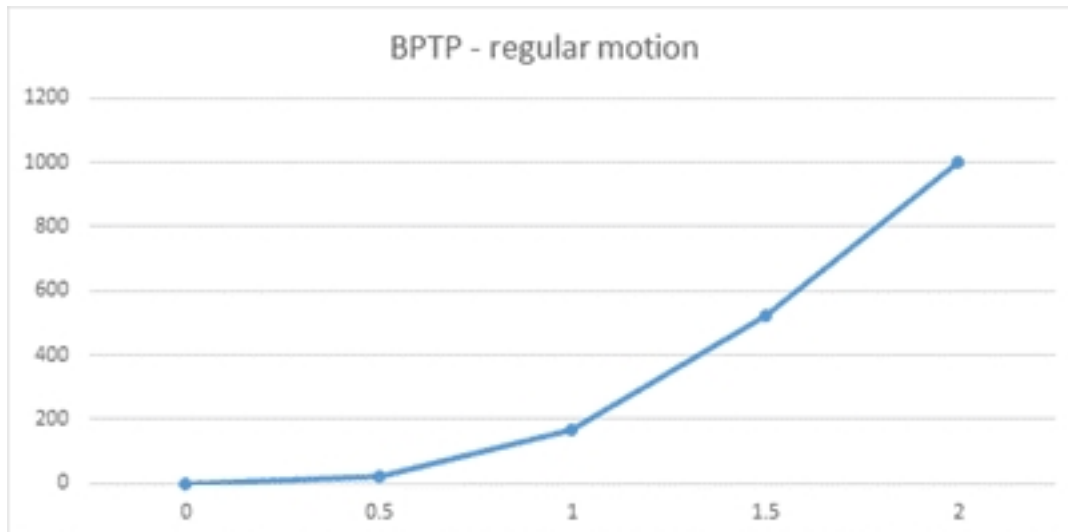
4.2.7 Motion Boost

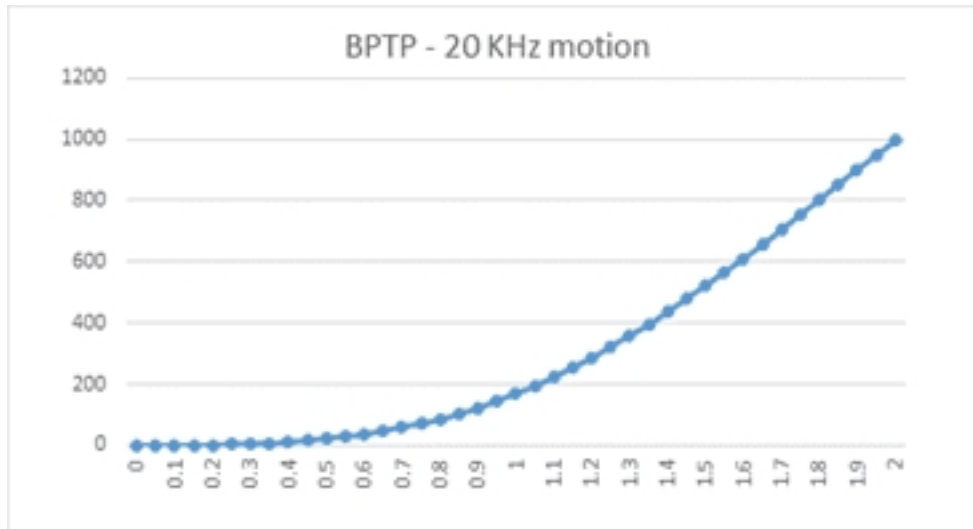
Motion Boost motion type reduces settling time for short moves with high accelerations. The motion profile has high level smoothness attributes even for Jerk values, with minimal energy injection, optimized for the best settling time.

4.2.7.1 20 kHz Motion

During execution, a discrete representation of a motion path is generated. Before the introduction of 20 kHz control, the controller would generate a new discrete point every controller cycle. Thus, the resolution of the path was determined by the controller cycle time (**CTIME**).

The new 20 KHZ motion generation feature allows the user to specify that a specific motion should be generated in a 20 KHZ resolution, regardless of the actual controller cycle. The feature is an extension of available motion types, and is currently only available for MotionBoost™ type motion.





When using MotionBoost™ on MC4U or SPiiPlusSA devices a custom DSP is required.

GMTYPE

A new **GMTYPE** is created for 20 KHz motions. GMTYPE for BPTP/2 is 43. $43 = 11 \text{ (BPTP)} + 32$.

GMTYPE over 32 will represent 20KHz motion.

Compatibility

The feature supports dynamic error compensation. The value used for compensation will be the one calculated once per controller cycle.

The feature uses the Reverse data collection mechanism to communicate with the DSP, which prevents it from working with other features that use the mechanism: **SPINJECT**, **SPRT**, Fast PEG loading.

The new feature will not support the following commands: **CONNECT**, **BREAK** and **HALT**.

DSP interface

DSP with ServoBoost is required.

To determine if the desired DSP supports the feature, a new variable is queried in the DSP- "desired_position_20i" of type int.

Data format

The data (reference positions) will be sent in the format of fixed point 32.32 (total 8 Byte)

The data written to the memory as two int variables.

The position data is sent to the DSP in the reverse data collection packet in the following position:

Reverse Data Collection Packet Offset

	CTIME = 0.5	CTIME = 0.25	CTIME = 0.2
DSP First Axis	0	0	0
DSP Second Axis	80	40	32

Data collection

The following variables may be collected for this motion type:

GPATH, GVEL, GACC, GJERK, APOS, PPOS, PPOSCOMP.

The controller calculates 20 reference positions per controller cycle, and passes the arrays **PPOS** to the DSP, which contains the reference position for use at 20 kHz. **PPOS** is used instead of **RPOS** to set the reference position. When using error compensation **PPOSCOMP** specifies the compensated reference position, equivalent to **RPOSCOMP**.

The values will be collected automatically, when a specified variable is collected in a frequency that is higher than the controller's.

The new item has two references, one to the standard variable (e.g. **APOS[6]**) and one to the custom 20 KHz array (e.g. **APOS_20KHZ[6]**).

When a 20 KHz motion generation is active for the axis, the custom data are collected and stored in **APOS_20KHZ**. When a standard motion generator is active or there is no motion at all, data is collected and stored in the standard variable **APOS** is collected instead. In this case there are no valid data in **APOS_20KHZ**.

4.2.7.2 BPTP

Description

BPTP defines a motion profile using the MotionBoost Feature.

Syntax

BPTP[/switch] axis_list, destination_point, [value of Tf, value of Vf, motor_motion_delay]

Switches

Arguments	Comments
None	Travel time will be calculated by the MPU to achieve a the minimum possible time
/t	Minimum travel time in seconds, The calculated travel time will be at least the specified value. Incompatible with the /d switch.
/d	Travel Time – specifies the exact travel time for the motion in seconds. All other considerations are ignored, which could cause a safety fault during motion execution. Incompatible with the /t switch.

Arguments	Comments
/f	User will enter final, nonzero velocity. In single axis motion the sign of the final velocity parameter has no effect, only the absolute value is considered.
/e	Wait for motion termination before executing next command.
/r	Relative motion
/v	Use velocity parameter instead of default velocity parameters.
/w	Create the motion, but to not start until the GO command is issued.
/z	Interpret entered coordinates according to the Local Coordinate System.
/m	Use the motion profile values of the axis group as a whole, rather than those of the leading axis, without exceeding any of the defined axes motion VEL , ACC , DEC , JERK values. Not compatible with /2 switch. Range is 0-25 ms.
/q	Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The maximum delay is 100 controller cycles: 100ms for CTIME=1ms or 20ms for CTIME=0.2ms. Allows delaying actual motor movement start (RPOS) for the specified delay, while motion profile generation (APOS) starts immediately.
/2	Use 20 kHz motion mode



Use of the /d switch to specify minimum travel time overrides all other parameters which might limit velocity and requires careful attention to safety considerations.



BPTP/2 is limited to at most 2 axes in a single function call.



The **BPTP/2** command is limited to at most 2 axes per Servo Processor and at most 4 axes per system.

Arguments

axis_list

a	Axis a will perform the motion
(axes)	Axes will perform a synchronized motion to the destination point along to a straight line connecting the start to the destination point. Axes may be any set of distinct numbers in the range 0 to 63.

destination-point

(af,bf,cf)	af is a destination point for axis a bf is a destination point for axis b cf is a destination point for axis c
------------	--

Value of Tf

Tf	Desired travel time
----	---------------------

Value of Vf

Vf	Desired final velocity
----	------------------------

Motion Delay

motor_ movement_delay	(Optional, used only with /q switch) Delay, in milliseconds, before motor motion actually starts.
--	---

GPHASE

Two options are available.

- > Four phases (For motion in positive direction; for motion in negative direction reverse the inequality signs)
 1. Acceleration buildup
 - > RJERK>0, RACC>0
 2. Acceleration finishing
 - > RJERK<0, RACC>0
 3. Deceleration buildup
 - > RJERK<0, RACC<0
 4. Deceleration finishing
 - > RJERK>0, RACC<0

Comments

This command is supported in ADK versions 2.70 and higher.

Examples

```
BPTP 0, 100
```

A simple example that moves axis 0 to position 100.

If the axis is moving when the command is issued, the controller creates the motion and inserts it into the axis motion queue. The motion waits in the queue until all motions before it finish, and only then starts.

```
BPTP/ed (0,1), 100, 200, 0.4
```

Move axes 0,1 to position 100,200.

Do not execute the next command in the program until the motion is done.

Execute the motion in exactly 0.4 seconds.

```
// execute a BPTP motion with 20 KHz resolution
// move axes 0,1 to absolute coordinates 1,1

BPTP/2 (0,1),1,1
```

GMTYPE

The motion GMTYPE is 11, or 43 if using the /2 suffix for 20 kHz control.

4.2.7.3 BPTPCalc

Description

The **BPTPCALC** function calculates and allows the user to set the motion variables according to a desired motion time. When the travel time and distance are known in advance, the **BPTPCALC** should be used to generate new **VEL**, **ACC** and **JERK** values.

Syntax

```
BPTPCALC real Motion_duration, real Distance, int index
```

Arguments

Motion_duration	The desired motion time in seconds. The time will be rounded up to a whole number of controller cycles.
Distance	The travel distance in user units
Index	1 - Velocity 2 - Acceleration 3 - Jerk

Comments

This command is supported in ADK versions 2.70 and higher.

Example

```
! Calculate the motion parameter to execute a 10 unit BPTP motion in 3 ms
VEL(0) = bptpcalc(0.003, abs(10), 1)
ACC(0) = bptpcalc(0.003, abs(10), 2)
```



```
JERK(0) = bptpcalc(0.003, abs(10), 3)

enable(0)

BPTP/R(0), 10 ! Move axis 0, 10 units, in 3 ms

STOP
```

4.3 JOG Command

The **JOG** command defines Jog motion, which is a motion with constant velocity and no defined end point. The motion continues until the next motion command stops it, or the motion fails because of limit switch activation or other condition.

Syntax:

JOG [/switch] axis_designator [,direction] [,velocity]

Where **SWITCH** can be one or a combination of:

w	Create the motion, but do not start until the GO command has been issued.
v	Use the velocity specified in the command instead of the default velocity.

and **DIRECTION** is indicated by:

+	Motion is in the positive direction.
-	Motion is in the negative direction.

The simplest **JOG** command is:

```
JOG 0
```

This command creates a jog motion of the 0 axis in positive direction using the default velocity **VEL (0)**.

Motion direction may be specified in the command:

```
JOG 0, -
```

This command creates a jog motion of the 0 axis in negative direction using the default velocity **VEL (0)**.

The command:

```
JOG 0, +
```

is the same as:

```
JOG 0
```

The **V** switch allows a specific velocity to be used instead of the default velocity **VEL**. The command:

JOG/V 0, 30000

ignores the default velocity and creates a jog motion with a velocity of 30000.

As with other types of motion, jog motion may be terminated by the **HALT**, **KILL**, or **BREAK** commands. Unlike any other motion, jog motion also terminates when the next motion command for the same axis executes. For example, the following program fragment:

```
JOG 0, +
WAIT 500
JOG 0, -
```

provides jogging in the positive direction for 500msec and then switches to the negative direction. The controller automatically ensures a smooth transition from motion to motion.

Jogging can also occur in an axis group. For example, the following program fragment

```
JOG (0,1,4), -++
```

creates jogging in three axes: 0 in the negative direction, and 1 and 4 in the positive direction. The motion uses the default velocity **VEL(0)** as a vector velocity for the three-axis motion.

4.4 TRACK Command

TRACK initiates track motion. In **TRACK** motion, a new point-to-point move is generated to a new target position whenever the variable **TPOS** (target position) changes. **TRACK** does not terminate automatically. If **TPOS** is not assigned a new value, motion stops at the last **TPOS** and waits. If a new **TPOS** value is assigned, motion continues.

TRACK terminates due to:

- > Any subsequent motion command (except **TRACK**) for the motion axis involved in a track motion, **except the case when the next motion is a group motion**.
- > Any fault activation that disables the drive or kills the motion.
- > User termination by **HALT**, **KILL**, or **DISABLE**.

Syntax

TRACK [/switch] axis, [motor_motion_delay]

Arguments

axis	Axis designation, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.
motor_motion_delay	(Optional, used only with /q switch) Delay, in milliseconds, before motor motion actually starts.

Switch

/switch can be:

/w	Create the motion, but do not start until GO .
/q	Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The maximum delay is 100 controller cycles: 100ms for CTIME=1ms or 20ms for CTIME=0.2ms. Allows delaying actual motor movement start (RPOS) for the specified delay, while motion profile generation (APOS) starts immediately.

The following command creates tracking motion of the 0 axis:

```
TRACK 0          Create track motion of 0 axis
```

If the axis is idle, the track motion is activated immediately. If the axis is moving, the controller creates the motion and inserts it into the axis motion queue. The motion waits in the queue until all previous motions in the queue are executed, and then starts.

When the track motion starts, the controller copies the current value of the reference position (**RPOS**) element to target position (**TPOS**) element. For example, when the command is executed, **RPOS(0)** is copied to **TPOS(0)**. No change of **RPOS** and no physical motion occur at this time.

Afterwards, the axis waits until the **TPOS** element is assigned a different new value. As soon as the program executes:

```
TPOS(0) = NewTarget    Assign a value to the TPOS element
```

the controller generates a PTP motion to the point designated by the value of the **NEWTARGET** user variable. After the 0 axis reaches **NEWTARGET**, the axis waits for the next change of **TPOS**. The next assignment to **TPOS(0)** automatically activates the next PTP motion and so on. Therefore, track motion is executed as a sequence of PTP motions.

The motion state bits **AST.#MOVE**, **AST.#ACC**, **MST.#MOVE**, and **MST.#ACC** reflect the state of each PTP motion in the track sequence exactly as they do for ptp motion. Between PTP motions, while the axis waits for the next **TPOS** assignment, the motion bits are zero (with the exception of the **MST.#MOVE** bit, which can be 1 if the position error exceeds the **TARGRAD** limit).

The following ACSPL+ program fragment defines sequential positioning to points 1000, 2000, 10000, 11000:

```
TRACK 2          Create track motion of axis 2
TPOS(2) = 1000    Move to point 1000
TILL ^AST(2).#MOVE Wait till the motion ends
TPOS2 = 2000      Move to point 2000
TILL ^AST(2).#MOVE Wait till the motion ends
TPOS2 = 10000     Move to point 10000
TILL ^AST(2).#MOVE Wait till the motion ends
TPOS2 = 11000     Move to point 11000
TILL ^AST(2).#MOVE Wait till the motion ends
HALT 2           Terminate track motion
```

The result is similar to the following fragment:

```
PTP 2,1000      Move to point 1000
PTP 2,2000      Move to point 2000
PTP 2,10000     Move to point 10000
PTP 2,11000     Move to point 11000
```

While the code with the **PTP** commands looks shorter and simpler, there are applications where track motion is preferable to point-to-point motion.

Track motion is not terminated automatically. If **TPOS** is not changed, the axis track motion remains at the last target point until **TPOS** is assigned a new value, and then motion continues.

TRACK terminates due to:

- > Any subsequent motion command (except **TRACK**) for the motion axis involved in a track motion, **except the case when the next motion is a group motion**.
- > Any fault activation that disables the drive or kills the motion.
- > User termination by **HALT**, **KILL**, or **DISABLE** command.

The motion profile while in Track mode, like in a standard PTP motion, is defined by the ACSPL+ variables **VEL**, **ACC**, **DEC** and **JERK**. The track command accepts the values of these variables at the beginning of each component PTP motion within the track motion. Therefore, if an application assigns a new value to **VEL**, **ACC**, **DEC** or **JERK**, while track mode is in effect, then the new value will be used the next time that the application initiates a motion (by assigning a new value to **TPOS**).

The following ACSPL+ program fragment sets a specific velocity for each PTP motion:

```
TRACK 1          Create track motion of axis 1
VEL(1) = 20000   Set motion velocity 20000 units/sec
TPOS(1) = 1000   Move to point 1000
TILL ^AST(1).#MOVE Wait till the motion ends
VEL1 = 5000      Set motion velocity 5000 units/sec
TPOS1 = 2000     Move to point 2000
TILL ^AST(1).#MOVE Wait till the motion ends
VEL1 = 10000     Set motion velocity 10000 units/sec
TPOS1 = 110000   Move to point 11000
TILL ^AST(1).#MOVE Wait till the motion ends
HALT 1           Terminate tracking motion
```

In the example above the application updates **TPOS** only after the previous PTP motion ends.

In the following example the application updates **TPOS** while the motion is still in progress:

```
TRACK 0          Create tracking motion of X axis
TPOS(0) = 2000   Move to point 2000
TILL GPHASE(0) >= 6 Wait till the motion comes to phase 6
                    (deceleration to final point)
TPOS(0) = 2500   Correct the final point
TILL ^AST(0).#MOVE Wait till the motion ends
HALT 0           Terminate tracking motion
```

In this case, the controller does not execute two separate motions. As soon as **TPOS** is changed to 2500 (before the controller reaches 2000), the controller changes the move on-the-fly to the new target position of 2500. The on-the-fly change is done smoothly, similar to end-point correction on-the-fly.

The same result is provided by the following fragment:

```
PTP 0, 2000           Move to point 2000
TILL GPHASE(0) >= 6   Wait till the motion comes to phase 6 (deceleration
to final point)
BREAK 0              Terminate the current motion and provide smooth
transition to the next motion
PTP 0,2500           Move to point 2500
```

The **TRACK** command may also be used for programming multi-axes motion, for example, the command

```
TRACK 0; TRACK 2; TRACK 3
```

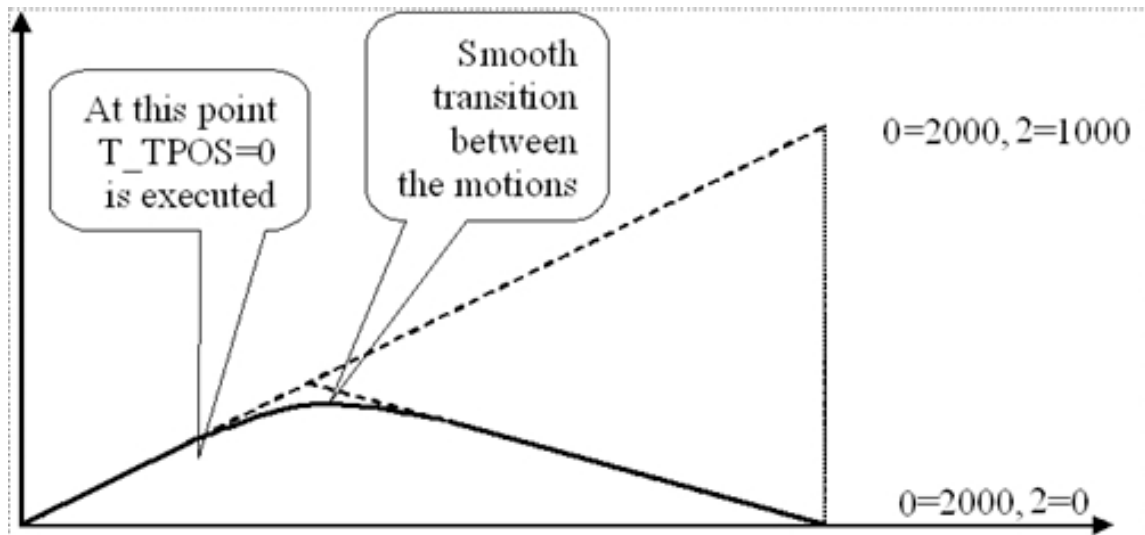
creates track motion of 0, 2 and 3 axes. The multi-axis track motion is executed as a sequence of PTP motions. A new PTP motion starts each time when one or more elements of **TPOS** that correspond to the involved axes are changed. Consider the following example:

```
TRACK 0;TRACK 2;TRACK 3           Create track motion of the 0, 2 and 3 axes
TPOS(0)=0; TPOS(2)=0; TPOS(3)=0   Move to point 0=0, 2=0, 3=0
TILL ^AST(0).#MOVE                Wait till the motion ends
TPOS(2)=1000                      Move to point 0=0, 2=1000, 3=0
TILL ^AST(0).#MOVE                Wait until the 0 motion ends
TPOS(0)=100;TPOS(3)=200           Move to point 0=100, 2=1000, 3=200
HALT (0,2,3)                     Terminate track motion
```

In the following example **TPOS** is updated while the previous motion is still in progress:

```
TRACK 0; TRACK 3                 Create track motion of the 0 and 3 axes
TPOS(0)=2000; TPOS(2)=1000       Move to point 0=2000, 2=1000
TILL GPHASE(0) = 4               Wait until the motion reaches constant velocity (phase 4)
TPOS(3)=0                       Set a new final point 0=2000, 3=0
TILL ^AST(0).#MOVE               Wait until the motion ends
HALT 0                           Terminate tracking motion
```

In the above case, the controller does not execute two separate motions. As soon as **TPOS** is updated, the controller changes on-the-fly from PTP motion towards 0=2000, 3=1000 to PTP motion towards 0=2000, 3=1000. The transition from the first motion to the second is done smoothly. While each PTP motion follows a straight trajectory, the transition between the motion is not straight, as shown in the following diagram:



4.5 Segmented Motion

Segmented Motion moves axes along a continuous path. The path is defined as a sequence of linear and arc segments on the plane. Although segmented motion follows a flat path, it may involve any number of axes because the motion plane can be connected to the axes at any projection transformation.

4.5.1 Understanding Slaved Segmented Motion

Motion generation for segmented motion may be considered as a two-stage process. In the first stage the controller generates a smooth motion diagram as a function of time:

$$\mathbf{S} = \mathbf{F}(\mathbf{T})$$

where $\mathbf{S}$ is a distance along the segmented path, $\mathbf{T}$ stands for time, and $\mathbf{F}$ is a function independent of the specified segments.

In the second stage the controller separates the $\mathbf{S}$ path into the involved axes:

$$\mathbf{0} = \mathbf{F}(\mathbf{S})$$

$$\mathbf{1} = \mathbf{F}_Y(\mathbf{S})$$

The second stage supplies the current values of the involved axes. The functions $\mathbf{F}_X$, $\mathbf{F}_Y$ depend only on the specified segments. Only the second stage builds the shape of the path in the XY plane. The first stage provides the motion progress along the path. If the function $\mathbf{F}$ of the first stage is modified, this affects the motion velocity and time, but does not alter the final shape of the path.

The including the $\mathbf{S}$ or $\mathbf{P}$ switch with the **MSEG** command affects the first stage of the motion generation process by causing the distance $\mathbf{S}$ to follow the value **MPOS** (Axis Master) of the leading axis in the group. For position lock (**P** switch), following is strict:

$$\mathbf{S} = \mathbf{MPOS}$$

For velocity lock (**S** switch), following allows a constant offset:

$$\mathbf{S} = \mathbf{MPOS} + \mathbf{C}$$

In both cases the second stage of the motion generation remains unchanged and depends only on the specified segment sequence.

Segment commands specify a path on the plane, and the **MPOS** value of the leading axis defines motion progress along the path.

Formulas that calculate the **MPOS** value must be defined before using the master command.

4.5.2 MSEG, LINE, ARC1, ARC2, STOPPER Commands

These commands are employed to define segment motion.

Syntax:

MSEG [/switch] axis_group, initial_start_point [,initial_start_point, initial_start_point][,projection matrix_designator]

LINE [/switch] axis_group, final_point [,final_point, final_point]

ARC1 [/switch] axis_group, center_point, final_point, rotation_direction [,velocity]

ARC2 [/switch] axis_group, center_point, rotation_angle, rotation_direction [,velocity]

STOPPER axis_group

ENDS axis_group

Where **switch** can be one or a combination of:

w	Create the motion, but do not start until the GO command
v	Use the velocity specified for each segment instead of the default velocity
c	Use the segment sequence as a cyclic array: after the last segment return to the first segment and so on.
s	Slaved motion - the motion advances in accordance to the master value of the leading axis (velocity lock).
p	Position lock - slaved motion, strictly conforming to the master value.
e	Extrapolated - if a master value travels beyond the specified path, the last or the first segment is extrapolated.
t	Stalled - if a master value travels beyond the specified path, the motion stalls at the last or first point.

The **E** and **T** switches are relevant only for slaved motion and must be used with **S** or **P** switch. For discussion of slaved motion see [SLAVE Command](#).

Segmented motion can be executed in an axis group with any number of controller axes.

The **MSEG** command specifies axis group and the initial starting point:

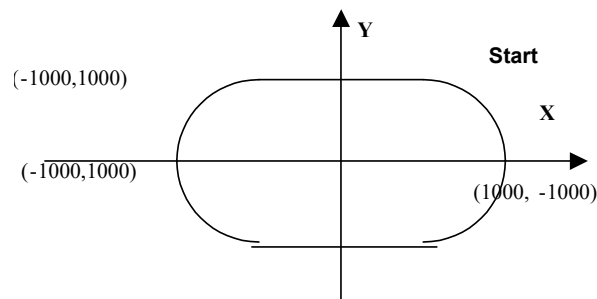
```
MSEG (0,1), 1000, 1000
```

This command creates a segmented motion of the X axis group and specifies the coordinates of initial point on the plane. The **MSEG** command itself does not specify any segment, so the created motion does not start immediately. A **LINE** or **ARC** command must follow the **MSEG** command to specify the segment sequence.

Consider the following program fragment:

<code>MSEG (0,1),1000,1000</code>	Create segmented motion in group (0,1) , coordinates of the initial point are (1000,1000).
<code>ARC1 (0,1), 1000,0,1000,-1000,-</code>	Add arc segment with center (1000,0), final point (1000,-1000), clockwise rotation.
<code>LINE (0,1),-1000,-1000</code>	Add line segment with final point (-1000,-1000).
<code>ARC2 (0,1),-1000,0,-3.141529</code>	Add arc segment with center (-1000,0) and rotation angle $-\pi$ radians.
<code>LINE (0,1),1000,1000,50000</code>	Add line segment with final point (1000,1000).
<code>ENDS (0,1)</code>	End the segment sequence

The **MSEG** command creates the segmented motion. The motion does not start, because no segment is defined yet. After the first **ARC1** command the motion starts if the axis group is idle (not involved in some previous motion). If the group is not idle, motion will start when the previous motion stops. The four segment commands specify the following path (where X is the direction of the 0 axis and Y is the direction of the 1 axis):



ARC1 and **ARC2** differ only by the required arguments. **ARC1** requires the coordinates of the center point, final point, and the direction of rotation. **ARC2** requires the coordinates of the center point and the rotation angle (in radians). Each command produces the same result, so selection of either **ARC1** or **ARC2** depends on the available data. If you know the coordinates of the center point, coordinates of the final point and the direction of rotation, **ARC1** is preferable. If you know the coordinates of the center point and rotation angle, **ARC2** is preferable.

The **ROTATION_DIRECTION** argument can be:

- > + (plus) – for counter clockwise
- > - (minus) – for clockwise rotation

The entire sequence of segmented motion must be terminated with an **ENDS** command. The **ENDS** command informs the controller that no more segments will be specified for the specified motion. The motion cannot finish until the **ENDS** command executes. If the **ENDS** command is omitted, the motion will stop in the last point of the sequence and wait for the next point. No transition to the next motion in the queue will occur until the **ENDS** command is executed.

Segmented motion usually starts with the first segment command (**LINE**, **ARC1**, or **ARC2**). The next segment command therefore executes while the motion is already in progress. This is generally not a problem, because the program execution rate is higher than typical motions time. However, sometimes you may need to delay starting the motion until all points are defined. In this case you append the **W** switch to the command to prevent the motion from starting until the **GO** command is issued. The motion, created by the command:

```
MSEG/w (0,1),1000,1000
```

will not start until the **GO (0,1)** command is issued.

The **R** switch is not allowed with segmented motion. All coordinates in the line, **ARC1** and **ARC2** commands are absolute in the plane. However, the whole path is relative to the point where the axes are located when the segmented motion starts. In other words, all coordinates are absolute in the plane, but the plane is relative to the starting point. The initial point, specified in the **MSEG** command, is also absolute in the plane. Therefore, the initial point does not cause any motion to that point, but only supplies starting coordinates for the first segment. The motion program should provide a motion to the desired initial point before executing the **MSEG** command. The following fragment illustrates a typical activation of the segmented motion:

```
PTP (0,1), 0, 100
```

This command causes a physical motion to the point (0,100) that will be an absolute starting point for the following segmented motion.

```
MSEG (0,1), 100, 100
```

Defines the starting coordinates for the first segment, which are absolute in the plane. The whole plane is located in such a way that the starting point in the plane (100,100) coincide with the present position of the motors (0,100).

The **MSEG** command uses the default velocity **VEL** for each segment. The **V** switch overrides the default velocity and allows you to define a specific velocity for each segment. The desired velocity must be specified in the **LINE**, **ARC1** and **ARC2** segment commands after all other arguments. If the velocity argument is omitted in a segment command, the velocity from the previous segment is used. If the velocity argument is omitted in the first segment, the default **VEL** is used. The previous example is modified for using individual velocities:

MSEG/v (0,1),1000,1000	Create segmented motion in group (0,1), coordinates of the initial point are (1000,1000).
ARC1 (0,1), 1000,0,1000,-1000,-,30000	Add arc segment with center (1000,0), final point (1000,-1000), clockwise rotation, vector velocity 30000.
LINE (0,1),-1000,-1000	Add line segment with final point (-1000,-1000). Vector velocity is not specified, previous value 30000 will be used.
ARC2 (0,1),-1000,0,-3.141529,10000	Add arc segment with center

<pre>LINE (0,1),1000,1000,5000 ENDS (0,1)</pre>	<pre>(-1000,0), rotation angle of $-\pi$, vector velocity 10000. Add line segment with final point (1000,1000), vector velocity 50000. End the segment sequence</pre>
---	--

Several switches can be attached to one command. For example, the command

```
MSEG/vw (0,1), 1000, 1000
```

creates a segmented motion with individual velocity for each segment. The motion does not start until the **GO (0,1)** command is issued.

4.5.3 PROJECTION Command

The **PROJECTION** command is an expansion to the **MSEG ...ENDS** set of commands that allows the controller to perform a three dimensional segmented motion such as creating arcs and lines on a user-defined plane. The method for this 3D segmented motion is to set a transformation matrix that defines a new plane for the segmented motion.

Syntax:

PROJECTION *axes, matrix_table*

Arguments

axes	List of axes.
matrix_table	Coordinates defining the new plane.

As mentioned above, all coordinates, specified in the segment commands, are absolute in the working plane. Projection is a matrix that connects the plane coordinates and the axis values as specified in the mseg command. If the axis group contains two axes, and no **PROJECTION** command is specified, the controller provides a default projection that corresponds to a 2x2 matrix:

1	0
0	1

The matrix directly connects the first coordinate of the working plane to the first axis of the axis group and the second coordinate to the second axis.

The matrix can also define rotation and scaling. The full transform also includes an implicit offset. The controller calculates the offset automatically in such a way that the initial coordinates specified in the mseg command match the desired axis values at the moment when the motion starts. The offset provides the full path to be relative to the starting point.

If an axis group contains N axes, the controller extends the default matrix to N lines. The additional lines are filled by zeros:

1	0
0	1

0	0
...	...
0	0

The matrix connects only the first two axes to the plane coordinates. Therefore the segmented motion will involve only the first two axes in the group.

If N = 1, the **MSEG** command applies to a single axis, and the matrix contains the first line only:

1	0
---	---

In this case the axis will follow the first coordinate of the working plane.

You can replace the default matrix with the **PROJECTION** command.

Example:

```

real M(3) (2)                                !Define Matrix
M(0) (0)=1;M(0) (1)=0
M(1) (0)=0;M(1) (1)=1
M(2) (0)=0;M(2) (1)=2.74                     !Set the transformation
                                              !matrix values.
VEL(0)=1000;ACC(0)=10000;DEC(0)=10000        !Axis motion parameters
ENABLE (0,4,5)                               !Required command.
GROUP (0,4,5)                                !Required command.
SET FPOS(0)=0;SET FPOS(4)=0;SET FPOS(5)=0    !Set axes' FPOS=0
MSEG (0,4),0,0                               !Define original plane.
PROJECTION (0,4,5),M                         !PROJECTION of the 0, 4 and 5
                                              !axes by matrix M
ARC2 (0,4),750,0,6,24                        !ARC2 performed on new plane.
ENDS (0,4)                                   !Concludes MSEG.
STOP                                          !End Program

```

If the group contains N axis, the matrix in the projection command must be of size Nx2.

4.5.4 Arguments as Expression

Any argument in the **LINE**, **ARC1**, **ARC2** commands can be specified by expression.

Using an expression instead of axis specification allows the involved axes to be calculated in the execution time as opposed to the programming time (axis-independent programming). The calculation must result in an integer between 0 and 7, corresponding to the eight axes.

Expression in place of coordinate allows calculating the segments on the fly. Consider the following program fragment:

```

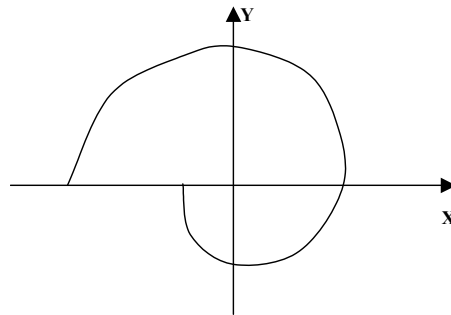
real P, K, S
P = 3.14159; K = 100 / P; S = P / 1000
PTP (0,1), -100, 0

```

Declare the real variables P, K and S.
Calculate P K and S.
Perform a point-to-point motion in the XY plane (X being the direction of the 0 axis and Y being the direction of the 2 axis) to the point: (-100,0).

MSEG (0,1), -100, 0	Create a segmented motion.
LOOP 2000	Perform the loop 2000 times.
P = P + S	
LINE (0,1), P*K*cos(P), P*K*sin(P)	Execute a linear segment based on a calculation.
END	End loop.
ENDS (0,1)	End segment motion.

The program executes the line command 2000 times. The line segments build up a curve close to the Archimedean spiral:



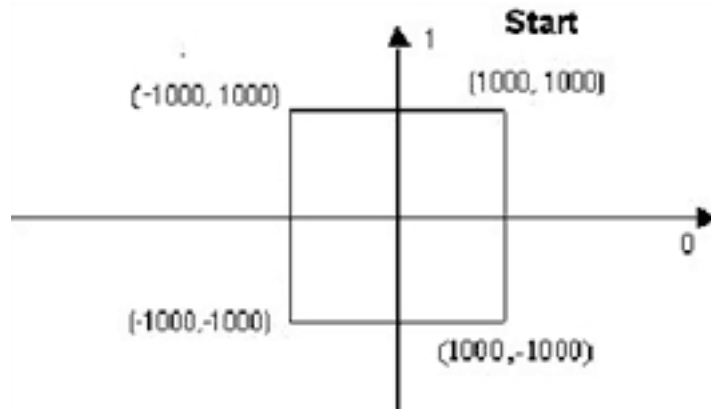
4.5.5 STOPPER Command

The controller builds the motion so that the vector velocity follows a smooth velocity diagram. If all segments are connected smoothly, axis velocity is also smooth. However, if you have defined a path with an inflection point, axis velocity jumps at this point. The jump might cause a motion failure due to the acceleration limit. Even if a failure does not occur, the abrupt change in velocity impairs accuracy and may be harmful to the machine.

The **STOPPER** command is used to avoid velocity jump in the inflection points. If a **STOPPER** command is specified between two segments, the controller provides smooth deceleration to zero before the stopper and smooth acceleration to specified velocity after the stopper. Consider the following program fragment:

PTP (0,1), 1000, 1000	Go to initial point.
MSEG (0,1), 1000, 1000	Create a segmented motion.
LINE (0,1), 1000, -1000	Execute linear segment.
STOPPER (0,1)	Slow down to zero.
LINE (0,1), -1000, -1000	Execute linear segment.
STOPPER (0,1)	Slow down to zero.
LINE (0,1), -1000, 1000	Execute linear segment.
STOPPER (0,1)	Slow down to zero.
LINE (0,1), 1000, 1000	Execute linear segment.
ENDS (0,1)	End the segment sequence.

The program provides a rectangular path without velocity jumps:



4.5.6 Cyclic Motion

The **C** switch provides cyclic execution of segmented motion. The **MSEG/C** command creates a motion that after executing the last segment, the motion begins again continues with the same sequence. The final point of the last segment is therefore the starting point of the first segment.



Cyclic segmented motion does not automatically finish. You must use one of the commands **HALT**, **KILL**, or **BREAK** to stop cyclic motion

The following is an example of the coding for cyclic motion:

```
ENABLE (0,1)
MSEG/c (0,1),1000,1000          !Create segmented motion in group X, coordinates of
                                !the initial point are (1000,1000)ARC1
(0,1), 1000,0,1000,-1000,-      !Add arc segment with center (1000,0), and
                                !final point (1000,-1000), clockwise rotation
LINE (0,1),-1000,-1000          !Add line segment with final point (-1000,- 1000)

ARC2 (0,1),-1000,0,-3.141529     !Add arc segment with center (-1000,0) and
                                !a rotation angle of  $-\pi$ 
LINE (0,1),1000,1000            !Add line segment with final point (1000,1000)
ENDS (0,1)                      !End the segment sequence
```

4.5.7 Slaved Motion at Extreme Points

Additional switches define behavior of slaved motion at extreme points, when the motion approaches the final point of the last segment or the starting point of the first segment. This occurs, when the **MPOS** master value falls out of the interval (0, L), where L is the overall length of the path. There are four possibilities, depending on the switch employed in the command:

- > **c** – Cyclic motion
The path is closed. The motion passes from the last segment to the first or from the first to the last as if they were adjacent.
- > **e** – Extrapolated motion
If the distance S is slaved to the **MPOS** master value becomes greater than the overall

length of the path, the motion continues along the extrapolated last segment. If the distance S slaved to the **MPOS** master value becomes less than zero, the motion continues along the extrapolated first segment. If the extrapolated segment is a circular arc, the motion will follow along the extrapolated circle.

- > **t** – Stalled motion
When the motion approaches the extreme point, the slave comes out from synchronism and stalls in the point until the **MPOS** master value allows to regain synchronism. For velocity lock synchronism is achieved when the **MPOS** changes its direction; after regaining the offset **C** may have a different value than before approaching the extreme point. For position lock synchronism regains when the **MPOS** falls into the interval $(0, L)$ again. The controller ensures smooth approaching the extreme points and smooth return to synchronism.
- > No switch – Bounded motion
The motion finishes when the slave approaches any extreme point. The controller activates the next motion in the queue (if any). If the **MPOS** master value changes only in positive direction, the behavior is very close to non-slaved motion. The difference is that non-slaved motion is based upon the time value and slaved motion is based upon the **MPOS** master value.

4.6 Extended Segmented Motion (XSEG)

XSEG provides several enhancements to MSEG for processing complex trajectory with higher throughput and accuracy. The following main capabilities are implemented in XSEG:

- > Corner detection
- > Detection of segments, where required velocity violates axis velocity/acceleration and jerk limits
- > Velocity limitation at corners and segments where required velocity violates axis velocity, acceleration and jerk limits
- > Building up velocity profiles using the multi-segment look-ahead algorithm
- > Corner rounding using different criteria
- > Support of up to 6 axes

4.6.1 Corner Processing

Corner processing includes automatic detection of the corner and calculating the velocity for passing the corner without exceeding acceleration and jerk limits.

4.6.1.1 Corner Detection

For each pair of adjacent segments, the controller calculates the tangent vector for each segment at the junction point. If both vectors are equal, the segments are tangent, and no special processing is required. Otherwise, both segments form a corner. In a corner, controller behavior follows the corner processing option selected by the customer for this XSEG motion.

4.6.1.2 Supported Options

- > Exact path: no deviation from the specified path is permitted. The user specifies two additional parameters: threshold angle and corner velocity. The controller compares the

corner angle and the threshold angle. If the corner angle is smaller, the controller ignores the corner and tries to move as if the junction is smooth (the threshold angle cannot be large, otherwise passing the junction at working velocity can produce mechanical jerk). If the corner angle is greater, the controller executes deceleration to achieve the junction point with the specified corner velocity.

The following picture illustrates the option:

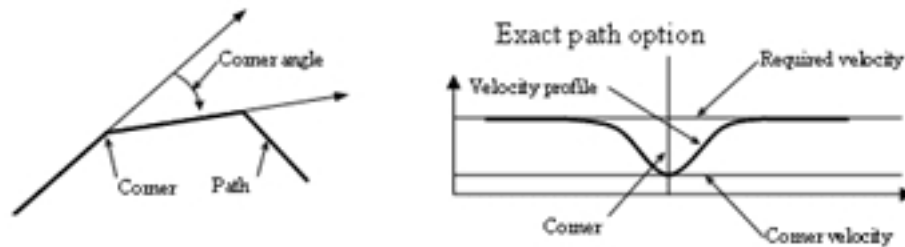


Figure 4-2. Corner Processing - Exact Path Option

- > Corner rounding according to permitted deviation: the user specifies the motion trajectory maximum permitted deviation from the corner point. The controller inserts an additional segment in the corner so that the resulting path is smooth and complies with the maximum deviation.
- > Corner rounding according to permitted radius: the user specifies the additional segment maximum permitted rounding radius. The controller inserts an additional segment in the corner so that the resulting path is smooth and complies with the maximum permitted radius.
- > Automatic corner rounding: the user specifies the maximum segment length for automatic corner rounding. The controller applies automatic corner rounding if the length of both segments in the pair is less than the maximum segment length.

The following picture illustrates the option:

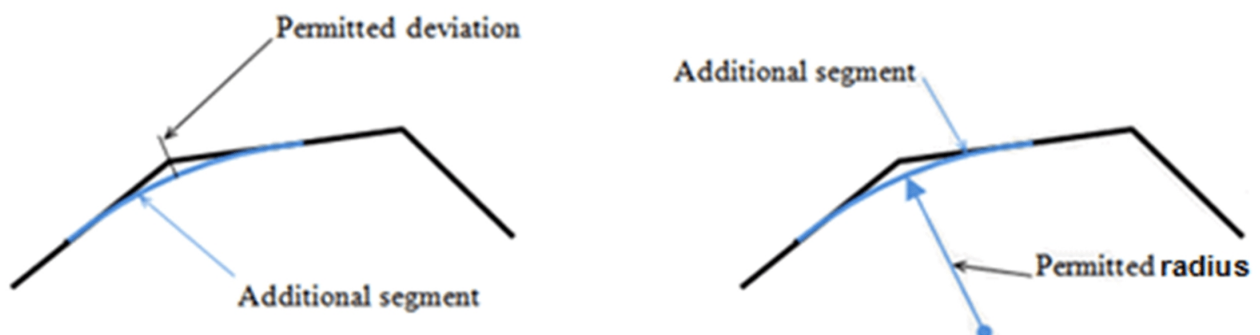


Figure 4-3. Corner Processing - Permitted Deviation, Permitted Radius and Corner Rounding Options

4.6.2 Automatic Corner Processing

If the **XSEG** command doesn't specify switch **J**, **A** or **D** the controller provides an automatic calculation of the corner parameters.

If switch **y** is specified, the controller provides automatic processing of corners and curvature discontinuity points as described in [Enhanced automatic corner and curvature discontinuity points processing \(switch/y\)](#).

If neither threshold angle or corner velocity are specified, the controller provides automatic calculation of the corner parameters.

The controller algorithm of automatic corner processing works as follows:

1. The controller detects a corner on the path.
2. The controller calculates the equivalent acceleration for all involved axes.
3. If for any involved axis, the absolute value of equivalent acceleration is less than allowed acceleration **ACC** for the axis, the controller ignores the angle and passes the angle without deceleration.
4. Otherwise, the controller calculates the vector velocity that for any involved axis provides the equivalent acceleration equal to or less than the allowed acceleration **ACC**. The controller provides deceleration to the calculated velocity before the corner using the look-ahead algorithm.



During movement on an arc segment, a coordinate acceleration (RACC) may exceed the acceleration limit (ACC) by factor of up to 1.41.

The allowed acceleration deceleration is defined by **ACC** only. The **DEC** parameter is not used by the algorithm.



Changing of the controller cycle time parameter (CTIME) may affect the automatic corner processing calculations. The user needs to check the system performance after the CTIME change.

4.6.3 Enhanced automatic corner and curvature discontinuity points processing (switch/y)

If switch **Y** is specified, the controller provides automatic processing of corners and curvature discontinuity points.

In this mode the controller limits velocity not only according to axis acceleration (**ACC**), but also according to axis jerk (**JERK**). The automatic velocity limitation is applied not only to corners, but also to curvature discontinuity points.

Unlike corner, smooth junction (linear-to-arc or arc-to-arc) shows no discontinuity in the velocity profile. However, in most smooth junctions curvature discontinuity occurs that causes discontinuity in axis acceleration. The controller detects and processes curvature discontinuity points only if switch **/D** is specified.

The controller algorithm of Enhanced automatic corner processing processes curvature discontinuities as follows:

1. Controller detects a curvature discontinuity on the path.
2. Controller calculates equivalent jerk for all involved axes.
3. If the absolute value of equivalent jerk, of any involved axis, is less than allowed **JERK** value for the axis, the controller passes the point without deceleration.
4. Otherwise, the controller calculates vector velocity for any involved axis that provides the equivalent jerk equal or less than the allowed **JERK** value. The controller provides deceleration to the calculated velocity at the point using look-ahead algorithm.

During accelerating/decelerating phases on an arc segment, a coordinate jerk (RJERK) may exceed the jerk limit (JERK) by factor of up to 1.5.



Changing of the controller cycle time parameter (CTIME) may affect the automatic corner processing calculations. The user needs to check the system performance after the CTIME change.

4.6.4 Velocity Control and Look-ahead Algorithm

The required velocity for extended segment motion is either specified in the **XSEG...ENDS** block or is taken from the **VEL** value of the leading axis. In a simple case, a third-order vector velocity profile looks as:

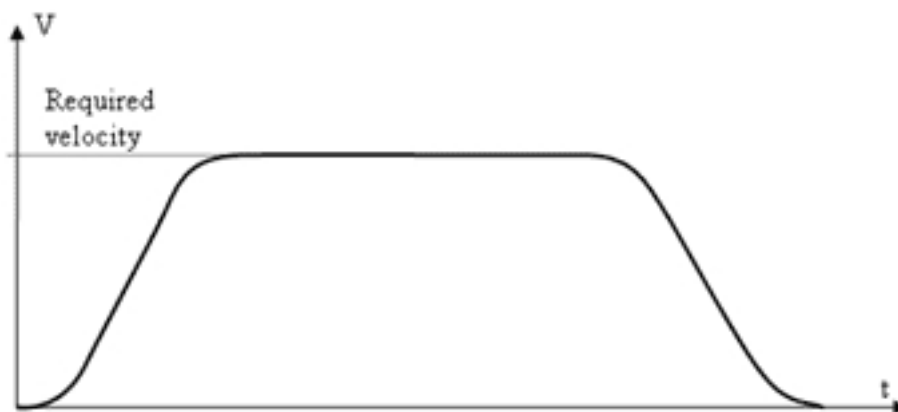


Figure 4-4. Third-Order Velocity Profile

The motion starts from zero velocity, accelerates to the required velocity, goes through the specified segments at the required velocity, and then decelerates approaching the final point of the last segment with zero velocity and acceleration. The diagram also explains that the required velocity actually defines maximal velocity through the path; on specific intervals the velocity is lower than required.

A number of factors can affect the velocity diagram. The following conditions apply additional restrictions to velocity in specific points or on intervals:

- > The user specifies a new required velocity in a segment definition. The new velocity remains active for the specified segment and for all subsequent segments.
- > The user specifies lower (or zero) velocity in the segment final point. The specification defines velocity in one point, but doesn't change required velocity for subsequent segments.
- > The user specifies required velocity greater than axis velocity (**VEL** value) of one or more involved axes. On some segments the required vector velocity can be achieved, on others the axis velocity would be surpassed. The controller reveals such segments, and limits the vector velocity accordingly.
- > The controller detects a corner. Depending on the specified options, the controller either uses specified corner velocity or calculates reduced velocity in the corner point to comply with acceleration/jerk limitations (**ACC** and **JERK**) for all axes.
- > The controller reveals an arc segment that requires either centripetal acceleration or jerk greater than axis acceleration (**ACC** parameter) or axis jerk (**JERK**) of one or more involved axes. The controller reduces velocity on this segment to comply with acceleration/jerk limitations (**ACC** and **JERK**) for all axes.
- > The controller detects a curvature discontinuity point (linear-to-arc or arc-to-arc junction). Depending on the specified options, the controller either uses specified curvature discontinuity velocity or calculates reduced velocity in the point to comply with jerk limitations (**JERK**) for all axes.

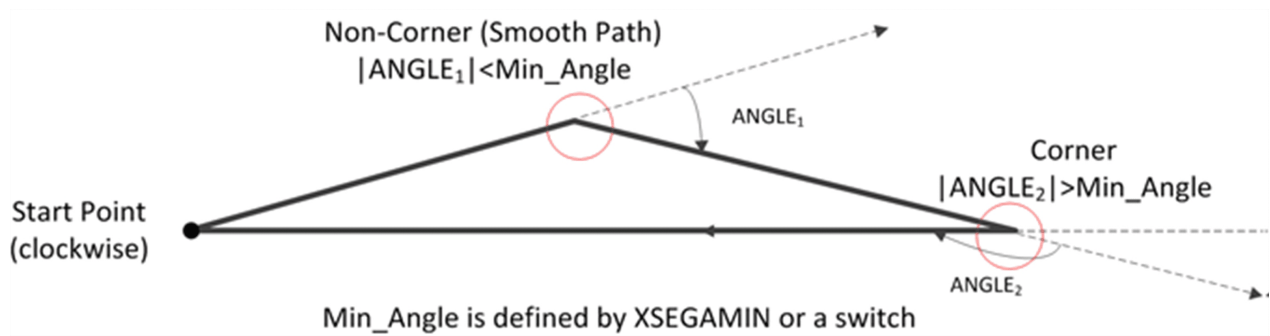
After all velocity restrictions are exposed, the controller builds a third-order velocity profile that never exceeds the required velocity.



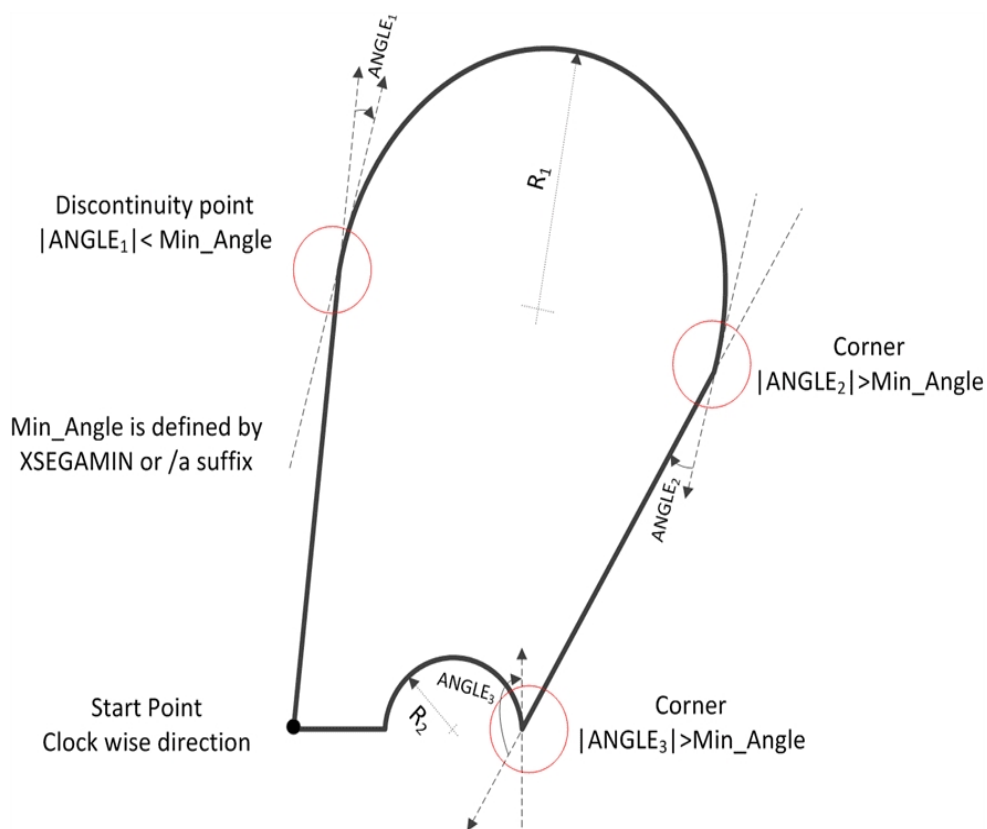
Frequently, acceleration/deceleration intervals appear longer than the related segment. When a CAD system calculates the motion segments, the resulting motion often contains numerous small segments. In this case, the acceleration/deceleration interval may span a number of segments. To calculate proper velocity profile, the controller needs to analyze segments well ahead of the current segment, reveal changes in required velocity in advance, and then trace the segments back to find starting points of deceleration in velocity profile.

4.6.5 Corners and Curvature Discontinuity Points

A corner is a non-smooth connection of two adjacent segments. In a corner, velocity vector changes its direction. If the velocity vector is non-zero, all related coordinates show discontinuity in the corner coordinate velocity.



A curvature discontinuity point is a smooth connection of two adjacent segments, if the two segments have different or differently-directed curvatures. Most smooth connections are actually discontinuity points. In a discontinuity point, all related coordinates show discontinuity in acceleration.



4.6.6 Dynamic Velocity Profile Generation

If all motion segments were specified before the motion start, the whole path is already known, and the whole velocity profile can be calculated from the beginning.

However, in a typical case, the controller starts executing XSEG motion while the program or host continues adding segments. A newly added segment can change the velocity profile of the previous segments. For example, a new segment specifies lower required velocity. The lower velocity is efficient from the first point of the segment; therefore the controller provides deceleration profile on the previous one or more segments.

The controller provides dynamic calculation and, if necessary, recalculation of the motion profile each time once a new segment is added.

To make look-ahead calculation possible, the program or host has to add segments in advance, before they are actually reached in the motion. This is not a problem, if segments are added from ACSPL+ program. However, if segments arrive from the host through a communication channel, the host may fail to supply a segment in time; a condition referred to as “segment starvation”. If segment starvation occurs, the controller raises the **AST.#STARV** flag and executes a special algorithm that prevents motion discontinuity. The algorithm avoids mechanical jerks, but moves at a slower rate as long as starvation continues; once the host begins supplying segments at sufficient rate, the controller returns to normal execution.

4.6.7 Controller Usage Considerations

As described in [Dynamic Velocity Profile Generation](#), the **XSEG** command implements an “on-the-fly” approach for velocity profile calculation. The calculations require significant processing time that might cause a controller overusage fault in cases such as:

- > Adding extremely short segments, when total execution time of the segment where the specified velocity is less than 5 milliseconds
- > Segment starvation conditions
- > “On-the-fly” change of velocity by executing the **IMM** command (see [The IMM Command](#)), where the specified velocity is lower than the previously defined velocity

In such cases, ACS Motion Control recommends using controllers with powerful processors such as:

- > SPiiPlusSC
- > SPiiPlusEC
- > SPiiPlusNTM- XX-H

Otherwise, you can use the **XSEG** command with the following controllers:

- > SPiiPlusNTM-08-S
- > SPiiPlusNT-LT-x-8, SPiiPlusNT-HP-x-8, SPiiPlusNT-LD-x-8
- > SPiiPlusCMnt-1-08, SPiiPlusCMnt-2-08, SPiiPlusCMnt-3-08



When using one of the above products, configure the controller time (**CTIME**) to 1 millisecond and ensure that a maximum of one **XSEG** motion is executed simultaneously.

4.6.8 Approaches for Adding Segments

4.6.8.1 Adding Segments In Advance

If the application calculates the whole (or part of) trajectory path in advance, the segments can be added once they are calculated.

The controller answers the needs of this approach by providing **AST.#NEWSEGM** flag, which is raised when the controller requires new segment to be added. In this case the application can wait till the flag is raised and then add the segment.

From within the ACSPL+ buffers the segments are added using the corresponding segment commands (**LINE**, **ARC1** with **XSEG**, **ARC2**). The ACSPL+ program need not take care of whether there is free space in the internal segments buffer or not.

If the buffer has free space, the segment is accepted and immediately inserted into the buffer; if the buffer is full, the ACSPL+ program waits on the corresponding line until a segment buffer has free space.

If segments are added from the host program, the host program should first query **GSFREE** in order to know how many cells are free in the segment buffer. The segments are added using the corresponding SPiiPlus Library functions.

This approach is simpler in implementation and ensures more optimal trajectory passing time, as starvation is avoided. However, the trajectory cannot be quickly changed on the fly as all already added segments should be executed first.

4.6.8.2 Adding Segments Only When Requested

The other approach to add segments when the application is not able to calculate the segments in advanced or the trajectory path can be changed only at the last moment is to add segments only when requested.

The controller answers the needs of this approach by providing the **AST.#NEWSEGM** flag, which is raised when the controller requires a new segment to be added. In this case the application can wait until the flag is raised and then add the segment.

From within ACSPL+ program the waiting for the flag is implemented using the **TILL**

TILL AST(axis).#NEWSEGM

If the segments are added from the host application, the application polls the bit and when it is raised, the segment is added.

In order to avoid the starvation, the user application is able to configure when the **#NEWSEGM** flag is raised by including the *starvation_margin* optional argument in the **XSEG** command.

4.6.9 Secondary Axes

To handle multi-axial CNC and laser cutting applications, a six-axial extension enables you, in **XSEG** motion, to add up to four secondary axes to the primary axes described above.

Primary axes support full look-ahead functionality, while secondary axes provide limited functionality, as described in this section.

4.6.9.1 Secondary Axes Functionality

Secondary axes provide the following functionality:

- > All secondary axes move concurrently and synchronously with primary axes.
- > Automatic corner processing takes care of secondary axes limits as well as of primary axes. As a result, vector velocity of primary axes may be reduced due to secondary axes limits.

The following primary axes functionality is NOT available for secondary axes:

- > Arc segments specification. Secondary axes implement linear interpolation .
- > Corner rounding.
- > Vector velocity and vector path calculation.

4.6.9.2 Using Secondary Axes in XSEG

Between two and six axes can be specified in the XSEG axis list as follows:

- > the first two axes are considered as primary axes
- > any additional axes (up to four) added to the axes list are considered as secondary axes

In the XSEG command, you specify initial coordinates for each axis specified in the XSEG axis list (see [XSEG...ENDS](#)).

4.6.9.3 Secondary Axes in ARC1, ARC2, LINE

Between two and six axes can be specified in the ARC1, ARC2 and LINE axis lists as follows:

- > the first two axes are considered as primary axes
- > any additional axes (up to four) added to the axes list are considered as secondary axes



The ARC1, ARC2 and LINE axis lists should be identical to the XSEG axis list.

For each axis, ARC1 and LINE specify the primary axes destination coordinate. ARC2 specifies the secondary axes destination coordinates (destination coordinates of the primary axes are calculated from the arc parameters). For details, see [ARC1 with XSEG, ARC2, LINE](#).

4.6.10 Supported Motion Parameters

The existing GSEG parameter indicates the currently executed actual segment, after trajectory processing. The trajectory processing may change segments numbering either by adding additional segments (in case of rounding) or by removing segments (in case of zero length segments). So, GSEG numbering may not corresponds to the numbering of originally defined by the user segments.

The newly introduced GSEGU parameter indicates the currently executed original user segment. Any added or removed segment does not effect the GSEGU numbering of originally defined by the user segments.

XSEG updates the following motion-related parameters:

- > Motion and Axis Statuses: **AST, MST**
- > Vector velocity, acceleration and jerk: **GVEL, GACC, GJERK**
- > Axis velocity: **GVEC**
- > Motion queue status and motion type: **GMQU, GMTYPE**
- > Trajectory vector distance from the beginning of the first segment: **GPATH**
- > Elapsed motion time: **GETIME**
- > Currently executed segment: **GSEG**
- > Currently executed "original user" segment: **GSEGU**
- > Segments buffer status: **GSFREE**
- > Currently executed segment final point coordinates: **TPOS**

XSEG builds the algorithm upon the following axis motion parameters as axis constraints: **VEL**, **ACC**, **JERK**



The deceleration (**DEC**) parameter is not used by the **XSEG** command.

4.6.11 Non Supported Motion Parameters

The following parameters are not updated by **xseg**:

- > Remaining motion time: **GRTIME**
- > Motion phase: **GPHASE**

4.6.12 XSEG...ENDS

Description

The **XSEG...ENDS** (Extended Segmented Motion) command block provides the following:

- > Corner detection
- > Detection of segments, where required velocity violates axis velocity/acceleration limits
- > Velocity limitation at corners and segments where required velocity violates axis velocity, acceleration and jerk limits
- > Building a velocity profile using multi-segment look-ahead algorithm
- > Corner rounding using different criteria
- > Support of up to 6 axes
- > Support for "Skywriting" - external loops at corners

Use the following commands to define the segmented motion:

- > **ARC1**- adds an arc segment to a segmented motion and specifies the coordinates of center point, coordinates of the final point, and the direction of rotation
- > **ARC2** - Adds an arc segment to a segmented motion and specifies the coordinates of center point, rotation angle and direction.
- > **LINE** - adds a linear segment to a segmented motion.
- > **ENDS**- terminates the point sequence

Syntax

XSEG

```
[/switches] (axis_list), initial_position_axis1,initial_position_axis2[,initial_position_axis3...,initial_
position_aixs6],
[,velocity][,end_velocity][,junction_velocity][,angle][,curvature_velocity][,deviation][,radius]
[,maximal_length]
[,starvation_margin [,segments_buffer]]
[,external_loop_type, minimal_segment_length, maximum_allowed_deviation]
[output_index, bit_number, polarity]
```

Segment commands (**ARC1**, **ARC2**, **LINE**)

ENDS

Arguments

<i>axis_list</i>	Axis group (specified as axes numbers separated by a comma), valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.  A minimum of two axes must be specified.
<i>initial_position_axis1</i>	Initial position of the first axis.
<i>initial_position_axis2</i>	Initial position of the second axis.
<i>initial_position_axis3...</i> <i>initial_position_axis6</i>	Mandatory only if axis_list contains more than 2 axes. Number of initial positions must correspond to the number of axes in axis_list .
<i>velocity</i>	[Optional, only used with /v switch] Defines required velocity instead of default velocity (VEL).
<i>end_velocity</i>	[Optional, only used with /f switch] Defines required velocity at the end of each segment.
<i>junction_velocity</i>	[Optional, only used with /j switch] Defines required velocity at the junction.
<i>angle</i>	[Optional, only used with /a switch] The junction will be treated as a corner if actual junction angle is more than defined.
<i>curvature_velocity</i>	[Optional, only used with switch /d] Defines required velocity at curvature discontinuity points. See Switches explanation for details.
<i>deviation</i>	[Optional, only used with /g switch] Defines maximal allowed trajectory deviation from the corner point. See Switches explanation for details.
<i>radius</i>	[Optional, only used with /u switch] Defines maximal allowed rounding radius of the additional segment. See Switches explanation for details.
<i>maximal_length</i>	[Optional, only used with /h switch] Defines the maximal length of the segment for smoothing processing. If the length of a segment that formed a corner exceeds the specified maximal length, the corner will not be smoothed.

starvation_ margin	[Optional] Starvation margin in milliseconds. The controller sets the AST.#NEWSEGM bit starvation_margin millisecond before the starvation condition occurs. By default, if the argument is not specified, the starvation margin is 500 milliseconds.
segments_ buffer	[Optional] One-dimensional user defined real array. The controller uses this array to store adding segments. By default, if the argument is not specified, the controller allocates internal buffer for storing 50 segments only. The argument allows the user application to reallocate the buffer for storing larger number of segments. The larger number of segments may be required if the application needs to add many very small segments in advanced.
external_loop_ type	0 - Cancel external loop 1 - Smooth External loop (line-arc-line) 2 - Triangle External loop (line-line-line)
minimum_ segment_length	If the lengths of both segments are more than this value, the skywriting algorithm will be applied.
maximum_ allowed_ deviation	The parameter limits the external loop deviation from the defined profile. If the value is negative – no limitation.
output_index	Index of digital output port to assigned to synchronization (read by OUT)
bit_number	Bit number (0 - 16) assigned to synchronization
polarity	0 or 1 - which value is considered the initial state
motor_ movement_ delay	(Optional, used only with /q switch) Delay, in milliseconds, before motor motion actually starts.

Switches

There are three types of optional switches:

- > General
- > Velocity look-ahead
- > Geometry look-ahead

The controller processes the specified switches in the following order:

1. The controller checks and applies geometry look-ahead options.
2. The controller checks and applies velocity look-ahead options.

Switches from different groups can be applied together. For example, it's possible to specify a velocity at curvature discontinuity points (switch **/d**) together with permitted deviation (switch **/g**).

In this case, the controller first applies corner rounding for the trajectory and then calculates velocity profile for already processed trajectory.

Optional switches are for use only with the **XSEG** command:

General	
/w	Do not start until the GO command. If the switch is not specified, the motion starts immediately after the first motion segment is defined.
/v	Specify required velocity. The switch requires additional parameter that specifies required velocity. If the switch is not specified, the required velocity is derived from the leading axis parameters.
/m	Use maximum velocity under axis limits. With this switch, no required velocity should be specified. The required velocity is calculated for each segment individually on the base of segment geometry and axis velocities (VEL values) of the involved axes.
/l	Velocity limitation under axis limits. As opposed to the switch /m, with this switch the required velocity can be specified either for all motion (switch /v for xseg command) or as segment velocity (switch /v for line, arc1, arc2 commands). In both cases, the controller tries to achieve the required velocity by taking into account also velocity limits of all involved axes. If the required velocity cannot be achieved within the axis velocity limits, the maximal velocity within the axis velocity limits is used. This switch cannot be specified together with switch /m.
/z	Interpret entered coordinates according to the Local Coordinate System. With this switch, use 2 axes motion coordinate only (X,Y). Using 3 or more coordinates causes a runtime error.
/q	Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The maximum delay is 100 controller cycles: 100ms for CTIME=1ms or 20ms for CTIME=0.2ms. Allows delaying actual motor movement start (RPOS) for the specified delay, while motion profile generation (APOS) starts immediately.
Velocity look-ahead	

/f	Decelerate to the end of each segment. The switch requires an additional parameter that specifies end velocity. The controller decelerates to the specified velocity in the end of each segment. The specified value should be less than the required velocity; otherwise the parameter is ignored. If the switch is not specified, deceleration in each segment is not required. However, in specific segments deceleration might occur due to corner processing or other velocity control conditions.
/j	Decelerate to corner. The switch requires an additional parameter that specifies corner velocity. The controller detects corner on the path and decelerates to the specified velocity before the corner. The specified value should be less than the required velocity; otherwise the parameter is ignored. If switch j is not specified while switch a is specified, zero value of corner velocity is assumed. If switches j, a, d, and y are not specified, the controller provides automatic calculation of the corner processing.
/a	Do not treat junction as a corner, if junction angle is less than or equal to the specified value in radians. The switch requires an additional parameter that specifies negligible angle in radians. If switch a is not specified while switch j is specified, the controller accepts default value of 0.01 radians that is about 0.57 degrees. If switches j, a, d, and y are not specified, the controller provides automatic calculation of the corner processing.
/d	Decelerate to curvature discontinuity point. The switch requires an additional parameter that specifies velocity at curvature discontinuity points. Curvature discontinuity occurs in linear-to-arc or arc-to-arc smooth junctions. If the switch is not specified, the controller does not decelerate to smooth junction disregarding curvature discontinuity in the junction. If the switch is specified, the controller detects curvature discontinuity points on the path and provides deceleration to the specified velocity. The specified value should be less than the required velocity; otherwise the parameter is ignored. The switch can be specified together with switches j and/or a. If switches j, a, d, and y are not specified, the controller provides automatic calculation of the corner processing.

/y	If the switch is specified the controller provides automatic calculations as described in Enhanced automatic corner and curvature discontinuity points processing (switch /y).
/r	Set the initial axis position as origin. Segment commands positions should be declared relative to the new origin.
Geometry look-ahead	
/g	Use a corner rounding option with the specified permitted deviation The switch requires additional parameter that specifies maximal allowed deviation of motion trajectory from the corner point. The switch cannot be specified together with switches /u and /h
/u	Use a corner rounding option with the specified permitted curvature The switch requires additional parameter that specifies maximal allowed rounding radius of the additional segment The switch cannot be specified together with switches /g and /h
/h	Use a corner smoothing option. The switch requires additional parameter that specifies the maximal length of the segment for smoothing processing. If a length of one of the segments that built a corner exceeds the specified maximal length, the corner will not be smoothed. Smoothing is only applied to pair of linear segments. If one of the segments in a pair is arc, the smoothing is not applied for this corner. The switch cannot be specified together with switches /g and /u .
/b	Use external loops at corners. The switch requires additional parameters that specify the external loop type, the minimum segment length, and the maximal allowed deviation from profile.
/s	Defines output bit to support external loop synchronization.



XSEG without switches does not require any additional parameters except the initial point coordinates, for example, **XSEG (0,1),0,0** creates segmented motion for axes 0 and 1 with initial point (0,0) with required velocity derived from the axis 0.

Related ACSPL+ Commands

GO – Starts the physical motion if **XSEG/W** is specified.

HALT – Stops the motion prematurely, even if not all segments have finished.

IMM – Changes the motion velocity, acceleration, or jerk. The new value takes effect immediately

Comments

XSEG without switches does not require any additional parameter, except initial point coordinates. For example, the command: **XSEG (0,1),0,0** creates Extended Segmented Motion for axes 0 and 1 with initial point (0,0) and required velocity derived from the axis 0.

Some switches, however, require an additional parameter to be specified. If more than one parameter is required, the parameters should be separated by comma, and the order of parameters is fixed in the following order:

1. Required velocity (used with /v)
2. Final velocity (used with /f)
3. Corner velocity (used with /j)
4. Angle (used with /a)
5. Curvature velocity (used with /d)
6. Deviation (used with /g)
7. Radius (used with /u)
8. Maximal segment length (used with /h)

Examples are:

XSEG (1,0),0,0	Segmented motion for axes 1 and 0. Required velocity is derived from the axis 1, i.e., the VEL (1) value. No deviation from the path is permitted. If the path contains a corner, and the junction angle is more than default value 0.01 radians, the velocity decelerates to zero in the corner point.
XSEG/vf (0,1),0,0,100,50	Segmented motion for axes 0 and 1 with initial point (0,0) with required velocity 100 units/sec; at the end of each segment, the motion should decelerate to 50 units/sec.
XSEG/vja (1,2),1000,1000, 100,20,0.05	Segmented motion for axes 1 and 2 with initial point (1000,1000) and required velocity is 100 units/sec. If the path contains a junction, and the junction angle is more than 0.05 radians, the velocity decelerates to 20 unit/sec in the junction point.

4.6.13 ARC1 with XSEG

Description

Use **ARC1** to specify the center point and final point coordinates of an arc and the direction of rotation. Direction is designated by a plus sign (+) or (–) for clockwise or counterclockwise rotation depending on the encoders' connections. When **ARC1** is used for Extended Motion, it must be initialized with **XSEG...ENDS**.

Syntax

ARC1 [/switches] (axis_list), center_point_axis1, center_point_axis2, destination_point_axis1, destination_point_axis2, [destination_point_axis3, ... destination_point_axis6,] direction[, velocity] [,end_velocity][,time][,values, variables[,index [,masks]]][, lci_segment_active]

Arguments

Argument	Description
axis_list	Defines one or two axes, specified as axes numbers separated by commas or as axes names separated by commas. The axes should only be those axes specified in the corresponding XSEG command.
center_point_axis1	Center point position for the first axis
center_point_axis2	Center point position for the second axis
destination_point_axis1	Destination position of the first axis
destination_point_axis2	Destination position of the second axis
destination_point_axis3 ... destination_point_axis6	Mandatory only if AXIS_LIST contains more than 2 axes. Destination position of the rest of axes. Number of destination positions must correspond to the number of axes in the AXIS_LIST .
direction	Direction is specified as + or -. It defines clockwise or counterclockwise rotation depending on the encoder connection: "+" for motion in the direction of increasing encoder counts, or "-" for motion in the direction decreasing encoder counts.
velocity	[Mandatory with /V switch]. Defines required velocity for the current and for all subsequent segments. See Using ARC1, ARC2 and LINE Switches . .
end_velocity	[Mandatory with /F switch]. Defines required velocity at the end of the current segment. See Using ARC1, ARC2 and LINE Switches .
time	[Mandatory with /T switch]. Defines segment processing time. See Using ARC1, ARC2 and LINE Switches .

Argument	Description
values	[Mandatory with /O switch]. Defines the values to be written to variables array at the beginning of the current segment execution. values is a one-dimensional user defined array of integer or real type with maximum size of 10 elements .
variables	[Mandatory with /O switch]. Defines the user-defined array, which will be written with values data at the beginning of the current segment execution. variables is a one-dimensional user defined array of the same type and size as the values array.
index	[Optional, only used with /O switch] Defines the first element (starting from zero) of the variables array, to which values data will be written. If argument is omitted, values data is written to the variables array starting from the first element (index 0).
masks	[Optional, only used if values and variables are integer] Defines the masks that are applied to values before the values are written to variables array at the beginning of the current segment execution. masks is a one-dimensional user-defined array of integer type and the same size as the values array. The masks are only applied for integer values: $\text{variables}(n) = (\text{variables}(n) \text{ AND } (\text{NOT mask}(n))) \text{ OR } (\text{values}(n) \text{ AND mask}(n))$ If argument is omitted, all values bits are written to variables. If values is a real array, the masks argument should be omitted.
lci_segment_active	[Mandatory with /p switch] Integer value. Fire LCI State or Pulse at the beginning of current segment. The function is available if the LCI segment-based mode was previously defined by the SegmentGate or SegmentPulse functions. The value defines the state value in SegmentGate mode (1 or 0). In SegmentPulse mode the value equal 1.



For information on optional switches for this command, see **Using ARC1, ARC2 and LINE Switches**.

4.6.14 ARC2

Description

Use **ARC2** to specify the center point and rotation angle in radians of an arc segment. Designate direction by positive or negative rotation angle, depending on the encoders' connections. When **ARC2** is used for Extended Motion, it must be initialized with **XSEG...ENDS**.

Syntax

ARC2 [/switches] (axis_list), center_point_axis1, center_point_axis2, rotation_angle [,destination_point_axis3, ... destination_point_axis6][,velocity][,end_velocity][,time][,values, variables[,index [,masks]]]

Arguments

Argument	Description
axis_list	Defines one or two axes, specified as axes numbers separated by comma or as axes names separated by comma. The axes should only be those axes specified in the corresponding XSEG command.
center_point_axis1	Center point position for the first axis
center_point_axis2	Center point position for the second axis
rotation_angle	Defines central angle of the arc, signed according to rotation direction: plus for a counter-clock-wise arc, minus for a clock-wise arc.
destination_point_axis3 ... destination_point_axis6	Mandatory only if AXIS_LIST contains more than 2 axes. Destination position of the rest of axes. Number of destination positions must correspond to the number of axes in the AXIS_LIST .
velocity	[Mandatory with /V switch]. Defines required velocity for the current and for all subsequent segments. See Using ARC1, ARC2 and LINE Switches
end_velocity	[Mandatory with /F switch]. Defines required velocity at the end of the current segment. See Using ARC1, ARC2 and LINE Switches .
time	[Mandatory with /T switch]. Defines segment processing time. See Using ARC1, ARC2 and LINE Switches .

Argument	Description
values	[Mandatory with /O switch]. Defines the values to be written to variables array at the beginning of the current segment execution. values is a one-dimensional user defined array of integer or real type with maximum size of 10 elements .
variables	[Mandatory with /O switch]. Defines the user-defined array, which will be written with values data at the beginning of the current segment execution. variables is a one-dimensional user defined array of the same type and size as the values array.

index	[Optional, only used with /O switch] Defines the first element (starting from zero) of the variables array, to which values data will be written. If argument is omitted, values data is written to the variables array starting from the first element (index 0).
masks	[Optional, only used if values and variables are integer] Defines the masks that are applied to values before the values are written to variables array at the beginning of the current segment execution. masks is a one-dimensional user-defined array of integer type and the same size as the values array. The masks are only applied for integer values: $\text{variables}(n) = (\text{variables}(n) \text{ AND } (\text{NOT mask}(n))) \text{ OR } (\text{values}(n) \text{ AND mask}(n))$ If argument is omitted, all values bits are written to variables . If values is a real array, the masks argument should be omitted.



For information on optional switches for this command, see [Using ARC1, ARC2 and LINE Switches](#).

4.6.15 LINE

Description

Use **LINE** to add a linear segment that starts at the current point and ends in the destination point to the motion. When **LINE** is used for Extended Motion, it must be initialized with **XSEG...ENDS**.

Syntax

LINE [/switches] (axis_list), destination_point_axis1,destination_point_axis2
[,destination_point_axis3 ... ,destination_point_axis6][,velocity][,end_velocity][,time]

[,values,variables[,index[,masks]]]

Arguments

Argument	Description
axis_list	Defines one or two axes, specified as axes numbers separated by comma or as axes names separated by comma. The axes should only be those axes specified in the corresponding XSEG command.
destination_point_axis1	Destination position of the first axis
destination_point_axis2	Destination position of the second axis
destination_point_axis3 ... destination_point_axis6	Mandatory if AXIS_LIST contains more than 2 axes. Destination position of the rest of axes. Number of destination positions must correspond to the number of axes in AXIS_LIST .
velocity	[Mandatory with /V switch]. Defines required velocity for the current and for all subsequent segments. See Using ARC1, ARC2 and LINE Switches .
end_velocity	[Mandatory with /F switch]. Defines required velocity at the end of the current segment. See Using ARC1, ARC2 and LINE Switches .
time	[Mandatory with /T switch]. Defines segment processing time. See Using ARC1, ARC2 and LINE Switches .
values	[Mandatory with /O switch]. Defines the values to be written to variables array at the beginning of the current segment execution. values is a one-dimensional user defined array of integer or real type with maximum size of 10 elements .
variables	[Mandatory with /O switch]. Defines the user-defined array, which will be written with values data at the beginning of the current segment execution. variables is a one-dimensional user defined array of the same type and size as the values array.
index	[Optional, only used with /O switch].

Argument	Description
	Defines the first element (starting from zero) of the variables array, to which values data will be written. If argument is omitted, values data is written to the variables array starting from the first element (index 0).
masks	[Optional, only used if values and variables are integer] Defines the masks that are applied to values before the values are written to variables array at the beginning of the current segment execution. masks is a one-dimensional user-defined array of integer type and the same size as the values array. The masks are only applied for integer values: $\text{variables}(n) = \text{variables}(n) \text{ AND NOT } (\text{mask}(n)) \text{ OR } (\text{values}(n) \text{ AND mask}(n))$ If argument is omitted, all values bits are written to variables. If values is a real array, the masks argument should be omitted.



For information on optional switches for this command, see [Using ARC1, ARC2 and LINE Switches](#)

4.6.16 Using ARC1, ARC2 and LINE Switches

The following optional switches may be used singularly or in combination with **ARC1**, **ARC2** and **LINE**:

/f	Decelerate to the end of segment. The switch requires an additional parameter that specifies the end velocity. The controller decelerates to the specified velocity at the end of segment. The specified value should be less than the required velocity; otherwise the parameter is ignored. The switch affects only one segment. The switch also disables corner detection and processing at the end of segment. If the switch is not specified, deceleration is not required. However, in special cases the deceleration might occur due to corner processing or other velocity control conditions.
/v	Specify required velocity. The switch requires an additional parameter that specifies the required velocity. The switch changes the required velocity for the current segment and for all subsequent segments. If the switch is not specified, the required velocity does not change.

/o	Synchronize user variables with segment execution. The switch requires additional two or three parameters that specify values , user variable and mask .
/t	Specify segment processing time The switch requires an additional parameter that specifies the segment processing time in milliseconds. Unlike the required velocity specification, the segment processing time defines velocity at the current segment only, and has no effect on the subsequent segments. The switch cannot be specified together with /V .
/b	Use external loops at corners. The switch requires additional parameters that specify the external loop type, the minimum segment length, and the maximum allowed deviation from profile. The /b switch may be defined with other corner processing options (/u, /g, etc.) . If the Skywriting algorithm is applied, other corner processing options are skipped. If Skywriting is skipped, other defined corner options will be applied. This switched is used for Extended Motion only (initialized with XSEG...ENDS)
/p	Specifies that the lci_segment_active parameter is required. This switched is used for Extended Motion only (initialized with XSEG...ENDS)

For **ARC1**, **ARC2**, and **LINE** some switches require an additional parameter to be specified. If more than one parameter is required, the parameters should be separated by a comma, and the order of parameters is fixed in the following order:

1. Required velocity (used with **/V**)
2. Final velocity (used with **/F**)
3. Segment processing time (used with **/T**)
4. **/O** requires specification of the **values**, **variables**, and **mask** parameters
5. **/B** requires specification of the External Loop Type, Minimum Segment Length, and Maximum Allowed Deviation parameters
6. **/P** requires specification of the **lci_segment_active** parameter

Examples:

LINE/v (1,0), 1000, -1000, 500	Add line segment with end point (1000, -1000) and segment velocity 500.
arc1/vf (0,1), 0, 0, 100, 100, +, 500, 100	Add arc segment with center (0,0), end point (100,100), clockwise direction, segment velocity 500 and end velocity 100

int Value(1) int Mask(1) Value(0) = 1; Mask (0) = 5 ARC2/o (0,1), 0, 0, 3.141529, Value, OUT, 2, Mask	Add arc segment with center (0,0) and 180 degree (π) angle. At the beginning of the segment execution, sets bit 0 and reset bit 2 of digital outputs OUT(2).
---	--

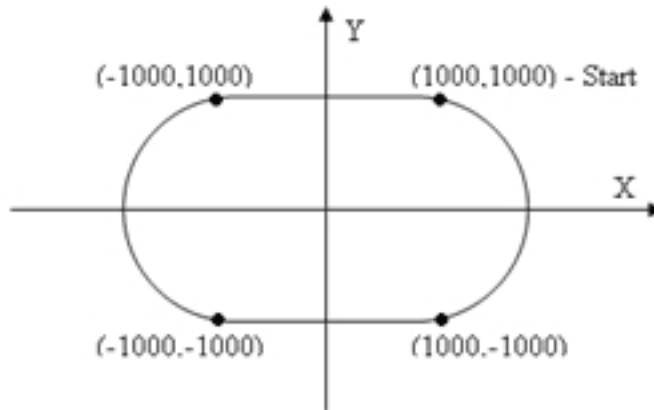
4.6.17 Examples

4.6.17.1 Simple Two Axes Example

The following is a simple program example employing Extended Segmented Motion:

XSEG (0,1),1000,1000	Create segment motion in axes XY (0,1) with initial point (1000,1000)
ARC1 (0,1),1000,0,1000,-1000,-	Add arc segment with center (1000,0), final point (1000, 1000), clockwise rotation
LINE (0,1),-1000,-1000	Add line segment with final point (1000, 1000)
ARC2 (0,1),-1000,0,-3.141529	Add arc segment with center (1000,0) and rotation angle of $-\pi$
LINE (0,1),1000,0	Add line segment with final point (1000,1000)
ENDS (0,1)	End the segment sequence

The **XSEG** command creates the segment motion. The motion does not start at this moment. Actual motion starts once the previous motion ends and one or more segments are added. The four segment commands specify the following path:



The **LINE** command may specify one axis that actually moves in this segment. Other axes specified in **XSEG** hold their positions while the linear segment is in progress.

The **ARC1** and **ARC2** commands always specify two axes.

The **ARC1** and **ARC2** commands differ by the required arguments. The **ARC1** command specifies the coordinates of the center point, coordinates of the final point and the direction of rotation (+ for counter-clockwise, – for clockwise rotation). The **ARC2** command specifies the coordinates of the center point and rotation angle (positive for counter clockwise, negative for clockwise rotation). The **ARC1** and **ARC2** commands may produce the same result, so the user may select the one that suits the available data. If the user knows the coordinates of the center point, coordinates of the final point and the direction of rotation, **ARC1** is preferable. If the user knows the coordinates of the center point and rotation angle, **ARC2** is preferable.

The **ENDS** command informs the controller that no more segments will be specified for the motion.

4.6.17.2 Simple Four Axes Example

<code>XSEG (0,1,2,3),1000,1000,500,500</code>	Create segment motion with primary axes 0,1 and secondary axes 2,3. Initial point for primary axes is (1000,1000), initial point for secondary axes is (500,500).
<code>ARC1 (0,1,2,3),1000,0,1000,-1000,500,-500</code>	Add arc segment for primary axes with center (1000,0), final point (1000, 1000), clockwise rotation ("") and line segment with final point (500,-500) for secondary axes.
<code>LINE (0,1,2,3),-1000,-1000,-500,-500</code>	Add line segment with final point (1000, 1000) for primary axes and (-500,-500) for secondary axes.
<code>ARC2 (0,1,2,3),-1000,0,-3.141529,-500,500</code>	Add arc segment for primary axes with center (1000,0) and rotation angle $-\pi$ and line segment with final point (-500,500) for secondary axes
<code>LINE (0,1,2,3),1000,1000,500,500</code>	Add line segment with final point (1000,1000) for primary axes and (500,500) for secondary axes
<code>ENDS (0,1,2,3)</code>	End the segment sequence

XSEG creates the segment motion for four axes: primary axes (0,1) and secondary axes (2,3). Actual motion starts once the previous motion ends and one or more segments are added. The four segment commands specify the following path:

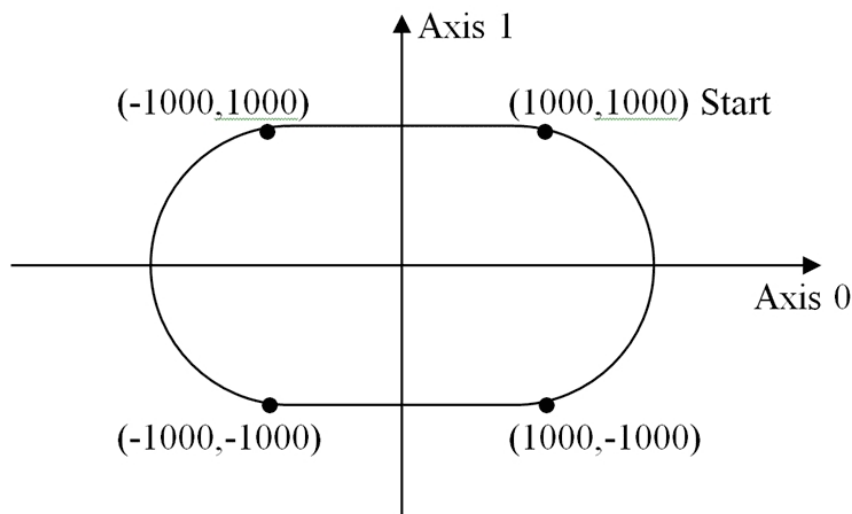


Figure 4-5. Primary Axes Motion

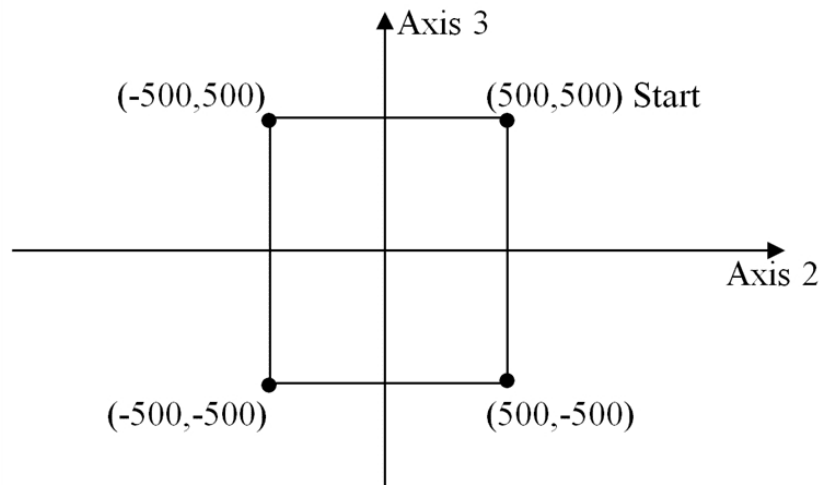


Figure 4-6. Secondary Axes Motion

In comparison with 2-axes example, xseg, line, arc1 and arc2 commands specify four axes, first two of them are treated as primary axes.



- > XSEG specifies the initial point coordinates for all four axes in the order the axes appear in the axis list.
- > ARC1 specifies the arc segment with a center, final point and rotation for primary axes, and a linear segment with the final point for secondary axes.
- > LINE specifies the final points of the linear segment for primary and secondary axes.
- > ARC2 specifies the arc segment with a center and angle for the primary segment, and a linear segment with the final point for secondary axes.
- > ENDS informs the controller that no more segment for this motion will be added.

4.6.18 New AST Bits

Two new bit statuses have been added to the **AST** (Axis State) variable to support the **XSEG** motion command:

Bit Name	No.	Description
#NEWSEGM	16	The controller sets the bit to inform that a new segment is required to be provided by the application. The bit is set starvation_margin ms before the starvation condition occurs. The starvation condition is indicated by #STARV bit.
#STARV	17	The controller sets the bit to indicate starvation condition. The starvation condition means that there are not enough further segments to continue the motion with required velocity. In this case, the controller starts decelerating the motion with $\frac{1}{2}$ jerk in order to prevent motion discontinuity and avoid mechanical jerks. Once the application begins supplying segments at a sufficient rate, the controller returns the motion back to normal condition. Often the starvation condition causes inefficient velocity generation and increases the time required for completing the required motion path.

4.6.19 The IMM Command

The **IMM** command specifies new values of velocity, acceleration, deceleration, and jerk. In typical case, the command is issued while a motion is in progress, and specifies new desired velocity. The new velocity is expected to take effect immediately, not waiting for the current motion termination.

The following considerations must be taken into account when issuing the **IMM** command in conjunction with **XSEG**.



If the **IMM** command occurs either before the start **XSEG**, or after **XSEG** termination, its action follows standard rules of **IMM** action in idle mode (or other motion whichever is executed in the moment of **IMM** command).

If the **IMM** command is issued, and **XSEG** motion is at a specific point of execution, all **XSEG** segments can be subdivided as follows:

- > Segments before the current segment; these segments has been passed and are of no interest.
- > Current segment the current point belongs to.
- > Segments after the current segments already included in segment queue and processed by look-ahead algorithm.
- > Future segments not included yet in segment queue.

If the **IMM** command specifies a new velocity, the velocity is expected to affect all **XSEG** segments after the current point, including the section of current segment after the current point. However, there are some limitations:

- > Individual velocity can be specified for any segment. Individual velocity should not be exceeded; actual velocity is required to be equal or (in some cases) less than the specified one.
- > The look-ahead algorithm may apply additional limitations to velocity on a segment or in a junction point. The limitations are not eliminated by **imm** command.

Taking the above considerations into account, the velocity specified by the **IMM** command is not a strict velocity setting from this point on; rather it operates as additional limitation that affects actual velocity along with other limitations.

If the **IMM** command specifies a new velocity while **XSEG** motion is in progress, the controller response includes the following actions:

- > The specified velocity replaces the required velocity for the rest of the **XSEG** motion. Therefore, the new value will act as the required velocity for a segment, if no individual velocity is specified.
- > The specified velocity defines velocity restriction for the rest of the **XSEG** motion. The velocity restriction will take part in any velocity calculation; so that actual velocity can appear equal or lower, but not higher than the specified velocity.
- > The velocity profile for the queued segments, including the section of the current segment after the current point, is recalculated taking into account the new required velocity and velocity restriction. This way, operation of **imm** command is identical for queued and future segments.

If the **IMM** command specifies new acceleration or jerk, the new value replaces the required value for the rest of the **XSEG** motion. However, unlike the velocity case, the queued segments are not recalculated. Therefore the new acceleration or jerk takes effect with some unspecified delay.

In some cases it is required to temporarily pause a segmented motion while remaining on the trajectory. The **imm** command allows specifying zero velocity for the leading axis in order to pause the currently executed segmented motion. The controller will immediately start decelerating to zero velocity and the motion will be paused on the current or one of the subsequent segments.

In order to resume the motion, the **imm** command for the leading axis should be sent with a non-zero velocity value. The controller will immediately start accelerating to the specified velocity value.

There is an additional restriction that if the newly specified velocity is greater than the current required velocity, the new velocity is not effective for already queued segments; it is effective for the future segments only. In other words, lower velocity takes effect immediately; greater velocity takes effect with some unspecified delay.

4.6.20 **XSEGAMIN** and **XSEGAMAX**

The controller provides two standard variables for configuring look-ahead processing angles:

- > **XSEGAMIN** - minimal angle
- > **XSEGAMAX** - maximal angle

The rounding option is applied only if the angle between segments is more than **XSEGAMIN** and less than **XSEGAMAX**.

XSEGAMIN also defines the minimal angle for corner processing, and its value is used by default for corner processing if **/a** is not specified.

4.6.21 XSEGRMIN and XSEGRMAX

The controller provides two standard variables for configuring arcs:

- > **XSEGRMIN** - minimal arc radius
- > **XSEGRMAX** - maximal radius difference



XSEGRMIN and XSEGRMAX are specified error threshold in encoder counts.

The controller returns an error if:

- > arc1 or arc2 specify an arc radius which is less than XSEGRMIN.
- > arc1 specifies center and final point coordinates with the difference between a radius from the center to the initial point and a radius from the center to the final point is more than XSEGRMAX.

4.7 Blended Segmented Motion

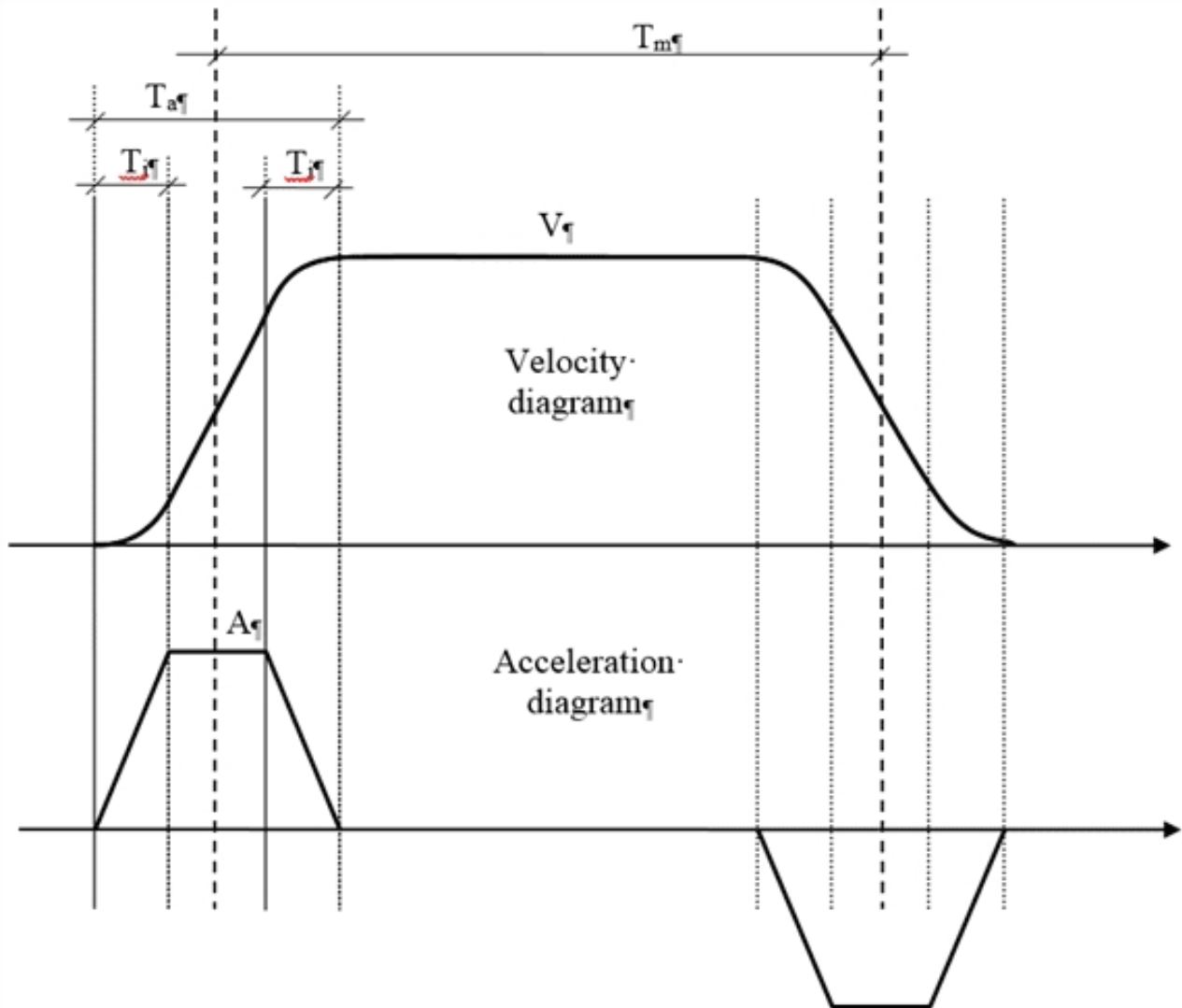
Blended segmented motion is a type of segmented motion that doesn't provide look-ahead capabilities, unlike Extended segmented motion. Both type of motions are intended for processing a complex multi-axis trajectory and smoothing corners between segments, but do it in different ways. The Extended segmented motion (XSEG) allows achieving highest throughput within the defined axis limitations and the defined accuracy. The Blended segmented motion (BSEG) allows passing along the trajectory with the defined timing constrains.

The following main capabilities are implemented in the Blended segmented motion:

- > Support of up to 6 axes (up to 2 axes the first stage)
- > Support of smoothing corners between two linear or linear and arc segments.
- > The blended profile guaranties following the timing constrains, defined by the user parameters, but doesn't guaranty geometry of the resulting trajectory.

4.7.1 Motion definition through time intervals

Third order motion profile is defined with four variables, Motion Distance (S), Motion Time (T_m), Acceleration Time (T_a), and Jerk Time (T_j):



Motion velocity V , acceleration A , and jerk J are derived from the time parameters as follows:

$$V = \frac{S}{T_m}$$

$$A = \frac{V}{T_a - T_j}$$

$$J = \frac{A}{T_j}$$

Total motion times is $T_m + T_a$

The specified times should comply with the following limitations:

$$T_m \geq T_a$$

$$T_a \geq 2T_j$$

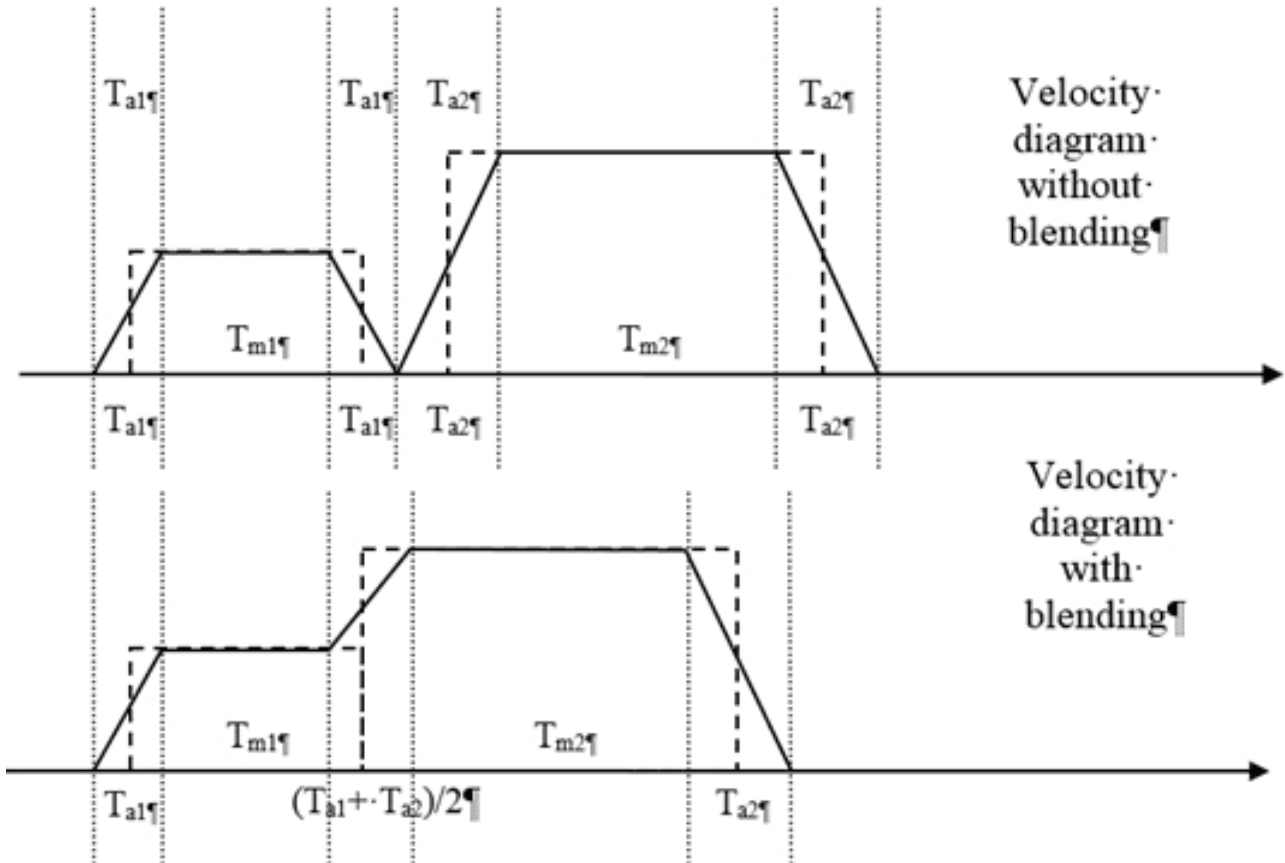
4.7.2 Motion blending

The following two motion diagrams shows:

1. Two sequential motions without blending
2. Two sequential motions with blending



In both diagrams, jerk sections are dropped for simplicity's sake.



If blending is active, sequential motions are executed without deceleration to zero. Actually, the next motion starts acceleration stage once the previous motion achieves the start point of deceleration stage. Then until the end of acceleration stage, the actual motion appears a mix of the previous and the next motions. In one-axis motion, this approach provides smooth transition from velocity of previous segment to velocity of the next segment. In multi-axis motion, blending additionally provides geometrical smoothing of corners, although exact control of the smoothing trajectory is not possible.

For n sequential motions without blending, total motion time (including jerk sections) is

$$\sum_{i=1}^n T_{mi} + \sum_{i=1}^n T_{ai}$$

If blending is active, total motion time (including jerk sections) is

$$\sum_{i=1}^n T_{mi} + \frac{T_{a1}}{2} + \frac{T_{an}}{2}$$

4.7.2.1 Motion blending in special cases

As it was stated in the section above, the specified times should comply with the following limitations:

$$T_m \geq T_a$$

$$T_a \geq 2T_j$$

If the user specify time parameters that don't comply with the specified limitations, the controller provides the following handling of these specific cases:

- > If the specified T_a is greater than T_m , the controller decreases T_a to be equal to T_m .
- > If the specified T_j is greater than $\frac{T_a}{2}$, the controller decreases T_j to be equal to $\frac{T_a}{2}$.

4.7.3 ACSPL+ interface

The Blended segmented motion is a sequence of linear and arc segments.

The following commands control segmented motion:

Commands

- > **BSEG** – Create blended segmented motion
- > **LINE** – Add linear segment
- > **ARC1** – Add arc segment
- > **ARC2** – Add arc segment
- > **ENDS** – Terminate the segment sequence

Related commands

- > **GO** – Starts physical motion, if the **BSEG**/w command was specified.
- > **HALT** – Stops the motion prematurely, even if not all segments were finished.
- > **KILL** – Emergency motion stop with maximal deceleration (KDEC)

4.7.3.1 Supported motion parameters

The **BSEG** updates the following motion related parameters:

- > Motion and Axis Statuses: AST, MST
- > Vector velocity, acceleration and jerk: GVEL, GACC, GJERK
- > Axis velocity: GVEC
- > Motion queue status and motion type: GMQU, GMTYPE
- > Trajectory vector distance: GPATH
- > Elapsed motion time: GETIME
- > Currently executed segment: GSEG
- > Currently executed "original user" segment: GSEGU
- > Segments buffer status: GSFREE

- > Currently executed segment final point coordinates, defined by the user: TPOS. With a blending, the final point is never reached.

4.7.3.2 Non-supported motion parameters

The following parameters are not updated by **BSEG**:

- > Remaining motion time: GRTIME
- > Motion phase: GPHASE

The motion parameters VEL, ACC, DEC, JERK are not used in **BSEG**.

4.7.3.3 BSEG...ENDS

Syntax

BSEG[/*switches*] (*axis_list*), *initial_position_axis1*, *initial_position_axis2*, [*motor_motion_delay*,] *segment_time*, *acceleration_time*, *jerk_time*[, *dwel_time*]

Arguments

Arguments	Comments
axis_list	Axes involved in BSEG motion, specified as axes numbers separated by comma or as axes names separated by comma
initial_position_axis1	Initial position of the first axis
initial_position_axis2	Initial position of the second axis
motor_motion_delay	(Optional, used only with /q switch) Delay, in milliseconds, before motor motion actually starts.
segment_time	Initial segment time (T_m) in milliseconds. The specified segment time will be used for all segments until <i>segment_time</i> argument is specified in segment LINE, ARC1 or ARC2 command.
acceleration_time	Initial acceleration time (T_a) in milliseconds. The specified acceleration time will be used for all segments until <i>acceleration_time</i> argument is specified in LINE, ARC1 or ARC2 command.
jerk_time	Initial jerk time (T_j) in milliseconds. The specified jerk time will be used for all segments until <i>jerk_time</i> argument is specified in LINE, ARC1 or ARC2 command.
dwel_time	Optional] Initial dwell time between segments in milliseconds. If this argument is specified, no blending will be done for all segments of the motion. That means that the motion will be stopped at the end of each segment for the specified <i>dwel_time</i> milliseconds.

Switches

Switch	Comments
/w	Do not start until the GO command is executed. If the switch is not specified, the motion starts immediately after the first motion segment is defined.
/r	Set the initial axis position as origin. Segment commands positions should be declared relative to the new origin.
/z	Interpret entered coordinates according to the Local Coordinate System.
/q	Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The maximum delay is 100 controller cycles: 100ms for CTIME=1ms or 20ms for CTIME=0.2ms. Allows delaying actual motor movement start (RPOS) for the specified delay, while motion profile generation (APOS) starts immediately.

4.7.3.4 LINE

This format of LINE is used for blended segment motion and in this form must be initialized with BSEG...ENDS. The command adds to the motion path a linear segment that starts in the current point and ends in the destination point.

Syntax

LINE[/suffixes] (*axis_list*),*destination_point_axis1*,*destination_point_axis2*
[,*segment_time* [,*acceleration_time* [,*jerk_time* [,*dwel_time*]]]]

Arguments

Arguments	Comments
<i>axis_list</i>	Defines one or two axes, specified as axes numbers separated by comma or as axes names separated by comma. The axes should only be from those axes specified in the corresponding BSEG command.
<i>destination_point_axis1</i>	Destination position of the first axis
<i>destination_point_axis2</i>	Destination position of the second axis
<i>segment_time</i>	Only if suffix /m is specified: Segment time (Tm) in milliseconds.
<i>acceleration_time</i>	Only if suffix /a is specified: Acceleration time (Ta) in milliseconds.
<i>jerk_time</i>	Only if suffix /s is specified: Jerk time (Tj) in millisecond

<i>dwell_time</i>	Only if suffix /d is specified: Dwell time at the final point of the segment in milliseconds. With this suffix no blending will be done at the segment final point.
-------------------	---

4.7.3.5 ARC1

This format of ARC1 is used for blended segment motion and in this form must be initialized with BSEG...ENDS. The command adds to the motion path an arc segment that starts in the current point and ends in the destination point with the specified center point.

Syntax

ARC1[/suffixes] (*axis_list*),
center_point_axis1,*center_point_axis2*,
destination_point_axis1,*destination_point_axis2*, *direction*
[,*segment_time* [,*acceleration_time* [,*jerk_time* [,*dwell_time*]]]]]

Arguments

Argument	Comments
<i>axis_list</i>	Defines one or two axes, specified as axes numbers separated by comma or as axes names separated by comma. The axes should only be from those axes specified in the corresponding BSEG command.
<i>center_point_axis1</i>	Center point position for the first axis
<i>center_point_axis2</i>	Center point position for the second axis
<i>destination_point_axis1</i>	Destination position of the first axis
<i>destination_point_axis2</i>	Destination position of the second axis
<i>direction</i>	Direction is specified as + or -. It defines clockwise or counterclockwise rotation depending on the encoder connection: "+" for motion in the direction of increasing encoder counts, or "-" for motion in the direction decreasing encoder counts.
<i>segment_time</i>	Only if suffix /m is specified: Segment time (T_m) in milliseconds.
<i>acceleration_time</i>	Only if suffix /a is specified: Acceleration time (T_a) in milliseconds.
<i>jerk_time</i>	Only if suffix /s is specified: Jerk time (T_j) in milliseconds

<i>dwell_time</i>	Only if suffix /d is specified: Dwell time at the final point of the segment in milliseconds. With this suffix no blending will be done at the segment final point.
-------------------	---

4.7.3.6 ARC2

This format of ARC2 is used for blended segment motion and in this form must be initialized with BSEG...ENDS. The command adds to the motion path an arc segment that starts in the current point and specified as the center point and rotation angle.

Syntax

ARC2[/suffixes] (axis_list),
 center_point_axis1,center_point_axis2,
 rotation_angle
 [,segment_time [,acceleration_time [,jerk_time [,dwell_time]]]]

Arguments

Arguments	Comments
<i>axis_list</i>	Defines one or two axes, specified as axes numbers separated by comma or as axes names separated by comma. The axes should only be from those axes specified in the corresponding BSEG command.
<i>center_point_axis1</i>	Center point position for the first axis
<i>center_point_axis2</i>	Center point position for the second axis
<i>rotation_angle</i>	Defines central angle of the arc, signed according to rotation direction: plus on counter-clock-wise arc, minus on clock-wise arc.
<i>segment_time</i>	Only if suffix /m is specified: Segment time (T_m) in milliseconds.
<i>acceleration_time</i>	Only if suffix /a is specified: Acceleration time (T_a) in milliseconds.
<i>jerk_time</i>	Only if suffix /s is specified: Jerk time (T_j) in milliseconds
<i>dwell_time</i>	Only if suffix /d is specified: Dwell time at the final point of the segment in milliseconds. With this suffix no blending will be done at the segment final point.

4.7.3.7 Suffixes with commands *LINE*, *ARC1*, *ARC2*

Suffix	Comments
m	Specifies segment time (Tm) in milliseconds. The suffix requires additional parameter that specifies required segment time in milliseconds. The suffix defines required segment time for the current segment and for all subsequent segments. If the suffix is not specified, the previously defined segment time is used.
a	Specifies acceleration time (Ta) in milliseconds. The suffix requires additional parameter that specifies acceleration time in milliseconds. The suffix defines required acceleration time for the current segment and for all subsequent segments. If the suffix is not specified, the previously defined acceleration time is used.
s	Specifies jerk time (Tj) in milliseconds. The suffix requires additional parameter that specifies jerk time in milliseconds. The suffix defines required jerk time for the current segment and for all subsequent segments. If the suffix is not specified, the previously defined jerk time is used.
d	Specifies dwell time in milliseconds at the final point of the segment. With this suffix no blending will be done at the segment final point.

Some suffixes require additional parameter to be specified. If more than one parameter is required, the parameters should be separated by comma, and the order of parameters is fixed in the following order:

1. Segment time (used with **/m** suffix)
2. Acceleration time (used with **/a** suffix)
3. Jerk time (used with **/s** suffix)
4. Dwell time (used with **/d** suffix)

4.7.3.8 HALT execution

HALT command is supported by **BSEG** motion and it is executed according with the following:

1. The motion is decelerated to zero velocity following the specified trajectory
2. **BSEG** motion is terminated and all subsequent segments are canceled
3. Halt deceleration time is taken from the currently executed segment as a currently effective Ta value.
4. If Halt operation occurs within the acceleration phase, or within blending process, the Halt process does not start immediately, but waits for the end of acceleration/blending, and then executes as described above. As a result, maximum time of Halt may appear $2 \cdot T_a$.

4.8 Two axes example

```

BSEG (0,1),1000,1000,100,20,5      ! Create blended motion for axes 0 and 1 with
                                     ! initial point (1000,1000),
                                     ! initial segment time 100msec, initial
                                     ! acceleration time 20msec,
                                     ! initial jerk time 5msec
ARC1 (0,1),1000,0,1000,-1000,-      ! Add arc segment with center (1000,0), final
                                     ! point (1000,-1000), clockwise rotation
                                     ! As no new time parameters are specified,
                                     ! the initial segment, acceleration and
                                     ! jerk time parameters are used for this segment
LINE (0,1),-1000,-1000              ! Add line segment with final point (-1000,-1000)
                                     ! As no new time parameters are specified,
                                     ! the initial segment, acceleration and
                                     ! jerk time parameters are used for this segment
ARC2/m (-0,1),-1000,0,-3.141529,50  ! Add arc segment with center (-1000,0) and
                                     ! rotation angle -180°
                                     ! New segment time 50msec is specified
LINE/ma (0,1),1000,1000,25,10        ! Add line segment with final point (1000,1000)
                                     ! New segment time 25msec and acceleration
                                     ! time 10msec are specified
ENDS (0,1)                           ! End the segments sequence

```

The program specifies the following path:

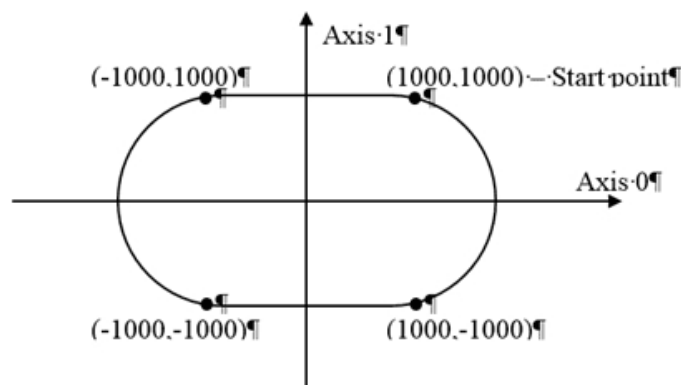


Figure 4-7. Example of motion with segment commands

The **BSEG** command creates the Blended segmented motion for axes 0 and 1 and initial point (1000, 1000). Initial blended motion parameters are specified as following: initial segment time 100msec, initial acceleration time 20msec and initial jerk time 5msec.

The motion does not start at this moment. Actual motion starts once the previous motion ends and one or more segments are added.

The **ARC1** command adds a first arc segment with center (1000,0), final point (1000,-1000), clockwise rotation. The initial segment, acceleration and jerk time parameters are used for the first arc segment.

The **LINE** command adds a second segment with final point (-1000,-1000). The initial segment, acceleration and jerk time parameters are used for the first arc segment.

The **ARC2** command adds a third arc segment with center (-1000,0) and rotation angle -180°. The new segment time 50msec is specified.

The last **LINE** command adds a fourth segment with final point (1000,1000). For this segment new segment time 25msec and new acceleration time 10msec are specified.

The **ENDS** command informs the controller, that no more segments will be specified for the motion.

4.9 Master/Slave Motion

4.9.1 MASTER Command

The **MASTER** command defines a formula for calculating the axis master value (**MPOS** – see *SPiiPlus Command & Variable Reference Guide*).

Syntax:

MASTER MPOS(axis_index)=value

Only one component of the **MPOS** (master position) variable is allowed as **MPOS(AXIS_INDEX)**.

In the simplest case the master value follows the feedback of another axis:

```
MASTER MPOS (0) = FPOS (1)
```

When the command executes, the controller stores the formula specified to the right of the equals sign, and then calculates the master value **MPOS(0)** according to this formula (in the example above, it simply assigns the current **FPOS(1)** value to **MPOS(0)**). The controller does this calculation every controller period, independent of program execution. Even if the program that executed the **MASTER** command terminates, the controller continues calculating the last specified master expression, until the another master command for the same axis executes.

The master value can be redefined at any time by the application. If the program that executed the above command then executes the command.

A more sophisticated use of the **MASTER** command connects the axis to another axis feedback with a scale factor:

```
MASTER MPOS (0) = 2.3 * FPOS (1)
```

The following example defines axis 2 to follow the **RPOS** (reference position) of axis 0 translated through a conversion table (cam motion):

```
MASTER MPOS (2) = MAPBY1 (RPOS (0) , Table)
```

In this example **TABLE** is a user-defined array that contains a table for conversion.

Similarly, the **MASTER** value may be connected to other sources such as the sum of two or more axes, or to an analog input.

4.9.2 SLAVE Command

The **SLAVE** command creates a motion slaved to the master value of the specified axis.

Syntax:

SLAVE [/switch] axis_designation [,start_interval,end_interval]

Where **switch** can be:

w	Create the motion, but do not start until the GO command has been issued.
p	Use position lock instead of velocity lock (see Velocity Lock vs. Position Lock).
t	Stall when approaching interval boundary (see Stalled Motion)

Slave motion is governed by the variables of the slaved axis. These include:

XSACC	Maximal allowed acceleration of the synchronous motion. If the master acceleration exceeds this value, the slave comes out from synchronism.
SYNV	Allowed difference in master and slave velocities. Used in asynchronous motion to determine if the synchronism can be re-established.
JERK	Default jerk. The slave uses this variable only in asynchronous motion to overtake the master.
ACC	Default acceleration. The slave uses this variable only in asynchronous motion to overtake the master.
VEL	Default velocity. The slave uses this variable only in asynchronous motion to overtake the master.

Once started, slaved motion terminates only if a failure occurs, or one of the commands **HALT**, **KILL**, or **BREAK** is executed. The **HALT** and **KILL** commands provide deceleration to zero and then the next motion starts. If no next motion was created, the axis becomes idle. The **BREAK** command provides smooth transition to the next motion without stopping, if a next motion is waiting in the queue.

4.9.2.1 Synchronization

In slaved motion the slave is usually synchronized to the master, meaning that the **APOS** axis reference follows the **MPOS** master value strictly or with a constant offset. However, there are two cases when synchronism is not attainable:

- > The slaved motion starts, and positions (position lock) or velocities (velocity lock) of the master and slave differ. The motion starts as asynchronous.
- > The motion was synchronized, but the acceleration of the master exceeds the allowed limit (the **XSACC** variable of the axis) for the slave. The slave comes out of synchronization.

In both cases, the motion continues asynchronously, and the correspondence between **APOS** and **MPOS** appears broken. The controller tries to regain synchronization by having the slave pursue the master within the maximal allowed motion parameters. When the slave overtakes the master, synchronization is re-established and the motion continues as synchronous.

Only individual axes are allowed to be used in a **SLAVE** command. Groups are not allowed. The created motion starts immediately if the axis is idle; otherwise the motion waits in the motion queue until all motions created before for the axis finish. The following command creates a slaved motion of the 0 axis:

SLAVE 0



The slave axis in a master-slave motion may show its state (through the **AST** variable) as accelerating and in motion even when the master axis is motionless. This reflects the fact that the axis is set to follow the motion of the other axis and is not following a motion profile of its own.

4.9.2.2 Velocity Lock vs. Position Lock

In velocity lock the slave velocity follows the master velocity. A constant offset between the master and slave position is allowed. In position lock the slave position strictly follows the master position.

The **SLAVE** command without the **P** switch activates a velocity-lock mode of slaved motion. When synchronized, the **APOS** axis reference follows the **MPOS** with a constant offset:

$$\text{APOS}(0) = \text{MPOS}(1) + C$$

Where **C** is constant in velocity lock mode and is zero in position lock mode.

When the **MSEG** command includes the **p** switch (see [MSEG, LINE, ARC1, ARC2, STOPPER Commands](#)), this activates the position lock mode of slaved motion. When synchronized, the **APOS** axis reference follows the **MPOS** strictly:

$$\text{APOS}(0) = \text{MPOS}(1)$$

When the motion is asynchronous for any reason (see above), the controller tries to regain synchronism by having the slave pursue the master with the maximal allowed motion parameters. The difference between position lock and velocity lock manifests itself at the moment of regaining synchronization:

- > Velocity lock motion switches to synchronized when the slave velocity reaches the master velocity (with allowed error defined by the **SYNV** variable of the slaved axis). At this moment the difference between the master position and the slave position is latched as the constant offset **C**, which then remains unchanged as long as the motion is synchronous.
- > Position lock motion switches to synchronized when the slave position overtakes the master position, i.e., when **APOS = MPOS**.

Each time the motion loses and regains synchronization, the velocity lock offset **C** may latch a different value. Under the same conditions, the position lock motion each time re-establishes the strict equality **APOS = MPOS**.

4.9.2.3 Stalled Motion

When the **SLAVE** command does not include the **T** switch, the command applies no limits to the slaved axis. The axis follows the master everywhere, unless a failure, such as limit switch activation, occurs.

The **SLAVE** command with the **T** switch requires two additional parameters that define a permitted interval for the slaved axis motion. For example, the command:

```
SLAVE/t 0, -1000, 2000
```

allows 0 axis motion only within the interval (-1000, 2000).

When the **APOS** axis reference approaches either of the two interval limit points, the slave comes out from synchronism and stalls at that point until the **MPOS** master value allows restoration of synchronism. In velocity lock, synchronization is regained when the **MPOS** changes its direction. After regaining the offset, **C** may have a different value than before approaching the extreme point. For position lock, synchronization is restored when the **MPOS** comes back into the permitted interval. The axis stalls when the master leaves the permitted range and regains synchronization when the master returns into the permitted range. The controller ensures a smooth approach to the extreme points and a smooth return to synchronization.

4.10 PATH Command

The **PATH** command is similar to **MPTP** in that it generates multi-point motion, but in this case it creates an arbitrary path motion.

Syntax:

PATH [/switch] axis_designators [,time_interval]

POINT axis_designators, coordinate [,coordinate] [,velocity] [,time_interval]

MPOINT axis_designators, point_matrix, number_of_points [,time_interval]

ENDS

Where **switch** can be one or a combination of:

r	The point coordinates are relative to the previous point.
w	Create the motion, but do not start until the GO command.
c	Use the point sequence as a cyclic array, that is, after positioning to the last point return to the first point and repeat.
t	Non-uniform time interval; the time interval is specified for each point along with the point coordinates.

Points for arbitrary path motion are defined by **POINT** and **MPOINT** commands (see [MPTP](#), [POINT](#), [MPOINT](#), and [ENDS Commands](#)).

The **ENDS** command terminates the point sequence. After the **ENDS** command, no **POINT** or **MPOINT** commands for this motion are allowed.

The trajectory of the motion follows through the defined points. Each point presents the instant desired position at a specific moment. Time intervals between the points are uniform, or non-uniform as defined by the **T** switch.

Motion generated by the **PATH** command does not use the standard motion profile. Typically, the time intervals between the points are short, so that the array of the points implicitly specifies the desired velocity in each point. For this reason, variables **VEL**, **ACC**, **DEC**, **JERK** have no effect on this motion.

If the time interval does not coincide with the controller cycle, the controller provides linear interpolation of the points.

Commands **HALT**, **KILL**, **KILLALL** (see [HALT Command](#) and [KILL and KILLALL Commands](#)) are executed in a specific way with this type of motion; as with other motion types, the controller provides a smooth deceleration profile using **DEC** (**HALT** command) or **KDEC** (**KILL**, **KILLALL** commands) for the leading axis. However, unlike other motions types, the controller does not provide following the motion trajectory during deceleration.

Arbitrary path motion created without the **T** switch implies uniform intervals between the points. If **PATH** is not specified with the **T** switch, the **TIME_INTERVAL** argument has to be included. The argument defines time interval between the motion points in milliseconds.

If **PATH** is specified with the **T** switch, it must not have **TIME_INTERVAL** specification. Instead, the **TIME_INTERVAL** must be specified for each point as an additional argument for the **POINT** command or as additional array column in the **MPOINT** command.



The **BREAK** command (see [BREAK Command](#)) is not supported when using the **PATH** command.

4.11 Spline Motion

4.11.1 Spline Motion Theory

General theory of spline interpolation is a topic of numerous books and articles. A classical introduction is *A Practical Guide to Splines* by Carl De Boor.

This section contains only basic facts required for understanding spline implementation in the SpiiPlus controller.

4.11.2 Main Definitions

PV (position-velocity) and PVT (position-velocity-time) refer to a motion mode that constructs the motion trajectory from spline segments. You specify the end position (P) and the end velocity (V) for each segment of motion. The difference between PV and PVT is that PVT motion also requires specification of the time interval for each segment, whereas PV motion uses a predefined constant time interval.

In PV/PVT mode the controller provides cubic spline interpolation between the specified points. As a spline mode, it minimizes the amount of data that the host-based program (or ACSPL+ program) needs to generate to produce the multi-axis arbitrary profile and provides precise trajectory generation.

A spline is a special function defined piecewise by polynomials. The spline is a piecewise polynomial function spread over an interval $[a, b]$ consists of polynomial pieces, such that:

$$a = t_0 < t_1 < t_2 < \dots < t_{k-2} < t_{k-1} = b$$

The points: t_i are called **knots**. The vector is called a **knot vector** for the spline. If the knots are equidistantly distributed in the interval $[a, b]$, we say the spline is **uniform**, otherwise we say it is **non-uniform**.

In many cases functional dependence between two or more values cannot be expressed as an analytic formula. The most common presentation of those functions is a table of function values in specific points.

For example, a machine axis was graduated with an external laser interferometer. The result of graduation is a table like the following:

Commanded position (x)	100	200	300	400	500	600	700	...
Actual position (p)	103	199	294	402	500	598	705	...

The table defines a functional dependence $p=f(x)$ that cannot be expressed analytically.

The argument values for x in the definition table are knots, and the function values for p are control points.

A 3rd order polynomial spline provides an approximation of the table-driven function that can provide the function value not only in the knots, but at any point. Between each two knots the spline is expressed as:

$$p = a_0 + a_1x + a_2x^2 + a_3x^3 = \sum_{i=0}^3 a_i x^i$$

where coefficients a_0, a_1, a_2, a_3 have different values at different intervals.

The SPiiPlus controller also supports two-dimensional splines. In this case, the definition table is a two-dimensional matrix. Knot points are defined for two arguments x and y , and the matrix contains corresponding p values. Knot values divide the XY plane into rectangular cells. The matrix defines the function values in the cell vertices. Within each cell, the interpolating spline is expressed as:

$$p = \sum_{i,j=0}^3 a_{ij} x^i y^j$$

4.11.3 PVSPLINE Command

The **PVSPLINE** command is used to create spline motion.

Syntax:

PVSPLINE [/switch] axis_designators [,time_interval]

POINT axis_designators, coordinate, [coordinate,] velocity [,time_interval]

MPOINT axis_designators, point_matrix, number_of_points [,time_interval]

ENDS

Where **switch** can be one or a combination of:

r	The point coordinates are relative to the previous point.
w	Create the motion, but do not start until the GO command.
c	Use the point sequence as a cyclic array, that is, after positioning to the last point return to the first point and repeat.
t	Non-uniform time interval; the time interval is specified for each point along with the point coordinates.

Points for PV and PVT motion are defined by the **POINT** and **MPOINT** commands (see [POINT Command](#) and [MPOINT Command](#)).



The **POINT** and **MPOINT** commands serve the same function for defining points along the path as the **POINT** and **MPOINT** commands for multiple point-to-point motion (see [MPTP](#), [POINT](#), [MPOINT](#), and [ENDS Commands](#)); however, the controller employs a different algorithm when calculating the spline motion.

The **ENDS** command terminates the point sequence. After the **ENDS** command, no **POINT** or **MPOINT** commands for this motion are allowed.

The trajectory of the motion follows through the defined points. Time intervals between the points are uniform, or non-uniform as defined by the **t** switch.

Motion generated by the **PVSPLINE** command does not follow the standard motion profile. Variables **VEL**, **ACC**, **DEC**, **JERK** have no effect on this motion. The motion profile is defined exclusively by the positions and velocities specified by the **POINT** and **MPOINT** commands.

The **HALT** command (see [HALT Command](#)) is executed in a specific way with this type of motion. As with other motion types, the controller provides a smooth deceleration profile with the **DEC** value of the leading axis. However, unlike other motions types, the controller does not follow the motion trajectory during deceleration.

Spline motion created without the **t** switch implies uniform intervals between the points. If **PVSPLINE** does not include the **t** switch, the **time_interval** argument has to be included. The argument defines time interval between the motion points in milliseconds.

If **PVSPLINE** includes the **t** switch, the **time_interval** argument must not be included. Instead, the time interval must be specified for each point as an additional argument for the **POINT** command or as additional array row in the **MPOINT** command.

The **PVSPLINE** command itself does not specify any point, so the created motion starts only after the first point is specified. The points of motion are specified by the **POINT** or **MPOINT** commands that follow the **PVSPLINE** command.

4.11.3.1 *POINT Command*

The **POINT** command for the spline interpolation specifies two values per axis that is involved: the first value defines the coordinate of the end point of the segment; the second value specifies the velocity at the end point. The coordinate is specified in the user units of the axis; the velocity is specified in user units per second.

The following fragment illustrates adding a point with PV spline motion.

```
PVSPLINE (0,1,3),10
```

```
POINT (0,1,3),200,100,300,1000,2000,1500
```

Create PV spline motion for axes 0, 1, and 3. Points are given at 10 millisecond intervals. Add a point with coordinates 0=200, 1=100, 3=300; velocities at the point are $V_0=1000$, $V_1=2000$, $V_3=1500$.

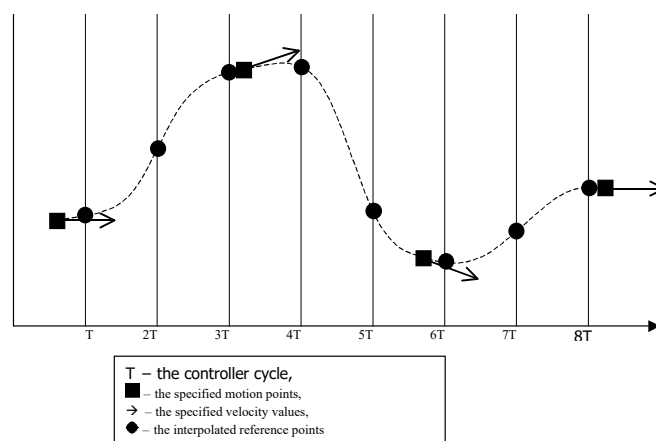
For each segment the controller constructs a third-order polynomial and calculates the reference coordinates using the polynomial.

The spline provides exact track through the specified points and exact specified velocity at the points. The spline also provides continuous velocity at all intermediate points.

In general the spline does not guarantee acceleration continuity at the connection points. However, the acceleration can be continuous if the proper velocity values are specified, and many host-based programs that prepare data for PV-interpolation actually calculate velocity values that will provide continuous acceleration.

The time interval between the points may be either uniform or non-uniform. In both cases the time interval is not required to be an integer or to be equal to an integer number of controller cycles. The controller uses the exact specified time interval to calculate the interpolated reference positions.

The following drawing illustrates the PV spline interpolation:



In the **POINT** command the **axis_designators** must specify the same axes in the same order as in the **axis_designators** of the corresponding **PVSPLINE** command.

The other arguments contain different values depending on corresponding **PVSPLINE** command.

If the related motion command is **PVSPLINE** without the **t** switch for M axes, there has to be $2 \cdot M$ arguments: 2 arguments per axis involved. The arguments specify the end point coordinates and the coordinate velocities at the end point in the following order:

- > The end point coordinate for each axis involved (M values)
- > The velocity at the end point for each axis involved (M values)

Each coordinate is specified in user units of the corresponding axis, each velocity is specified in user units per second.

If the related motion command is **PVSPLINE/t** for M axes, there have to be $2*M+1$ arguments: $2*M$ arguments specify the end point coordinates and velocities, and the last argument specifies the time interval between the previous and the current point. The time is specified in milliseconds.

4.11.3.2 MPOINT Command

The **MPOINT** command adds an array of points to PV or PVT spline motion.

The arguments of the **MPOINT** command are:

- > **axis_designators** - must specify the same axes in the same order as in the axes-specification of the corresponding mptp or path command.
- > **point_matrix** - name of declared two dimensional array.
- > **number_of_points** - specifies how many points are added to the motion by the command.

Before the **MPOINT** command can be executed, an array must be declared and filled with the point coordinates. Each row of the array contains coordinates of one point.

If the related motion command is **PVSPLINE** without the **t** switch, for M axes the matrix must contain $2*M$ rows, 2 rows per axis involved. The values in each column specify:

- > in row 1: the end point coordinate for each axis involved (M values)
- > in row 2: the velocity at the end point for each axis involved (M values)

Each coordinate is specified in user units of the corresponding axis, each velocity is specified in user units per second.

If the related motion command is **PVSPLINE/t**, for M axes the matrix must contain $2*M+1$ rows:

- > M rows for end point coordinates
- > M rows for end point velocities
- > plus additional row for the time interval between the points. The time is specified in milliseconds.

4.11.4 Spline Motion Variables

As mentioned, the spline motion does not use the values of the **VEL**, **ACC**, **DEC**, **JERK** variables for motion profile construction. The motion profile is constructed using the coordinates and velocities specified in the segment end points.

While the motion is in progress the controller updates the following read-only variables every controller cycle.

- > **APOS** and **RPOS** are updated as for any other motion and read the motion result.
- > Bits in **AST**, **MST** are updated as for any other motion and read the motion state.
- > **GSEG** is updated for the leading axis with the number of the currently executed segment.
- > **GSFREE** is updated for the leading axis with the number of free cells in the segment queue. If **GSFREE** is zero, the segment queue is full and the next coming point or mpoint command will be delayed until the required number of cells will be freed.
- > **GVEC** is updated with the instant velocity for each axis involved. The **GVEC** values build up a vector of instant velocity and also can be used for retrieving a tangent vector.
- > **GPATH**, **GVEL**, **GACC**, **GJERK**, **GPHASE**, **GRTIME** are updated while the motion is in progress.

The following example illustrates how the **PVSPLINE** command can be used for adding points on-the-fly. The example also shows a simple approach to synchronization between the host-based program that calculates the points and the controller that executes the motion.

The host-based program calculates the desired points and transmits the coordinates via Ethernet link. The motion involves 6 axes. Therefore, each point specification contains 12 real numbers (6 coordinates and 6 velocities).

Because transmitting each point separately would be a very inefficient use of the Ethernet, the host calculates the desired points in advance and transmits them in batches of 50 points. The controller then executes the motion. As soon as the controller is ready to accept the next batch of points and the host is ready to send that batch, the next transmission occurs, and so on.

The pace of the host and the controller do not have to be identical. However, the host is assumed to be fast enough to calculate the next 50 points before the controller has moved through the previous 50 points.

The controller executes the following program:

<pre> real Points(12) (50) int N,HSync,NBuf PVSPLINE (0,1,2,3,4,5,6), 10 WHILE Sync >= 0 TILL Sync IF Sync > 0 MPOINT (0,1,2,3,4,5,6), Points, Sync Sync = 0 END END ENDS STOP </pre>	Declare an array of 50 points. The host will write the coordinates and velocities to the array. Declare a synchronization variable (initiated to zero by default). Create PV for axes 0, 1, 2, 3, 4, 5 and 6. Points are given at 10-millisecond intervals. Continue until the host writes negative number to Sync. Wait until the points are received. Once the host has filled the Points array, it writes the Sync variable with a number of points written to the Points array. Sync greater than 0 indicates that the host has finished the point generation. Add points to the Points matrix. The controller informs the host that the next batch is expected. The motion through the points already received from the host has not completed, but the controller is ready to receive more points. End if. End while. (0,1,2,3,4,5,6) End PVSPLINE.
--	---

The program running on the host looks like the following pseudo code:

```

double HPoints(12) (50);
int N, HSync;
HANDLE Com;
open communication, start program in buffer NBuf of the controller;
while (Continue)
calculate N (<= 50) points in array Hpoints;
acsc_WriteReal(Com, NBuf, "Points" , 0, 11, 0, N-1, HPoints, 0) ;
acsc_WriteInteger(Com, NBuf, "Sync", -1, -1, -1, -1, &N, 0);

```

```
do
acsc_ReadInteger(Com, NBuf, "Sync", -1, -1, -1, -1, &HSync, 0);
while HSync;
reset Continue to zero if all points have been calculated;
end;
N = -1
acsc_WriteInteger(Com, NBuf, "Sync", -1, -1, -1, -1, &N, 0);
```

Synchronization between the host and the controller is provided by the **Sync** user variable. When the host has finished transmitting the next batch of points to the controller, it writes to **Sync** the number of points in the batch. The controller waits for non-zero **Sync** and then adds the points to the motion. When the controller has added the points, it writes zero to **Sync**, which signals to the host to transmit the next batch of points.

When the host comes to the end, it writes -1 to **Sync**, to indicate the end of the motion.

4.12 Multi-Axis Point-to-Point Motions With Smooth Transitions (SMOVE)

The ACSPL+ **SMOVE** command enables execution of a series of **PTP** motions with the ability to smoothly transition from one motion to the next before reaching an endpoint.

Example application workflow:

1. A multi-axis **PTP** motion command to an endpoint is issued and motion starts.
2. The controller or host application monitors the status of a device attempting to acquire a measurement during motion.
3. When the measurement is validated, a motion command to the next endpoint (may have been precalculated, or may be calculated at time of measurement validation) is issued.
4. Repeat steps 2 and 3 until all motions in the process are executed.

4.12.1 SMOVE

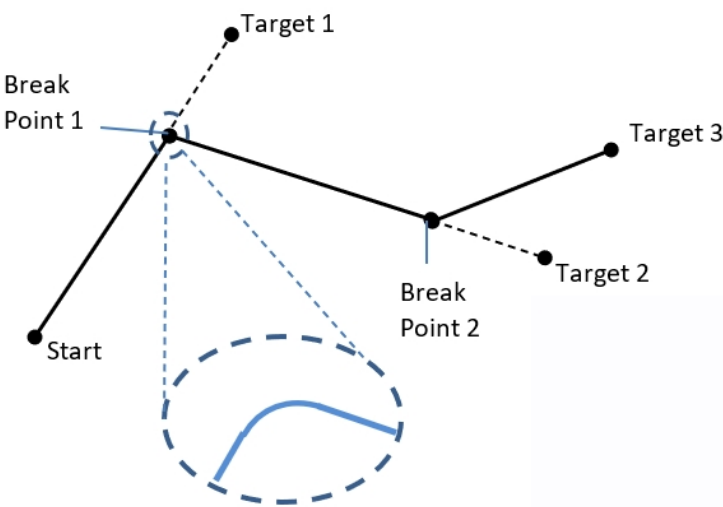
Description

The **SMOVE** command provides for positioning to specific target. **SMOVE** commands work in sequence and the next **SMOVE** command changes the previous target and provide a smooth transition from one motion direction to another, based on **ACC**, **DEC** and **JERK** values. The motion profile is optimized to pass on a rounded path near the breaking point, minimizing changes in speed and direction that would cause unwanted vibrations in the system.

Syntax

```
SMOVE[/switch] axis_list, target_point[, velocity]
```

Example



4.13 SmoothPath

SmoothPath is a new spline-based motion profile capability that provides smooth, constant speed, multi-axis motion. The 2-axis SmoothPath feature is available in SPiiPlus controllers with a MotionBoost license. 3+ axis SmoothPath is supported by special controller configurations only - please contact your ACS sales or support representative for more information.

SmoothPath motion may be implemented by ACSPL+ commands ([SmoothPath ACSPL+](#)) and by GSP Commands ([SmoothPath G-code](#)).

4.13.1 SmoothPath ACSPL+

4.13.1.1 SPATH

Description

Initiate a path smoothing motion.

Syntax

SPATH [/switches] (axis_list)), starting_coordinates[,velocity][,exc_angle][,exc_length][,segments_buffer][,motor_motion_delay]

Arguments

axis_list	Axes involved in the path smoothing motion, specified as axes numbers separated by comma or as axes' names separated by comma. A minimum of two axes must be specified.
starting_coordinates	A list of coordinate values, separated by commas. The list must specify one value for each axis in axis_list . This array specifies the starting position for the SPATH motion.

velocity	Optional, only used with /v switch Defines required feed rate. If not specified, the feed rate is derived from the leading axis parameters.
exc_angle	Optional, only used with /a switch The value defines exceptions from spline interpolation. If for an internal control point directions to the previous and the next control points require direction change more than the specified angle (by modulo), the control point is processed as a corner. Actually, such point divides the spline into two independent splines.
exc_length	Optional, only used with /l switch. The value defines exceptions from spline interpolation. If a distance between two control points appears longer than the specified length, the trajectory between the points is considered straight. Actually, two independent splines are built before and after the segment.
segments_buffer	[Optional] One-dimensional user-defined real array. The controller uses this array to store added segments. By default, if the argument is not specified, the controller allocates internal buffer for storing 50 segments only. The argument allows the user application to reallocate the buffer for storing a larger number of segments. The larger number of segments may be required if the spline is defined with a large number of closely specified control points; the case is typical when path smoothing is used to process minute segment prepared with a CAD system. The buffer is for the controller's internal use only and shouldn't be used by the user application
motor_motion_delay	(Optional, used only with /q switch) Delay, in milliseconds, before motor motion actually starts.

Switches

w	Do not start until the go command. If the switch is not specified, the motion starts once the first three spline points are specified.
v	Specify required velocity. The switch requires an additional parameter that specifies the required velocity. If the switch is not specified, the feed rate is derived from the leading axis parameters.

a	Specify exception angle. The switch requires an additional parameter that specifies maximum angle in a control point. The value defines exceptions from spline interpolation. If for an internal control point, directions to the previous and the next control points require direction change more than the specified angle (by modulo), the control point is processed as a corner. Actually, such point divides the spline into two independent splines.
l	Specify exception length. The switch requires an additional parameter that specifies maximum segment length. The value defines exceptions from spline interpolation. If a distance between two control points appears longer than the specified length, the trajectory between the points is considered straight. Actually, two independent splines are built before and after the segment.
G	Acceleration consideration. Allow the motion generator to deviate from the specified axes acceleration parameter during velocity profile generation.
q	Defines actual motor movement delay in microseconds. The delay resolution is 50 microseconds. The maximum delay is 100 controller cycles: 100ms for CTIME=1ms or 20ms for CTIME=0.2ms. Allows delaying actual motor movement start (RPOS) for the specified delay, while motion profile generation (APOS) starts immediately.

Return Value

None

Comments

This command is supported in version 3.10 and higher.

Example

```
! Start an SPATH motion from current position (0,0),
! velocity = 1000, wait for go command before starting motion
SPATH/VW(0,1), 0, 0, 1000

SEGMENT (0,1), 50, 0
SEGMENT (0,1), 100, 0
SEGMENT/f (0,1), 100, 100 , 500 ! velocity for segment = 500
SEGMENT/c (0,1), 200, 200 ! come to an exact stop at this point,
                        ! start a new spline after this point

SEGMENT/v (0,1), 300, 200, 2000 ! change motion vel to 2000
SEGMENT (0,1), 300, 300
SEGMENT (0,1), 400, 300
SEGMENT (0,1), 400, 400
```

```
GO (0,1) !start motion

ENDS (0,1)
STOP
```

4.13.1.2 SEGMENT

Description

Add a new control point to the **SPATH** motion generator

Syntax

SEGMENT[/switches](axis_list),coordinates[,velocity][,segment_velocity]

Arguments

axis_list	The list of axes must be the same as in the corresponding SPATH command.
coordinates	A list of coordinate values, separated by commas/ The list must specify one value for each axis in axis_list . The list defines coordinates of the end control point of the spline.
velocity	Optional, only used with /v switch. The value changes required velocity from this point forward. The new value is valid for all spline segments after the corresponding control point.
segment_velocity	Optional, only with the /f switch The required velocity for this segment only. At the end of the segment, the velocity reverts to the value previously in use.

Switches

/v	Specify new velocity. The switch is not compatible with /f . The switch requires an additional parameter that specifies the velocity value for the current and all subsequent segments.
/f	Specify required segment velocity. The switch is not compatible with /v . The switch requires an additional parameter that specifies the required segment velocity. The value defines the required velocity in the current segment, but does not change the required velocity for subsequent segments.
/c	Mark the current point as a corner. The control point is processed as a corner. Actually, such point divides the spline into two independent splines.

Return Value

None

Comments

This command is supported in version 3.10 and higher.

4.13.2 SmoothPath G-code

There are two general approaches to using the G-code interface:

- > Direct **NURBS** programming, parallel to ACSPL+ interface – allows for specification of all interpolation parameters
- > Indirect **NURBS** programming

The first option is mainly future-oriented, for that time when CAD/CAM programs produce **NURBS** programs. The second option is designed to use minute-segment programs of existing CAD/CAM software with minimal changes.

See the *GSP Reference Guide* for information on implementing SmoothPath Motion in G-code.

4.14 NURBS

NURBS stands for Non-Uniform Rational B-Spline.

SPiiPlus implements third order **NURBS** interpolation.

NURBS interpolation provides the following unique features:

- > Continues velocity and acceleration through the whole trajectory
- > Locality: one control point or knot has impact on restricted number of adjacent segments
- > Smoothing: **NURBS** tends to interpolate smoothly, without oscillations, loops, and cusps
- > Filtering: **NURBS** is an approximating spline, i.e. the spline does not go through the control points. Usually seen as disadvantage, the feature is actually useful when the control points contain some kind of noise.

NURBS is a generic spline; as special cases it may also encompass non-rational and uniform B splines.

4.14.1 NURBS Command

Description

Create a NURBS motion.

Syntax

NURBS[/switches] (axis_list)[,velocity][,exc_angle][,exc_length][,segments_buffer]

Arguments

axis_list	Axes involved in NURBS motion, specified as axes numbers separated by comma or as axes names separated by comma. Minimum two axes must be specified.
velocity	Optional, only used with switch /v.

	Defines required feed rate. If not specified, the feed rate is derived from the leading axis parameters.
exc_angle	Optional, only used with switch /a. The value defines exceptions from spline interpolation. If for an internal control point directions to the previous and the next control points require direction change more than the specified angle (by modulo), the control point is processed as a corner. Actually, such points divide the spline into two independent splines.
exc_length	Optional, only used with switch /l. The value defines exceptions from spline interpolation. If a distance between two control points appears longer than the specified length, the trajectory between the points is considered straight. Actually, two independent splines are built before and after the segment.
segments_buffer	[Optional] One-dimensional user-defined real array. The controller uses this array to store added segments. By default, if the argument is not specified, the controller allocates internal buffer for storing 50 segments only. The argument allows the user application to reallocate the buffer for storing larger number of segments. The larger number of segments may be required if the spline is defined with a big number of closely specified control point; the case is typical when NURBS is used for processing minute segments prepared with a CAD system. The buffer is for the controller's internal use only and shouldn't be used by the user application

Switches

/w	Do not start until the go command. If the suffix is not specified, the motion starts once the first three spline points are specified.
/v	Specify required velocity. The suffix requires an additional parameter that specifies required velocity. If the suffix is not specified, the feed rate is derived from the leading axis parameters.
/a	Specify exception angle. The suffix requires additional parameter that specifies maximum angle in a control point. The value defines exceptions from spline interpolation. If for an internal control point, directions to the previous and the next control points require direction change more than the specified angle (by modulo), the control point is processed as a corner. Actually, such point divides the spline into two independent splines.
/l	Specify exception length. The suffix requires an additional parameter that specifies maximum segment

	length. The value defines exceptions from spline interpolation. If a distance between two control points appears longer than the specified length, the trajectory between the points is considered straight. Actually, two independent splines are built before and after the segment.
G	Acceleration consideration. Allow the motion generator to deviate from the specified axes acceleration parameter during velocity profile generation.

Return Value

None

Comments



The first **NPOINT** command must define the starting point of the NURBS motion.

Example

```
Set FPOS(0) = 0
SET FPOS(1) = 0
ENABLE (0,1)
NURBS/V (0,1), 1000
npoint(0,1), 0, 0
npoint(0,1), 100, 0
npoint(0,1), 100, 100
npoint(0,1), 200, 100
npoint(0,1), 200, 200
npoint(0,1), 300, 200
npoint(0,1), 300, 300
npoint(0,1), 400, 300
npoint(0,1), 400, 400
ENDS (0,1)

STOP
```

4.14.2 NPOINT Command

Description

Add next control point and knot

Syntax

NPOINT[/switches](axis_list),coordinates,[velocity][,knot][,weight][,segment_velocity]

Arguments

axis_list	The list of axes must be the same as in the corresponding NURBS command.
coordinates	The list of coordinate values separated by commas. The list must specify one value for each axis in axis_list. The list defines coordinates of one

	control point of the spline.
velocity	Optional, only used with switch /v. The value changes the required velocity. The new value is valid for all spline segments after the corresponding control point.
knot	Optional, only used with switch /k. The value specifies a delta for the new knot calculation. The delta must be positive or zero. Each npoint command adds a new knot to the knot vector. The new knot is calculated as a previous knot plus delta (positive or zero). If suffix k is not specified, the delta default value is one; a new knot is calculated as the previous knot plus one. A special case occurs in the very first npoint command, if knots command was not specified, so that the previous knot does not exist. In this case, the knot parameter specifies the absolute value of the first knot; if switch /k is omitted, a zero value is assumed as the first knot. If none of the npoint commands specifies a /k switch and knots commands are omitted, the controller builds a uniform spline with evenly spread knots (except for the duplicated knots at the beginning and end).
Weight	Optional, only used with /w switch. The value specifies control point weight. Only positive weights are accepted. If a /w switch is not specified, the default value of weight is one. If none of the point commands specifies a /w switch, all control points have the same weight (one); in this case, the spline is actually a non-rational B spline.
segment_ velocity	Optional, only with /f switch The required velocity for this segment only

Switches

/v	Specify new required velocity. The switch is not compatible with /f. The switch requires an additional parameter that specifies the required velocity. The value is used as the required velocity for the current and all subsequent points.
/f	Specify segment velocity. The switch is not compatible with /v. The switch requires an additional parameter that specifies required velocity. The value is used as the required velocity for the current point, but does not change the required velocity for subsequent points.

/k	Specify knot delta. The switch requires an additional parameter that specifies the knot delta from the previous knot. The new knot is calculated as the previous knot plus delta.
/w	Specify control point weight. The switch requires an additional parameter that specifies the weight of the control point.
/d	Mark the point specification as dummy. Dummy point specifications can either precede the first control point specification, or follow the last control point specification. Dummy points specification is required in rare cases where default calculation of starting/trailing knots is not appropriate.
/c	Mark the current point as a corner. The control point is processed as a corner. Actually, such a point divides the spline into two independent splines.

Return Value

None



The first **NPOINT** command must define the starting point of the NURBS motion.

4.15 Open-Loop Operation (Torque Control)

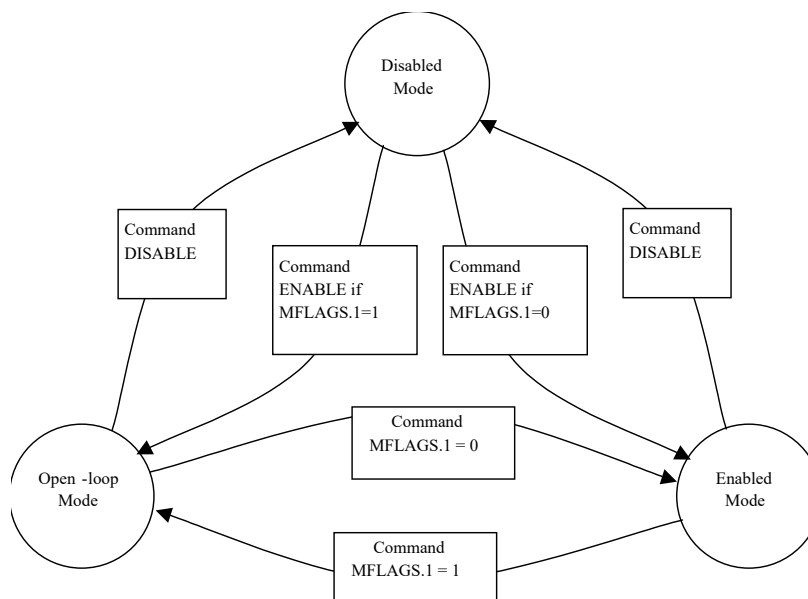
The open-loop motor mode is often referred as a torque-control mode. The following table summarizes differences between three motor modes:

Table 4-1. Motor Modes

Mode	Disabled	Open-loop	Enabled
State of the Drive Enable output	Off	On	On
Calculation of control loop algorithm	No	No	Yes

Mode	Disabled	Open-loop	Enabled
Correspondence of Reference Position (RPOS) and Feedback Position (FPOS)	RPOS follows FPOS	RPOS follows FPOS	RPOS is calculated according to the commanded motion. FPOS follows the RPOS as provided by control loop algorithm.
Position Error (PE)	Zero	Zero	The control algorithm calculates PE = RPOS - FPOS
Voltage at the drive output	Zero	Proportional to DCOM	As calculated by the control algorithm

Bit 1 in the **MFLAGS** variable enables or disables open-loop mode. The following diagram explains transition between the three modes:



The circles in this diagram represent the motor modes and the arrows with rectangles show the controller commands that switch the controller from one mode to another.

As shown in the diagram, a motor can be switched from the enabled mode to the open-loop mode and back without changing to the disabled mode. The controller provides a smooth transition between the modes. Even if the motor experiences uncontrolled movement while in the open-loop mode, switching back to the enabled mode does not cause any motor jump. However, be careful if you execute a motion while the controller is in open-loop mode. Once the command switches back to enabled mode, the controller continues the motion from the point where it finds the motor. No

motor jump occurs, but the motion trajectory and the final point may be shifted by the value of uncontrolled offset in the open-loop mode.

While in open-loop mode, the controller calculates the drive voltage output based on the **DCOM** variable. The **DCOM** variable sets the drive output as a percentage of the maximum available drive output voltage. For example, the UDI provides differential drive output in the range from -10V to +10V. Therefore, assigning 100 to **DCOM** provides +10V on the drive output, assigning -100 provides -10V, and assigning 0 provides 0V.

The following program fragment shows an example of torque control implementation through the open-loop mode.

<pre> ENABLE 0 PTP/f 0,400,100 JOG/v 0,100 TILL IN0.4; KILL 0; MFLAGS0.1=1; DCOM0=30 WAIT 50; DCOM0=10 WAIT 100; MFLAGS0.1=0; PTP 0,400 </pre>	Enable the 0 motor. Move to the point where the contact search begins. Provide low (search) velocity of 100 count/sec in the final point. Move to the contact point using the search velocity. Wait for the signal from the contact sensor. Kill the motion. Switch to open-loop mode. Apply 30% of the maximum torque. Wait 50 milliseconds and then change torque to 10% of the maximum torque. Wait 100 milliseconds, then switch off the open-loop mode and move away from the contact point.
--	---

4.16 Step Velocity Profile (Non-Zero Minimal Velocity)

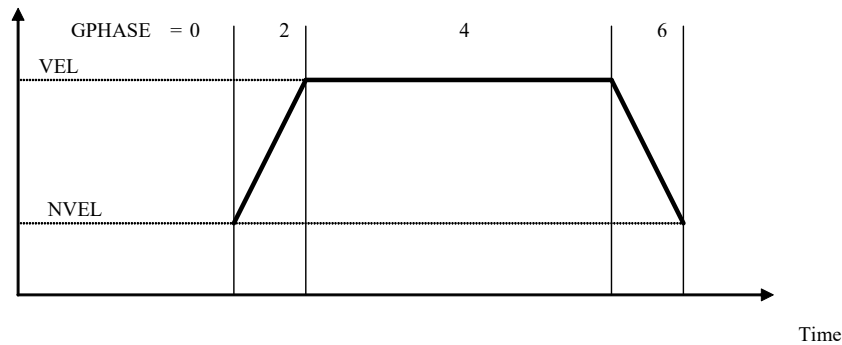
Many applications with step motors require the motion velocity to be restricted from below. In this case, the motion profile has non-zero start and finish velocity. To handle this case you can use the **NVEL** variable (see *SPiiPlus Command & Variable Reference Guide*) that specifies minimal velocity for each axis. If an application sets a non-zero **NVEL** for some axis, the controller uses the value as start and finish velocity in any motion related to the axis except for an axis that is governed by the **PATH** or **PVSPLINE** command.

4.16.1 The **NVEL** Variable

NVEL is an array of 8 real values, one per axis. A non-zero value in any **NVEL** element determines that the motion of the corresponding axis will be executed with non-zero start and finish velocities.

If an **NVEL** element is zero, the normal motion profile starts from zero velocity and finishes at zero velocity. If an element is non-zero, in the beginning of motion the velocity immediately jumps to the value specified in the **NVEL** element and then continues normal motion profile. In the end, the motion approaches the final point at velocity specified in the **NVEL** element, then the velocity immediately drops to zero.

In a typical application, the step motor does not require acceleration build-up phases; the motion profile is trapezoidal:

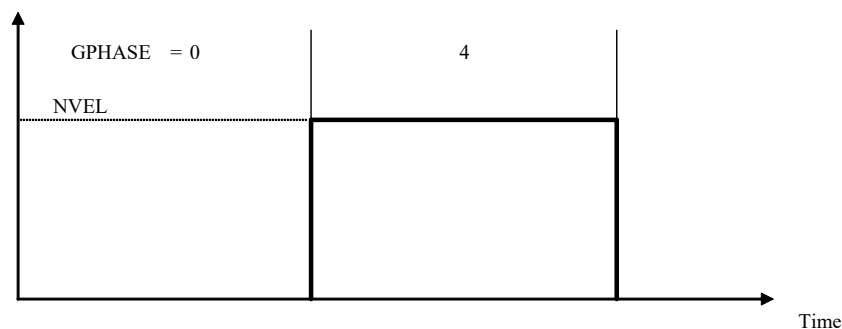


KILL and **HALT** commands are also affected, i.e., they slow down the velocity to the value specified in the **NVEL** element and then the velocity drops to zero.

4.16.2 Special NVEL Cases

4.16.2.1 Specified Velocity Less Than NVEL

If the absolute value of the **VEL** value is less than the **NVEL** value, the **VEL** value is ignored. In this case the motion profile is rectangular:



Also ignored are the velocity specified in a motion command or in an **IMM** command if its absolute value is less than **NVEL**. In all cases, the actual motion velocity is not less than **NVEL**.

4.16.2.2 Multi-Axis Motion

Multi-axis motion is not typical for an axis with non-zero minimal velocity. If, however, the axis is involved in multi-axis motion, the controller uses **NVEL** as follows:

- > If the axis is a leading axis in the motion, its **NVEL** is used as a minimal vector velocity in the motion profile
- > If the axis is not leading, its **NVEL** is ignored.

In both cases, the axis velocity can be lower than the **NVEL** value. Therefore, be careful in defining multi-axis motion that involves both stepper motors and servo motors.

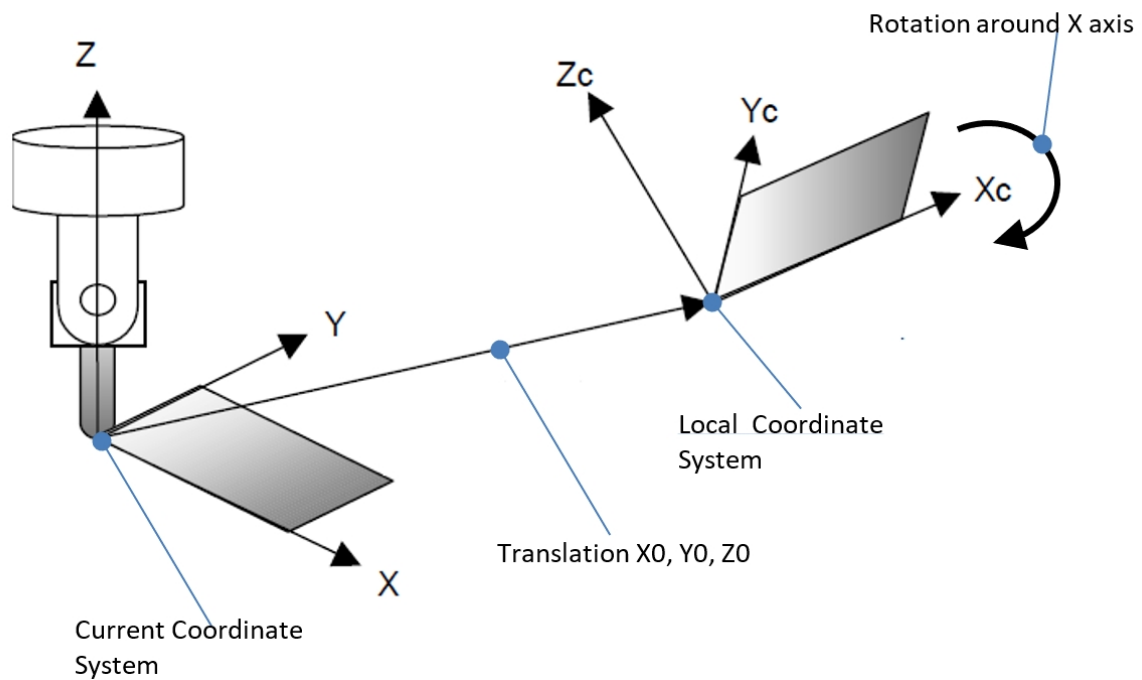
4.16.2.3 NVEL and Non-Default Connection

If a non-default connection is specified (see [Non-Default Connections](#)), the axis and the motor are different entities. In this case, **NVEL** refers to the axis, but not to the motor. For example, if **NVELO** is

set to 500, the velocity of any motion of the X axis will be limited from below to 500 units per second irrespective of which motor is affected by the motion.

4.17 Local Coordinate System

The Local Coordinate System (LCS) coordinate system is always a Cartesian coordinate system and assigned to the specific workpiece feature. The LCS is the result of an allowed transformation (translation and rotation) of the current coordinate system.



4.17.1 CSCREATE

Description

The **CSCREATE** command creates the new Local Coordinate System (LCS) relative to the Machine Coordinate System or the previous LCS, depending on the applied switches.

Syntax

```
CSCREATE[/r] axis_list, x_trans, y_trans[, rot_axis, rot_angle]
```

or

```
CSCREATE[/r] axis_list, x_trans, y_trans, z_trans[, rot_axis, rot_angle]
```

Arguments

axis_list	The group of 2 or 3 axes. Valid values are: 0, 1, 2 ... up to the number of axes in the system minus 1.
x_trans	The new X position of the LCS in user units
y_trans	The new Y position of the LCS in user units
z_trans	The Z position of the LCS in user units. This parameter is included when axis_list includes 3 axes.
rot_axis	(Optional) The rotation axis: 0 – X, 1 – Y, 2 – Z
rot_angle	(optional) Rotation angle value: (-3.14159 : +3.14159) in radians

Switches

/r	The new LCS is relative (additive) to the existing LCS (otherwise the new LCS is relative to the Machine Coordinate System).
-----------	--

Comments

- > In function calls which include rotation parameters, the translation parameters are applied to the system first, and then the rotation parameters.
- > The enumeration of axes for the **rot_axis** parameter is a numbered list of axes in the newly created coordinate system. This enumeration relates to the virtual axes, not the physical axes of the system.
- > This command is supported in version 3.10 and higher.

4.17.2 CSDESTROY

Description

The **CSDESTROY** command cancels the active Local Coordinate System and sets the Machine Coordinate System or previous Local Coordinate System.

Syntax

```
CSDESTROY axis_list[, restore_flag]
```

Arguments

axis_list	The group of 3 axes. Valid values are: 0, 1, 2 ... up to the number of axes in the system minus 1.
restore_flag	(Optional) Set to 1 to restore the previous LCS; 0 or omitted value restores the MCS.



Unpredictable and dangerous motion may result if **CSDESTROY** is called before the motion involving the coordinate system created by **CSCREATE** is complete.

Comments

This command is supported in version 3.10 and higher.

4.17.3 LCS Examples

Example 1

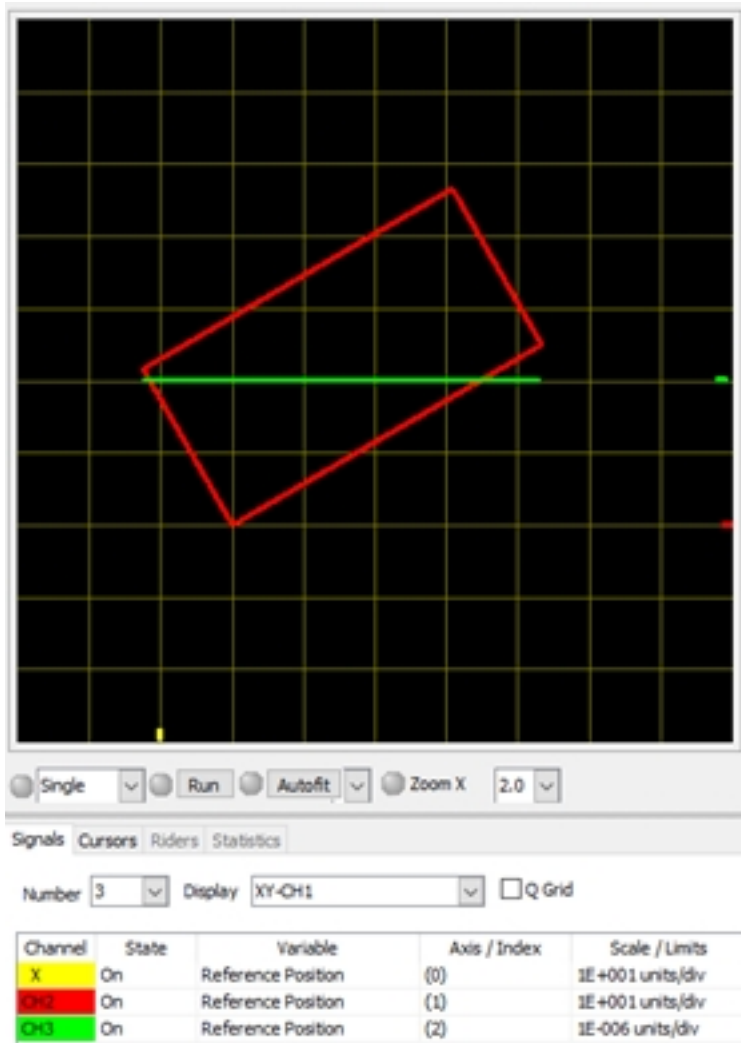
This example demonstrates rectangular motion in PTP mode.

```
!Create LCS with origin (10, 0, 0) and rotated 30 degrees around Z
REAL ang = 30* 3.141592 /180
CSCREATE (X,Y,Z), 10, 0, 0, 2, ang

PTP/ze (X,Y), 0, 0      !go to the beginning position

!Make rectangular motion
PTP/ze (X,Y),  0, 25
PTP/ze (X,Y), 50, 25
PTP/ze (X,Y), 50,  0
PTP/ze (X,Y),  0,  0

CSDESTROY (X,Y,Z)      !restore machine coordinate system
```



Example 2

This example demonstrates rectangular motion in MPTP mode.

```
!Create LCS with origin (10, 0, 0) and rotated 30 degrees around Z
REAL ang = 30* 3.141592 /180
CSCREATE (X,Y,Z), 10, 0, 0, 2, ang
PTP/ze (X,Y), 0, 0      !go to the beginning position
MPTP/z (X,Y)
    POINT (X,Y), 0, 25
    POINT (X,Y), 50, 25
    POINT (X,Y), 50, 0
    POINT (X,Y), 0, 0
ENDS (X,Y)
TILL GSEG(X) = -1      !Wait until motion complete
CSDESTROY (X,Y,Z)      !restore machine coordinate system
```

Example 3

This example demonstrates round rectangular motion in XSEG mode.

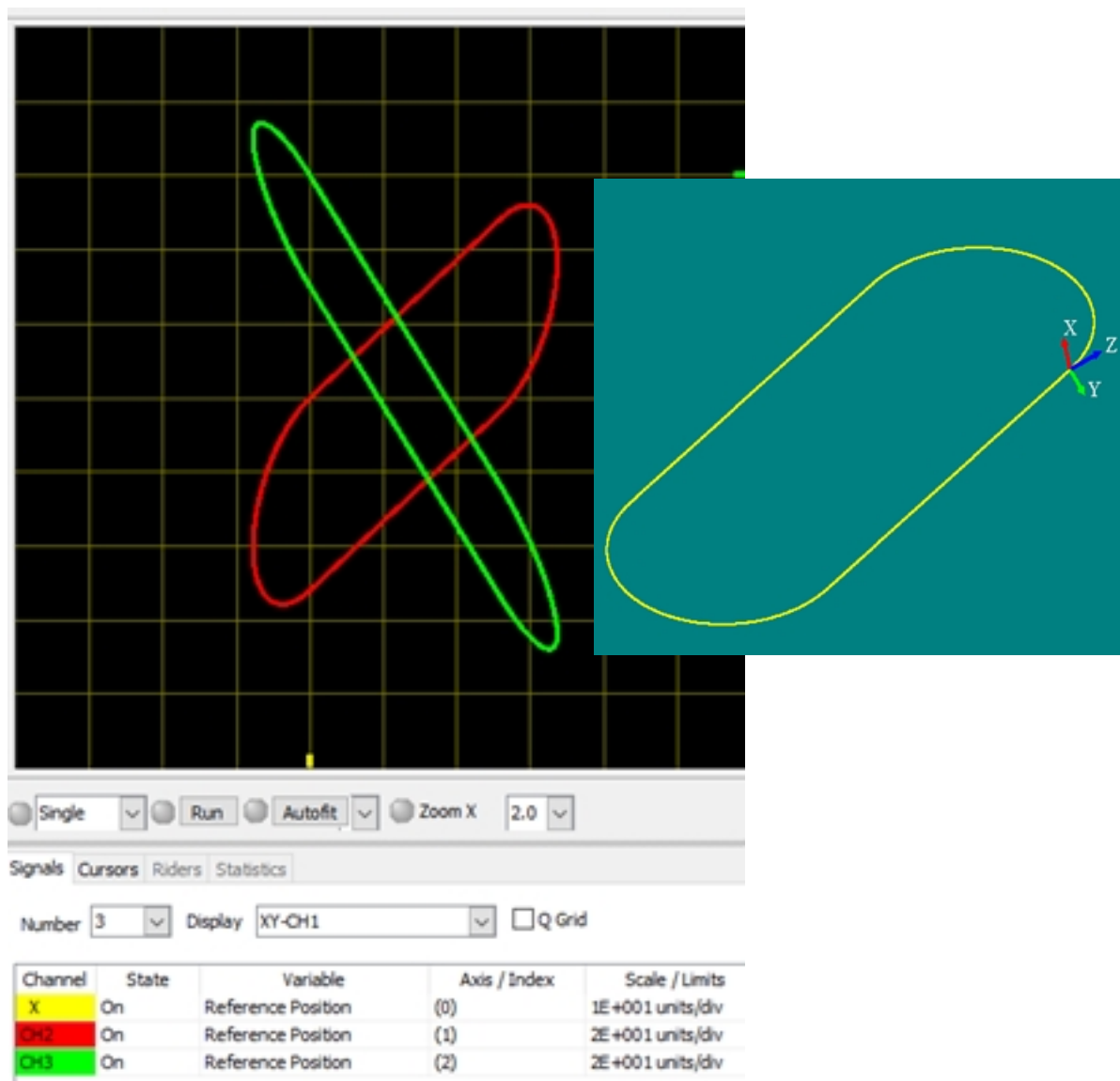
```

!Create LCS rotated 30 degrees around X
REAL ang = 30* 3.141592 /180
CSCREATE (X,Y,Z), 10, 0, 0, 0, ang
ang = 75* 3.141592 /180
CSCREATE/r (X,Y,Z), 0, 0, 0, 1, ang !Additional rotation 75 deg. Around Y

PTP/ze (X,Y), 0, 0      !go to the beginning position

XSEG/z (X,Y), 0, 0
    LINE (X,Y), 100, 0
    ARC2 (X,Y), 100, -30, -3.14159
    LINE (X,Y), 0, -60
    ARC2 (X,Y), 0, -30, -3.14159
ENDS (X,Y)
TILL GSEG(X) = -1      !Wait until motion complete
CSDESTROY (X,Y,Z)      !restore machine coordinate system

```

5. Inputs and Outputs

The controller includes digital and analog inputs and outputs. This chapter discusses the following:

- > General purpose digital inputs and outputs
- > General purpose analog inputs and outputs

Safety inputs and digital encoder implementations are discussed in other sections of this guide (see [Fault Handling](#)).

The controller provides a set of general-purpose inputs and outputs that have no predefined function. You can assign a function to any input/output as required by your application.

The exact number of general purpose digital inputs and outputs depends on the controller configuration.

5.1 Digital Inputs and Outputs

Digital Inputs - A digital input is a binary signal in the form of low or high voltage that the controller accepts from an external source such as a switch or a relay.

Digital Outputs - A digital output is a binary signal that the controller provides to an external acceptor such as a LED or actuator.

5.1.1 Addressing Digital I/Os

Digital Inputs are presented by the ACSPL+ read-only integer array variable: **IN**. Digital Outputs are presented by the ACSPL+ integer array variable: **OUT**.

Each member of the array is a bitmask of input or output states, respectively. During the system configuration process, array members are bound to a certain unit, according to amount of I/O that the unit supports.



When you are operating a system whose configuration is currently unknown, you can use the Terminal command: **#SI** (see *SPIIPlus ACSPL+ Command & Variable Reference Guide* for details on Terminal commands) to find out the correlation between the **IN/OUT** array indexes and I/O.

You address a digital I/O using the following format:

IN(PORT_#).BIT_# or **INPORT_#.BIT_#** – Input

OUT(PORT_#).BIT_# or **OUTPORT_#.BIT_#** – Output

Where:

IN	Integer array IN , a read-only array
OUT	Integer array OUT
port_#	The port number to which the bits belong. The controller's digital input/output ports are numbered from 0 to N-1, where N is the number of controller ports (see the controller's Hardware Guide for the number of input/output for your controller).
.bit_#	The specific bit within the port. Each port is divided into 32 bits that are numbered from 0 to 31. For example: IN0.1 – input 1 of port 0 IN3.19 – input 19 of port 3 OUT0.5 – output 5 of port zero OUT3.19 – output 19 of port 3

Rather than explicitly designating the port number, you can use an integer user-defined variable that equates to the number. In this case you have to include the parentheses, for example:

IN(U_VAR).1 - where **U_VAR** is an integer variable the value of which equates to the port number.



If the controller provides only 32 inputs or less, all inputs/outputs are located in port zero. In this case the port number can be omitted, and input is referred as: **IN.0** (for input 0), **IN.22** (for input 22), **OUT.0** (for output 0), **OUT.2** (for output 2), etc.

5.1.2 Querying Digital I/Os

The **IN** and **OUT** arrays can be queried like any other variable in the SPiiPlus MMI Application Studio **COMMUNICATION TERMINAL**. Each element of the array is read as a 32-bit binary number.

Example:

: ?IN0	What is the status of input 0
10111001,00011010,00000100,00000000	
: ?IN0.1,IN0.2	What is the status of bits 1 & 2 of input 0
0	
1	
: ?OUT(0,3)	What is the status of outputs 0 and 3

10111001,00011010,00000100,00000000 10111001,00011010,00000100,00000000
10111001,00011010,00000100,00000000 10111001,00011010,00000100,00000000

5.1.3 Assigning Outputs

You can assign only **OUT** elements. The **IN** array is read-only.

Each digital input/output is treated as one binary bit. The low voltage level corresponds to zero (or “clear”) and high voltage level corresponds to one or (“set”).

Examples of assignment to the elements of an **OUT** are:

<code>OUT0.1 = 0</code>	Set output OUT0.1 to zero
<code>OUT0.1 = 1</code>	Set output OUT0.1 to one
<code>OUT0.1 = V0</code>	If <code>V0 = 0</code> , set OUT0.1 to zero. Otherwise, set OUT0.1 to 1
<code>OUT0.15 = IN0.10</code>	Copy state of input IN0.10 to output OUT0.15
<code>OUT0.15 = ~ IN0.10</code>	Copy inverse state of input IN0.10 to output OUT0.15
<code>OUT6.1 = IN0.0 & IN0.1</code>	Set OUT6.1 to logical AND of inputs IN0.0 and IN0.1
<code>OUT0 = 0x0101</code>	Set signals OUT0.0 and OUT0.8 to one. Set all other bits of OUT0 to zero
<code>OUT0 = OUT0 0x0101</code>	Set signals OUT0.0 and OUT0.8 to one. Do not alter other bits of OUT0
<code>OUT0 = OUT0 & ~0x0101</code>	Set signals OUT0.0 and OUT0.8 to zero. Do not alter other bits of OUT0.
<code>OUT0 = (OUT0 & ~0x0101) (V1 & 0x0101)</code>	Copy bits 0 and 8 from V1 to OUT0. Do not alter other bits of OUT0

5.1.4 Digital I/O in Conditional Commands

Commands such as **if**, **while** and **till** are always followed by a logical expression. Using I/O in the logical expression provides program branching options that are I/O state-dependent.

Examples:

<code>if ^IN0.1 goto L</code>	Go to label L only if IN0.1 is zero
<code>while IN0.1 & IN0.2</code>	Execute the subsequent commands up to command end while both IN0.1 and IN0.2 are one
<code>till IN0.10</code>	Wait until IN0.10 becomes one
<code>till IN0 & 0x0101</code>	Wait until at least one of IN0.0 and IN0.8 becomes one.

5.1.5 PLC Implementation

The Programmable Logic Controller (PLC) is often used to manage digital inputs and outputs. SPiiPlus controllers provide implementation of PLC without separate PLC hardware. The techniques described in this section provide implementation of PLC functionality by the controller. This approach provides an easy integration of PLC program with motion control. For example, a motion can be started when a condition calculated by the PLC program is satisfied, and an output can be activated when a motion starts or terminates.

There are several options for implementing a PLC program:

- > Implement the PLC program in a separate buffer. This is the most suitable approach if the PLC program must not interfere with motion, and has few connections to motion programs. The PLC program runs in parallel with motion programs in other buffers, and any desired connections are provided via global variables.
- > Mix motion programs and PLC program in the same buffers. This approach provides a very close interaction between PLC and motion programs, resulting in faster reaction time, but in general has a more complex structure.
- > Split the PLC program into two different parts running in two different buffers. This approach is most suitable when a time-critical part of the program has to operate faster than the rest of the program. PLC programs run at either a fast or slow scanning rate, and you must assign a greater priority one buffer using the **PRATE** variable.
- > Implement a part of the PLC program as a set of autoroutines. This approach provides a very fast and interrupt-like response to critical conditions, because the autoroutine condition is checked each controller cycle.

The following is an example of a PLC program implemented in a separate buffer using autoroutines for fast response:

```
1.      real T1
2.      int Bits
3.      Start:
4.      OUT0.0 = MST0.#INPOS
5.      if T1 <= TIME
6.      if Bits.0  T1 = T1 + 30000  else  T1 = T1 + 15*60000  end
7.      Bits.0 = ^Bits.0
8.      end
9.      OUT0.4 = IN0.4 & Bits.0
10.     goto Start
11.     on IN0.15; killall; ret
```

Line1 – Definition of local variable **T1** that is used to store the next switch time. **T1** may be defined as integer, but as a real, it can provide continuous running for an extended period without overflow. The program relies on the automatic initialization of all local variables to zero when they are declared.

Line2 – Definition of a local variable: **Bits**. In this program only one bit of **Bits** is used. One temporary integer variable can be used for storing 32 temporary bits.

Line3 – A label: **Start**. A typical case in PLC programming is a long program cycle that executes to the end and returns to the beginning. In the example shown above, the execution period is quite short

even with default rate of 'one Line per each controller cycle'. In a long program, the execution cycle can reach hundreds of milliseconds.

This is a good reason to divide a typical PLC program into slow and fast sections.

Line 4 – **OUT0.0** reflects the 'in position' state of the motor. If the motor is not in position, the output is 0. If it is in position, the output is 1.

Lines 5-9 – **OUT0.4** controls a periodic activity that must be executed every 15 minutes for a 30-second period. It is executed only if **IN0.4** is active. In a typical application, the output might be connected to lubrication pump.

Line10 – Returns the motion to the **Start** point.

Line11 – An autoroutine that provides extra fast response to **IN0.15**, typically an emergency input. The whole autoroutine is implemented in one line providing an immediate kill of all motions within one controller cycle when input port 0 bit 15 is 1.

5.1.6 Digital I/O in Autoroutines

You may use digital I/O as conditions for autoroutines (see [Autoroutines](#)). The autoroutine can be very useful for PLC implementation and fault handling.

For example:

```
ON IN0.0           ! When input#0=0
OUT0.4=1          ! Set output#4 to 1
disp "Activates motor"
RET              ! Ends the autoroutine
```

5.1.7 Using HSSI I/O Extension

Use the High-Speed Synchronous Serial Interface (HSSI) channels available in the SPiiPlus controllers for connecting additional inputs and outputs. ACSPL+ supports access to HSSI through the standard arrays **EXTIN** and **EXTOUT**.

The arrays can be queried, indexed and used in expressions like other ACSPL+ variables. For detailed information about the HSSI interface see the *HSSI Modules Hardware Guide*.

5.1.8 SPI Communication

An SPI interface connected to the DSP is supported by ACSPL+ Commands in the IDMsM and ECMsM devices.

5.1.8.1 ACSPL+ Commands and Variables for SPI

The following ACSPL+ Commands and variables are used in implementing an SPI interface.

5.1.8.1.1 SPICFG

Description

SPICFG configures and initializes the SPI interface.

Syntax

SPICFG (SlaveIndex, Mode, NumberOfWords, Polarity,Size,Frequency)

Arguments

SlaveIndex	Index of the EtherCAT slave in the EtherCAT network, or 0 in case of IDMsM/ECMsM/UDMsM
Mode	The mode of the SPI interface. The following modes are supported: > 0 - Slave > 1 - Master > 2 - SlaveListenOnly > 3 - Disable > 4 - Master Single Transaction (Used by ACSPL+ SPIWRITE)
NumberOfWords	Number of SPI Data Words used by the application (FW to SPI). Range: {0,8} Not relevant in case of Master Single Transaction Mode
Polarity	Clock Polarity. Four types are available: > Rising Edge – 0 > Rising Edge with Delay – 1 > Falling Edge – 2 > Falling Edge with Delay - 3
Size	Data size in bits, the range is {1-16}.
Frequency	An integer number which defines the frequency. The range is 200KHz-10MHz, limited values are supported (defined in the table below).

Frequency Values Supported

Value	Frequency (kHz)
2	800
3	1000
4	1500
5	2000
6	2500
7	3000
8	3500

Value	Frequency (kHz)
9	4000
10	5000

Return Value

None

Comments

When the SPI interface is not required anymore, SPICFG should be called with Mode=3 (disable) parameter.

Example**5.1.8.1.2 SPIRXN****Description**

SPIRXN is a variable that shows the number of actual words that contain data transmitted from the SPI external interface. The range is from 0 to 8.

Accessibility

Read-only

Tag

384

Comments

SPIRXN value is 0 if the SPI is disabled. This variable is supported by the UDMxx, IDMxx and ECMxx products only.

5.1.8.1.3 EXTIN**Description**

EXTIN is an integer array, the size of which is determined by the total number of SPI input signals in the system, and reads the current state of the inputs. The number of inputs depends on the number of SPI modules in the system.

Comments

The **SPIRXN** variable is updated every cycle with the number of active elements.

Tag

42

Accessibility

Read-Only

Related ACSPL+ Variables**EXTOUT, IN****COM Library Methods and .NET Library Methods**

ReadVariable, GetExtInput, GetExtInputPort

C Library Functions

acsc_ReadInteger, acsc_GetExtInput, acsc_GetExtInputPort

*5.1.8.1.4 EXTOUT***Description**

EXTOUT is an integer array, the size of which is determined by the total number of SPI output signals in the system, which can be used for reading or setting the current state of the outputs. The number of outputs depends on the number of SPI inputs in the system.

Syntax**EXTOUT(*index*) = *value*****Arguments**

<i>index</i>	A number between 0 and 511.
<i>value</i>	value ranges from -2147483648, 2147483647, Default = 0.

Comments

When used with an SPI interface in master mode, the **EXOUT** function should be used.

The **SPICFG** function sets the number of elements which contain data.

Tag

43

Accessibility

Read-Write

Related ACSPL+ Variables**EXTIN, OUT****COM Library Methods and .NET Library Methods**

ReadVariable, WriteVariable, GetExtOutput, SetExtOutput, GetExtOutputPort, SetExtOutputPort

C Library Functions

acsc_ReadInteger, acsc_WriteInteger, acsc_GetExtOutput, acsc_SetExtOutput, acsc_GetExtOutputPort, acsc_SetExtOutputPort

5.2 Analog Inputs and Outputs

The controller provides a set of analog inputs and outputs. This section discusses the general purpose analog I/Os whose number is controller dependent.

- > **Analog input** - accepts analog signal from an external source, such as a sensor or a potentiometer.
- > **Analog output** - provides analog signal to an external receiver, such as an actuator or a measuring device.

Analog inputs/outputs have no predefined function in the controller. You can connect signals to inputs/outputs and process them as required by the application.

5.2.1 Addressing Analog I/Os

Analog Inputs are presented by the ACSPL+ read-only real array variable: **AIN**. Analog Outputs are presented by the ACSPL+ real array variable: **AOUT**.

Each member of the array is a value of input or output, respectively. During the system configuration process, array members are bound to a certain unit, according to the number of inputs or outputs supported by the particular.



When you are operating a system whose configuration is currently unknown, you can use the Terminal command: **#SI** (see *SPiiPlus ACSPL+ Command & Variable Reference Guide* for details on Terminal commands) to find out the correlation between the **AIN/AOUT** array indexes and I/O.

You address an analog I/O using the following functions:

```
value = AIN(index)
AOUT(index) = value
```

Where:

AIN	Real array AIN , a read-only array
AOUT	Real array AOUT
index	A number between 0 up to the maximum number of analog input signals minus one.

The value returned is scaled by percent and ranges from -100 to +100.



If an analog output is connected to a drive, it has a dedicated destination and cannot be used as a general purpose analog output.

For model-dependent analog I/O information (for example, the number and range of inputs and outputs) see the controller's Installation Guide.

5.2.2 Assigning Analog Outputs

You can assign any variable (ACSPL+ or user-defined) to an entity of **AOUT**. For example, ACSPL+ variables:

- > **FPOS** – Feedback position

- > **FVEL** – Feedback velocity
- > **FACC** – Feedback acceleration



FVEL is calculated by digital differentiation of **FPOS**.

FACC is calculated by digital differentiation of the **FVEL** variable.

The **ACSPL+ FVFIL** variable defines a power of the smoothing filter used in the **FVEL** calculation (see *SPiiPlus Command & Variable Reference Guide*).

Example:

The following ACSPL+ program assigns X-axis feedback position to output #3 and acceleration to output #1. In the example the user variables: **SF1** and **SF2** are scale factors.

```
real SF1, SF2;  
SF1=0.01 ; SF2=0.001;           ! Define scaling factors  
While 1;  
  AOUT3 = FPOS0 * SF1;  
  AOUT1 = FACC0 * SF2;  
END
```

If the acceleration signal is too noisy, increase **FVFIL**.

6. Fault Handling

Fault handling is not only a concern for isolating motion faults, it is also a vital concern to personal and equipment safety.

Safety features are necessary to protect both the equipment and you from potential injury. SPiiPlus controllers include numerous safety-related features, but the final responsibility for the safe use of the controller in a particular application lies with you. Before you create your application make sure that you thoroughly read and understand this chapter.

This chapter addresses:

- > Safety Controls
- > Working with Fault Procedures

6.1 Safety Control



Some alarms, limits, and errors involve protection against potentially serious bodily harm and equipment damage. Be aware of the implications before changing or disabling any alarm, limit, or error.

Safety control is one of the highest-priority tasks of the controller. The controller continually monitors safety conditions each controller cycle, in parallel to its other activities.

The controller sets one of the fault bits of the ACSPL+ **FAULT** variable when it detects a malfunction. The response to a fault may vary from sending a fault message to complete termination of all activities. For each fault type you can enable/disable the default controller response or define your own autoroutine response.

6.1.1 Types of Malfunctions

The controller monitors numerous safety conditions that may indicate different hardware or software malfunctions (faults). The most frequent causes of faults are:

Table 6-1. Types of Malfunctions

Type of Fault	Examples
User error	Defining a required velocity that is invalid or beyond the limits.
Improper or broken wiring	A loose connection in the feedback encoder wiring.
Power amplifier malfunction	The power amplifier malfunctions and sends a fault signal to the controller.
Motor malfunction	A motor overheats.

Type of Fault	Examples
Controlled plant malfunction	The Emergency Stop input is activated.
Controller hardware malfunctions	The Main Processor Unit (MPU) and the Servo Processors (SPs) work together to detect malfunctions in the controller. Examples include the servo processor alarm and the servo interrupt.

6.1.2 How the Controller Detects Malfunctions

To detect malfunctions the controller monitors safety inputs, such as limit switches, and internal safety conditions, such as comparing the reference position with the software limits.

Internal safety conditions may consist of a static formula, such as checking the acceleration limit, or may include time dependencies, such as measuring the time interval between two interrupts to detect servo interrupt faults.

Some safety conditions are a mixture of both techniques. For example, position error control is based on both a static condition, whether a motor is positioned at a certain location, and a time dependency, how long the motor is positioned at the location.

6.1.3 Faults

When the controller detects a malfunction, it raises a specific fault bit. Fault bits are grouped into ACSPL+ **FAULT** and **S_FAULT** variables.

In certain cases, you may want to define which fault conditions are examined in a specific application. The ACSPL+ **FMASK** and **S_FMASK** variables specify which fault conditions must be examined in a particular application.

See *SPiiPlus Command & Variable Reference Guide* for complete details of these variables.

6.1.3.1 The FAULT Variable

The **FAULT** variable is an integer array containing eight elements (corresponding to the number of motors), where each element is made up of a set of bits. Each bit indicates a motor fault. Motor faults are related to a specific motor, power amplifier, or Servo Processor. Examples include tracking error and motor overheat.

6.1.3.2 The S_FAULT Variable

S_FAULT is a scalar variable, where each bit represents the aggregate status of a particular fault. The bits of **S_FAULT** are divided into two categories:

- > Aggregated motor faults – Once the controller raises a bit in any element of **FAULT**, it immediately raises the corresponding bit of **S_FAULT**. Therefore, each bit of **S_FAULT** is an OR aggregate of the corresponding bits in all elements of **FAULT**.
- > System faults such as Emergency Stop and Time Overuse that are not related to any specific motor.

6.1.4 Controller Response

The controller response to a fault can vary according to the requirements of your application:

- > No response.
- > Default response – One or more predetermined actions. You can disable the default response for any fault.
- > Autoroutine response – User-defined actions implemented in an autoroutine. In the autoroutine you select a controller fault and controller responses to the fault. An autoroutine can replace the default response or supplement it with additional actions.

6.2 Safety Control Summaries

A fault is a critical error for which the controller provides a default response. You can control the response: deactivating it or changing it.

6.2.1 Summary of Faults and Default Responses

A fault can be either a motor or system fault. Motor faults refer to a specific motor, power amplifier, or Servo Processor and affect the state of the corresponding bit for that element of the **FAULT** variable. System faults do not refer to a specific motor. The corresponding bits are located in the **S_FAULT** variable.

Faults may be simulated for purposes of testing and debugging by use of the **FAULTSIM** variable. Setting any of the bits in the variable actuates the corresponding fault.

For most controller-detected faults there is a default response, which is normally executed automatically when the fault occurs. Your ACSPL+ application can simply allow the default response for a fault or you can do either or both of the following:

- > Disable the default response by using ACSPL+ variables **FDEF** and **S_FDEF**.
- > Create an autoroutine (implementing your preferred response) activated by the occurrence of the fault. If desired, you can leave the default response enabled so that it will execute together with the autoroutine.

In addition to the above, each motor fault is either:

- > Latched in the **MERR** variable (see *SPiiPlus Command & Variable Reference Guide*) indicating that the motor is disabled or killed, in which case you can build a little routine that checks the value of **MERR** and resets it by running the **FCLEAR** or **ENABLE (ENABLEALL)** command, or
- > Set in the **MERR** variable only so long as the fault condition exists.



Right Limit restricts the motion in the positive direction and the Left Limit in the negative direction.

The faults reported by the ACSPL+ fault variables, as well as those latched in **MERR** are:

Table 6-2. Faults and the Controller's Default Response

Bit	Designator	Type	Fault Description	Default Response
0	#RL	Motor	RIGHT LIMIT. The controller raises the fault bit when the right limit switch is activated.	The controller kills the violating motor. As long as the fault is active, the controller kills any motion that tries to move the motor in the direction of the limit. Motion to return to the allowed range of motion is allowed.
1	#LL	Motor	LEFT LIMIT. The controller raises the fault bit when the left limit switch is activated.	The controller kills the violating motor. As long as the fault is active, the controller kills any motion that tries to move the motor in the direction of the limit. Motion to return to the allowed range of motion is allowed.
2	#NT	Network	NETWORK ERROR. The controller raises the fault bit when a loss of network is detected.	The controller disables all axes until a valid network Sync signal is received.

Bit	Designator	Type	Fault Description	Default Response
5	#SRL	Motor	SOFTWARE RIGHT LIMIT. The controller raises the fault bit when the motor reference position (RPOS) is greater than the software right limit margin (SRLIMIT).	The controller kills the violating motor. As long as the fault is active, the controller kills any motion that tries to move the motor in the direction of the limit. Motion in the direction away from the limit is allowed.
6	#SLL	Motor	SOFTWARE LEFT LIMIT. The controller raises the fault bit, when the motor reference position (RPOS) is less than the software left limit margin (SLLIMIT).	The controller kills the violating motor. As long as the fault is active, the controller kills any motion that tries to move the motor in the direction of the limit. Motion in the direction away from the limit is allowed.
7	#ENCNC	Motor	ENCODER NOT CONNECTED. The controller raises the fault bit when the primary encoder is not connected.	The controller disables the violating motor. The error code is latched in the MERR variable and remains active until you resolve the problems and enable the motor again or issue the FCLEAR command.

Bit	Designator	Type	Fault Description	Default Response
8	#ENC2NC	Motor	ENCODER 2 NOT CONNECTED. The controller raises the fault bit when the secondary encoder is not connected.	No default response. The error code is latched in the MERR variable, and remains active until you resolve the problems and enable the motor again or issue the FCLEAR command.
9	#DRIVE	Motor	DRIVE ALARM. The controller raises the fault bit when the signal from the drive reports a failure.	The controller disables the violating motor.
10	#ENC	Motor	ENCODER ERROR. The controller raises the fault bit when the primary encoder malfunctions.	The controller disables the violating motor. The error code is latched in the MERR variable and remains active until you resolve the problems and enable the motor again or issue the FCLEAR command.
11	#ENC2	Motor	ENCODER 2 ERROR. The controller raises the fault bit when the secondary encoder malfunctions.	The controller disables the violating motor. The error code is latched in the MERR variable and remains active until you resolve the problems and enable the motor again or issue the FCLEAR command.

Bit	Designator	Type	Fault Description	Default Response
12	#PE	Motor	NON-CRITICAL POSITION ERROR. The controller raises the fault bit when the position error (PE) limit is exceeded. The limit depends on the motor state and is defined by the following variables: ERRI if the motor is idle (not moving) ERRV if the motor is moving with constant velocity ERRA if the motor is accelerating or decelerating	None.
13	#CPE	Motor	CRITICAL POSITION ERROR. The controller raises the fault bit when the position error (#PE) exceeds the value of the critical limit. Whereas #PE errors occur during normal operation, #CPE is assumed to occur outside normal operation parameters and #CPE is greater than #PE.	The controller disables the violating motor.

Bit	Designator	Type	Fault Description	Default Response
			The critical limit depends on the motor state and is defined by the following variables: CERRI if the motor is idle (not moving) CERRV if the motor is moving with constant velocity CERRA if the motor is accelerating or decelerating	
14	#VL	Motor	VELOCITY LIMIT. The controller raises the fault bit when the absolute value of the reference velocity (RVEL) exceeds the limit defined by the XVEL variable.	The controller kills the violating motor.
15	#AL	Motor	ACCELERATION LIMIT. The controller raises the fault bit when the absolute value of the reference acceleration (RACC) exceeds the limit defined by the XACC variable.	The controller kills the violating motor.

Bit	Designator	Type	Fault Description	Default Response
16	#CL	Motor	CURRENT LIMIT. The controller raises the fault bit, when the RMS current calculated in the Servo Processor exceeds the limit value defined by either the XRMS , XRMSD , or XRMSM variable.	The controller disables the violating motor.
17	#SP	Motor	SERVO PROCESSOR ALARM. The controller raises the fault bit when the axis Servo Processor loses its synchronization with the main processor. The fault indicates a fatal problem in the controller.	The controller disables the violating motor. The error code is latched in the MERR variable and remains active until you resolve the problems and enable the motor again or issue the FCLEAR command.
20	#HSSINC	Motor	HSSI NOT CONNECTED. The controller raises the fault bit if the HSSI expansion module is not connected.	None.
23	#EXTNT	External Network Error	There is no EtherCAT device connected.	The controller disables all axes.
24	#TEMP	MPU Overheat	Activated at CPU temperature > 90°C or System temperature > 70°C	S_SETUP.#USGTEMP = 0: No response S_SETUP.#USGTEMP = 1: Default response is to disable all axes

Bit	Designator	Type	Fault Description	Default Response
25	#PROG	System	PROGRAM FAULT. The controller raises the fault bit when a run time error occurs in one of the executing ACSPL+ programs.	The controller kills all motors.
26	#MEM	System	MEMORY FAULT. The user application requires too much memory.	The controller kills all motors.
27	#TIME	System	TIME OVERUSE. The user application consumes too much time in the controller cycle.	No default response.
28	#ES	System	EMERGENCY STOP. The controller raises the fault bit when the ES signal is activated.	The controller disables all motors.  It does not stop the program buffers.
29	#INT	System	SERVO INTERRUPT. The servo interrupt that defines the controller cycle is not generated. The fault indicates a fatal controller problem.	The controller disables all motors.

Bit	Designator	Type	Fault Description	Default Response
30	#INTGR	System	INTEGRITY VIOLATION. The controller raises the fault bit when an integrity problem is detected.	No default response

6.2.2 Summary of Safety Inputs

Safety inputs and internal safety conditions are the building blocks for safety control. Safety inputs receive binary signals (low, represented by "0", or high voltage, represented by "1"), from external sources such as a switch or a relay. Unlike general-purpose inputs that have no predefined function, each safety input is dedicated to specific function.

There are six different motor safety inputs and one system safety input. The controller provides a complete set of safety inputs for each motor. For example there are eight left limit inputs: one per motor.

The state of the motor safety inputs is stored in the ACSPL+ **SAFIN** variable, while the current state of the Emergency Stop input is stored in the ACSPL+ **S_SAFIN** variable. A high level of a physical signal (voltage) raises the corresponding bit and a low level drops the corresponding bit.

The safety inputs occupy the same bit numbers in **SAFIN** and **S_SAFIN** as the corresponding faults in **FAULT** and **S_FAULT**. Therefore, the same constants are used for bit addressing.

The physical signal connected to a safety input may indicate a safety violation with either high or low level. For instance on one axis, the right limit switch may indicate a safety violation with high voltage, and the left limit switch with low voltage. Use the ACSPL+ **SAFINI** and **S_SAFINI** variables to define which level is active, thereby eliminating the need for hardware inverters.

Table 6-3. Safety Inputs

Bit	Fault	Fault Category	Fault Description
0	#RL	Motor	RIGHT LIMIT SWITCH
1	#LL	Motor	LEFT LIMIT SWITCH
9	#DRIVE	Motor	DRIVE ALARM - alarm signal from a drive.
28	#ES	System	EMERGENCY STOP - alarm signal from the controlled plant.

6.2.3 Summary of Safety-Related Variables

The **FAULT**, **S_FAULT**, **SAFIN**, **S_SAFIN** variables are read-only (**SAFIN**, **S_SAFIN** can be assigned values, but these apply only when the Simulator is used). The **SAFINI**, **S_SAFINI**, **FMASK**, **S_FMASK**,

FDEF, **S_FDEF** variables are protected and can be assigned only in protected mode (see [Application Protection](#)).

Table 6-4. Safety Related Variables

Name	Size	Access	Remarks
FAULT	8 (one per axis)	Read-only	MOTOR FAULTS. Each motor fault occupies one bit. Not all bits are occupied by faults. Only those bits that correspond to motor faults are meaningful.
FDEF	8 (one per axis)	Read-write (protected mode)	FAULT DEFAULT MASK. The variable bits control availability of the default responses to motor faults. The default value for all the bits, 1, enables the default response. If a bit is 0, the default response is disabled. Only those bits that correspond to motor faults are meaningful.
FMASK	8 (one per axis)	Read-write Protected	MOTOR FAULT MASK. The variable bits control whether the controller checks for motor faults. The default value 1 causes the controller to check for the fault associated with that bit. Only those bits that correspond to motor faults are meaningful.
S_FAULT	Scalar	Read-only	SYSTEM FAULTS. Each system fault and each aggregated motor fault occupies one bit. Only those bits that correspond to the faults are meaningful.
S_FDEF	Scalar	Read-write (protected mode)	SYSTEM FAULT DEFAULT MASK. The variable bits control availability of the default responses to system faults. The default value for all the bits, 1, enables the default response. If a bit is 0, the default response is disabled.

Name	Size	Access	Remarks
S_FMASK	Scalar	Read-write (protected mode)	SYSTEM FAULT MASK. The variable bits control whether the controller checks for system faults. The default value 1 causes the controller to check for the fault associated with that bit. Only those bits that correspond to system faults are used.
S_SAFIN	Scalar	Read-only (read/write for Simulator)	SYSTEM SAFETY INPUTS. Bit #ES reads the current state of the Emergency Stop input. Other bits are meaningless.
S_SAFINI	Scalar	Read-write (protected mode)	SYSTEM SAFETY INPUTS INVERSION. Bit #ES defines which value of S_SAFIN.#ES bit causes a fault. Other bits are not used.
SAFIN	8 (one per axis)	Read-only (read/write for Simulator)	MOTOR SAFETY INPUTS. Each meaningful bit reads the current value of a motor safety input. Only those bits that correspond to the motor safety inputs are meaningful.
SAFINI	8 (one per axis)	Read-write (protected mode)	MOTOR SAFETY INPUTS INVERSION. A bit of the variable defines which value of the corresponding SAFIN bit causes a fault. Only those bits that correspond to the meaningful SAFIN bits are used.
STODELAY	8 (one per axis)	Read-write	Used to configure the delay time between the STO fault indication and the default response (disable) to the fault.

6.2.4 Integrity Control

Integrity Control validates the firmware and the user application stored in the controller. The following groups of files are stored in the internal file system of the nonvolatile memory:

- > Firmware: files SB4.EXE, SB4.BIN, SBAUTO.BT
- > Default configuration values: files PAR.### and PARn.### , where n = 0, 1...
- > Default ACSPL+ programs: files ACSPLnn.###, where nn = 00, 01, 02...
- > Default Servo Processor programs: files SP.### and/or SPn.###, where n = 0, 1...

- > Saved configuration values: files PAR.\$\$\$ and PARn. \$\$\$, where n = 0,1...
- > Saved ACSPL+ programs: files ACSPLnn. \$\$\$, where nn = 00, 01, 02...

Firmware and the default files are present in the controller from the beginning and can be replaced only by the **Upgrade and Recovery Wizard** of the SPiiPlus MMI Application Studio.

The saved files compose the user application. Saved files are created or replaced by the memory management commands (See *SPiiPlus Command & Variable Reference Guide*).

Integrity Control is active for the all files specified above. The controller stores the size and checksum of each file, existing or created. The controller then compares the stored size/checksum with size/checksum of the actual file to expose damaged files. Validation is performed automatically on power-up. After power-up you can use the **IR** command to validate files (see [Integrity Report Command](#)).

6.2.4.1 Integrity Violation Fault

The bit of the Integrity Violation fault resides in the **S_FAULT** variable, and can be addressed as: **S_FAULT.#INTGR** or **S_FAULT.30**.

The fault has no default response. The masks **S_FMASK** and **S_FDEF** do not affect processing of the bit.

The controller automatically validates integrity on power-up before loading the user application. Therefore, you are able to define an **AUTOEXEC** program that checks the Integrity Violation fault and reports the error as required.

6.2.4.2 Integrity Report Command

The **#IR** Communication Terminal command activates integrity validation and provides a report of current integrity state.

If any integrity problem is detected, the command raises fault bit S_FAULT.#INTGR.

The report displays a list of files. Each list entry displays a file name, expected file size and checksum of the file and actual file size and checksum.

The following is an example of an integrity report::

#IR				
	Size		Checksum	
	Registered	Actual	Registered	Actual
C:\				
sb1218pc.frm	001DA050	001DA050	DF4F97F0	DF4F97F0
model.inf	000014C3	000014C3	C5CC6B93	C5CC6B93
array.txt	00000010	00000010	00D4FF89	00D4FF89
1.prg	000002CB	000002CB	D19BF636	D19BF636
ECAT.XML	00052B0A	00052B0A	4D35D447	4D35D447
C:\SB4\SP\				
sp.###	0004FF0C	0004FF0C	BCBB37F5	BCBB37F5
ADJ0. \$\$\$	0000017D	0000017D	8E1A3690	8E1A3690
sp.##1	0003DC8E	0003DC8E	0B678F5D	0B678F5D
c:\sb4\startup\				
Acspl_e. \$\$\$	0000010B	0000010B	6C441150	6C441150
Par. \$\$\$	000001E9	000001E9	704245FA	704245FA

Par0. \$\$\$	00000D7C	00000D7C	21F7589A	21F7589A
Par1. \$\$\$	00000D7C	00000D7C	471078B2	471078B2
Par2. \$\$\$	00000D7D	00000D7D	12FA61B8	12FA61B8
Par3. \$\$\$	00000D7C	00000D7C	9142BBDC	9142BBDC
Par4. \$\$\$	00000D7C	00000D7C	B65BDCF2	B65BDCF2
Par5. \$\$\$	00000D7C	00000D7C	DB74FE08	DB74FE08
Par6. \$\$\$	00000D7C	00000D7C	008E1F1E	008E1F1E
Par7. \$\$\$	00000D7C	00000D7C	25A74034	25A74034
Par8. \$\$\$	00000D7C	00000D7C	4BC06150	4BC06150
Par9. \$\$\$	00000D7C	00000D7C	70D98266	70D98266
Par10. \$\$\$	00000DF1	00000DF1	8142E4DE	8142E4DE
Par11. \$\$\$	00000DF1	00000DF1	A561FBF9	A561FBF9
Par12. \$\$\$	00000DF1	00000DF1	C3801414	C3801414
Par13. \$\$\$	00000DF1	00000DF1	E79F2A2F	E79F2A2F
Par14. \$\$\$	00000DF1	00000DF1	0BBE414A	0BBE414A
Par15. \$\$\$	00000DF1	00000DF1	2FDD5865	2FDD5865
Par16. \$\$\$	00000DF1	00000DF1	53FC6F80	53FC6F80
Par17. \$\$\$	00000DF3	00000DF3	2D5F0297	2D5F0297
Par18. \$\$\$	00000DF1	00000DF1	9C3A9DB6	9C3A9DB6
Par19. \$\$\$	00000DF1	00000DF1	C059B4D1	C059B4D1
Par20. \$\$\$	00000DF1	00000DF1	966603F5	966603F5
Par21. \$\$\$	00000DF1	00000DF1	BA851B10	BA851B10
Par22. \$\$\$	00000DF1	00000DF1	DEA4322B	DEA4322B
Par23. \$\$\$	00000DF1	00000DF1	02C34946	02C34946
Par24. \$\$\$	00000DF1	00000DF1	26E26061	26E26061
Par25. \$\$\$	00000DF1	00000DF1	4B01777C	4B01777C
Par26. \$\$\$	00000DF1	00000DF1	6F208E97	6F208E97
Par27. \$\$\$	00000DF1	00000DF1	933FA5B2	933FA5B2
Par28. \$\$\$	00000DF1	00000DF1	B75EBCCD	B75EBCCD
Par29. \$\$\$	00000DF1	00000DF1	DB7DD3E8	DB7DD3E8
Par30. \$\$\$	00000DF1	00000DF1	B18A230C	B18A230C
Par31. \$\$\$	00000DF1	00000DF1	D5A93A27	D5A93A27
Acsp101. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
Acsp102. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
Acsp103. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
Acsp104. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
Acsp105. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
Acsp106. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
Acsp107. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
Acsp108. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
Acsp100. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
Acsp109. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
Acsp110. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
Acsp111. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
Acsp112. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
Acsp113. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
Acsp114. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
Acsp115. \$\$\$	00004E27	00004E27	6F167A54	6F167A54
C:\sb4\user\ oleg	00000024	00000024	4144536B	4144536B

```

I          00000330    00000330    011453BC    011453BC
V          00002EF0    00002EF0    414453A7    414453A7
System Integrity is OK
:
```

6.2.5 Report of Realtime Usage Command

The **#U** Terminal command provides a report of Real Time Usage. The controller continuously measures the time taken by realtime tasks. For details on realtime tasks see [Realtime and Background Tasks](#).

When the **#U** command is received, the controller analyzes the measured times during the last 50 controller cycles and calculates minimal, maximal and average time. The results are reported in percentages.



You can also use the ACSPL+ **USAGE** variable (see *SPIIPlus Command & Variable Reference Guide*) to monitor the usage. This variable is particularly useful in autoroutines for halting a program if the MPU usage is excessive.

6.2.6 Application Protection

Application protection does the following:

- > "Protects the user application from unintentional modification.
- > "Prevents harmful operator intervention while the application is running.
- > "Restricts erroneous changes to critical data and execution of potentially dangerous operations while the application is running.

At any time the user can enable or disable application protection. When application protection is disabled, none of the protections specified above apply.

When application protection is enabled, the controller is said to be in protected mode. When application protection is disabled, the controller is said to be in configuration mode.

At any time the user can intentionally delete the application in the controller. This operation brings the system to factory default state and disables all protections.

6.2.6.1 Protected Features

In protected mode the following operations are disabled:

- > Assignment to any protected variable.
- > Editing, or opening, an ACSPL+ buffer (only if the **PFLAGS.#DISABLE** bit of the affected buffer is set).
- > Commands **#C**, **#X**, **#S**, **#P**, **#SR**, **#XS**, **#XD**, **#BR** and **#BS** (if the **PFLAGS.#DISABLE** bit of the affected buffer is set).
- > All memory management commands, for example, **#SAVE**, **#LOAD**, etc.
- > All operations that change the flash memory.
- > Changing SP program or adjustment data.

- > SETSP function.
- > **Connect** command.
- > Any command received via any communication channel if the Communication Shutdown bit is set.

Protected mode affects all controller commands, including:

- > Any command sent to the controller for immediate execution.
- > Any command executed in a controller buffer (unless the Privileged bit is set).
- > Any operation initiated by SPiiPlus C Library or by an SPiiPlus Tools.

Any command that involving a prohibited action will be rejected with the error "Protection violation."

Protection does not affect power-up operations of the controller. Therefore, the user application stored in the flash memory will be loaded on power-up, including the saved values of all protected variables and all saved ACSPL+ programs. If one or more programs contains an AUTOEXEC label, the program will automatically start executing from this label.

6.2.6.2 Enhanced Application Protection

The following are enhanced application protection features:

1. Saving unprotected buffers to non-volatile memory
2. Using protected commands, such as **SETSP / GETSP / SETCONF / GETCONF**, in the Communication Terminal
3. **SETPROTECTION** settings (which standard variables are selected to be protected) remains saved in non-volatile memory after removing protection

The following backward compatibility feature is introduced in this version:

- > The addition of an Enhanced Application Protection" bit in **S_SETUP**
 - > If OFF, execution of protected commands is prevented (backward compatible behavior)
 - > If ON, execution of protected commands is allowed (default option)

6.2.6.3 CFG Variable

The standard controller variable CFG (Configuration) indicates whether the controller is in protected or configuration mode. The variable is read-only.

In configuration mode, CFG is 1. In protected mode, CFG is 0.

6.2.6.4 Protection of Variables

When the controller is in protected mode, any assignment to a protected variable is prohibited. By default the controller defines a set of protected variables. These are:

AFLAGS BAUD BOFFTIME BONTIME CERRA CERRI CERRV CONID DELI DELV E_FREQ E_SCMUL E_TYPE
E2_SCMUL E2_TYPE E2FAC EFAC ENTIME ERRA ERRI ERRV FDEF FMASK FVFIL GATEWAY IMASK
IOMASK LOWD LOWV MFF MFLAGS NVEL ONRATE PFLAGS PRATE RVFIL S_FDEF S_FMASK S_SAFINI S_
SETUP SAFINI SCCOFFS SCGAIN SCPHASE SCSOFFS SETTLE SLLIMIT SMCCPAR SRLIMIT STEPF STEPW
SUBNET SYNIV TARGRAD TCP2 TCP2PORT UDPPORT VELBRK XACC XCURI XCURV XRMS XRMSD
XRMSM XRMST XRMSTD SRMSTM XSACC XSEGRMAX XSEGRMIN XVEL SLABITS SLAFF SLBIASA
SLBIASB SLCBITS SLCFIELD SLCHALL SLCNP SLCOFFS SLCORG SLCPRD SLCRAT SLCROUT SLCSLIP

SLDBITS SLDRA SLDRAIF SLDROUT SLDRX SLDZMIN SLDZMAX SLEBIASA SLEBIASB SLFRC SLFRCD
SLHRS SLIFAC SLIFILT SLIKI SLILI SLIKP SLIOFFS SLPBITS SLPKI SLPKP SLPKPIF SLPKPSF SLPKI SLPKPSF SLPKI SLPKPSF
SLPMIN SLPROUT SLSTHALL SLVB0DD SLVB0DF SLVBITS SLVBOND SLVBONF SLVKI SLVKIIF SLVKISF
SLVKP SLVKPIF SLVKPSF SLVLI SLVNFRQ SLVNWID SLVNATT SLVSOF SLVSOFD SLVRAT SLVROUT
SLXROUT SLZFF SLVB1DD SLVB1DF SLVB1ND SLVB1NF E_MODE E_PAR_A E_PAR_B SLFRCN SLDZTIME
SLVKPDCF SLPKDCF SLVKIDCF SLGCAXN

The user is able to change the protection attribute for each variable individually.

The following command specifies the variable to be protected:

setprotection *variable* = 1

The following command specifies the variable to be not protected:

setprotection *variable* = 0

The variable is any standard variable name except the read only variables.

Only standard variables can be protected. Assignment to a user variable is allowed in any controller mode.

If a variable represents an array, all elements of the array share the same protection attribute.

Therefore the array as a whole can be protected or not protected. Protection cannot be specified for individual elements of the array.

The setprotection command can be executed only in configuration mode.

6.2.6.5 Protection of ACSPL+ Programs

While the controller is in configuration mode, the user can enter and edit a program in any buffer.

Protected mode restricts access to the program buffers. Restriction can change from buffer to buffer and is defined by the bit of **PFLAGS** variable.

Each element of the **PFLAGS** array corresponds to one buffer. The following bit defines protection of a buffer:

Variable **PFLAGS.#NOEDIT** (bit 1) when raised disables buffer editing while in protected mode. The default of the bit is 0.

Variable **PFLAGS.#NOVIEW** (bit 6) when raised disables buffer viewing while in protected mode. The default of the bit is 0.

By default, the **PFLAGS** variable is protected and cannot be changed in protected mode.

6.2.6.6 Privileged Buffer

One or more buffers can be marked as privileged. An ACSPL+ program in a privileged buffer is executed irrespective of the protection mode. In other words, in protected mode the program in a privileged buffer is executed as if there is no protection.

Variable **PFLAGS.#PRIVLG** (bit 4) marks the corresponding buffer as privileged. The default of the bit is 0.

The program in the privileged buffer can change any protected variable, write to SP variables, start and stop any other ACSPL+ program and execute any other action that in a regular buffer would cause protection violation.

6.2.6.7 Communication Shutdown

The user can disable executing commands and queries received via communication channels while the controller is in **protected mode**.

The following bits of the **COMMFL** variable control the communication shutdown:

- > Bit 7, Disable Commands: controls the communication in protected mode. If the bit is raised, the controller ignores any command received via communication channels except the queries that start from '?' character. The bit is not effective if the controller is in **configuration mode**. The default of the bit is 0.
- > Bit 8, Disable Queries: controls the communication in **protected mode**. If the bit is raised, the controller ignores any query received via communication channels. The bit is not effective if the controller is in **configuration mode**. The default of the bit is 0.

If both bits are raised, the controller in protected mode ignores any command or query received via any communication channel.

Communication shutdown does not affect executing the ACSPL+ programs in the buffers.

Communication shutdown does not prevent sending unsolicited messages from the controller as a result of executing the **DISP** or **SEND** command. Also communication shutdown does not affect executing the **INPUT** command, i.e., the controller accepts from a communication channel messages that are in response to the **INPUT** command.

Therefore, if communication shutdown is in effect, the communication with the controller is restricted to messages sent by the **DISP** or **SEND** command and messages accepted by the **INPUT** command.

6.2.7 Report Safety Configuration

The **#SC** command reports the current safety system configuration.

The controller response includes the following:

- > active safety groups
- > the configuration of each fault for each motor

For example:

#SC	Code	Fault	0	1	2	3	4	5	6	7	
0	#RL	Right Limit	K	K	K	K	K	K	K	K	K
1	#LL	Left Limit	K	K	K	K	K	K	K	K	K
2	#NT	Network error			D	D	D	D	D	D	-
3	#FAN	Cooling Fan Fault			-	-	-	-	-	-	-
4	#HOT	Overheat			-	-	-	-	-	-	-
5	#SRL	Software Right Limit			K	K	K	K	K	K	K
6	#SLL	Software Left Limit			K	K	K	K	K	K	K
7	#ENCNC	Encoder Not Connected			D	D	D	D	D	D	D
8	#ENC2NC	Encoder2 Not Connected			-	-	-	-	-	-	-
9	#DRIVE	Drive Alarm			KD	KD	KD	KD	KD	KD	KD
10	#ENC	Encoder Error	D		D	D	D	D	D	D	D
11	#ENC2	Encoder 2 Error	-		-	-	-	-	-	-	-
12	#PE	Position Error									
13	#CPE	Critical Position Error			KD	KD	KD	KD	KD	KD	KD
14	#VL	Velocity Limit	K		K	K	K	K	K	K	K
15	#AL	Acceleration Limit			K	K	K	K	K	K	K
16	#CL	Overcurrent	KD		KD	KD	KD	KD	KD	KD	KD

17	#SP	Servo Processor Alarm	D	D	D	D	D	D	D	D	D
25	#PROG	Program Error	K	K	K	K	K	K	K	K	
26	#MEM	Memory Overuse									
27	#TIME	Time Overuse									
28	#ES	Emergency Stop	KD	KD	KD	KD	KD	KD	KD	KD	
29	#INT	Servo Interrupt	D	D	D	D	D	D	D	D	
30	#INTGR	Integrity Violation	D	D	D	D	D	D	D	D	D
31	#FAILURE	Component Failure	D	D	D	D	D	D	D	D	D

The following designations are used in the report:

--- – fault detection is disabled (**FMASK=0**)

(blank) – fault response is disabled (**FDEF=0**) or no default response is defined

K – response is kill

D – response is disable

KD – response is kill-disable

+ – generalized fault

6.3 Working with Faults

6.3.1 Addressing the Fault Bits

Faults are represented as bits in the ACSPL+ variables **FAULT** and **S_FAULT**.

FAULT is an integer array containing eight elements (corresponding to the number of motors), where each element is made up of a set of bits. Each bit indicates one motor fault. Motor faults are related to a specific motor, power amplifier, or Servo Processor. Examples include Tracking Error, and Motor Overheat.

To address a specific motor fault bit, start with the specification of the **FAULT** element, followed by the bit selection operator (dot) and then the corresponding fault designator.

For example:

FAULT(2).#LL	Addresses the left limit fault bit of axis 2. The bit is raised if the 2 left limit switch is activated.
FAULT (3).#DRIVE	Addresses the drive fault bit of axis 3. The bit is raised if the 3 drive safety input is active.

S_FAULT is a scalar variable with two categories of bits:

- > Aggregated motor faults. Once the controller raises a bit in any element of **FAULT**, it immediately raises the corresponding bit of **S_FAULT**. Therefore, each bit of **S_FAULT** is an OR aggregate of the corresponding bits in all elements of **FAULT**.
- > System faults that are not related to any specific motor, such as Emergency Stop and Time Overuse.

The aggregated motor fault bits occupy the same bit positions as the corresponding motor fault bits in the **FAULT** variable. Use the designators of the motor faults to address the aggregated motor fault bits.

Examples:

S_FAULT.#LL	Addresses the aggregated Left Limit fault bit. The bit is raised if the Left Limit switch of any motor is activated.
S_FAULT.#DRIVE	Addresses aggregated Drive fault bit. The bit is raised if the Drive safety input of any motor is active.

Use the bit designators of the system faults to address the system fault bits.

Examples:

S_FAULT.#ES	Addresses the Emergency Stop fault bit. The bit is raised when the Emergency Stop safety signal is active.
S_FAULT.#PROG	Addresses the Program fault bit. The bit is raised when any program has failed due to a run-time error.

6.3.2 Querying Faults

The variables **FAULT** and **S_FAULT** are queried like any other variable. The controller reports the status of each meaningful bit.

Example:

```
?S_FAULT
0  OFF Right Limit (#RL)
1  ON  Left Limit (#LL)
2  OFF Network error (#NT)
3  OFF Cooling Fan Fault (#FAN)
4  OFF Overheat (#HOT)
5  OFF Software Right Limit (#SRL)
6  OFF Software Left Limit (#SLL)
7  OFF Encoder Not Connected (#ENCNC)
8  OFF Encoder 2 Not Connected (#ENC2NC)
9  OFF Driver Alarm (#DRIVE)
10 OFF Encoder Error (#ENC)
11 OFF Encoder 2 Error (#ENC2)
12 OFF Position Error (#PE)
13 OFF Critical Position Error (#CPE)
14 OFF Velocity Limit (#VL)
15 OFF Acceleration Limit (#AL)
16 OFF Overcurrent (#CL)
17 OFF Servo Processor Alarm (#SP)
20 OFF HSSI Not Connected (#HSSINC)
25 OFF Program Error (#PROG)
26 OFF Memory Overuse (#MEM)
27 OFF Time Overuse (#TIME)
28 OFF Emergency Stop (#ES)
29 OFF Servo Interrupt (#INT)
30 OFF Integrity Violation (#INTGR)
31 OFF Component Failure (#FAILURE)
```

The number in the left column is the bit number, followed by an ON/OFF indicator and the fault description and the bit name in parentheses. In the above example all the faults are OFF except for the Left Limit fault of one or more axes.

The **S_FAULT** variable indicates that there is a motor fault, but does not specify which motor has failed. To determine which motor has failed, query the **FAULT** variable, or use **?\$** to query the state of all motors.

Fault bits can be queried individually:

```
?S_FAULT.#LL
1
?FAULT(0).#LL, FAULT1.#LL
0
1
```

The controller answers a query of an individual bit by showing the numerical value of the bit: either 0 or 1.

6.3.3 Using the Fault Bits in if, while, till Commands

Using the fault variables in the condition of commands **if**, **while**, or **till** provides a decision making mechanism that is based on the present state of the faults.

Examples:

if FAULT(0).#HOT OUT(0).6 = 1end	Activate an output (OUT.6) if X motor is overheated
if FAULT(0).#LL FAULT(0).#RL disp "X limit switch"end	Display a warning if any limit switch of X motor is active
till ^FAULT(0).#LL	Wait until the X Left limit switch is released
if S_FAULT disp "Failure";end	Display a warning if any fault is active

The condition **IF S_FAULT** is satisfied if **S_FAULT** is non-zero, i.e., if any bit of **S_FAULT** is raised. Any fault, either system or motor, raises a bit in **S_FAULT**. Therefore a non-zero **S_FAULT** indicates that one or more faults are active.

The variables **FAULT** and **S_FAULT** display the current state of the faults. A conditional command based on these variables uses the fault state at the instant when the command is executed. For example, if the X left limit was activated but then released, **FAULT0.#LL** is zero, and the command **if FAULT0.#LL** considers the condition unsatisfied.

6.3.4 Creating Fault-Processing Autoroutines

To create an autoroutine that processes a fault, specify the fault bit in the autoroutine condition.

Example:

```
on FAULT(0).#LL          Start the autoroutine when an 0 axis Left Limit fault occurs
```

A fault-processing autoroutine can reside in any program buffer. When the buffer is compiled, the controller checks the autoroutine condition each controller cycle. When the condition is satisfied, the controller interrupts the program that is currently executing in the buffer that the autoroutine resides in, and starts the autoroutine execution.

A fault-processing autoroutine can supplement or replace the default response to a fault. If the corresponding **FDEF** or **S_FDEF** bit enables the default response, the autoroutine starts and executes in parallel with the default response. If the corresponding **FDEF** or **S_FDEF** bit is zero, the default response is disabled, and the autoroutine is the only controller response.



The controller examines all autoroutine conditions each controller cycle. However, if an autoroutine is executing in a buffer, and a condition of the second autoroutine in the same buffer is satisfied, the second autoroutine will start only after termination of the subroutine currently executing. Therefore, if an application includes a time-consuming autoroutine, avoid placing safety autoroutines that require short response times in the same buffer with the time-consuming autoroutine.

Examples:

The following autoroutine displays a message when the Drive Alarm signal becomes active for the 0 axis motor:

```
on FAULT(0).#DRIVE
disp "Axis 0 Drive Alarm"
ret
```

In the following autoroutines, the 0 and 2 axes motors must be disabled simultaneously. Therefore, if one of the drives fails, the second must be disabled as well. The default response disables the 0 axis motor if the 0 Drive Alarm occurs and disables the 2 axis motor if the 2 Drive Alarm occurs. The following pair of autoroutines supplements the default response by disabling a motor if the other motor fails:

```
on FAULT(0).#DRIVE disable 2; ret
on FAULT(2).#DRIVE disable 0; ret
```

When a 0 axis drive fault occurs, the following autoroutine terminates the controller activity for all motors:

on FAULT(0).#DRIVE	When 0 axis drive fault occurs
disableall	Disable all motors
stopall	Stop all other programs
stop	Stop the current program
ret	End of autoroutine

The **S_FAULT** variable contains the bits of the aggregated motor faults. These bits provide a convenient alternative to the motor faults if an application requires common processing of a motor fault irrespective of which motor caused the fault.

For example, the following autoroutine displays a message when the Left Limit switch of any motor is activated:

```
on S_FAULT.#LL
disp "One of the Left Limit Switches is Activated"
ret
```

Autoroutine conditions can contain more than one fault bit, as is shown here in the first line of the example:

```
on S_FAULT.#LL | S_FAULT.#RL
disp "Some Limit Switch Activated"
ret
```

The **S_FAULT** variable (used without a bit extension) indicates whether a fault has been detected by the controller. The following example shows an autoroutine that provides an alarm message if any fault occurs in the controller:

```

on S_FAULT
disp "Something happened"
ret

```

The controller activates an autoroutine when the condition of the autoroutine changes from false to true. If the condition remains true, the autoroutine is not activated again until the condition becomes false, and then true again. Therefore the above autoroutine displays the alarm message only on the first fault. If one fault bit is already raised, and another fault occurs, the second fault does not generate an alarm message.

The following autoroutine displays a fault message each time a fault occurs:

```

int LastFault
on LastFault <> S_FAULT
if (LastFault ~ S_FAULT) & S_FAULT
    disp "Something happened"
end
LastFault = S_FAULT
ret

```

In the above example the local variable **LastFault** stores the current value of **S_FAULT**. The exclusive OR (~) of **LastFault** and **S_FAULT** detects the bits of **S_FAULT** that changed. The AND (&) with **S_FAULT** retains only the bits that changed from zero to one, and not from one to zero.

6.3.5 Disabling Fault Processing



Certain safety variables provide protection against potential serious bodily injury and damage to equipment. Be aware of the implications before disabling any alarm, limit or error.

The ACSPL+ variables define which faults are examined and processed. If a bit of **FMASK** or **S_FMASK** is zero, the corresponding fault is disabled and the bit of **FAULT** or **S_FAULT** is not raised.

FMASK and **S_FMASK** are queried like any other variable, and the controller reports the status of each meaningful bit.

Example:

```

?FMASK(0)
0 ON Right Limit (#RL)
1 ON Left Limit (#LL)
2 ON Network error (#NT)
4 ON Overheat (#HOT)
5 ON Software Right Limit (#SRL)
6 ON Software Left Limit (#SLL)
7 ON Encoder Not Connected (#ENCNC)
8 ON Encoder 2 Not Connected (#ENC2NC)
9 ON Driver Alarm (#DRIVE)
10 OFF Encoder Error (#ENC)

```

```

11 OFF Encoder 2 Error (#ENC2)
12 ON Position Error (#PE)
13 ON Critical Position Error (#CPE)
14 ON Velocity Limit (#VL)
15 ON Acceleration Limit (#AL)
16 OFF Overcurrent (#CL)
17 OFF Servo Processor Alarm (#SP)
20 OFF HSSI Not Connected (#HSSINC)
?S_FMASK
25 ON Program Error (#PROG)
26 OFF Memory Overuse (#MEM)
27 OFF Time Overuse (#TIME)
28 ON Emergency Stop (#ES)
29 OFF Servo Interrupt (#INT)
30 OFF Integrity Violation (#INTGR)
31 OFF Component Failure (#FAILURE)

```

Normally, you enable or disable fault detection through the Adjuster wizard of the SPiiPlus MMI Application Studio (see the *SPiiPlus MMI Application Studio User Guide*) when initially configuring the controller. The configured values of **FMASK** and **S_FMASK** are then stored in the flash memory and left unchanged during the application lifetime.

Changes to safety variables after initial controller configuration may affect your application. The following section is relevant only if you need to enable or disable faults after initial configuration.

Example:

<code>?FAULT(0).#DRIVE , SAFIN(0).#DRIVE</code>	Display the status of the 0 drive alarm fault, and the safety signal
1	Drive Alarm fault bit is set.
1	Drive Alarm safety signal is set.
<code>FMASK(0).#DRIVE = 0</code>	Disable 0 axis Drive Alarm fault
<code>?FAULT(0).#DRIVE , SAFIN(0).#DRIVE</code>	Display the status of the 0 axis Drive Alarm
fault, and the safety signal	
0	Drive Alarm fault bit is reset.
1	Drive Alarm safety signal is still set.

6.3.6 Defining the Active Level of Safety Input

Safety inputs receive physical signals from various sources, such as limit switches and relays. Each safety signal is sent in one of two forms (binary):

- > High voltage level, no current in the controller input circuit.
- > Low voltage level, outflowing current in the controller-input circuit.

By default, the high voltage level is defined as the active state of the signal, i.e., the state that triggers a fault. This is called the normal polarity.

By using the ACSPL+ variables **SAFINI** and **S_SAFINI**, which define what level is active for each safety input, you can change the default defining the low voltage level as active for a specific safety input. The low voltage level will now trigger a fault. This is called inverse polarity. If a bit of **SAFINI** or **S_SAFINI** is zero (default value), the corresponding input accepts the high level as active.

The bits of **SAFIN** and **S_SAFIN** reflect the physical states of the signals, while the bits of **SAFINI** and **S_SAFINI** define the logical processing of the signals. **SAFINI** and **S_SAFINI** variables have no effect on the physical signal, and the bits of variables **SAFIN** and **S_SAFIN**, which display the raw values of the safety inputs are unaffected by the bits of **SAFINI**, **S_SAFINI**

The variables **SAFINI** and **S_SAFINI** are queried like any other variable. The controller reports the status of each meaningful bit that corresponds to a safety signal.

Example:

```
?SAFINI(0)
  0 ON Right Limit (#RL)
  1 ON Left Limit (#LL)
  4 OFF Overheat (#HOT)
  9 OFF Driver Alarm (#DRIVE)
?S_SAFINI
28 OFF Emergency Stop (#ES)
31 OFF Component Failure (#FAILURE)
```

In the above example, the fact that the response to the **SAFINI(0)** query shows that **RL** and **LL** are ON (bits 0 and 1) indicates that you have defined inverse polarity (low active level) for signals **#RL**, **#LL** of the 0 axis.

Normally, you define the signal polarity through the Adjuster wizard of the SPiiPlus MMI Application Studio (see the *SPiiPlus MMI Application Studio User Guide*) when initially configuring the controller. The configured values of **SAFINI** and **S_SAFINI** are then stored in the flash memory and are not changed during the application's lifetime.

Example:

?FAULT(0).#LL , SAFIN(0).#LL	Display the status of Left Limit and the Left Limit Safety signal for the 0 axis.
1	Left Limit fault bit is raised.
1	Limit safety signal is high.
SAFINI(0).#LL = 1	Set inverse polarity
?FAULT(0).#LL , SAFIN(0).#LL	Display the status of Left Limit and the Left Limit Safety signal for the 0 axis.
0	Left Limit fault bit changes to zero.
1	Left Limit safety signal remains high.

6.3.7 Fault Processing Modes

The controller defines two modes of behavior after a failure: regular and strict. Bit **#FCLEAR** of the **S_FLAGS** variable selects the fault processing mode:

- > If **S_FLAGS.#FCLEAR = 0** (default) the controller is in regular mode
- > If **S_FLAGS.#FCLEAR = 1** the controller is in strict mode.

The difference between the modes manifests when a fault occurs that kills a motor:

- > In the regular mode the next motion command simply clears the reason for the previous kill for all involved motors and starts the new motion.

- > In the strict mode the next motion command cannot activate the motion and fails. The motion cannot be activated as long as the reason for the previous kill is non-zero for any involved motor.

The reason for a kill operation is stored in the **MERR** variable. In the strict mode as long as a **MERR** element is non-zero, the corresponding motor cannot be put in motion. Commands **ENABLE** and **FCLEAR** clear the **MERR** elements for the specified motors and enable the next motion.

The same rules apply to the results of a **KILL** command with non-zero second argument (the reason for the kill - see [KILL and KILLALL Commands](#)). The reason is stored in the **MERR** element and in the strict mode the next motion cannot be activated until the reason is cleared.

In the regular mode the behavior is simple and totally compatible with previous versions. However, you may prefer the strict mode, especially during application development. The following example gives a hint why the strict mode may be preferable:

```
Reciprocated:  
PTP/r 0,10000  
PTP/r 0,-10000  
GOTO Reciprocated
```

Under normal conditions the motor continuously moves forward and backward by 10,000 units. Assume, however, that the first motion brings the motor to the right limit switch. The first motion terminates prematurely, because the motor is killed. However, the program continues running and executes the second motion command. In the regular mode the second motion starts successfully because it is directed out of the limit. Then the first motion command again brings the motor to the limit. Therefore, in the regular mode the reciprocated motion continues and there is no clear indication of abnormal condition.

Assume further, for the same application, that a broken connection to the right limit switch causes the controller to mistakenly continuously detect that the right limit has been passed. The first motion fails immediately after start, but the second one executes. The result is that the motors move in a negative direction by steps of 10,000 units.

In the strict mode, the behavior is more predictable. After the first motion failed, the second one cannot start and the program itself terminates with error. You can check the information in **MERR** and **PERR** to disclose the reason for the failure.

If at any point of the application a fault is an expected condition and the program must continue, the program in the strict mode must analyze the **MERR** element and execute the **FCLEAR** command before activating the next motion.

6.4 Network Faults

There are three types of possible faults:

- > Initialization failure – the EtherCAT stack could not start properly
- > Network failure during the normal work – SP software keeps running
- > SP software in a slave does not run properly or in reset state

6.4.1 Axis Network-Related Faults

Each axis has two network-related faults:

- > **FAULTX.2** - Network Fault

> **FAULTX.17** - Servo Processor Alarm

The possible reasons for Network Fault are:

- > Reset of EtherCAT slave chip
- > Physical Ethernet line disconnection
- > Power Down of a single slave

The Servo Processor Alarm is set whenever the handshake counter between the master and the slave is different from $CTIME*20$, i.e., the SP worked exactly $CTIME*20$ times between two consequent MPU cycles.

The possible reasons for Servo Processor Alarm are:

- > Communication loss with SP for any reason
- > SP SW failure (Network might keep working OK)
- > SP SW over usage
- > Loss of synchronization between MPU and SP
- > SP reset by Watch Dog or Power down

Both faults have a default response of disabling the axes that are affected. If there is a Network Fault on an axis, it will be always followed by Servo Processor Alarm. Malfunction of a single node will raise the Network Fault bit in all axes, in order to give the ability of immediate reaction.

6.4.2 Initialization Failure

The stack may fail to start up for several reasons. The ACSPL+ **ECST** variable shows different stages of stack initialization. As long as the stack has not reached full network initialization followed by successful load of an SP program, all axes of the corresponding SP will be in constant Servo Processor Alarm.

In this case, **ECST** and **ECERR** will show the cause of the failure.

6.4.3 Network Failure During Operation

In the event of a loss of communication, for example, due to broken cable, power down, poor contact, etc., after successful initialization, the Firmware will analyze which nodes are out of order and will activate the Network Fault on related axes. If the SP program is valid and running (for example in case of cable out), the SP recognizes that the master is not controlling the bus and disables all its axes; in addition the handshake **SYNC** counter is frozen.

After successful reconnection, the SP sees that the master has sent a recent **SYNC** value and returns to normal functioning. **FCLEAR** or **ENABLE** will reset the Network Fault and Servo Processor Alarm and allow normal axis operation.

6.4.4 SP Software Failure

In case of node reset due to watchdog or power down or in case of an unpredictable SP malfunction, the Servo Processor Alarm will always be activated on the related axes. In most cases it will also cause Network Fault. There is no way to clear this fault, because the SP does not have a valid program and it is not properly synchronized with the MPU.

To overcome this, run **#HWRES** (Reboot controller) in the SPiiPlus MMI Application Studio **Communication Terminal**, or perform a complete power down, and then power up to reset this node.



Other nodes may keep functioning, if their response to Network Fault is masked.

6.5 Detailed Description of Faults

This section provides a detailed description of each fault, including a description of the bits involved, the default response, and examples of autoroutines.

6.5.1 Limit Switches: #LL, #RL

The exact usage of limit switches depends on the application. A specific axis may require only one pair of limit switches or no limit switches at all. The following diagram illustrates a typical use of two pairs of limit switches:

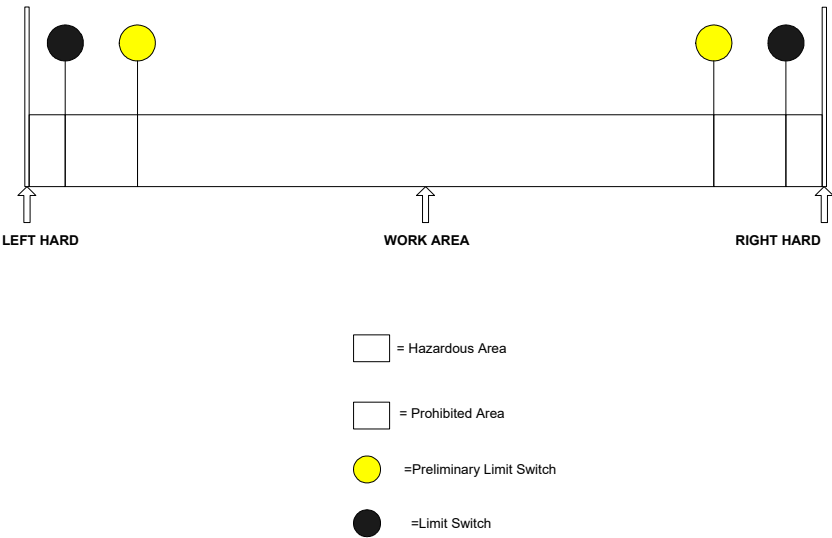


Figure 6-1. The Use of Limit Switches

Fault bits	FAULT.#LL, FAULT.#RL (in each element of FAULT)
Mask bits	FMASK.#LL, FMASK.#RL (in each element of FMASK)
Based upon safety signals	SAFIN.#LL, SAFIN.#RL (in each element of SAFIN)
Inversion bits	SAFINI.#LL, SAFINI.#RL (in each element of SAFINI)
Internal safety condition	None

Fault bits	FAULT.#LL, FAULT.#RL (in each element of FAULT)
Default response bits	FDEF.#LL, FDEF.#RL (in each element of FDEF)
Default response	The controller kills the violating motor. As long as the fault is active, the controller kills any motion that tries to move the motor in the direction of the limit. Motion to return to the allowed range of motion is allowed.

Autoroutine examples:

The first example supplements the default processing of X limit faults with alarm messages:

ON FAULT(0).#LL	When a Left Limit fault occurs in the 0 axis motor.
DISP "0 Left Limit switch activated"	Display the message: 0 Left Limit switch activated.
RET	
ON FAULT(0).#RL	When a right limit fault occurs in 0 axis motor.
DISP "0 Right Limit switch activated"	Display the message: 0 Right Limit switch activated.
RET	

This second example implements an autoroutine that disables the motor rather than the default response of killing the motion in case of a right limit or left limit fault. This response may be superior to the default response if the motor is equipped with a brake that is activated by the disable command because the brake may stop the motor faster than a kill command.

ON FAULT(2).#RL FAULT(2).#LL.	When there is a right limit or left limit fault in the 2 axis motor.
DISABLE 2	Disable axis 2
RET	

6.5.2 Network Fault: #NT

Fault bits	FAULT.#NT
Mask bits	FMASK.#NT
Based upon safety signals	None
Inversion bits	
Internal safety condition	

Fault bits	FAULT.#NT
Default response bits	None
Default response	Disables the axis

6.5.3 Software Limit Switches: #SLL, #SRL

Fault bits	FAULT.#SLL, FAULT.#SRL (in each element of FAULT)
Mask bits	FMASK.#SLL, FMASK.#SRL (in each element of FMASK)
Based upon safety signals	None
Inversion bits	None
Internal safety condition	See explanation below.
Default response bits	FDEF.#SLL, FDEF.#SRL (in each element of FDEF)
Default response	The controller kills the violating motor. As long as the fault is active, the controller kills any motion that tries to move the motor in the direction of the limit. Motion in the direction out of the limit is allowed.

Software limit switches use the following ACSPL+ variables:

- > **SLLIMIT** – Software Left Limit (Lower limit of working area)
- > **SRLIMIT** – Software Right Limit (Upper limit of working area)

The condition for software limit switches is based on the motor reference **RPOS** variable, not the motor feedback **FPOS** variable. Therefore, the fault provides protection against errors in the ACSPL+ application, not against hardware malfunctions.

The controller monitors the reference position **RPOS** and reference velocity **RVEL** and implements the following verifications:

- > If **RPOS** < **SLLIMIT**, the controller detects **#SLL** fault.
- > If **RPOS** > **SRLIMIT**, the controller detects **#SRL** fault.
- > If **RPOS** is within the range and **RVEL** is non-zero, the controller calculates the distance required to decelerate **RVEL** to zero using **KDEC** deceleration. If the final point of the calculated deceleration process is < **SLLIMIT**, the controller detects **#SLL** fault. If the final point of the calculated deceleration process is > **SRLIMIT**, the controller detects a **#SRL** fault.

This logic provides the moving edge of the software limit fault, depending on the instant velocity. As the controller kills the motor when the fault is detected, the termination point of the kill process will be very close to the corresponding software limit point.

The termination point is not exactly the software limit point because the controller checks the condition every controller cycle, i.e., at discrete time points. The termination point complies with the following conditions:

- > The termination point lies beyond the corresponding software limit.
- > Overrun is not more than $2 \cdot \mathbf{Vel} \cdot \mathbf{Cycle}$,
where **Vel** is an instant velocity and **Cycle** is the controller cycle.

For example, if the **FAULT(0).#SRL** fault is detected, the requested velocity of 0 axis is 10,000 count/sec and the controller cycle is 1 msec. The controller will overrun the software right limit for not more than $2 \cdot 10000 \cdot 0.001 = 20$ counts.

Autoroutine examples:

The following autoroutines supplement the default processing of X software limit faults with an alarm messages:

```
ON FAULT(0).#SLL
    DISP "0 Software Left Limit violated"
RET
```

```
ON FAULT(0).#SRL
    DISP "0 Software Right Limit violated"
RET
```

6.5.4 Non-Critical Position Error: #PE

Fault bits	FAULT.#PE (in each element of FAULT)
Mask bits	FMASK.#PE (in each element of FMASK)
Based upon safety signals	None
Inversion bits	None
Internal safety condition	See explanation below.
Default response bits	FDEF.#PE (in each element of FDEF)
Default response	None

Use the **#PE** fault to detect non-critical violation of position accuracy, and the **#CPE** (see [Critical Position Error: #CPE](#)) fault to detect uncontrolled, excessive error that indicates loss of control.

The following ACSPL+ variables are associated with position errors:

- > **ERRI** – Maximum position error while the motor is idle (not moving)
- > **ERRV** – Maximum position error while the motor is moving with constant velocity
- > **ERRA** – Maximum position error while the motor is accelerating or decelerating
- > **DELI** – Delay on transition from ERRA to ERRI

> **DELV** – Delay on transition from **ERRA** to **ERRV**

The controller raises the **FAULT.#PE** bit if the position error exceeds the maximum specified value, which is equal to **ERRI**, **ERRV** or **ERRA** depending on the motion state.

The variables **DELI** and **DELV** are used in a similar manner with the **#CPE** fault.

The following diagram illustrates the use of these variables for a typical motion profile that includes acceleration, constant velocity and deceleration:

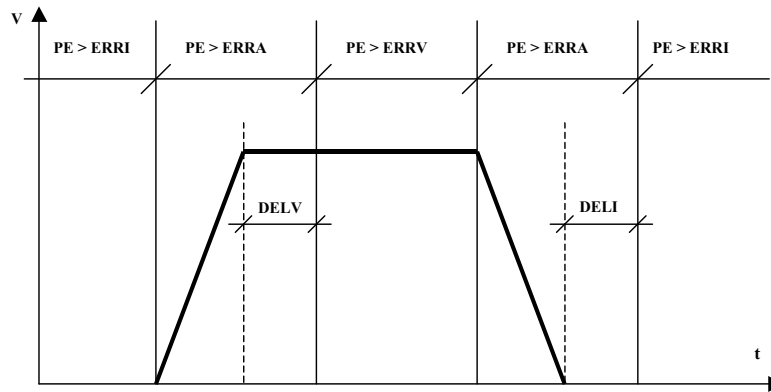


Figure 6-2. Use of Variables in a Typical Motion Profile

The allowed position error limit is:

- > **ERRI** if the motor is idle
- > **ERRV** if the motor is moving with constant velocity
- > **ERRA** if the motor is accelerating or decelerating
- > **DELV** defines delay on transition from **ERRA** to **ERRV**.
- > **DELI** defines delay on transition from **ERRA** to **ERRI**.

Autoroutine examples:

The following autoroutine supplements the default response to a position error with an alarm message.

```
ON FAULT(1).#PE
DISP "Accuracy violation - the motion was killed"
RET
```

The next example corrects the motion conditions by reducing the velocity (**VEL1**) until the error returns to within limits, instead of killing the motion.

<pre>ON FAULT(1).#PE WHILE FAULT(1).#PE IMM VEL(1) = 0.9 * VEL(1) WAIT 10 END RET</pre>	When there is a position error fault in the 1 axis motor. As long as there is a position error. Reduce the velocity of the 1 axis motor by 10%. Delay.
---	---

The controller automatically provides a smooth transition to the new velocity.

An application that incorporates the above autoroutine must satisfy the following conditions:

- > All motions of the 1 axis are single-axis, or 1 is a leading axis in a group. If another axis is leading, all motions will use the velocity of that axis, and the command **VEL(1) = ...** will have no effect.
- > All motions of the 1 axis use the default velocity **VEL(1)** and do not specify individual velocity.
- > The specific error monitored is the position error only while the motor is moving with constant velocity. To avoid the fault while the motor is idle or moves with acceleration, you have to initialize the variables **ERRI(1)** and **ERRA(1)** to sufficiently large values.



The above autoroutine executes an undefined number of loops with delays in each loop. Therefore, the execution time may be significant. As long as the autoroutine executes, no other autoroutine in the same buffer can be activated. Do not place this autoroutine in the same buffer that contains any time-critical autoroutine.

6.5.5 Critical Position Error: #CPE

Fault bits	FAULT.#CPE (in each element of FAULT)
Mask bits	FMASK.#CPE (in each element of FMASK)
Based upon safety signals	None
Inversion bits	None
Internal safety condition	See explanation below.
Default response bits	FDEF.#CPE (in each element of FDEF)
Default response	The controller disables the violating motor.

Use **#PE** fault (see [Non-Critical Position Error: #PE](#)) to detect non-critical violations of position accuracy, and the **#CPE** fault to detect uncontrolled, excessive error that indicates loss of control. **#CPE** should be greater than **#PE**.

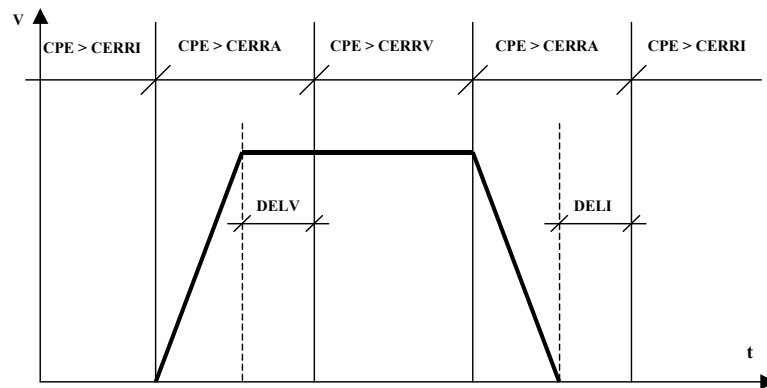
The following ACSPL+ variables are associated with critical position error:

- > **CERRI** - Critical position error if the motor is idle (not moving)
- > **CERRV** - Critical position error if the motor is moving with constant velocity
- > **CERRA** - Critical position error if the motor is accelerating or decelerating
- > **DELI** - Delay on transition from **CERRA** to **CERRI**
- > **DELV** - Delay on transition from **CERRA** to **CERRV**

The variables **DELI** and **DELV** are used also in the condition for the **#PE** fault.

The controller raises the fault bit if the position error exceeds the critical value. The critical value is equal to **CERRI**, **CERRV** or **CERRA** depending on the motion stage. The following diagram illustrates

the use of these variables for a typical motion profile that includes acceleration, constant velocity and deceleration:



The Critical limit for position error is:

- > **CERRI** if the motor is idle
- > **CERRV** if the motor is moving with constant velocity
- > **CERRA** if the motor is accelerating or decelerating.
- > **DELV** defines delay on transition from CERRA to CERRV.
- > **DELI** defines delay on transition from CERRA to CERRI.

A **#CPE** fault implies a serious problem in motor control. Do not disable the default response unless it is absolutely necessary in your application, i.e., keep **FDEF.#CPE = 1**.

Autoroutine examples

The following autoroutine supplements the default response with an alarm message:

```
ON FAULT(3).#CPE
    DISP "Axis 3 shows abnormal error. The motor was disabled."
RET
```

6.5.6 Encoder Error: #ENC, #ENC2

Fault bits	FAULT.#ENC, FAULT.#ENC2 (in each element of FAULT)
Mask bits	FMASK.#ENC, FMASK.#ENC2 (in each element of FMASK)
Based upon safety signals	None
Inversion bits	None
Internal safety condition	The controller latches fault #ENC if the phase shift between the signals of the primary encoder is lost, indicating a faulty encoder or noisy environment.

Fault bits	FAULT.#ENC, FAULT.#ENC2 (in each element of FAULT)
	The controller latches fault #ENC2 if the phase shift between the signals of the secondary encoder is lost, indicating a faulty encoder or noisy environment.
Default response bits	FDEF.#ENC, FDEF.#ENC2 (in each element of FDEF)
Default response	The controller disables the violating motor. The faults remain active until the user resolves the problems and enables the motor again or executes the FCLEAR command.

Unlike most faults, **#ENC** and **#ENC2** faults are latched. The fault bits remain raised even after the cause of the fault has been eliminated. Only the next enable command resets the fault bits.

Occurrence of an **#ENC** fault indicates a serious problem in motor control. Do not disable the default response unless it is absolutely necessary in your application, i.e., keep **FDEF.#CPE = 1**.

Autoroutine examples

The following autoroutine supplements the default response with an alarm message:

```
ON FAULT(2).#ENC
    DISP "Encoder Error in 2 axis. The motor was disabled."
RET
```

6.5.7 Encoder Not Connected: **#ENCNC, #ENC2NC**

Fault bits	FAULT.#ENCNC, FAULT.#ENC2NC (in each element of FAULT)
Mask bits	FMASK.#ENCNC, FMASK.#ENC2NC (in each element of FMASK)
Based upon safety signals	None
Inversion bits	None
Internal safety condition	The controller raises fault bit #ENCNC if a primary encoder is not connected. The controller raises fault bit #ENC2NC if a secondary encoder is not connected.
Default response bits	FDEF.#ENCNC, FDEF.#ENC2NC (in each element of FDEF)
Default response	The controller disables the violating motor.

If the controller detects a pair of differential encoder inputs that are not in opposite states (high and low level), it raises the fault because this may indicate a problem such as a short circuit or unconnected wire.

An **#ENCNC** fault indicates a serious problem in motor control. Do not disable the default response unless it is absolutely necessary in your application, i.e., keep **FDEF.#CPE = 1**.

Autoroutine examples

The following autoroutine supplements the default response with an alarm message:

```
ON FAULT(0).#ENCNC
    DISP "Axis 0: Encoder Not Connected. The motor was disabled."
RET
```

6.5.8 Drive Alarm: #DRIVE

Fault bits	FAULT.#DRIVE (in each element of FAULT)
Mask bits	FMASK.#DRIVE (in each element of FMASK)
Based upon safety signals	SAFIN.#DRIVE (in each element of SAFIN)
Inversion bits	SAFINI.#DRIVE (in each element of SAFINI)
Internal safety condition	The controller never sets the fault bit while the motor is disabled. The controller starts monitoring the fault condition when the period of time defined by variable ENTIME elapses after the motor has been enabled.
Default response bits	FDEF.#DRIVE (in each element of FDEF)
Default response	The controller disables the violating motor.

The condition involves the following ACSPL+ variable:

ENTIME – Motor's enable time in milliseconds

Even if the **SAFIN.#DRIVE** bit is in an active state, the controller never raises the fault bit while the motor is disabled. When the **enable** command is issued, the controller waits for the period of time defined by the **ENTIME** variable, and only then starts monitoring the **SAFIN.#DRIVE** bit. If the Drive Alarm signal is still active at that time, the fault condition is satisfied.

The controller continues monitoring the fault condition until the motor is disabled by a **disable** command or a fault that disables the motor.

Occurrence of a **#DRIVE** fault indicates a serious problem in the motor control. Do not disable the default response unless it is absolutely necessary in your application, i.e., keep **FDEF.#CPE = 1**.

General autoroutine example:

The following autoroutine supplements the default response with an alarm message:

```

ON FAULT(2).#DRIVE
    DISP "Axis 2 Drive Alarm. The motor was disabled"
RET

```

6.5.9 Motor Overheat: #HOT

Fault bits	FAULT.#HOT (in each element of FAULT)
Mask bits	FMASK.#HOT (in each element of FMASK)
Based upon safety signals	SAFIN.#HOT (in each element of SAFIN)
Inversion bits	SAFINI.#HOT (in each element of SAFINI)
Internal safety condition	None
Default response bits	FDEF.#HOT (in each element of FDEF)
Default response	None

Autoroutine examples:

The first autoroutine activates the **OUT(0).1** output, which could be wired to switch on an additional motor ventilation fan. The second routine switches off the fan when the fault is no longer active:

```

ON FAULT(1).#HOT
    OUT(0).1 = 1
RET
ON ^FAULT(1).#HOT
    OUT(0).1 = 0
RET

```

6.5.10 Velocity Limit: #VL

Fault bits	FAULT.#VL (in each element of FAULT)
Mask bits	FMASK.#VL (in each element of FMASK)
Based upon safety signals	None
Inversion bits	None
Internal safety condition	If ABS(RVEL) > XVEL , raise FAULT.#VL
Default response bits	FDEF.#VL (in each element of FDEF)
Default response	The controller kills the violating motor.

The condition involves the following ACSPL+ variable:

XVEL – Maximum allowed velocity for each motor

#VL uses the motor reference velocity **RVEL**, not the motor feedback velocity **FVEL**. Therefore, the fault bit is raised if an application command calls for excessive velocity, even if the motor has not reached this velocity. The fault can also be used for program testing without physical motion, while motors are disabled.

Autoroutine example:

The autoroutine informs you about the violation.

```
ON FAULT(2).#VL
    DISP "Axis 2 velocity limit was exceeded"
RET
```

6.5.11 Acceleration Limit: #AL

Fault bits	FAULT.#AL (in each element of FAULT)
Mask bits	FMASK.#AL (in each element of FMASK)
Based upon safety signals	None
Inversion bits	None
Internal safety condition	If $ABS(RACC) > XACC$, raise FAULT.#AL
Default response bits	FDEF.#AL (in each element of FDEF)
Default response	The controller kills the violating motor.

Acceleration limit uses the following ACSPL+ variable:

XACC – Maximum allowed acceleration for each motor

#AL uses the motor reference acceleration **RACC**, not the motor feedback acceleration **FACC**. Therefore, the fault bit is raised if an application command calls for excessive acceleration, even if the motor has not reached this acceleration. The fault also can be used for a program testing without motion, while motors are disabled.

Autoroutine example:

The following autoroutine supplements the default response with an alarm message:

```
ON FAULT(0).#AL
    DISP "Axis 0 Acceleration limit exceeded. The motor was disabled."
RET
```

6.5.12 Current Limit: #CL

Fault bits	FAULT.#CL (in each element of FAULT)
Mask bits	FMASK.#CL (in each element of FMASK)
Based upon safety signals	None
Inversion bits	None
Internal safety condition	If RMS current > XRMS , raise FAULT.#CL , or If RMS current > XRMSD , raise FAULT.#CL , or If RMS current > XRMSM , raise FAULT.#CL or
Default response bits	FDEF.#CL (in each element of FDEF)
Default response	The controller kills the violating motor.

The current limit fault is based on the Servo Processor algorithm that calculates the RMS value of the motor current. When the calculated RMS current exceeds the allowed value the Servo Processor reports an error that the MPU translates into a current limit fault.

Current limit processing uses the following ACSPL+ variables:

- > **XRMS** – Maximum allowed RMS current for each drive
- > **XRMSD** – Maximum allowed RMS current for each drive
- > **XRMSM** – Maximum allowed RMS current for each motor
- > **XCURI** – Maximum instantaneous current if the motor is idle (not moving)
- > **XCURV** – Maximum instantaneous current if the motor is moving

Use the SPiiPlus MMI Application Studio **SAFETY AND FAULTS CONFIGURATOR** to configure the specified variables.

Autoroutine example:

The following autoroutine kills the motion (and displays an alarm message) instead of the motor. (The default response can be disabled by adding **FDEF.#CL = 0** to the ACSPL+ program.)

```
ON FAULT(1).#CL
  KILL 1
  DISP "Axis 1 RMS current limit exceeded. Motor halted."
RET
```

6.5.13 Servo Processor Alarm: #SP

Fault bits	FAULT.#SP (in each element of FAULT)
Mask bits	FMASK.#SP (in each element of FMASK)
Based upon safety signals	None
Inversion bits	None
Internal safety condition	If the Servo Processor lost synchronization with MPU, raise FAULT.#SP
Default response bits	FDEF.#SP (in each element of FDEF)
Default response	The controller kills the violating motor.

#SP indicates that communication between the MPU and one of the servo processors failed. The occurrence of the **#SP** fault indicates a serious hardware problem.

Do not disable the default response unless it is absolutely necessary in your application, i.e., keep **FDEF.#SP = 1**.

This fault may be caused by a problem in the SP program. If the SP program hangs, the fault remains permanent. If the SP program time exceeds the tick time (50 μ sec), the fault is intermittent.

The disable reason reported by the controller is 5027 'Servo Processor Alarm'.

Autoroutine example:

The following autoroutine supplements the default response with an alarm message.

```
ON FAULT(1).#SP
    DISP "Axis 1 Servo Processor Alarm"
RET
```

6.5.14 HSSI Not Connected: #HSSINC

Fault bits	FAULT.#HSSINC (in each element of FAULT)
Mask bits	FMASK.#HSSINC (in each element of FMASK)
Based upon safety signals	None
Inversion bits	None
Internal safety condition	The controller raises fault bit #HSSINC if HSSI is not connected.
Default response bits	FDEF.#HSSINC (in each element of FDEF)
Default response	None

See also [Using HSSI I/O Extension](#).

Autoroutine example:

The following autoroutine displays an alarm message.

```
ON FAULT(0).#HSSINC
    DISP "Axis 0: HSSI not connected."
RET
```

6.5.15 Emergency Stop: #ES

Fault bits	S_FAULT.#ES
Mask bits	S_FMASK.#ES
Based upon safety signals	S_SAFIN.#ES
Inversion bits	S_SAFINI.#ES
Internal safety condition	None
Default response bits	S_FDEF.#ES
Default response	The controller disables all motors.

Autoroutine example:

The following autoroutine kills all motions but does not disable the motors (this assumes that the default response has been disabled by **S_FDEF.#ES = 0**).

```
ON S_FAULT.#ES
    KILLALL
RET
```

6.5.16 Program Error: #PROG

Fault bits	S_FAULT.#PROG
Mask bits	S_FMASK.#PROG
Based upon safety signals	None
Inversion bits	None
Internal safety condition	The controller latches the fault when any ACSPL+ program encounters a run-time error.

Fault bits	S_FAULT.#PROG
Default response bits	S_FDEF.#PROG
Default response	The controller kills all executed motions.

Unlike most faults, the **#PROG** fault is latched. Once raised, the bit remains raised until the controller resets it on execution of any command that compiles or starts a program in any buffer.

Autoroutine examples:

The following autoroutine supplements the controller's default response, terminating all concurrent programs and displaying an alarm message.

```
ON S_FAULT.#PROG
  STOPALL
  DISP "Run-time error"
RET
```

A run time error in a buffer stops all activity in the buffer. Therefore, the above autoroutine cannot intercept an error that occurred in the buffer where the autoroutine is located. However, it intercepts an error in any other buffer.

This autoroutine can supplement the default response (**S_FDEF.#PROG = 1**) or can replace it (**S_FDEF.#PROG = 0**).

The following autoroutine does the same (stops all programs) and also provides a diagnostic message.

```
ON S_FAULT.#PROG
  STOPALL
  IO = 0
  LOOP IO
    IF PERR(IO) >= 3020
      DISP "Program ", IO, " failed. Error ", PERR(IO)
    END
    IO = IO + 1
  END
RET
```

The ACSPL+ **PERR** variable contains the termination codes of the ACSPL+ programs. Each element of **PERR** contains a termination code for a different buffer. At power-up all elements of **PERR** are reset to 0. When a program in any buffer finishes or terminates for any reason, the corresponding element of **PERR** is assigned with a code that specifies the termination reason. The element resets to zero again once the corresponding buffer is compiled or its program execution starts.

Termination codes from 3000 to 3020 indicate normal termination. Codes greater than or equal to 3020 indicate run-time errors (see *SPiiPlus Command & Variable Reference Guide* for a complete breakdown of the termination codes).

6.5.17 Memory Overflow: #MEM

Fault bits	S_FAULT.#MEM
Mask bits	S_FMASK.#MEM
Based upon safety signals	None
Inversion bits	None
Internal safety condition	The controller latches the fault bit when the user application requires more memory than is available in the controller.
Default response bits	S_FDEF.#MEM
Default response	The controller kills all executed motions.

Unlike most faults, the **#MEM** fault is latched. Once raised, the bit remains raised until the controller resets it on execution of any command that compiles or starts a program in any buffer.

Because the controller uses dynamic memory handling, the amount of the memory available for a specific user application cannot be exactly determined. If an application raises this fault, you need to reduce the size of the application or add memory.

The following recommendations may be useful in eliminating the error:

- > Reduce the length of ACSPL+ programs.
- > Reduce the volume of user local and global variables. Pay special attention to arrays.
- > Limit the length of commands that are sent to the controller. Do not use commands that exceed 2032 characters.
- > Simplify the formulae used with the **CONNECT** and **MASTER** commands.

Autoroutine example:

The following autoroutine terminates all executing programs and displays an error message when the fault occurs.

```
ON S_FAULT.#MEM
  STOPALL
  DISP "Memory overflow"
RET
```

This routine can supplement the default response (**S_FDEF.#PROG = 1**) or can replace it (**S_FDEF.#PROG = 0**).

6.5.18 Time Overuse: #TIME

Fault bits	S_FAULT.#TIME
Mask bits	S_FMASK.#TIME
Based upon safety signals	None
Inversion bits	None
Internal safety condition	The user application demands too much processing time.
Default response bits	S_FDEF.#TIME
Default response	The controller kills all executed motions.

The controller raises the fault bit when the user application consumes too much processing time and **S_FMASK.#TIME** bit is raised.

The structure of the controller's realtime cycle is discussed in [Realtime and Background Tasks](#).

As the realtime processing time varies between cycles, the fault may occasionally occur and requires no special attention. However, frequent or permanent occurrence of the fault requires measures to correct the situation.

The controller has no default response to the fault. To monitor this fault, you must define your own autoroutine.

The following recommendations may be useful in reducing real time processing time, thereby eliminating the fault:

- > Reduce the number of concurrently executed programs.
- > Reduce the program execution rates (variables **PRATE**, **ONRATE**).
- > Reduce the number of command specified in one program line.
- > Reduce the number of autoroutines.
- > Simplify the conditions in the autoroutines.
- > Reduce the number of concurrently executed motions.
- > Avoid short-time motions.
- > Use segmented motion instead of a series of short PTP motions.
- > Simplify the formula used in the **CONNECT** and **MASTER** commands.

Autoroutine example:

The following autoroutine accumulates statistics on the fault. The routine measures a time of 1000 fault occurrences and then displays the average time between faults. The routine relies on zero initialization of the local variables.

<pre>Local int N Local real ATIME</pre>	<pre>Declare local variable for counting the number of faults. Declare local variable that will show the time of 1000 faults.</pre>
---	---

<pre> on S_FAULT.#TIME N=N+1 >IF N >=1000 DISP "#TIME occurs once per ", (TIME - ATIME) / N, " ms" faults. ATIME=TIME N=0 END RET </pre>	Activate routine when the TIME fault occurs. Increment the number of faults If 1000 faults were accumulated... Display average time between Prepare for the next accumulation Zero to prepare for the next accumulation
---	---

6.5.19 Servo Interrupt: #INT

Fault bits	S_FAULT.#INT
Mask bits	S_FMASK.#INT
Based upon safety signals	None
Inversion bits	None
Internal safety condition	The controller raises the fault bit when the servo interrupt is not generated or is irregular.
Default response bits	S_FDEF.#INT
Default response	The controller disables all motors.



The Servo Interrupt fault indicates a serious failure. Do not disable the default response unless it is absolutely necessary in your application, i.e., keep **FDEF.#INT = 1**.

The MPU sets the fault if the 1ms interrupt is not received. The probable cause is a hardware problem. The controller response to the fault is to disable all motors. The disable reason reported by the controller is 5029 'Servo Interrupt'.

Autoroutine example:

The following autoroutine supplements the default response with termination of all ACSPL+ programs and an alarm message.

```

ON S_FAULT.#INT
    STOPALL
    DISP "Main interrupt is missing"
RET

```

6.5.20 Component Failure Faults: #FAILURE

Some components other than Drive, for example, power supply, I/O extension card or encoder card, may have fault outputs that are common for all components. The controller provides system faults that by default disable all axes if fault occurs.

The controller provides special functions that retrieve the card malfunctioned information (according to address on the I²C bus) and the fault reason. This allows to user to write an ACSPL+ application that provides user-defined responses for different component faults. The components that have such outputs provide a special jumper that connects or disconnects the fault output to the controller.

6.5.20.1 Safety Variables



Currently only MC4U systems support these variables.

Component Failure fault is treated as any other system fault. There is a dedicated bit 31 (**#FAILURE**) for this fault in **S_FAULT**, **S_FMASK**, **S_SAFIN**, **S_SAFINI** and **S_FDEF**.

S_FAULT.#FAILURE (or S_FAULT.31)	Indicates if there is fault or not. 1 = An MC4U hardware component other than the drive, such as the Power Supply, I/O card, or encoder card, has failed.
S_FMASK.#FAILURE (or S_FMASK.31)	Defines if the Component Failure Fault will be examined by the controller. 1 = Enables S_FAULT.#FAILURE .
S_FDEF.#FAILURE (or S_FDEF.31)	You use this variable for triggering a response that you have programmed. 1 = An MC4U hardware component other than the drive, such as the Power Supply, I/O card, or encoder card, has failed.
S_SAFIN.#FAILURE (or S_SAFIN.31)	Indicates the actual status of the Component Failure controller input - see Examining System Fault Conditions .
S_SAFINI.#FAILURE (or S_SAFINI.31)	Used for inverting the Component Failure input logic.

6.5.20.2 Component Failure Fault Handling in ACSPL+

The **GETCONF(247,<AXIS>)** function serves for retrieving the component malfunction information (according to address on the I²C bus) and the fault reason. Using the 247 key triggers the function to retrieve a 32-bit integer number containing the following information about the failed component: Error Code, Card Address and Card Type. Below is the structure of the error data number. In the event that more than one component failed, Bit 15 is set to "1". The malfunctioning device is automatically reset.

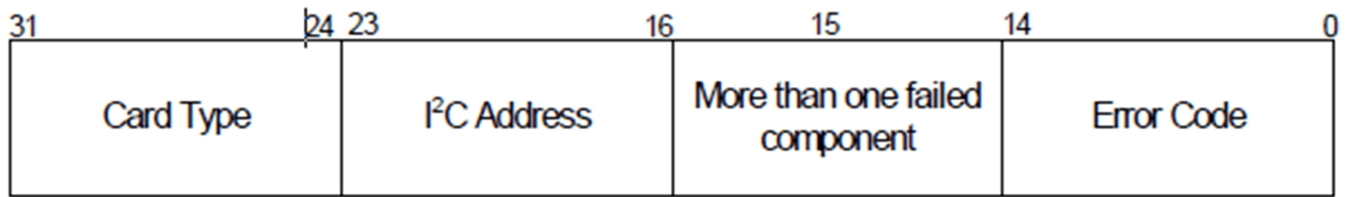


Figure 6-3. 32-bit Error Data Number

The following ACSPL+ example shows how to treat the return value of the **GETCONF(247,<AXIS>)** function:

```
# Local variables
INT res, error, i2c_addr, dev, nexterr
ON S_FAULT.#FAILURE
REP:
    res = GETCONF(247, 0)
    error = res & 0x7FFF
    nexterr = res.15
    i2c_addr.0 = res.16
    i2c_addr.1 = res.17
    i2c_addr.2 = res.18
    dev.0 = res.24
    dev.1 = res.25
    dev.2 = res.26
    dev.3 = res.27
    dev.4 = res.28
    dev.5 = res.29
    dev.6 = res.30
    dev.7 = res.31
    DISP "ERROR =",error
    DISP "I2C ADDRESS =",i2c_addr
    DISP "DEVICE =",dev
    IF(nexterr)
        GOTO REP
RET
```

6.6 Detailed Description of Safety Controls

This section explains the details of safety control implementation in the controller software. Read this section if you want a deeper understanding of how the controller analyzes safety inputs, examines safety conditions and processes faults.



The **SAFIN** and **S_SAFIN** variables are normally read-only. However, they can be written to when working with the Simulator, to simulate safety inputs.

6.6.1 Examining Fault Conditions - Flow Chart

This is how the controller examines fault conditions:

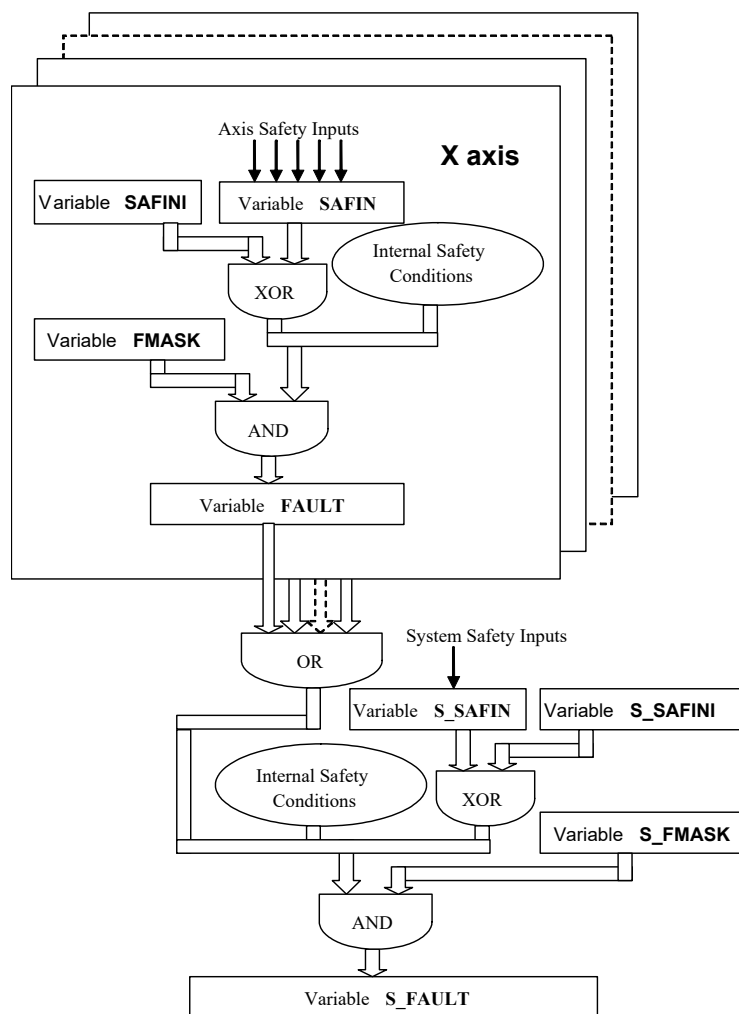


Figure 6-4. Fault Examination Flow Chart

The upper part of the diagram shows how motor faults are examined. The list of faults is identical for each motor and the controller repeats the process for each motor. The end product is the ACSPL+ **FAULT** variable.

The lower part of the diagram shows the elements that go into constructing the **S_FAULT** variable. Part of its bits are set as the OR-aggregate of the **FAULT** elements, and other bits are determined by examining the system faults.

6.6.2 Examining Motor Fault Conditions

The controller monitors motor fault conditions each controller cycle for each axis. There are two sources of motor faults:

- > Motor safety inputs
- > Internal safety conditions

The controller samples motor safety inputs each controller cycle and stores the values in the **SAFIN** variable. High voltage of a signal is stored as one in the corresponding bit of **SAFIN**. Low voltage is stored as zero. Only the following bits of **SAFIN** are meaningful:

- > **#LL** - Left Limit

- > **#RL** - Right Limit
- > **#LL2** - Preliminary Left Limit
- > **#RL2** - Preliminary Right Limit
- > **#HOT** - Overheat
- > **#DRIVE** - Drive Fault

For example the command:

```
IF SAFIN(0).#DRIVE  V0 = 1 ELSE V0 = -1
END
```

assigns a value of 1 to variable **V0** if the Drive Alarm signal of the 0 axis motor is high and -1 if low.

The **SAFINI** configuration variable defines which level of motor safety input causes a fault. In the above diagram XOR is a bit-wise operation. Therefore, if a bit of **SAFINI** is zero, high voltage of the corresponding signal causes fault. If a bit of **SAFINI** is 1, low voltage causes fault. Only those bits of **SAFINI** that correspond to the meaningful bits of SAFIN are used in fault processing. Other bits have no effect.

In addition to the safety inputs, the controller examines a number of internal safety conditions for each motor each controller cycle. The faults caused by the motor safety inputs and the faults detected by internal conditions provide a set of motor faults.

A detected motor fault is stored in a bit of variable **FAULT** only if the corresponding bit of variable **FMASK** is 1. If a bit of **FMASK** is zero, the controller does not raise the corresponding fault bit even if the fault condition or safety input is true. If a bit of **FMASK** is set to 1, the corresponding bit of **FAULT** is immediately set when the fault occurs. The bit rises to 1 or drops to zero in the same controller cycle as the corresponding safety input or internal safety condition shows change in the fault state.

Only those bits of **FAULT** that correspond to the motor faults are meaningful.

When a bit is raised, it activates the default response to the fault. An Autoroutine that processes the fault must use the bit of **FAULT** in as the condition.

6.6.3 Examining System Fault Conditions

System safety inputs and internal system safety conditions are monitored similarly to motor fault conditions.

There are three sources of **S_FAULT** bits:

- > Aggregated motor faults
- > System safety inputs
- > Internal system safety conditions

Aggregated motor faults are implemented as a combination of motor faults for all axes. For example, if bit **FAULT.#LL** is one, bit **S_FAULT.#LL** also rises to one. Bit **S_FAULT.#LL** drops to zero only if the **#LL** bits in all **FAULTs** for all motors are zero. Each meaningful bit of **FAULT** has its counterpart in **S_FAULT**.

Emergency Stop is the only system safety input. The controller samples the input each controller cycle and stores the value in the **S_SAFIN** variable. Therefore, only one bit, **S_SAFIN.#ES**, is

meaningful. High voltage of the signal is stored as one in the bit, low voltage is stored as zero. An application uses **S_SAFIN** if a raw immediate value of Emergency Stop input is required.

The configuration variable **S_SAFINI** defines which level of Emergency Stop input causes a fault. In [Examining Fault Conditions - Flow Chart](#) the XOR is a bit-wise operation. Therefore, if bit **S_SAFINI.#ES** is zero, high voltage of Emergency Stop causes a fault. If bit **S_SAFINI.#ES** is one, low voltage causes fault. Only one bit of **S_SAFINI** is used in fault processing. Other bits have no effect.

Each controller cycle the controller examines a number of internal system safety conditions. The aggregated motor faults, the Emergency Stop fault and the faults detected by internal conditions provide a set of system faults.

A system fault is stored in a bit of variable **S_FAULT** only if the corresponding bit of variable **S_FMASK** is one. If a bit of **S_FMASK** is zero, the controller does not raise the corresponding **S_FAULT** bit even if the fault is detected. If a bit of **S_FMASK** is one, the corresponding bit of **S_FAULT** follows the current state of the fault. The bit rises to one or drops to zero in the same controller cycle as the corresponding fault changes its state.

Only those bits of **S_FAULT** that correspond to the aggregated motor faults and to the system faults are meaningful.

When a bit of **S_FAULT** is raised, it activates the default response to the fault. An autoroutine that processes the fault must use the bit of **S_FAULT** as the condition.

6.7 Extended Fault Configuration



The controller's default safety configuration and responses fit the needs of most applications.

Only an experienced user should make modifications to the safety configuration. Improper configuration may result in unsafe operation, may damage the equipment, and may constitute a hazard to personnel.

As mentioned earlier in this chapter, the controller response to a fault can be modified with the ACSPL+ **FDEF/S_FDEF** variables and with autoroutines.

The **#SC** terminal command reports the current safety system configuration (see [Report Safety Configuration](#)).

There are several other options for safety system configuration:

Safety groups

Two or more axes can be combined in a safety group. All axes belonging to the safety group respond to any fault synchronously. If a fault affects one axis in the group, it immediately affects all the other axes in the group (refer to the **SAFETYGROUP** command in the *SPiPlus Command & Variable Reference Guide*).

Kill -disable response

In response to a fault, the controller executes **KILL**, decelerates the motor to zero velocity, and then disables the motor.

Changing response without an autoroutine

An axis can respond to a fault in one of the following basic ways:

No response

Kill response

Disable response

Kill-disable response

For each type of fault, the controller defines a default response, which can be overridden with an autoroutine. The **SAFETYCONF** command (refer to the **SAFETYCONF** command in the *SPiiPlus Command & Variable Reference Guide*) switches an axis between the four basic fault responses. An autoroutine is only required if the desired response is not one of the these.

Fault generalization

The fault response for a specific fault and a specific axis can be generalized to affect all the axes. For example, by default the 0 Drive Alarm fault disables the 0 axis motor, but has no affect on the 1 axis motor or any other motor. However, if you generalize the Drive Alarm fault for axis 1, the 1 axis motor will be affected by a Drive Alarm fault on any axis.

Specific motor configuration

The default configuration for all axes is identical. For example, the default response to a Limit Switch fault is to kill motion. However, the response can be modified individually for each motor. For example, if a Limit Switch fault occurs, the 0 axis motor can be killed while the 1 axis motor is disabled.

7. Connection to the Plant

This chapter details the connection between the motion controller and the controlled plant.

7.1 General Diagram

All connections between the controller and the controlled plant are provided through the SP processors as shown:

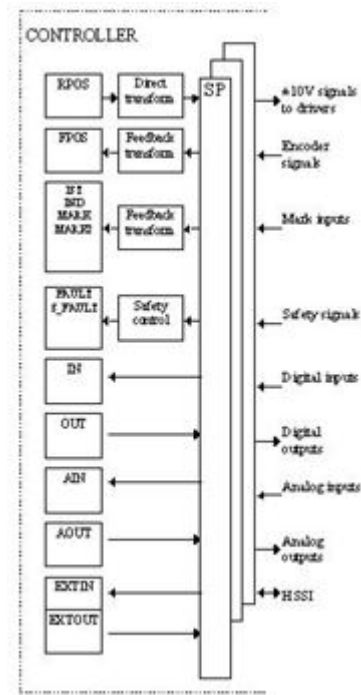


Figure 7-1. SPiiPlus-Plant Connections and Related Parameters

The ACSPL+ variables that provide access from your application to the control object are shown to the left. The following variables are available:

- > **RPOS** – Reference Position, array of eight elements, contains the desired motor position calculated by the controller.
- > **FPOS** – Feedback Position, array of eight elements, reads the current motor position.
- > **IND** – Index Position, array of eight elements, reads the position of the last encountered encoder index.
- > **MARK** – Mark Position, array of eight elements, reads the position of the last encountered Mark signal.
- > **M2ARK** – Secondary Mark Position, array of eight elements, reads the position of the last encountered Mark2 signal.
- > **FAULT** – Faults, array of eight elements, the bits of the variable report the axis-related faults detected by the safety control

- > **S_FAULT** – System Faults, scalar, the bits of the variable report the system faults detected by the safety control.
- > **IN** – General Purpose Inputs, array of eight elements, each bit reports the state of one general-purpose input.
- > **OUT** – General Purpose Outputs, array of eight elements, each bit defines the state of one general-purpose output.
- > **AIN** – Analog Inputs, array of 16 elements, each element reads the numerical value of the voltage supplied to one analog input. The number of analog inputs varies depending on the controller model.
- > **AOUT** – Analog Outputs, array of 16 elements, each element defines the voltage generated by one analog output. The number of analog inputs varies depending on the controller model.
- > **EXTIN** – Extended Inputs, array of eight elements, each bit represents a bit in the input or HSSI register.
- > **EXTOUT** – Extended Outputs, array of eight elements, each bit represents a bit in the HSSI register

7.2 User-Defined Units

The controller allows you to define the units used for motion programming and monitoring. This is done by setting the value of the **EFAC** variable.

During the configuration stage, you can set the value of the **EFAC** variable to specify the ratio between the desired unit and an encoder count for each axis. For example, if the encoder resolution is 1000 counts per millimeter, and you desire to program in millimeters, the corresponding **EFAC** element must be set to 0.001.

The user-defined units apply to all motion commands, so that all position-related arguments must be specified in user-defined units. The user-defined units also affect all position-related standard variables. For example, if the user-defined units are millimeters, then the unit of **RPOS**, **FPOS**, **APOS**, **MPOS**, **IND**, **MARK**, etc., will be in millimeters.

The user unit also affects velocity, acceleration and jerk variables. For example, if the **EFAC** value defines the unit as millimeter, then the units of **VEL**, **RVEL**, **FVEL**, **XVEL**, etc., will all be millimeters per second. The unit of the variables **ACC**, **DEC**, **KDEC**, **RACC**, **FACC**, etc., will be millimeters per second as well as the unit of the variables **JERK**, **GJERK**.

You can define the same or different units for each axis, irrespective of the encoder resolution.

Example (rotary motor):

$$EFAC(0) = 360 / (2000 \times 4)$$

Where:

Feedback resolution	2000 lines/rotation
Internal Multiplier	x4
User units	360 degrees per rotation

EFAC value	0.045 (i.e., $360/(2000*4)$)
-------------------	-------------------------------

$$EFAC(0) = 1 / (500*4)$$

Where:

Feedback resolution	2 micron (500 lines/mm)
Internal Multiplier	x4
User units	mm
EFAC value	0.005 (i.e., $1/(500*4)$)

Example (linear motor, sin-cos encoder)

$$EFAC(0) = 1 / (500*64)$$

Where:

Feedback resolution	500 lines/mm
Internal Multiplier	x64
User units	mm
EFAC value	0.00003125 (i.e., $1/(500*64)$)

Example (rotary motor connected to ball screw)

$$EFAC(0) = 1 / (2000*4*(1/0.5))$$

Where:

Feedback resolution	2000 lines/ rotation
Gear ratio	1 motor rotation=0.5mm motion of load
Internal Multiplier	x4
User units	mm
EFAC value	0.000625 (i.e., $1/(2000*4*(1/0.5))$)

7.3 Direct and Feedback Transform

The SPiiPlus controller accepts the encoder signals and calculates the feedback position in encoder counts. All servo algorithms in the SPiiPlus controller are based on encoder counts, not user units. Therefore, when reading the feedback position from the SP, the controller executes feedback transforms according to the formula:

$$FPOS = FP*EFAC + EOFFS$$

where **FPOS** is the controller feedback position in user units, **FP** is an SP-calculated feedback position in encoder counts, **EFAC** is a user-defined value of the corresponding **EFAC** factor, and **EOFFS** represents an offset.

EOFFS is a read-only standard variable that provides the offset between the SP-calculated position and the controller feedback position. The value of **EOFFS** changes when the set command defines a new origin for an axis.

When writing the reference position to the SP, the controller executes a direct transform given by the formula:

$$RP = (RPOS - EOFFS) / EFAC$$

where **RPOS** is the controller reference position in user units, and **RP** is the SP reference position in encoder units.

7.4 Index and Mark Values

Index and Mark processing is based on the SP hardware latch of the encoder reading when the index or mark input signal occurs. The latched position is accurate to within one encoder count at speeds of up to 5 million counts per second.

The following input signals are used for position latching:

- > Encoder Index – One index signal is available per axis.
- > **MARK** and **MARK2** – These signals are available only for axes 0, 1, 2, and 3.

Each of the three signals has an independent circuit and a separate latch register. The circuit latches the encoder reading in the register on the positive edge of the corresponding signal.

The controller further processes the latched signal. In this example we will use the index to explain the process. Processing the mark values is similar.

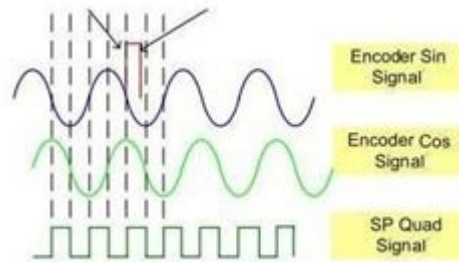
When the index signal latches the encoder reading, the controller raises a flag (the bit **IST.#IND**), converts the value into user units (scale and offset), and stores the value in the **IND** variable.

Afterwards, as long as the **IST.#IND** bit is set, the controller ignores any new index occurrences. Therefore, the **IND** variable does not change even if a new index was encountered. To reactivate the index latching, you must explicitly reset the **IST.#IND** bit.

The above is only accurate for Digital encoders. In the case of SIN-COS encoders, **FPOS(0)** is an interpolated value that contains a position in between zero-crossing of the SIN-COS signals.

Interpolation is done in the software and the hardware has the ability of latching only the zero crossings of the sine-cosine. So when you reset **IST(0).#IND** on the rising edge of the pulse, it latches the current value of the zero-crossing counter.

This is a low resolution counter that counts the quadrants of the SIN-COS. If the rising edge is detected a little bit before the zero crossing, the counter still has the previous zero-crossing position as shown in the following diagram:



The latching is done on the "Quad" signal that has a coarse resolution. So **IND** holds the zero-crossing position of SIN-COS only.

The following program fragment reports the index value each time that the index is encountered:

<code>IST0.#IND = 0</code>	Reset any index encountered previously
<code>INDREP</code>	Start of the loop
<code>TILL IST0.#IND</code>	Wait until the index is latched
<code>DISP IND0</code>	Report the index position
<code>IST0.#IND = 0</code>	Reactivate index latching
<code>GOTO INDREP</code>	Go to the start of the loop

Maximum index/mark detection frequencies by an ACSPL+ program for different Controller Cycle Time (CTIME) values:



- > 1 msec: 50Hz
- > 0.5 msec: 100Hz
- > 0.2 msec: 250Hz

7.5 Safety Inputs

The following safety inputs are available for each axis:

- > **LEFT LIMIT** – Signal from Left Limit switch.
- > **RIGHT LIMIT** – Signal from Right Limit switch.
- > **DRIVE ALARM** – Alarm signal from the drive; normally is on when the drive is disabled, and is off when the drive is enabled.

- > **NETWORK** – Loss of network connectivity has been detected.
- > **OVERHEAT** – Signal from a temperature sensor (not supported in all controller models).

The following safety input is not related to a particular axis and indicates a general failure of the control object:

- > **EMERGENCY STOP** – General failure of the control object.

The controller processes each safety input and raises the corresponding fault bit in the **FAULT** or **S_FAULT** variable accordingly. The following bitmapped variables are involved in the processing of the safety inputs:

- > **SAFIN** and **S_SAFIN**, read-only variables (except when used with Simulator) – Indicate the raw state of safety inputs before any processing.
- > **SAFINI** and **S_SAFINI**, configuration variables – Define the active state of each signal, specifying inversion of a signal if required.
- > **FMASK** and **S_FMASK**, configuration variables – Enable or disable using each signal in the safety control.
- > **FAULT** and **S_FAULT**, read-only variables – Indicate the faults detected by the safety control.
- > **FDEF** and **S_FDEF**, configuration variables – Enable or disable the controller default response when the fault occurs.

7.6 Digital Inputs/Outputs Repetitive

A digital input accepts a binary signal from an external source, such as a switch or a relay. A digital output provides a binary signal to an external acceptor such as an LED or actuator. Unlike the safety inputs, a digital input or digital output has no predefined function in the controller. You can connect signals to inputs or outputs and process them as required by the application.

The inputs are represented by the integer array: **IN**. The outputs are represented by the integer array: **OUT**. Each digital input or output is treated as one binary bit. The low voltage level corresponds to zero and high voltage level corresponds to one. Each element of the **IN** and **OUT** arrays is a 32-bit integer number that can represent up to 32 inputs or outputs. In all current SPiiPlus models, the number of inputs and outputs is less than 32; therefore all actual inputs and outputs are represented in **IN0** and **OUT0**. All other elements of **IN** and **OUT** are reserved for future extension. For the exact number of inputs/outputs see the specifications of the controller model.

The following example shows an autoroutine that changes the state of output 5 once the state of input 3 changes from 0 to 1.

```
ON IN0.3           Activate autoroutine on positive edge of input 3.
OUT0.5 = ^OUT0.5   Invert the state of output 5.
RET               End of the autoroutine
```

7.7 Analog Inputs/Outputs

An analog input accepts analog signal from an external source, such as a sensor or a potentiometer. An analog output provides analog signal to an external receiver, such as an actuator or a measuring device. Analog inputs and outputs have no predefined function in the controller. You can connect signals to inputs and outputs and process them as required by the application.

The analog inputs are represented by the integer array: **AIN** and the analog outputs are represented by the integer array: **AOUT**. Each analog input/output is represented by one array element. The range of the **AIN** and **AOUT** arrays depends on the type of the input or output and the bit resolution of the **ADC** or **DAC**.

Example: for $\pm 10\text{V}$ analog outputs, the **AOUT** range is from -100 (for -10V) to +100 (for +10V).

Example: for $\pm 10\text{V}$ analog inputs, the **AIN** range is from -100 (for -10V) to +100 (for +10V).

If an analog output is connected to a drive, it has a dedicated destination and cannot be used as a general-purpose analog output.

For model-dependent analog I/O information (for example, the number and range of inputs and outputs) see the controller's Hardware Guide.

The following example represents an autoroutine in the UDI that displays a message when the voltage in the 1st analog input rises above +7.5V.

```
ON AIN1 > 75      !Activate autoroutine once the value of AIN1 exceeds 75
(+7.5V)
DISP "AIN1 > 7.5V" !Display a message
RET              !End of the autoroutine
```

7.8 High-Speed Synchronous Serial Interface

The High-Speed Synchronous Serial Interface (HSSI) provides a simple interface for various external devices.

The HSSI provides up to 256 input bits and up to 256 output bits. Not every controller model supports all 512 bits. For the exact number of inputs and outputs supported, see the controller model's Hardware Guide.

The HSSI bits are represented by an integer array: **EXTIN** or **EXTOUT**. Each HSSI bit corresponds to one bit in one element of the **EXTIN** and **EXTOUT** array.

The HSSI bits have no predefined function in the controller. You are free to use HSSI as required in the application. In the simplest case, the HSSI bits can be used for an extension of the digital inputs/outputs. In a more sophisticated application, any sensor or actuator including encoders and drives can be connected through the HSSI.

The HSSI interface is described in detail in the *HSSI Expansion Modules Guide*.

8. Advanced Features

This chapter describes various advanced ACSPL+ programming features that are available to you. Topics covered are:

- > [Data Collection](#)
- > [PEG Overview](#)
- > [Sin-Cos Encoder Multiplier Configuration](#)
- > [Interrupts](#)
- > [Dynamic Braking](#)
- > [Constant Current Mode](#)
- > [Hall Sensor Commutation](#)
- > [Communicating with the SPiiPlus C Library](#)
- > [Communicating with Non-ACS Devices](#)
- > [TRIGGER](#)
- > [Dynamic TCP/IP Addressing](#)
- > [Non-Default Connections](#)
- > [Input Shaping](#)
- > [DRA Algorithm](#)
- > [BI-Quad Filter](#)

8.1 Data Collection

Data collection is useful in the following applications:

- > Troubleshooting
- > Adjusting servo control loops
- > Applications that require detailed information about internal controller processes
- > Teach-and-Go applications

8.1.1 DC Command

Description

The **DC** command is used for executing data collection.

Syntax

DC [/switch] [axis], array_name, #_of_samples, period, list_of_variables

STOPDC [/switch] [integer]

Where **switch** can be one or a combination of:

s	Start data collection synchronously to a motion.
w	Create the synchronous data collection, but do not start until a GO command is issued. W can only be used with the S switch.
t	Temporal data collection: sampling period is calculated automatically according to collection time.
c	Start cyclic data collection (can be used with switches S and W).

- > Data collection started by the **DC** command without the **S** switch is called system data collection.
- > Data collection started by the **DC/S** command is called axis data collection.
- > Data collection started by the **DC/C** command is called cyclic data collection.

Unlike the standard data collection that finishes when the collection array is full, cyclic data collection does not self-terminate. Cyclic data collection uses the collection array as a cyclic buffer and can continue to collect data indefinitely. When the array is full, each new sample overwrites the oldest sample in the array. Cyclic data collection can only be terminated by the [STOPDC Command](#).

Arguments

axis	Axis to which the data collection must be synchronized. The parameter is required only for axis data collection (S switch).
array_name	Array that stores collected data. The array must be previously defined as global, integer or real. The array size must be compatible with the number of samples and number of stored variables (see Understanding System Data Collection).
#_of_samples	The number of data samples to collect.
period	Sampling period in milliseconds. Actual sampling period may differ from this value, because the controller rounds it to an integer number of controller periods. For example if you set the period to 3.3 milliseconds, the controller will round it to 3 milliseconds. If switch T is included in the command, the parameter defines a minimal period (see explanation of temporal data collection below).
list of variables	Up to eight variable names, whose values are to be collected. Each variable can be a scalar variable, or an element of an array. Both local and global variables can be specified. Irrespective of the storage array type, any combination of integer and real variables is allowed - the controller automatically executes type conversion if required.

8.1.2 SPDC - High-Speed Data Collection

Description

The **SPDC** command starts data collection from an Servo Processor variable for a given Servo Processor.

Syntax

SPDC [/r] *Array, number_of_samples, sampling_period, SP_number, SP_Address1, [SP_Address2]*

Arguments

Array	Array name, up to XARRSIZE variable value. By default, ARRAY is assumed to be an integer array, if the /R switch is added, it defines the array as real.
number_of_samples	The number of samples to collect, the maximum value depends on the size of the array.
sampling_period	The time, in milliseconds, that each sample is taken.
SP_number	The number of the Servo Processor to be sampled
SP_Address1	The address of the Servo Processor variable in the Servo Processor to sample,
SP_Address2	As an option, you can add another address of an other Servo Processor variable in the Servo Processor to sample,

The minimum sampling period is 0.05 millisecond, which defines a sampling frequency of 20kHz.

The maximum sampling period is limited by **CTIME** value. By default, **CTIME** is 1 millisecond, which restricts sampling period to 1 millisecond maximum.

The controller rounds the specified period to an integer number of minimal periods and restricts it to the permitted range.

8.1.3 ACSPL+ Variables Involved in Data Collection

Data collection uses the following ACSPL+ variables:

AST	Axis State – Bit #DC in this bit-mapped variable is ON while an axis (started with /S) data collection for the corresponding axis is in progress. The bit is OFF if no axis collection for the axis is executed.
DCN	DC Number of Samples (axis variable) – While an axis data collection is in progress the variable displays the index of the array element that stores the next sample. When an axis data collection for the corresponding axis terminates, the variable stores the number of actually collected samples. If the data collection terminates automatically, the variable is always equal to the requested number of samples specified in the DC command. If the data collection terminates due to the STOPDC command, the variable may be less than the requested number of samples.
DCP	DC Period (axis variable) – When an axis data collection for the corresponding axis terminates, the variable stores the sampling period. Unless a temporal data collection was executed, the variable is always equal to the requested period specified in the DC command. For temporal data collection (/T) the variable may be greater than the requested minimal period.
S_DCN	General DC Number of Samples – While a system data collection is in progress the variable displays the index of the array element that stores the next sample. When a system data collection terminates, the variable stores the number of actually collected samples. If the data collection terminates automatically, the variable is always equal to the requested number of samples specified in the dc command. If the data collection terminates due to the STOPDC command, the variable may be less than the requested number of samples. For cyclic data collection S_DCN displays the current number of collected samples and changes as follows: 1. At the start of data collection S_DCN is assigned with zero 2. Each sampling tick S_DCN is incremented until it reaches the size of the sample array 3. S_DCN then remains unchanged (the new sample overwrites the oldest and the total number of samples remains the same) As long as the cyclic data collection is in progress, the application cannot use the sample array. After the cyclic data collection finishes, the controller repacks the sample array so that the first element represents the oldest sample and the last element represents the most recent sample.
S_DCP	System DC Period – When a system data collection terminates, the variable stores the sampling period. Unless a temporal data collection was executed, the variable is always equal (see explanation of the PERIOD parameter above) to the requested period specified in the dc command, unless it was rounded to the controller cycle. For temporal data collection (/T) the variable may be greater than the requested minimal period.

S_ST

System State – Bit: **#DC** in this bit-mapped variable is ON while a general (started without **/S**) data collection, either standard or cyclical, is in progress. The bit is OFF if no general collection is executed.

8.1.4 Understanding System Data Collection



Data collection is disabled when the SPiiPlus MMI Application Studio **SCOPE** is operating.

When the controller executes the **DC** command, it starts a separate real-time data collection process that progresses in parallel with ACSPL+ programs and motion execution. Each sampling period the process latches the values of all specified variables and stores them in the specified array. The process continues until the specified number of samples is stored, or the command **STOPDC** is executed. After process termination the array contains a time series of the specified variables that may then be used for data analysis.

This is shown in the following example:

```
global real DCA(2)(1000)
DC DCA, 990, 3, FPOS(0), FPOS(1)
```

- > In the first line, a matrix consisting of two lines and 1,000 columns is set up for data collection
- > The second line starts the data collection of the Feedback Position values for axes 0 and 1. 990 samples are to be collected, with a period of three milliseconds. The first step of the data collection stores the current value of **FPOS0** in **DCA(0)(0)** and **FPOS1** in **DCA(1)(0)**. The second step stores **FPOS0** in **DCA(0)(1)** and **FPOS0** in **DCA(1)(1)**.

Each variable is stored in one line of the array. Therefore the first dimension of the array (the number of lines) must be equal or greater than the number of variables. If the number of lines is greater than the number of variables, the extra array lines remain unaffected by the data collection. If only one variable is specified for data collection, a one-dimensional array is allowed.

Each sample of data collection fills up one column of the array. Therefore the second dimension of the array (number of columns) must be equal or greater than the requested number of samples. If the number of columns is greater than the number of samples, the extra array columns remain unaffected by the data collection. The following examples show incorrect usages of the **DC** command.

The following **DC** command is not allowed because the number of variables exceeds the number of array lines:

```
global int IA, IC(1000)
DC IC, 1000, 1, IA, FPOS(0)
```

The following **DC** command is not allowed, because the number of required samples exceeds the number of array columns:

```
global int IA, IC(1000)
DC IC,1001, 1, IA
```

If **/S** is not specified, the system data collection process starts immediately after executing the **dc** command. Normally, data collection stops automatically after <number of samples> x <period> milliseconds. If the **STOPDC** command executes while the data collection is in progress, the data collection stops prematurely, and the remaining array columns remain unaffected. To terminate system data collection, the **STOPDC** command must contain no parameters.



The variable **S_DCN** provides the number of samples stored during data collection.

The following are examples of **DC** commands:

global int IA, IC(1000)	Set up a one-dimensional 1000-line array for collecting data about a user-defined integer variable.
global real RA(2) (2000)	Set up a two dimensional array with two columns and 2000 lines for collecting data about a real user-defined variable.
DC IC, 1000, 1, AOUT(0)	Collect 1000 samples of AOUT on the 0 axis at a rate of one per millisecond.
DC RA, 2000, 2.5, IA, FPOS(7)	Collect 2000 samples of FPOS on axis D, at a rate of 3 per millisecond (it should be noted that the command calls for 2.5 milliseconds, but the controller rounds it up to the nearest whole number).
DC RA, 500, 1, TIME	Collect 500 samples of the ACSPL+ variable TIME at a rate of one per millisecond

8.1.5 Axis Data Collection

When switch **S** is included in the **DC** command, data collection starts synchronously with a motion. The first parameter must specify an axis with which the data collection is synchronized.

If the axis is idle at the moment when the **DC/S** command executes, no synchronization is provided and the data collection starts immediately just as if the **DC** command was executed without the switch. If the axis or axis group is in motion when the **DC** command executes, the data collection lines up in the motion queue and waits for all motions that were planned before it, to finish. When the motion queue comes to the data collection, the data collection starts, and immediately the next motion in the motion queue starts. Data collection therefore introduces no delay in the motion queue and does not affect motion execution.

Having started, data collection continues in parallel to the executed motion. Data collection finishes when the specified number of samples is stored, or the **STOPDC** command executes.

Data collection initiated by the **DC** command with no associated parameters is stopped by the **STOPDC** command with no associated parameters. To stop synchronous data collection initiated by the **DC/S** command, the **STOPDC** command must also include the **S** switch and the axis specification.

The **DC/S** command synchronizes the data collection start to the motion of an axis, but is not limited to collecting data only of that axis. Any parameter for any other axis may be specified for data collection. For example, the command

```
global real Array(2) (5)
dc/s 0,Array,500,1,FPOS(0),FPOS(1)
```

synchronizes data collection to the start of motion of axis 0, but collects data on both axes 0 and 1.

Only one data collection process, started by the **DC** command, can execute at a time. The next **DC** command can execute only after the data collection started by the previous **DC** command finishes. However, data collection, initiated by the **DC/S** command, may progress in parallel with the data collection initiated by the **DC** command. Moreover, several data collection processes initiated by the **DC/S** command may progress in parallel, providing they refer to different axes or axis groups. For example these two commands are executed serially:

```
DC/s 0, Array,500,1,FPOS(0)
DC/s 0, Array,500,1,FPOS(1)
```

While these commands are executed in parallel (unless the 0 and 1 axes belong to the same axis group):

```
DC/s 0,Array,500,1,FPOS(0)
DC/s 1,Array,500,1,FPOS(1)
```

The following is a full example of using the **DC/S** command:

```
GLOBAL REAL DC_DataX(2) (15000)
GLOBAL REAL DC_DataY(2) (15000)
GLOBAL REAL DC_DataA(2) (15000)
GLOBAL REAL DC_DataAsync(3) (10000)
ENABLE 0; ENABLE 1; ENABLE 2
!MASTER MPOS(1) = RPOS(0)+200
!SLAVE/p 1
DISP "DCN = %D, %D, %D", DCN(0), DCN(1), DCN(2)
!DC/s X, DC_DataX, 15000, 1, APOS(0), RPOS(0) ! commented
DC/s Y, DC_DataY, 15000, 1, APOS(1), RPOS(1)
DC/s A, DC_DataA, 15000, 1, APOS(4), RPOS(4)
!DC DC_DataAsync,
DISP "DCN = %D, %D, %D", DCN(0), DCN(1), DCN(2)
wait 1000000
PTP/re X, 10
DC/s X, DC_DataX, 15000, 1, APOS(0), RPOS(0) ! added
STOPDC
```

8.1.6 STOPDC Command

Description

The **STOPDC** command is also applicable to cyclic data collection.

An additional integer argument can be specified in the **STOPDC** command; **STOPDC** without arguments terminates the data collection immediately. **STOPDC** with an argument creates delayed termination of the data collection. For example:

STOPDC 50	Collect additional 50 samples and then finish.
-----------	--

It should be noted that the syntax of **STOPDC** that terminates synchronous data collection has been changed. If your application uses synchronous data collection, slight changes may be required. To terminate a synchronous data collection that was commanded by a **DC/S** command, the **STOPDC** command must also have switch **S**. For example:

DC/s 0,ARR,50,10,VAR1	Start synchronous data collection by axis 0 to a global real array ARR of size 50, sampling period 50, collect global variable VAR1.
STOPDC/s 0	Terminate synchronous data collection by axis 0.

8.2 PEG Overview

The purpose of the Position Event Generator (PEG) mechanism is to generate accurate high speed position-based events.



PEG is not supported for absolute encoders.



SinCos encoders can be fired on zero crossing only.

The SPiiPlus EtherCAT-based motion controllers provide the user with up to 6 identical PEG engine units. Each PEG engine can operate in one of the following two modes:

Incremental PEG mode

- > Provides the ability to generate a fix width pulse whenever a fixed position interval has passed, starting at a predefined start point and ending at a predefined end point
- > Incremental PEG commands are summarized in [Incremental PEG mode provides the ability to generate a fixed-width pulse whenever a fixed position interval has passed at a predefined position range \(from a start point to an end point\). Each PEG engine can generate a PEG pulses signal \(PEGx_PULSE\), based on exact positions..](#)
- > [Running Incremental PEG](#) leads you through a step-by-step approach to setting and activating Incremental PEG
- > A programming example is given in [PEG_I](#).

Random PEG mode

- > Provides the ability to control a PEG pulse and a four-bit vector at pre-defined positions, which are stored as a 256/1024 member user-defined array
- > Random PEG commands are summarized in [Random PEG Commands](#). It also leads you through a step-by-step approach to setting and activating Random PEG.
- > A programming example is given in [PEG_R](#).



1024 member user-defined array is supported by SPiiPlusCMba-X/SPiiPlusCMhp-Xs/UDMba-X/UDMhp-X (rev. B2 and later)/UDMxa-X, SPiiPlusNT(DC)-LT/HP/LD/NP (rev. D1 and later) only.

Each PEG engine can generate 1 PEG pulse: **PEGx_PULSE** (both in Incremental and Random modes) signal and 4 state signals (in Random mode only): **PEGx_STATEy** - a 4-bit output vector, on each random position PEG event. State signals are set to a defined logical level or set to generate a pulse on transition, as defined by a 256/1024 member **PEGx_STATE_ARRAY** integer array.

The PEG engines can be configured to be triggered by a position of any of the controller Encoders, with certain restrictions that result from the board's architecture.

The PEG engine outputs can be assigned to 10 physical interface outputs and the PEG pulse width and polarity are programmable.

Routing of Fast General Purpose Outputs to the Physical PEG Outputs

The PEG pulse signals can be assigned in parallel to opto-isolated General Purpose digital outputs. There are 10 physical output pins that are shared between the PEG engines' outputs. A special command: **ASSIGNPOUTS** provides the means for routing Fast General Purpose outputs to the physical PEG outputs.

The schematic block diagram of the PEG engines' architecture (see [8.2.1.2](#)) is useful for understanding the signal paths and routing. To configure the PEG operation, see [ASSIGNPOUTS](#) for the mapping of PEG engine outputs to physical outputs.

8.2.1 PEG Modes and Operation

Incremental PEG mode provides the ability to generate a fixed-width pulse whenever a fixed position interval has passed at a predefined position range (from a start point to an end point). Each PEG engine can generate a PEG pulses signal (PEGx_PULSE), based on exact positions.

Incremental PEG is activated periodically by starting from the `first_point` position event and stopping at the `last_point` position event. All pulses, including the last pulse, are of equal duration (width). PEG is generated only after `first_point` is reached. If the `first_point` is not reached during the motion, PEG will not be generated. PEG is defined only for one direction of the motion. To set PEG pulses for motion in the negative direction, set the `first_point` as the maximum position and the `last_point` as the minimum current position, and define a negative interval.

Example

Define incremental PEG pulses every 1000 [user-unit] from position 3500 [user-unit] to 6200 [user-unit], while each pulse width is 0.01 msec. [Figure 8-1](#) describes the process of incremental PEG generation, and [Figure 8-2](#) describes the output PEG pulses.

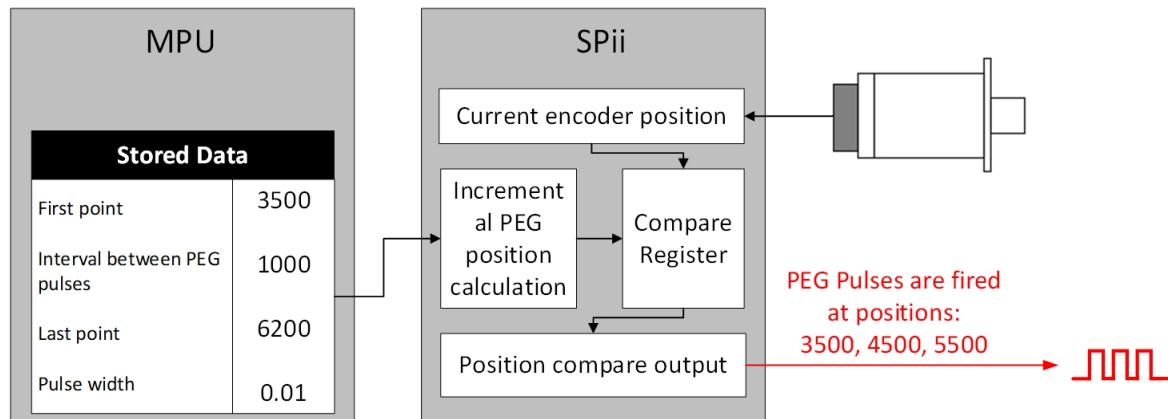


Figure 8-1. Incremental PEG Generation Process

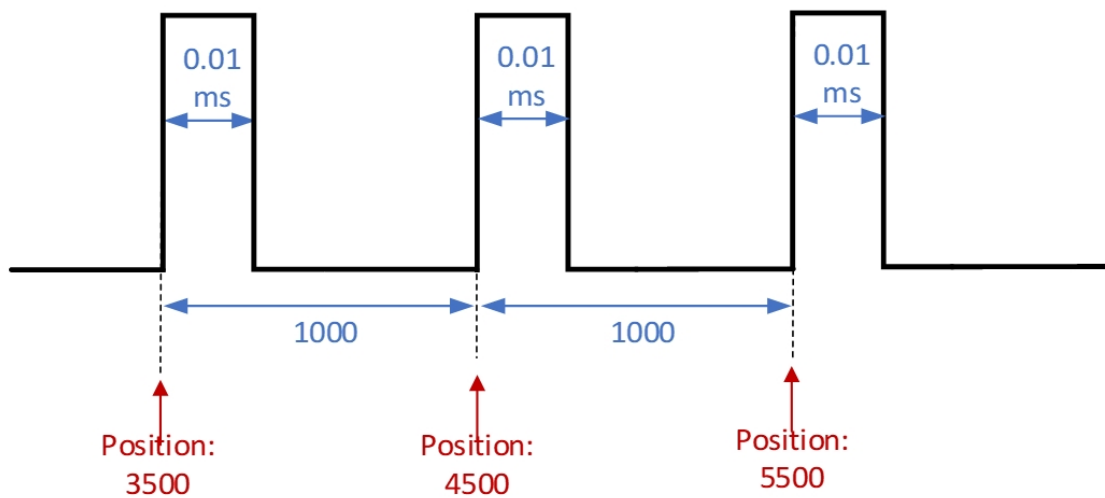


Figure 8-2. Incremental PEG Pulses Examples

For a programming example, see the Incremental PEG example in the *PEG and MARK Operations Application Note*

8.2.1.0.1 Running Incremental PEG

The Incremental PEG is set and activated by issuing the following commands:

1. Configuring specific PEG engines to specific encoders through the **ASSIGNPEG** command.
2. Assigning the physical output pins through the **ASSIGNPOUTS** command.
3. Setting Incremental PEG for the specific axis using **PEG_I axis, width, first_point, interval, last_point**
4. Waiting for **PEGREADY** to be '1' before activating PEG.
5. Starting PEG.

PEG firing is initiated if the specific PEG engine is configured properly by **ASSIGNPEG** and **PEG_I** is issued. PEG continues to be fired periodically until *last_point* is reached.

Incremental PEG is activated periodically starting from the *first_point* position event, and stops at the *last_point* position event. All pulses, including the last pulse, are of equal duration (*width*).

PEG is generated only after *first_point* is reached. If *first_point* is not reached during the motion, PEG will not be generated.

It is recommended that *first_point* position be the maximum current position for movement in the positive direction and the minimum current position for movement in the negative direction, before PEG engine activation.

Stopping the PEG motion with **STOPPEG** (optional).

6. **STOPPEG** is a synchronous delayed command which stops the PEGs from firing.

Since the Incremental PEG mechanism does not keep track of the direction of the movement, you may have to issue **STOPPEG** after *last_point* has been reached. This is needed in order to avoid unintentional PEG firing due to a rapid reversal of a movement's direction once *last_point* has been passed.

If **STOPPEG** has been executed before *last_point* has been reached, you can use **STARTPEG** to continue PEG from the current position.

8.2.1.1 Random PEG Commands

Random PEG mode provides the ability to generate pulses at predetermined positions. In addition, a 4-bit state vector is generated at the position events. All pulses, including the last pulse, are of equal duration (*width*).

The positions and the states are stored as user-defined arrays:

- > PEGx_POS_ARRAY: a real array defining the PEG event positions.
- > PEGx_STATE_ARRAY: 4-bit logical states associated with the PEG event position.

For each PEGx_POS_ARRAY[i] position, PEGx_STATE_ARRAY[i] defines the PEGx_STATEy outputs.

Example:

Define random PEG pulses in positions 1000, 1200, 1800, 2200 [user-unit], each pulse width is 0.01 msec. The 4 states associated with the position events are:

- > State bit 0: 1 at position 1000, 0 at position 1200, 0 at position 1800, and 0 at position 2200.
- > State bit 1: 0 at position 1000, 1 at position 1200, 0 at position 1800, and 1 at position 2200.
- > State bit 2: 0 at position 1000, 0 at position 1200, 1 at position 1800, and 1 at position 2200.
- > State bit 3: 0 at position 1000, 0 at position 1200, 1 at position 1800, and 0 at position 2200.

Figure 8-3 describes the process of random PEG generation, and Figure 8-4 describes the output PEG pulses.

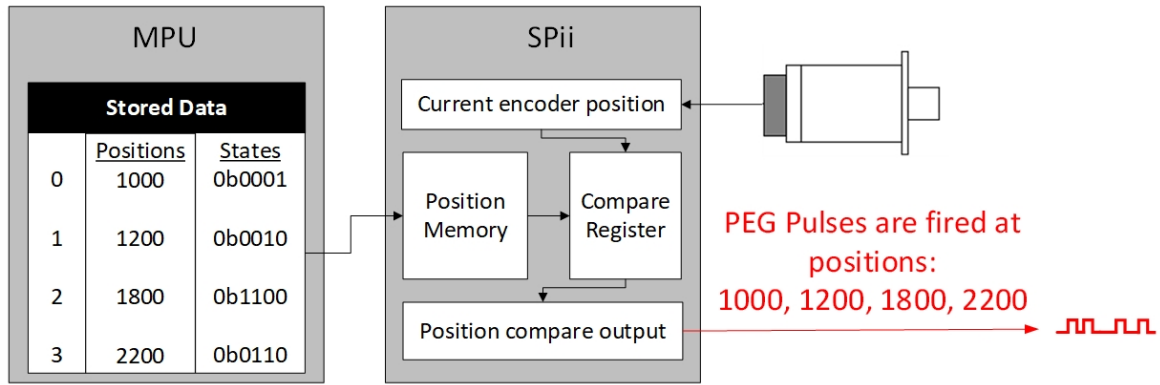


Figure 8-3. Random PEG Generation Process

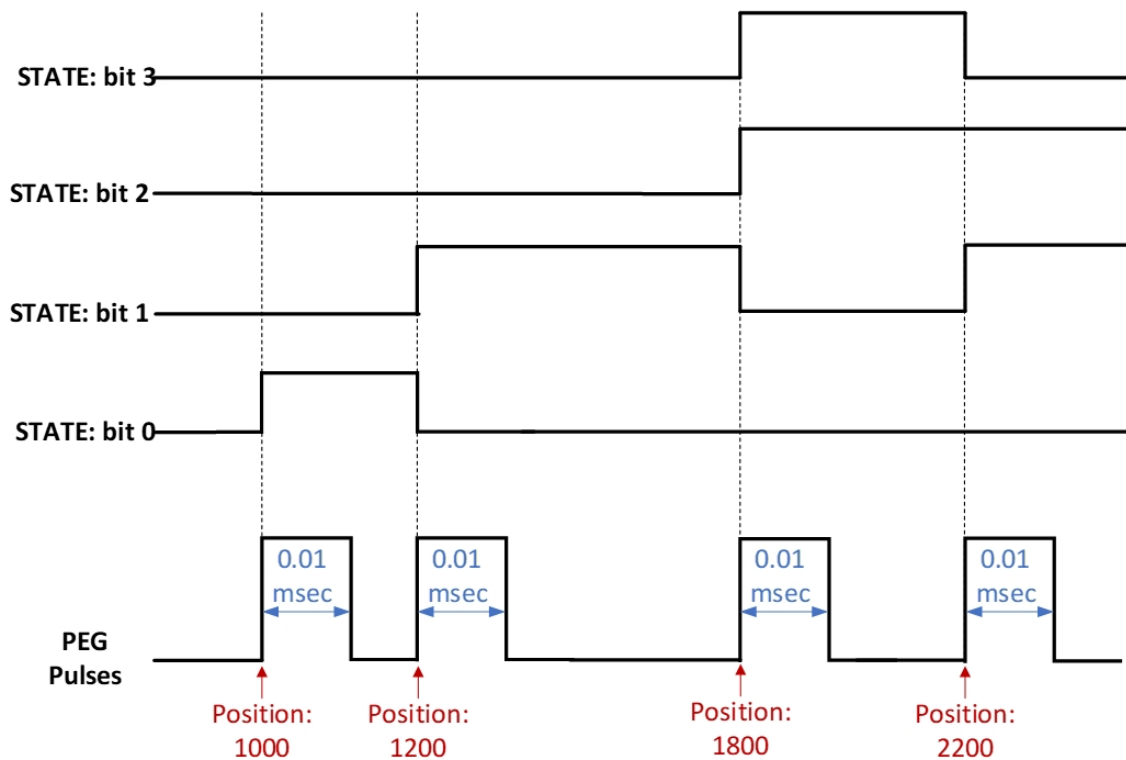


Figure 8-4. Random PEG Pulses

For a programming example, see the Random PEG example in the *PEG and MARK Operations Application Note*.

Loading Random PEG Arrays

The Random PEG Arrays are loaded by issuing the following commands:

1. Load arrays.
2. Activate PEG.
3. Start movement.

The movement will cause the PEG signals to be fired each time a position match event occurs. Following the last position event, the sequence above is repeated with the movement activated in the opposite direction.

This still allows loading 3 axes of the same Servo Processor in parallel, for example:

```
PEG_R PEG_ENGINE0, 1, 0x4444, 0, 255, Y_ARR, Y_STAT
PEG_R PEG_ENGINE1, 1, 0x4444, 0, 255, Y_ARR, Y_STAT
PEG_R PEG_ENGINE2, 1, 0x4444, 0, 255, Y_ARR, Y_STAT
TILL AST(PEG_ENGINE0).#PEGREADY
TILL AST(PEG_ENGINE1).#PEGREADY
TILL AST(PEG_ENGINE2).#PEGREADY
```

Table [Table 8-1](#) shows the typical times for loading PEG engines on the condition that the products match the versions in . Otherwise, the typical times for loading PEG engines are as listed in Table [Table 8-2](#).

Table 8-1. Typical PEG Engine Loading Times on Products with Support for Fast Loading of Random PEG Arrays

Number of Points to load	Typical Time to Load 1 PEG Engine [msec * CTIME]		Typical Time to Load 2 PEG Engines [msec * CTIME]		Typical Time to Load 3 PEG Engines [msec * CTIME]	
	without specifying PEG States	specifying PEG States	without specifying PEG States	specifying PEG States	without specifying PEG States	specifying PEG States
1	13	13	14	14	15	15
2	13	13	14	14	15	15
4	13	13	14	14	15	15
8	13	13	14	14	15	15
16	13	14	14	16	16	18
32	14	15	17	18	19	21
64	16	17	21	22	24	27
128	19	22	27	32	33	42
256	25	32	39	51	52	71
512	38	52	65	90	91	129
1024	64	91	117	167 [*]	168	244



HSSI devices (HSSI-IO16, HSSI-ED2, etc.) cannot be used with a Servo Processor on which fast loading of Random PEG arrays is activated.



SPRT and **SPINJECT** commands cannot be used for a Servo Processor on which fast loading of Random PEG arrays is activated.

Table 8-2. Typical PEG Engine Loading Times on Products without Support for Fast Loading of Random PEG Arrays

Number of Points to load	Typical Time to Load 1 PEG Engine [msec * CTIME]		Typical Time to Load 2 PEG Engines [msec * CTIME]		Typical Time to Load 3 PEG Engines [msec * CTIME]	
	without specifying PEG States	specifying PEG States	without specifying PEG States	specifying PEG States	without specifying PEG States	specifying PEG States
1	16	17	24	25	29	30
2	17	18	26	28	31	33
4	19	21	30	34	37	42
8	23	27	38	46	49	60
16	31	39	55	70	73	100
32	48	64	87	119	122	169
64	81	112	151	216	218	315
128	146	210	281	410	413	605
256	276	406	539	798	800	1185
512	537	796	1056	1573	1576	2350
1024	1060	1579	2090	3124	3127	4677



The priority of loading PEG engine command is low, so using '**SETSP**' / '**GETSP**' commands and changing tuning parameters in parallel with the loading process may cause loading times to be higher than the times mentioned in the table. For this reason the loading commands should be run separately.

8.2.1.1.1 Running Random PEG

The Random PEG is set and activated by issuing the following commands:

1. Setting the event position array: **PEGx_POS_ARRAY**. The array has a maximum of 256/1024 members.
2. Setting the output states array: **PEGx_STATE_ARRAY**. This array contains the state vectors which are to be issued per position event of the **PEGx_POS_ARRAY**.
3. Assigning the specific PEG engine to a specific encoder through the **ASSIGNPEG** command.
4. Assigning the physical output pins through the **ASSIGNPOUTS** command.
5. Setting Random PEG for the specific axis using **PEG_R axis, width, mode, first_index, last_index, PEGx_POS_ARRAY, PEGx_STATE_ARRAY**.
6. Waiting for **PEGREADY** to be '1' before activating PEG.
7. Starting PEG.

PEG firing is initiated if the specific PEG engine is configured properly by **ASSIGNPEG** and **ASSIGNPOUTS** and **PEG_R** is issued. PEG continues to be fired upon position match until *last_point* is reached.

8. Stopping PEG with **STOPPEG** (optional).

STOPPEG stops the PEGs from firing. If **STOPPEG** has been issued and the motion has not reached *last_point*, you can use **STARTPEG** to continue PEG from the current position.

8.2.1.1.2 Time-Based PEG Support

Time-based PEG support provides the ability to generate time-based pulses following each position-based pulse. Time-based PEG support is achieved using **PEG_I** and **PEG_R** commands.

The commands use the following arguments:

- > **Time-based-pulses** - Optional parameter - The number of time-based pulses generated after each encoder-based pulse, the range is from 0 to 65,535.
- > **Time-based-period** - Optional parameter - The period of time-based pulses (milliseconds), the range is from 0.00005334 to 1.7476.
The time-based period must be at least pulse width + 26.6667 nsec (minimum distance between two pulses).

Example

Define incremental PEG pulses every 1000 [user-unit] from position 3500 [user-unit] to 6200 [user-unit], and 2 time-based pulses every 0.02 msec in following to each position-based pulse. The pulse width is 0.01 msec. Figure 6 displays the output signal, while the black pulses are position-based, and the blue pulses are time-based.

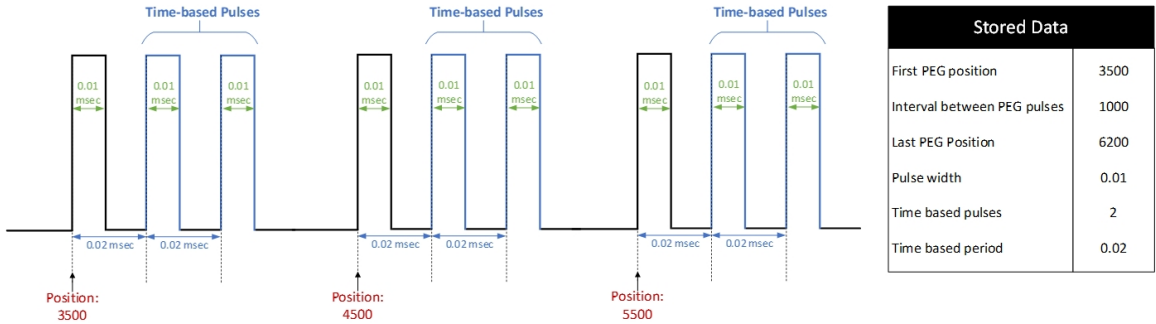


Figure 8-5. Time-Based PEG Pulses Example

See the Time-based Pulses example showing the use of Time-based PEG pulses in the *PEG and MARK Operations Application Note*.

8.2.1.1.3 Loading Random PEG Arrays

The Random PEG Arrays are loaded by issuing the following commands:

1. Load arrays.
2. Activate PEG.
3. Start movement.

The movement will cause the PEG signals to be fired each time a position match event occurs. Following the *last_point* event, the sequence above is repeated with the movement activated in the opposite direction.

This still allows loading 3 axes of the same Servo Processor in parallel, for example:

```
PEG_R AXIS0, 1, 0x4444, 0, 255, Y_ARR, Y_STAT
PEG_R AXIS1, 1, 0x4444, 0, 255, Y_ARR, Y_STAT
PEG_R AXIS2, 1, 0x4444, 0, 255, Y_ARR, Y_STAT

TILL AST (AXIS0) .#PEGREADY
TILL AST (AXIS1) .#PEGREADY
TILL AST (AXIS2) .#PEGREADY
```

4. In contrast to SPiiPlusCM/SPiiPlusSA/SPiiPlus3U, the PEG pulse is inversed. Use PEG_R/i or PEG_I/i to overcome this issue.

Table 8-3 shows the typical times for loading PEG engines on condition that the products match the versions in 8.2.1.

Otherwise the typical times for loading PEG engines are as listed in Table 8-4.

Table 8-3. Typical Times to Load PEG Engines for the products that support fast loading of Random PEG arrays

Number of Points to load	Typical Time to Load 1 PEG Engine [msec * CTIME]		Typical Time to Load 2 PEG Engines [msec * CTIME]		Typical Time to Load 3 PEG Engines [msec * CTIME]	
	without specifying PEG States	with specifying PEG States	without specifying PEG States	with specifying PEG States	without specifying PEG States	with specifying PEG States
1	13	13	14	14	15	15
2	13	13	14	14	15	15
4	13	13	14	14	15	15
8	13	13	14	14	15	15
16	13	14	14	16	16	18
32	14	15	17	18	19	21
64	16	17	21	22	24	27
128	19	22	27	32	33	42
256	25	32	39	51	52	71
512	38	52	65	90	91	129
1024	64	91	117	167	168	244



HSSI devices (HSSI-IO16, HSSI-ED2, etc.) cannot be used for the same Servo Processor when fast loading of Random PEG arrays is activated. **SPRT** and **SPINJECT** commands cannot be used for the same Servo Processor when fast loading of Random PEG arrays is activated.

Table 8-4. Typical Times to Load PEG Engines for the products that do not support fast loading of Random PEG arrays

Number of Points to load	Typical Time to Load 1 PEG Engine [msec * CTIME]		Typical Time to Load 2 PEG Engines [msec * CTIME]		Typical Time to Load 3 PEG Engines [msec * CTIME]	
	without specifying PEG States	with specifying PEG States	without specifying PEG States	with specifying PEG States	without specifying PEG States	with specifying PEG States
1	16	17	24	25	29	30
2	17	18	26	28	31	33
4	19	21	30	34	37	42
8	23	27	38	46	49	60
16	31	39	55	70	73	100
32	48	64	87	119	122	169
64	81	112	151	216	218	315
128	146	210	281	410	413	605
256	276	406	539	798	800	1185
512	537	796	1056	1573	1576	2350
1024	1060	1579	2090	3124	3127	4677



Since the priority of loading PEG engine command is low, it is recommended **not** to use 'SETSP' / 'GETSP' commands and change tuning parameters in parallel with the loading process, otherwise loading times can be higher than the times mentioned in the table

8.2.1.1.4 SPiiPlusCM / SPiiPlusSA / SPiiPlus3U PEG Compatibility

In distinction to SPiiPlusCM/SPiiPlusSA/SPiiPlus3U PEG operation, the following changes should be noted:

- > Time-based-pulses and time-based-period which are optional parameters of the PEG_R command used by SPiiPlusCM/SPiiPlusSA/SPiiPlus3U products are supported as mentioned in 8.2.1.
- > **ASSIGNPEG** - The command's parameters have been changed.

- > PEG interrupts are not supported.
- > In contrast to SPiiPlusCM/SPiiPlusSA/SPiiPlus3U, the PEG pulse is inversed. Use PEG_R/i or PEG_I/i to overcome this issue.

8.2.1.2 PEG Commands

This chapter specifies all PEG commands.

8.2.1.2.1 ASSIGNPEG

Description

The **ASSIGNPEG** function is used for engine-to-encoder assignment as well as for the additional digital outputs assignment for use as PEG pulse outputs. It allows mapping of PEG engines to be triggered by the feedback of a specific encoder.

Syntax

ASSIGNPEG[/f] *axis, engines_to_encoders_code, gp_out_assign_code*

Arguments

axis	The axis index, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1. Axis parameter can be any axis number of the same unit. The axis parameter determines the Servo Processor used.
engines_to_encoder_s_code	Bit code for engines-to-encoders mapping according to: SPiiPlusNT/DC-LT/HP/LD Processor 0 SPiiPlusNT/DC-LT/HP/LD Processor 1 SPiiPlusCMnt/UDMpm/UDMpc/CMba/CMhp/CMxa/UDMba/UDMxa/CMhv/UDMhv/UDMnt/UDMpa/UDMcb UDMlc/UDlIt/UDlhp/UDMmc/PDIcl NPMpm/NPMpc IDMsm, ECMsm, and UDMsm
gp_out_assign_code	General purpose outputs assignment to use as PEG pulse outputs according to: SPiiPlusNT/DC-LT/HP/LD SP 0 SPiiPlusNT/DC-LT/HP/LD SP 1 SPiiPlus CMnt/CMhv/UDMpm/UDMhv UDMnt/UDMpa/UDMcb IDMsm, ECMsm, UDMsm



The **axis** parameter actually serves to determine the Servo Processor used.

Comments

- > **ASSIGNPEG** is a blocking command - the ACSPL+ program moves to the next line or command only after this command has been fully executed or an error is generated.
- > The **axis** parameter can be any of the axes controlled by the same servo processor, the result will be the same.
- > If the "/f" switch is included, fast loading of Random PEG arrays is activated. This feature allows definition of state-arrays with more than 1024-members by using Random PEG. The **PEG_R** command must be called with the "/d" switch.
- > The **Bit Code** shown in the Mapping PEG Engines to Encoders tables affects all of the connectors in the row.



HSSI devices (HSSI-IO16, HSSI-ED2, etc.) cannot be used for the same Servo Processor when fast loading of Random PEG arrays is activated.
SPRT and **SPINJECT** commands cannot be used for the same Servo Processor when fast loading of Random PEG arrays is activated.

Related ACSPL+ Commands

PEG_I, ASSIGNPOUTS, PEG_R, STARTPEG, STOPPEG

COM Library Methods

None

C Library Functions

acsc_AssignPegNT

8.2.1.2.2 ASSIGNPOUTS

Description

The **ASSIGNPOUTS** function is used for assigning PEG engine output signals to physical output pins. In addition, the function allows assigning Fast General Purpose output pins and mapping between **FGP_OUT** signals to the bits of the ACSPL+ **OUT(x)** variable, where **x** is the index that has been assigned to the controller in the network during System Configuration.



The assignments can be obtained by running **#SI** in the SPiiPlus MMI Application Studio **Communication Terminal**. For example, the following is a fragment from the response to this command:

```
Axes Assignment: 8,9,10,11
Inputs/Outputs Assignment:
    Digital inputs (IN): 1.0,1.1,1.2,1.3,1.4,1.5,1.6,1.7
    Digital outputs (OUT): 1.0,1.1,1.2,1.3,1.4,1.5,1.6,1.7
```

OUT is an integer array that can be used for reading or writing the current state of the General Purpose outputs - see *SPiiPlus ACSPL+ Command & Variable Reference Guide*.

Each PEG engine has 1 PEG pulse output and 4 state outputs for a total of 5 outputs per PEG engine and a total of 30 outputs for the whole PEG generator. The controller supports 10 physical output pins that can be assigned to the PEG generator. The user defines which 10 outputs (of the 30) of the PEG generator are assigned to the 10 available physical output pins. Some of the output pins are shared between the PEG and the HSSI.

The tables in Appendix A.2 define how each of the 30 outputs of the 6 PEG engines can be routed to the 10 physical output pins - 4 PEG out signals, 3 PEG state signals, and 3 HSSI signals. Note that some of the signals cannot be routed to physical pins.



Bit Code: 111 is used for switching the physical output pins to Fast General Purpose Outputs, see [ASSIGNPOUTS](#).

Syntax

```
ASSIGNPOUTS axis, peg_output, bit_code
```

Arguments

axis	The axis index, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1. For controllers with firmware version 2.15 or higher, the axis parameter can be any axis number of the unit.
peg_output	The peg output number according to the Mapping of Engine Outputs to Physical Output tables below

bit_code	Bit code for engine outputs to physical outputs mapping according to: SPiiPlusNT/DC-LT/HP/LD SP 0 SPiiPlusNT/DC-LT/HP/LD SP 1 CMnt/UDMpm/UDMpc/CMhv/UDMhv CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa (OUT0-4) CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa (OUT5-9) UDMnt/UDMpa/UDMcb UDMIc/UDMmc/UDIlt/UDIhp/PDIcl NPMpm/NPMpc IDMsm/ECMsm/UDMsm Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, Outputs 0-5) IDMsm/ECMsm/UDMsm Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, Outputs 6-11) ECMsa/IDMsa/UDMsa Mapping of Engine Outputs to Physical Outputs (Servo Processor 0) ECMma / IDMma / UDMma Mapping of Engine Outputs to Physical Outputs (Servo Processor 0) ECMdx / IDMdx / UDMdx Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, Outputs 0-5) ECMdx / IDMdx / UDMdx Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, Outputs 6-11)
------------------------	--

Comments

ASSIGNPOUTS is a blocking command in the sense that the ACSPL+ program moves to the next line or command only after this command has been fully executed or an error is generated.

A separate **ASSIGNPOUTS** command should be called for every GP output or PEG output.

Examples

The following examples illustrate using the **ASSIGNPOUTS** in order to use PEG outputs as GP outputs

Example 1:

```
ASSIGNPOUTS 0, 2, 0b111
```

This defines the **Z_PEG** output as **FGP_OUT2** and maps it to the bit 18 of the ACSPL+ **OUT** variable (see [ASSIGNPOUTS](#)).

If you run, for example:

```
OUT (x) .18=0
```

Where **x** is the index assigned to the controller during System Configuration, **FGP_OUT2** output will be activated.

Then if you run:

```
OUT (x) .18=0
```

FGP_OUT2 will be deactivated.

Example 2:

```
ASSIGNPOUTS 4, 7, 0b111
```

This defines the **X_STATE2** output as **FGP_OUT6** and maps it to the bit 22 of the ACSPL+ **OUT** variable (see [ASSIGNPOUTS](#)).

Related ACSPL+ Commands

[ASSIGNPEG](#), [PEG_I](#), [PEG_R](#), [STARTPEG](#), [STOPPEG](#)

COM Library Methods

None

C Library Functions

`acsc_AssignPegOutputsNT`

8.2.1.2.3 STARTPEG

Description

The **STARTPEG** command initiates the PEG process on the specified engine. The command is used in both the Incremental and Random PEG modes by using /w switch in **PEG_I** or **PEG_R** command. If this switch is included, the execution of the **PEG_I** and **PEG_R** commands is delayed until the execution of the **STARTPEG** command.

Syntax

STARTPEG *peg_engine*

Arguments

peg_engine

The PEG engine number

Comments

STARTPEG is a blocking command in the sense that the ACSPL+ program moves to the next line or command only after this command has been fully executed or an error is generated.

If [STOPPEG](#) has been issued before the last PEG position, you have to use **STARTPEG** to resume PEG engine firings from the current point.

Valid numbers of the *peg_engine* parameter can be found in the #SI report. In case of multiple network units, the first axis number of each node indicates the first peg engine of the node.

Example

```
PEG_I/w 0, 0.003, 1000, 1000, 3000, 2, 0.01
PTP 0, 3000      !The program initiates synchronous PEG, with PTP
```

```
!motion on axis 0.
WAIT 2    !Two milliseconds after motion starts, PEG
          !will be initiated by STARTPEG
STARTPEG 0
```

Related ACSPL+ Commands

[ASSIGNPEG](#), [ASSIGNPOUTS](#), [STOPPEG](#)

Related ACSPL+ Variables**AST****COM Library Methods**

None

C Library Functions

acsc_StartPegNT

8.2.1.2.4 STOPPEG**Description**

The **STOPPEG** command terminates the PEG process immediately on the specified engine. The command is used in both the Incremental and Random PEG modes.

Syntax

STOPPEG *peg_engine*

Arguments

<i>peg_engine</i>	The PEG engine number
--------------------------	-----------------------

Comments

STOPPEG is a blocking command in the sense that the ACSPL+ program moves to the next line or command only after this command has been fully executed or an error is generated.

Valid values of the *peg_engine* parameter can be found in the #SI report. In case of multiple network units, the first axis number of each node indicates the first peg engine of the node.

Example

```
PEG_I 0, 0.003, 1000, 1000, 3000, 2, 0.01
PTP 0, 3000          !The program initiates synchronous PEG, with PTP
                    !motion on axis 0.
WAIT 2              !Two milliseconds after motion starts, PEG
                    !will be terminated by STOPPEG.
STOPPEG 0
```

Related ACSPL+ Commands

[ASSIGNPEG](#), [ASSIGNPOUTS](#), [PEG_I](#), [PEG_R](#), [STARTPEG](#)

COM Library Methods

None

C Library Functions

acsc_StopPegNT

8.2.1.2.5 PEG_I

Description

The **PEG_I** command is used for setting the parameters for the Incremental PEG mode.

Syntax

PEG_I[/awiecyzo] (*peg_engine*, *width*, *first_point*, *interval*, *last_point* [, *err_map_static_axis1*, *static_axis_coord1*, [, *err_map_static_axis2*, *static_axis_coord2*], [*err_map_max_size*,]], [*stable_dir_dist*,] [*time_based_pulses*, *time_based_period*])

Arguments

<i>peg_engine</i>	The PEG engine number
<i>width</i>	Width of the pulse in milliseconds. Valid range is 26.6 ns to 1.745 ms.
<i>first_point</i>	A real scalar value in user units indicating the first point for the PEG generation.
<i>interval</i>	A real scalar value in user units indicating the distance between PEG events.
<i>last_point</i>	A real scalar value in user units indicating the last point for PEG generation. Optional if the /e switch is used.
<i>err_map_static_axis1</i>	The index of the first static axis when error mapping is used with PEG. Use with the /y or /yz switches.
<i>static_axis_coord1</i>	The predefined location value of the first static axis when error map correction compensation is used with PEG. Use with the /y or /yz switches.
<i>err_map_static_axis2</i>	The index of the second static axis when error mapping is used with PEG. Use with the /yz switch.
<i>static_axis_coord2</i>	The predefined location value of the second static axis when error map correction compensation is used with PEG. Use with the /yz switch.
<i>err_map_max_size</i>	Maximum array size contained error correction data. Default is 512. Range from 1 to XARRSIZE . A negative value or 0 is interpreted as default. Available only if switch /m is used in combination with /y or /yz switches.
<i>stable_dir_dist</i>	(Optional) Limit defining actual motion as opposed to jitter. If motion along the axis is greater than this value, that motion is used to determine the direction of motion.

<i>time_ based_ pulses</i>	Optional parameter - a real scalar value indicating the number of time-based pulses generated after each encoder-based pulse, the range is from 0 to 65,535.
<i>time_ based_ period</i>	Optional parameter, a real scalar in milliseconds - period of time-based pulses, the range is from 0.00005334 to 1.7476. The time-based period must be at least pulse width + 26.6667 ns (minimum distance between two pulses).



PEG is generated only after the first pre-defined start point is reached. If the current encoder position exceeds pre-defined start point no PEG pulses are fired. It is recommended to activate the PEG engine before the maximum current position for movement in the positive direction and the minimum current position for movement in the negative direction.

Comments

- > If the switch: /w is included, the execution of the command is delayed until the execution of the **STARTPEG** command.
- > If the switch: /i is included, the PEG pulse output signal is inverted.
- > If the switch: /a is included, error accumulation is prevented by taking into account the rounding of the distance between incremental PEG events.
You must use this switch if **interval** does not match the whole number of encoder counts. Using this switch is recommended for any application that uses the **PEG_I** command, regardless if **interval** matches the whole number of encoder counts.
- > Valid numbers of the **peg_engine** parameter can be found in the #SI report. In case of multiple network units, the first axis number of each node indicates the first PEG engine of the node.
- > If the switch /e is included, the **last_point** parameter is optional and ignored, and the PEG process continues until the **STOPPEG** command is issued.
- > The /c switch supports 1D dynamic error compensation for Incremental PEG. 1D error compensation can be defined by the **ERRORMAP1D** or **ERRORMAP1ND** functions.
- > The /y switch supports 2D dynamic error compensation for Incremental PEG. 2D error compensation can be defined by **ERRORMAP2D**, **ERRORMAP2ND** functions. This switch requires 2 additional function arguments: *static axes index*, *static axes coordinates*.
- > The /z switch supports 3D dynamic error compensation for Incremental PEG. 3D error compensation can be defined by **ERRORMAP3DX** functions. This switch must be used in combination with switch /y (/yz) and requires 4 additional function arguments: *static axes 1 index*, *static axis 1 coordinate*, *static axes 2 index*, *static axis 2 coordinate*.
- > The /o switch is used with the **stable_dir_dist** parameter. The PEG process does not count motions smaller than this value as motion, for the purpose of determining the next

PEG firing. Only motion greater than this value will be used to determine the direction of axis motion.

Product Support

The /c, /e, /y, /yz, and /o switches are supported in the following products:

- > IDM/ECM/UDMsm
- > IDM/ECM/UDMsa
- > IDM/ECM/UDMma
- > IDM/ECM/UDMdx

Example

See the Incremental PEG example in the *PEG and MARK Operations Application Note*.

Error Mapping Example

```
! PEG engine 4, Pulse width=0.1ms , Start Pos. 500,
! Increment 200, Stop Pos. 1000, Use 1D error mapping
PEG_I/c 4, 0.1, 500, 200, 1000

! PEG engine 0, Pulse width=0.1ms , Start Pos. 500,
! Increment 200, Stop Pos. 1000,
! Use 2D error mapping for axis 1 coordinate = 500.
! Maximum error mapping table size = 1024
PEG_I/ym 0, 0.1, 500, 200, 1000, 1, 500, 1024
! PEG engine 0, Pulse width=0.1ms , Start Pos. 500,
! Increment 200, Stop Pos. 1000,
! Use 3D error mapping for axis 1 coordinate = 100,
! for axis 2 coordinate = 200
PEG_I/yz 0, 0.1, 500, 200, 1000, 1, 100, 2, 200
```

Related ACSPL+ Commands

PEG_R, ASSIGNPEG, ASSIGNPOUTS, STARTPEG, STOPPEG

Related ACSPL+ Variables

AST

COM Library Methods

None

C Library Functions

acsc_PegIncNT, acsc_WaitPegReady

8.2.1.2.6 PEG_R

8.2.1.2.7 PEG_R

Description

The **PEG_R** command is used for setting the parameters for the Random PEG mode.

Syntax

PEG_R[/wdyzmo] (*peg_engine, width, mode, first_index, last_index, POS_ARRAY, [STATE_ARRAY,] [, err_map_static_axis1, static_axis_coord1, [err_map_static_axis2, static_axis_coord2], [err_map_max_size,]] [time_based_pulses, time_based_period]*)

Arguments

<i>peg_engine</i>	The PEG engine number
<i>width</i>	Width of the pulse in milliseconds. Valid range is 26.6 ns to 1.745 ms.
<i>mode</i>	PEG state output signals configuration according to PEG_R. Bits 0-3 relates the PEG State 0 of the specific PEG engine Bits 4-7 relates the PEG State 1 of the specific PEG engine Bits 8-11 relates the PEG State 2 of the specific PEG engine Bits 12-15 relates the PEG State 3 of the specific PEG engine The most commonly used value is 0x4444 - PEG State Outputs 0-3 are configured with the 'State' option (bits 2, 6, 10, 14 are ON).
<i>first_index</i>	Index of first entry in the array for PEG generation
<i>last_index</i>	Index of last entry in the array for PEG generation
<i>POS_ARRAY</i>	The Random Event Positions array, maximum of 256/1024 members. If a longer array is required, use both PEG_R/d and ASSIGNPEG/f switches.
<i>STATE_ARRAY</i>	Optional parameter - the Outputs States array defining the four PEG output states, maximum of 256/1024 members. If a longer array is required, use both PEG_R/d and ASSIGNPEG/f switches.
<i>err_map_static_axis1</i>	The index of the first static axis when error mapping is used with PEG. Use with the /y or /yz switches.
<i>static_axis_coord1</i>	The predefined location value of the first static axis when error map correction compensation is used with PEG. Use with the /y or /yz switches.
<i>err_map_static_axis2</i>	The index of the second static axis when error mapping is used with PEG. Use with the /yz switch.
<i>static_axis_coord2</i>	The predefined location value of the second static axis when error map correction compensation is used with PEG. Use with the /yz switch.
<i>stable_dir_dist</i>	(Optional) Limit defining actual motion as opposed to jitter. If motion along the axis is greater than this value, that motion is used to determine the direction of motion.

<i>time-based-pulses</i>	Optional parameter - number of time-based pulses generated after each encoder-based pulse, the range is from 0 to 65,535.
<i>time-based-period</i>	Optional parameter - period of time-based pulses (milliseconds), the range is from 0.00005334 to 1.7476. Time-based period must be at least pulse width + 26.6667 nsec (minimum distance between two pulses).

Table 8-5. PEG Output Signal Configuration

PEG state output types:

"Three state" - PEG state output is not in use

"State" - PEG state output will be changed according to the STATE_ARRAY values

"Pulse" - PEG state output will be changed according to PEG pulse value

"Pulse & State" - PEG state output will be changed according to the result of AND operation between STATE_ARRAY values AND PEG pulse value

Pulse Polarity:

If positive or negative pulse is used as PEG pulse value for the specific "PEG State Output"

State Polarity:

If positive or negative state is used as PEG pulse value for the specific "PEG State Output"

Comments

- > If the switch **/w** is included, the execution of the command is delayed until the execution of the **STARTPEG** command.
- > If the switch **/i** is specified, the PEG pulse output signal is inverted.
- > If the switch **/d** is specified, dynamic loading of positions is used. Dynamic loading can only be implemented on a PEG engine with fast loading (**ASSIGNPEG /f**). In this case, it allows definition of a state-array which has more than 1024-members.
- > The **/y** switch supports 2D dynamic error compensation for Random PEG. 2D error compensation can be defined by **ERRORMAP2D**, **ERRORMAP2ND** functions. This switch requires 2 optional function arguments: *static axes index*, *static axes coordinates*.
- > The **/z** switch supports 3D dynamic error compensation for Random PEG. 3D error compensation can be defined by **ERRORMAP3DX** functions. This switch must be used in combination with switch **/y (/yz)** and requires 4 optional function arguments: *static axes 1 index*, *static axis 1 coordinate*, *static axes 2 index*, *static axis 2 coordinate*.
- > The parameters that can be set by the command differ from those that could be set for SPiiPlusCM/SPiiPlusSA/SPiiPlus3U controllers with the addition of the new *first_index* and *last_index* parameters.
- > When the PEG pulse is activated, the voltage between the two differential **PEG** outputs (+) and (-) drops to -5V. When the **PEG** pulse is de-activated, the voltage between the two differential **PEG** outputs is 5V.

- > In **PEG_R**, the number of position-based pulses is limited to eight pulses per controller cycle and the **minimum time interval must be >200nsec**.
- > When using a Sin-Cos encoder, **PEG** is triggered at the zero crossing of the sine-cosine waves and not at the precise interpolated position.
- > The last three arguments are optional. If **STATE_ARRAY** is omitted, the controller generates the PEG pulses at each position but does not change the state of any output. If time-based-pulses and time-based-period are omitted, the controller does not generate time based pulses.
- > The dynamic loading feature is limited by the loading frequency. If a high loading frequency is required, the loading capacity may not suffice to keep the FIFO loaded.
- > If the FIFO is emptied before all data arrays have been loaded, a memory overflow fault will be thrown.
- > Valid numbers of the `peg_engine` parameter can be found in the #SI report. In case of multiple network units, the first axis number of each node indicates the first peg engine of the node.
- > The `/m` switch supports random PEG on a modulo axis. The **POS_ARRAY** is loaded once and provides pulses every Modulo cycle. The Position values should be inside the **SLPMIN...SLPMAX** range. The PEG engine must be assigned to a Modulo axis by the **ASSIGNPEG** command.
- > The `/o` switch allows the definition of a minimum distance for which motion is considered actual motion as opposed to jitter. The name of the optional parameter is `stable_dir_dist`; motion greater than the value of this parameter is used to calculate motion direction.

Product Support

The `/m` and `/o` switches are supported in the following products:

- > IDM/ECM/UDMsm
- > IDM/ECM/UDMsa
- > IDM/ECM/UDMma

Loading Frequency Table

CTIME (ms)	Frequency (Hz)
1	200
0.5	400
0.25	800
0.2	1000

Example

See the Random PEG example in the *PEG and MARK Operations Application Note*.

Related ACSPL+ Commands

PEG_I, ASSIGNPEG, ASSIGNPOUTS, STARTPEG, STOPPEG

Related ACSPL+ Variables**AST****COM Library Methods**

None

C Library Functions

acsc_PegRandomNT, acsc_WaitPegReady

8.2.1.2.8 ASSIGNMARK**Description**

The **ASSIGNMARK** function allows assignment of Mark inputs to encoder. It allows a mapping of encoder latching to be triggered by using different physical input pins.

Syntax**ASSIGNMARK**[/*i*] *axis*, *mark_type*, *inputs_to_encoder_bit_code*

In newer products the *inputs_to_encoder_bit_code* parameter is actually a byte code. See [IDMxx/ECMxx/UDMsm/UDMsa/UDMma Encoder Mapping](#) table.

Arguments

Axis	The axis index, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1. The axis parameter determines which node unit is used. Axis parameter can be any axis number of the same unit.
mark_type	1 for Mark-1 assignment 2 for Mark-2 assignment
inputs_to_encoder_bit_code	Bit code for inputs-to-encoders mapping according to the following tables. The bit code determines which physical input pin leads to each encoder MARK latching. Mark-1 Inputs to Encoder mapping for Mark-1 Inputs to Encoders Mapping for SPiiPlusNT/DC-LT/HP/LD IDMxx/ECMxx/UDMsm/UDMsa/UDMma Encoder Mapping Mark-1 Inputs to Encoders Mapping for with SPiiPlus CMnt/UDMpm-x/UDMpc/CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa/CMhv/UDMhv Mark-2 Inputs to Encoder mapping for Mark-2 Inputs to Encoders Mapping for SPiiPlusNT/DC-LT/HP/LD Mark-2 Inputs to Encoders Mapping for with SPiiPlus CMnt/UDMpm/UDMpc/CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa/CMhv/UDMhv

Comments

Latching of Encoder <index> means **IST**(<index>).#MARK=1, and the **MARK**(<index>) variable value stores the feedback position of encoder <index> (**FPOS**(<index>)).

The Bit Code shown in the tables affects all of the connectors in the row.

Supported products that are not listed in the tables include only the default case.

If the switch: /i is included, the MARK input signal is inverted.

In IDMxx/ECMxx products, the latching for all encoders happens simultaneously.

8.2.1.3 ACSPL Plus Functions and Definitions

This section covers changes that have been incorporated in the ACSPL+ function set to accommodate PEG operations in NT.

8.2.1.3.1 GETCONF

Description

GETCONF retrieves system configuration data that was configured by **SETCONF**.



Some keys relate to data that is set by the system and not by **SETCONF**, for keys set by **SETCONF** see **SETCONF** Arguments.

Syntax

GETCONF(key,index)

Arguments

key	Specifies the configured feature.
index	Specifies the axis, buffer, or type of information requested.

GETCONF keys 71, 72, and 73 return values describing the PEG setup of a given system.

GETCONF(71, index)

Description

GETCONF(71, index) is used to view the actual assignment of digital outputs to PEG states and PEG pulses outputs.

Arguments

The index argument contains the axis index, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.

Return value

Depending on the axis, the function returns the bit code according to the relevant table below, depending on the system.

[Mapping PEG Engines to Encoders \(Servo Processor 0\) for SPiiPlusNT/DC-LT/HP/LD](#)

[Mapping PEG Engines to Encoders \(Servo Processor 1\) for SPiiPlusNT/DC-LT/HP/LD](#)

Mapping PEG Engines to Encoders (Servo Processor 0) for SPiiPlus
CMnt/CMhv/CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa/UDMhv/UDMnt/UDMpa/UDMpm/UDMpc/UDMcb

Mapping PEG Engines to Encoders (Servo Processor 0) for UDMlc/UDlIt/UDlhp/UDMmc/PDIcl

Mapping PEG Engines to Encoders (Servo Processor 0) for NPMpm/NPMpc-

GETCONF(72, index)

Description

GETCONF(72, index) is used to view the actual encoder PEG engine assignment.

Arguments

The index argument contains the axis index, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.

Return value

Depending on the axis, the function returns the bit code according to the relevant table below, depending on the system.

General Purpose Outputs Assignment for Use as PEG Pulse Outputs (Servo Processor 0) for SPiiPlusNT/DC-LT/HP/LD

General Purpose Outputs Assignment for Use as PEG Pulse Outputs (Servo Processor 1) for SPiiPlusNT/DC-LT/HP/LD

General Purpose Outputs Assignment for Use as PEG Pulse Outputs (Servo Processor 0) for SPiiPlus
CMnt/UDMpm/CMhv/UDMhv-

General Purpose Outputs Assignment for Use as PEG Pulse Outputs (Servo Processor 0) for UDMnt/UDMpa/UDMcb

Engine to Encoder Assignment for IDMxx, ECMxx, and UDMxx

GETCONF(73, index)

Description

GETCONF(73, index) is used to view the actual output pins assignment.

Arguments

The index argument has to following meaning:

Bits 0 - 7 - Axis Index

Bits 8 - 16 - 0 for OUT_0, 1 for OUT_1, ..., 9 for OUT_9

Return value

Depending on the axis, the function returns the bit code according to the relevant table below, depending on the system.

Mapping of Engine Outputs to Physical Outputs (Servo Processor 0) for SPiiPlusNT/DC-LT/HP/LD

Mapping of Engine Outputs to Physical Outputs (Servo Processor 0) for
CMnt/UDMpm/UDMpc/CMhv/UDMhv

Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, OUT 0-4) for
CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa

Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, OUT_5-9) for CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa

Mapping of Engine Outputs to Physical Outputs (Servo Processor 0) for UDMnt/UDMpa/UDMcb

8.2.1.3.2 *AST.#PEGREADY*

Description

AST is an integer array containing a set of bits used for displaying the current axis state. Bit 4 (**#PEGREADY**) has been added to indicate the readiness of a PEG engine.

Syntax

AST(*AXIS#*).**#PEGREADY**

AXIS# identifies the axis and can take on values of *AXIS0* through *AXIS_Last*, where *_Last* is the number of axes in the system minus 1.

Return value

- > 0 - Indicates that the engine is not configured and motion should not be started.(default)
- > 1 - Indicates that all values are loaded and the PEG engine is ready to respond to movement (the user can start a motion).

Comments

The **#PEGREADY** bit is reset to '0' when PEG is activated.

8.3 Sin-Cos Encoder Multiplier Configuration

The sin-cos encoders are encoders with analog outputs.

For convenient modification of encoder related variables, use the SPiiPlus MMI Application Studio:

Toolbox → Setup → System and Faults Configurator, see *SPiiPlus MMI Application Studio User Guide* for details..

8.3.1 Sin-Cos Encoder Multiplier

The sin-cos encoder multiplier provides a combination of high speed and high resolution that cannot be achieved with an external encoder multiplier producing a digital quadrature signal.

Sin-cos encoder multiplier is an optional feature. You need to specify the number of sin-cos encoder multipliers when you order the controller.

8.3.1.1 Technical Data

- > Maximum multiplication factor: 65536 counts per encoder sine signal period.
- > Maximum velocity: 42×10^9 counts/sec or 641×10^3 sine periods/sec.
- > Maximum acceleration: 65536×10^8 counts/sec² or 10^8 sine periods/sec².
- > The encoder-related controller features: **index**, **Mark**, and **PEG**, do not support the full resolution of the multiplier. Resolution for these features is limited to 4 counts per encoder sine signal period.

8.3.1.2 Configuring the Sin-Cos Multiplier

You specify the sin-cos encoder multiplier as a feedback source using the ACSPL+ **E_TYPE** variable (see *SPiiPlus Command & Variable Reference Guide*). The variable is an array sized according to the number of axes. If an element in **E_TYPE** is set to 4, the corresponding axis will use the sin-cos encoder multiplier as a feedback source. You can select any axis to use the sin-cos encoder multiplier, but the total number of multipliers is limited to the number ordered with the controller (the allowed number is hardwired in the PAL). If you try to select more multipliers than allowed, the controller issues error 1148.

The ACSPL+ variable **E_SCMUL** (see *SPiiPlus Command & Variable Reference Guide*) specifies the desired value of multiplication as a power of 2. The maximum value of 16 corresponds to a multiplication of $65536 = 2^{16}$. The minimum value of 2 corresponds to a multiplication of $4 = 2^2$.

An axis that uses the sin-cos encoder multiplier is not different from any other axis in the controller. Any motion can involve the multiplier axis. The multiplier and non-multiplier axes can be used simultaneously in one multi-axis motion.

The following example shows how the encoder multiplier affects the **EFAC** (see [User-Defined Units](#)) calculation:

Assume the encoder has 250 lines per mm and you assign **E_SCMUL** the value 10, which provides multiplication $\times 1024$ (i.e., 2^{10}). The desired programming unit is millimeter. In this case you have to specify **EFAC** as:

$$1/(250 \times 1024) = 0.00000390625$$

Encoder-related features: index, Mark, PEG use the same user unit. However, actual resolution of these functions is lower than the resolution of encoder feedback. As mentioned above, the resolution of index, mark and PEG corresponds to 4 counts per encoder sine period. In the former example the resolution will be $1/(250 \times 4) = 0.001\text{mm}$.

The controller continuously checks the integrity of the encoder multiplier feedback. If any error occurs, the controller activates the Encoder Error fault.

8.4 Interrupts

Typically, you work with the SPiiPlus interrupts using the SPiiPlus C Library and do not need the low-level details specified in this section. Refer to the *SPiiPlus C Library Reference* document for explanation of the interrupt managing functions.

8.4.1 Hardware Interrupts

The hardware interrupt status registers contain the following bits:

Bit	Description
3	PEG 0
4	PEG 1
5	PEG 3
6	PEG 4

Bit	Description
7	MARK 1 0
8	M2ARK 0
9	MARK 1 2
10	M2ARK 2
11	MARK 1 3
12	M2ARK 3
13	MARK 1 4
14	M2ARK 4
15	Emergency Stop

8.4.2 Software Interrupts

The software interrupt status register contains the following bits:

Bit	Description
16	Physical motion end
17	Logical motion end
18	Motion failure (Motion interruption due to a fault)
19	Motor failure (Motor disable due to a fault)
20	Program termination
21	ACSPL+ interrupt with user parameter (by INTERRUPTEX command)
22	ACSPL+ interrupt (by INTERRUPT command)
23	Digital input
27	New motion segment is required to be provided by the application
28	System error (the controller raises this interrupt if system error occurred)
29	EtherCAT error (the controller raises this interrupt if EtherCAT error occurred)

Bit	Description
30	Controller cycle (the controller raises this interrupt in the beginning of each controller cycle)
31	Communication interrupt (the controller raises this interrupt after sending a complete message to the FIFO)

When a software interrupt occurs, the corresponding tag contains detailed information about the interrupt source. For example, the tag of the Physical Motion End interrupt specifies the axes that caused the interrupt. When a specific software interrupt has occurred, the next interrupt of the same type can be generated only after the host drive reads both interrupt status register and writes zero to the corresponding tag.

8.4.3 Software Interrupt Tags

The following tags are available:

Bit	Description
16	Details of Physical motion end interrupt. Bit 0 specifies that axis 0 has finished, bit 1 - axis 1, and so on.
17	Details of Logical motion end interrupt. Bit 0 specifies that axis 0 has finished, bit 1 - axis 1, and so on.
18	Details of Motion failure interrupt. Bit 0 specifies that axis 0 has failed, bit 1 - axis 1, and so on.
19	Details of Motor failure interrupt. Bit 0 specifies that axis 0 has failed, bit 1 - axis 1, and so on.
20	Details of Program termination interrupt. Bit 0 specifies that buffer 0 has finished, bit 1 - buffer 1, and so on.
21	Details of ACSPL+ interrupt with user parameter (by INTERRUPTEX command). Bits 0..63 specify the user parameter.
22	Details of ACSPL+ interrupt (by INTERRUPT command). Bit 0 specifies interrupts from buffer 0, bit 1 - buffer 1, and so on.
23	Details of Digital input interrupt. Bit 0 specifies interrupts from input 0, bit 1 - input 1, and so on.

Bit	Description
27	Details of new motion segment interrupt. Bit 0 specifies that a new motion segment must be provided by the application for axis 0, bit 1 - axis 1, and so on.
28	Details of system error interrupt. Bits 0..63 specify the error that occurred.
29	Details of EtherCAT error interrupt. Bits 0..63 specify the error that occurred.

8.4.4 Interrupt Configuration Variables

The following ACSPL+ variables enable/disable interrupts from a specific source:

- > **IENA** - Scalar variable that enables/disables all interrupts that belong to a specific interrupt status bit.
- > **ISENA** - Array that enables/disables interrupts within a specific interrupt status bit. Each elements corresponds to one interrupt status bit and specifies which axes or buffers or inputs are enabled to cause interrupt.

8.4.4.1 IENA Variable

The **IENA** variable contains the following bits:

Bit	Description
3	Enable PEG axis 0 interrupt
4	Enable PEG axis 0 interrupt
5	Enable PEG axis 2 interrupt
6	Enable PEG axis 2 interrupt
7	Enable MARK 1axis 0 interrupt
8	Enable M2ARKaxis 0 interrupt
9	Enable MARK 1 axis 1 interrupt
10	Enable M2ARK axis 1 interrupt
11	Enable MARK 1 axis 2 interrupt
12	Enable M2ARK axis 2 interrupt

Bit	Description
13	Enable MARK 1 axis 3 interrupt
14	Enable M2ARK axis 3 interrupt
15	Enable Emergency Stop interrupt
16	Enable Physical motion end interrupt
17	Enable Logical motion end interrupt
18	Enable Motion failure (Motion interruption due to a fault) interrupt
19	Enable Motor failure (Motor disable due to a fault) interrupt
20	Enable Program termination interrupt
21	Enable ACSPL+ interrupt with user parameter (by INTERRUPT command)
22	Enable ACSPL+ interrupt (by INTERRUPT command)
27	Enable new motion segment interrupt
28	Enable system error interrupt
29	Enable EtherCAT error interrupt

8.4.4.2 ISENA Variable

The **ISENA** variable is an array containing the following elements:

Bit	Description
16	Controls Physical motion end interrupt. Bit 0 enables interrupts from axis 0, bit 1 - axis 1, and so on.
17	Controls Logical motion end interrupt. Bit 0 enables interrupts from axis 0, bit 1 - axis 1, and so on.
18	Controls Motion failure interrupt. Bit 0 enables interrupts from axis 0, bit 1 - axis 1, and so on.
19	Controls Motor failure interrupt. Bit 0 enables interrupts from axis 0, bit 1 - axis 1, and so on.

Bit	Description
20	Controls Program termination interrupt. Bit 0 enables interrupts from buffer 0, bit 1 - buffer 1, and so on.
22	Controls ACSPL+ interrupt (by INTERRUPT command). Bit 0 enables interrupts from Buffer 0, bit 1 - buffer 1, and so on.
23	Controls Digital input interrupt. Bit 0 enables interrupts from input 0, bit 1 - input 1, and so on.
27	Controls new motion segment interrupt. Bit 0 enables interrupt that new motion segment is required to be provided by the application for axis 0, bit 1 – for axis 1, and so on.

8.5 Dynamic Braking

The dynamic brake reduces the motor runoff if the motor becomes disabled during motion. In dynamic braking the controller short-circuits the motor armature.

If dynamic braking is enabled for an axis, the controller applies the braking when the feedback velocity falls below the value specified by the **VELBRK** parameter (default - 0).

The **MFLAGS.#DBRAKE** bit enables dynamic braking (default - off).

8.6 Constant Current Mode

The Constant Current mode provides extra safety. When the mode is activated, and the emergency stop signal is on, the motor is kept at a standstill by the drive.

- > The function, **SETCONF (133, 1, 1)** enables constant current mode for axis 1. The function, **SETCONF (133, 1, 0)** disables constant current mode for axis 1.
- > To retrieve the Constant Current mode status for axis 1, **GETCONF (133, 1)** is used. It retrieves a non-zero value if Constant Current mode is on and zero if Constant Current mode is off.

The drive activates constant current mode only if all of the following conditions are true:

- > Emergency Stop (**ES**) signal is on.
- > All axes are disabled.
- > Dynamic brake mode is off (**MFLAGS(AXIS).#DBRAKE = 0**).

Only once these conditions are true can the constant current mode be enabled by the function **SETCONF (133, 1, 1)**. If any of the conditions is changed, it will deactivate the constant current mode.

To set/get the current level used for constant current mode, the following **SETCONF/GETCONF** parameters are used:

- > **SETCONF (130, {1|5}, current level)**, where current level defines that the constant current level as a percentage of the maximum current for the specified axis (1 or 5 only). The default maximum value that can be set is 16% for axis 1 and 30% for axis 5 (this value can be changed, see description of key 131 below).

- > **GETCONF (130, {1|5})** retrieves the present value of current level for the specified axis.

The default value of current level is 0, so it must be defined before constant current mode can be used.

To set/get the maximum value that can be set with **SETCONF (130, . . .)**, the following **SETCONF/GETCONF** parameters are used:

- > **SETCONF (131, {1|5}, max_current_level)**, where **MAX_CURRENT_LEVEL** defines the maximum allowable value of the current level as a percentage of the maximum current for the specified axis (1 or 5 only).
- > **GETCONF (131, {1|5})** retrieves the present maximum allowable current level for the specified axis.

The following ACSPL+ program illustrates how to implement constant current mode:

```
! Constant current mode implementation
on S_FAULT.#ES
disableall
wait 100
MFLAGS(1).#DBRAKE = 0; MFLAGS(5).#DBRAKE = 0 !Disable Brake Mode for axes
1, 5
setconf(130, 1, 10) ! set constant current level (10%) for axis 1
setconf(130, 5, 20) ! set constant current level (20%) for axis 5
setconf(133, 1, 1) ! enable constant current mode
ret
on ^S_FAULT.#ES | S_FAULT.#DRIVE
setconf(133, 1, 0) ! disable constant current mode
ret
```

8.7 Hall Sensor Commutation

- > Hall sensor commutation requires a first time adjustment from the commutation dialog in the SPiiPlus MMI Application Studio Adjuster wizard (see *SPiiPlus MMI Application Studio User Guide* for details).
- > In subsequent power-ups, the motor will start moving according to the Hall sensors until it encounters the first change in the Hall sensors. At this point, commutation will switch to full sinusoidal commutation.
- > The quality of commutation relies on the physical alignment of the Hall sensors relative to the magnetic field of the motor, and in most cases is done very accurately by the motor manufacturers. Using this method, factors like high friction, vertical load, etc. have no effect on the commutation quality.
- > The connection sequence of the three Hall sensors is not important. Simply verify that the three sensors are connected, and that the Hall counter counts 0,1,2,3,4,5. It does not matter if the Hall counters count is opposite of the encoder. This situation is identified and dealt with during the initial adjustment.

The variables and functions for supporting Hall operations are:

Table 8-6. Variables and Functions for Hall Support

Variable/Function	Description
SLCHALL	Holds the Hall shift. The Adjuster Wizard commutation program calculates this parameter and saves it. Do not change this parameter manually.
MFLAGS.27	If this bit = 1, commutation will be based on the Hall sensors.
MFLAGS.28	Hall direction inversion. The Adjuster Wizard commutation program calculates this parameter and saves it. Do not change this parameter manually. 1 = Controller inverts Hall direction.
GETCONF(110, Index)	Returns the Hall counter of the axis specified by Index. The Hall direction bit is not taken into account. This function is used by the Adjuster commutation program.
GETCONF(111, Index)	This function is used by the Adjuster Wizard during commutation.
GETCONF(262, Index)	Returns the current Hall state, which can be 0, 1, 2, 3, 4, or 5, of the axis given by Index (a number: 0, 1, 2, ... up to the number of axes in the system minus 1). It returns -1 for invalid states



After hardware reset, if the Hall commutation was successful, the firmware automatically sets bit **MFLAGS.9** = 1, if **MFLAGS.27** = 1.
For proper Hall commutation, the encoder resolution and number of poles should be defined correctly. The current loop should be adjusted.

8.8 Communicating with the SPiiPlus C Library

This section provides an overview of the SPiiPlus C Library communication channels.

8.8.1 Remote Connection

The C Library installed on a specific computer supports not only applications running on the same computer, but also applications on remote computers. The only requirement is for a TCP/IP connection between the computers.

This illustrates the simultaneous connection of two local and one remote applications.

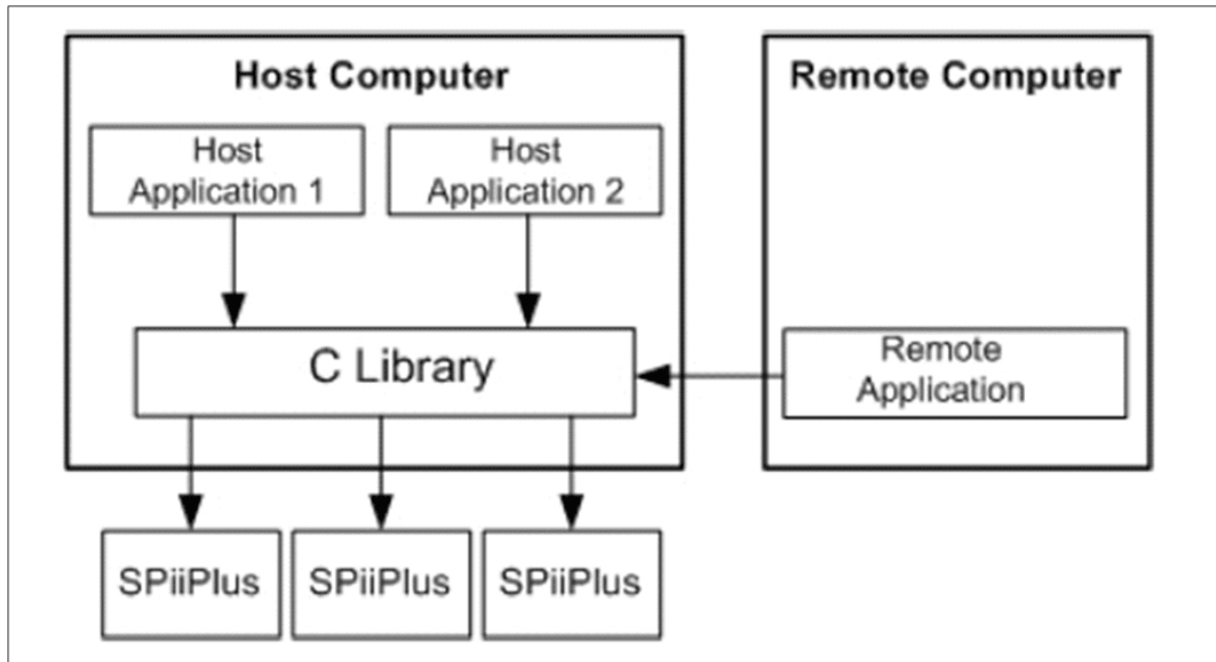


Figure 8-6. Simultaneous Connection for Remote Support

8.8.2 Callbacks in all Communication Channels

The Callback mechanism provides the fastest response to the controller events. The implementation is transparent for user applications. In all communication channels, the callback API consists of the functions, **SetCallback** and **SetCallbackExt**.

8.8.2.1 Timing

Callback operations include sending/receiving a message that requires much more time. Specific rates depend on the communication channel rate.

From the viewpoint of callback mechanism, all communication channels are functionally equivalent, but differ in timing.

8.8.2.2 Software Interrupts

The following interrupts are generated by the controller firmware and therefore are called software interrupts:

- > Physical motion end
- > Logical motion end
- > Motion failure (Motion interruption due to a fault)
- > Motor failure (Motor disable due to a fault)
- > Program termination
- > Command execution
- > ACSPL+ interrupt (by **INTERRUPT** command)

- > Digital input
- > Logical motion start
- > Motion phase change
- > Trigger

8.8.2.3 Hardware Interrupts

The following interrupts are generated by the controller hardware and therefore are called hardware interrupts:

- > Emergency stop
- > Mark 1 (axes 0, 1, 2, 3)
- > Mark 2 (axes 0, 1, 2, 3)
- > PEG (axes 0, 1, 2, 3)

The hardware interrupt callback conditions in different communication channels are:

Table 8-7. Hardware Interrupt Callback Conditions

Callback	Condition of Alert Message
Emergency stop	The message is sent when bit S_FAULT.#ES changes from zero to one. The message is disabled if S_FMASK.#ES is zero.
Mark 1 and Mark 2	The message is sent when corresponding IST.#MARK or IST.#MARK2 bit changes from zero to one.
PEG	The message is sent when the corresponding AST.#PEG bit changes from one to zero.

8.8.3 TCP/IP Port Assignment for Remote Connection

The C Library installed on a specific computer supports not only applications running on the same computer, but also applications on remote computers. The only requirement is for a TCP/IP connection between the computers.

8.8.3.1 TCP/IP Port Assignmnet



For a description of all the UMD functions see *SPiiPlus Utilities User Guide*.

To establish a remote connection using TCP/IP, select **Enable Access from Remote Application** on the User Mode Driver (UMD) **Remote Connection** tab.

If the default port is not busy, no communication error messages are encountered, and no problem is anticipated. In this case, use the function `acsc_SetServerExt` with the **ACSC_DEFAULT_REMOTE_PORT** parameter to set the remote port address from the user application.

If the default port (9999) is busy, the UMD will return the following error message:

Requested port 9999 is in use by another application. Select another port in the Remote Connection tab.

In this case, proceed as follows:

1. From the **Remote Connection** tab, select **Change** from the **Remote Port Connection** list.
2. Enter the remote port address in the **Enter valid port number** dialog.
3. From the **Remote Connection** tab, select **Enable Access from Remote Application**. As soon as the check box is selected, communication with the remote port is attempted. If communication does not succeed, the following error message appears:

Requested port [port number] is in use by another application. Select another port in the Remote Connection tab.
4. Repeat Steps 1-3 until communications are established. When communications are successfully established, the UMD stores the settings.
5. Click **OK** and then **Close**.
6. In the user application, use the function `acsc_SetServerExt` and specify the same port number that was entered in the UMD GUI.



Every time the User Mode Driver (UMD) initializes, the availability of the specified port is checked. If the system configuration or port number have changed, the UMD generates an error message and the **Enable Access from Remote Application** on the UMD **Remote Connection** tab check box will be cleared.

8.8.3.2 Disabling Remote UMD Connections

After installation, the remote connection is disabled.

To enable the remote connection, select **Enable Access from Remote Application** on the UMD **Remote Connection** tab. The remote application can now connect to the UMD, until it is disabled.

Disable the remote connection as follows:

1. Clear **Enable Access from Remote Application** on the UMD **Remote Connection** tab.
2. Restart the UMD for the changes to take effect.

8.8.3.3 UMD Log Types

The UMD logs constantly at run-time. The data is stored in binary format in an internal cyclic buffer and is translated to text just before it is written to file.

You may choose one of two mutually exclusive log types:

- > **Dump on Request** – all the binary data that is stored in the internal binary buffer and is flushed by explicit request to the file, see `acsc_FlushLogFile`.
- > **Continuous** – this is a background thread that takes care of periodic file updates. It reads the binary buffer and performs text formatting to the file.

Perform the **Dump on Request** as follows:

1. From the UMB **Log Settings** tab, select **Dump on request** as the **Log File Mode**.

2. Select the Log file path

Refer to the screen shot and note that **Dump** button appears and that **Maximum logging time** is disabled.

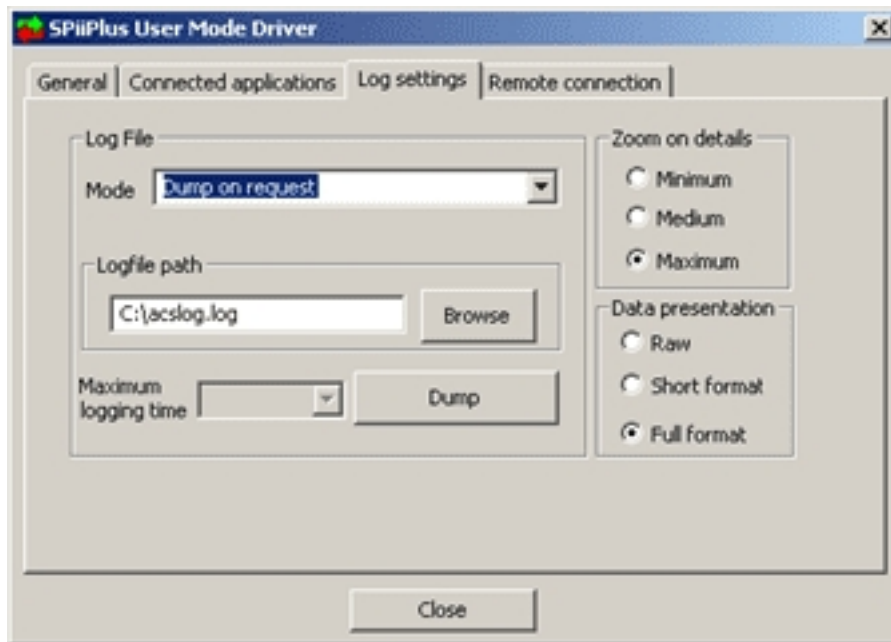


Figure 8-7. UMD Log Settings - Dump on Request

Perform a **Continuous Log File** as follows:

1. From the UMD **Log Settings** tab, select **Continuous** as the **Log File Mode**.
2. Select the Log file path.
Refer to the screen shot below and note that the **Start Log** button appears and that **Maximum logging time** is enabled.
3. Set **Maximum logging time**. Possibilities range from one hour to One week.



The **Infinite** setting is not recommended because of a possible disk overflow situation.

4. Click **Start Log**.

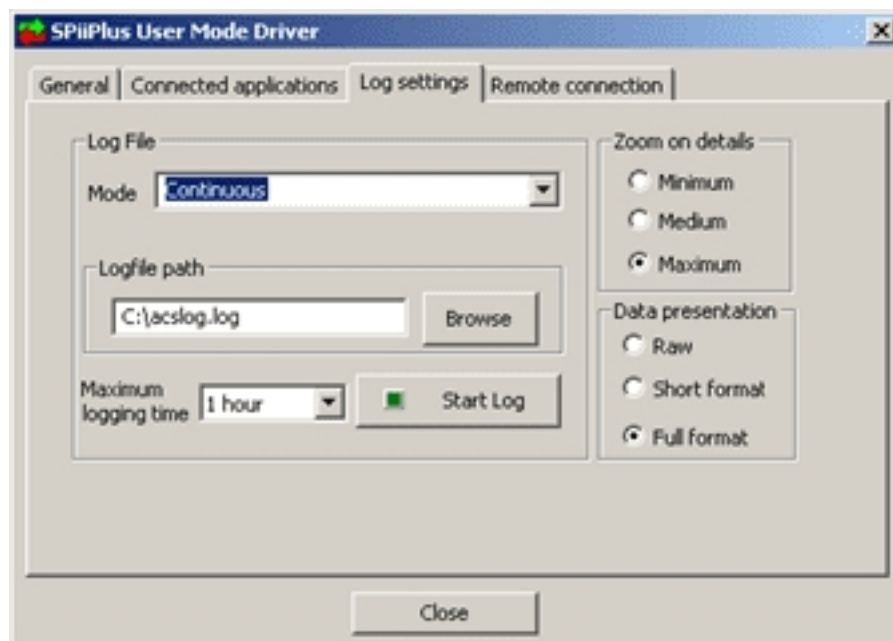


Figure 8-8. UMD Log Settings - Continuous

5. After clicking **Start Log**, and the log is active, and the **Stop Log** button appears.

8.8.3.4 Unloading the UMD from Memory

Unload the UMD from memory as follows:

1. Right click the UMD icon in the status area
2. Click **Unload**.
3. Click **OK** on the confirmation message. The UMD is then unloaded from the memory.

8.9 Communicating with Non-ACS Devices

Another use of the **SETCONF** and **GETCONF** functions (see *SPIiPlus Command & Variable Reference Guide*) relates to external communication channels. The functions are used to facilitate the following tasks:

- > Connection to user panel (teach pendant, console) that make use of special communication protocols (Modbus, etc.)
- > Connection to devices with special interfaces (laser interferometer, intelligent drives, etc.)



Currently only serial channels can be connected to devices such as these. All functions described below are implemented for RS232 serial channels only.

8.9.1 Channel Configuration Report

The **#CC** terminal command reports the current configuration of communication channels.

The controller responds with a table that specifies the configuration for each supported communication channel.



For any specific hardware configuration, not all listed channels may be physically available.

The following example illustrates the report:

```
#CC
Channel Type      Commandant
1 Serial          Command Rate:115200
2 Serial          Special Rate:9600 Parity:Even
10 TCP/IP         Command Connection:point-to-point Peer:10.0.0.96
```

The report describes the following channels:

- > 1 - COM1, in command interpretation mode, rate 115200 kbps, no special options, i.e. no parity bit, normal stop bit, no break.
- > 2 - COM2, in special mode, rate 9600 kbps, even parity bit, normal stop bit, no break.
- > 10 – Ethernet (TCP/IP protocol), in command interpretation mode, point-to-point connection, currently connected to peer address 10.0.0.96.

8.9.2 Assigning COM Channel for Special Input

The **SETCONF** function with key 302 assigns a communication channel for special input as follows:

SETCONF(302, channel, {0|1})

Where:

channel	Indicates the assigned COM channel, it can be one of the following values: 1 – COM1 2 – COM2
----------------	--

The values that can be used in connection with the channel are:

- > 1 – assigns the channel for special input
- > 0 – returns the channel to regular command processing.

If a channel is assigned for special input, the controller does not process commands from this channel. Any input from this channel can be processed only by the **INP** function (see [INP Function](#)). Output to the channel is provided by regular **DISP** and **SEND** commands (see *SPIIPlus Command & Variable Reference Guide*).



While a channel can be set to special communication mode by the **SETCONF** command through the same communication channel, the channel cannot be returned to normal mode through the same channel. The reason is that in special communication mode the channel does not process commands and therefore cannot execute **SETCONF**. You have to use another channel to return the channel to the regular command processing mode.

The **GETCONF** function with key 302 retrieves the state of a communication channel as follows:

GETCONF(302, channel)

The function returns:

- > 0 – if the channel is in normal command-processing mode
- > 1 – if the channel is in special-communication mode

8.9.3 Setting Communication Parameters

Currently, the **SETCONF** function with key 303 is supported only for serial channels (channel number 1 or 2). The function configures the communication rate for the specified channel as follows:

SETCONF(303, channel, baud)

Where:

channel	Indicates the assigned COM channel, it can be one of the following values: 1 – COM1 2 – COM2
baud	Specifies the communication rate in the channel.

The most popular communication rates are the following (kbps):

- > 115200
- > 57600
- > 19200
- > 9600
- > 4800
- > 2400
- > 1200
- > 600
- > 300

The function **GETCONF** with key 303 retrieves the communication rate of the specified channel as follows:

GETCONF(303, channel)

8.9.4 Parity, Stop Bit

The **SETCONF** function with key 304 set communication options for the channel.

```
setconf(304, channel, options)
```

Parameters	Description
channel	Channel specifies the channel (1- COM1, 2 - COM2)
options	Option is bit mask of the communication options. The bits are interpreted as follows: > Bit 2: 0 - normal stop bit, 1 - extended stop bit > Bit 3: 0 - no parity, 1 - check parity > Bit 4: 0 - odd parity, 1 - even parity

Example

In order to save the specified setting in the controller, the following ACSPL+ program should be prepared and saved in one of the available buffers.



The program has an AUTOEXEC label that means that it will automatically start after the controller power-up.

```
AUTOEXEC:
CONID = 32                ! Set Slave address 32
setconf(302, 2, 2)        ! Set COM2 for MODbus communication
setconf(303, 2, 19200)    ! Set baud rate 19200
setconf(304, 2, 0x18)     ! Set even parity
STOP
```

8.9.5 INP Function

Description

The **INP** function reads data characters from the specified channel and stores them into integer array.

Syntax

```
int INP (channel [,variable] [,start_index] [,number] [,timeout])
```

Arguments

channel	Indicates the assigned COM channel, it can be one of the following values: 1 – COM1 2 – COM2
variable	Name of user-defined integer array.

start_index	Index of the array.
number	Specifies number of characters to read.
timeout	Maximum waiting time (in milliseconds) for response from the channel.

The function returns the number of actually assigned characters.

If the **variable** argument is omitted, the function dumps all characters received before in the channel. If the **variable** argument is specified, the function accepts one or more characters from the specified channel and assigns them to the sequential elements of the **variable** array.

Each ASCII character is represented by its numerical value and is stored in a separate element of the array.

If **start_index** is specified, the first received character is assigned to the array element with the specified index. If **start_index** is omitted, the assignment starts from the first element of the array.

If **number** is specified, the function does not return until the exact number of characters is received. Any received character, including carriage return and other non-printable characters, is stored in the array. In this case the function return value is strictly equal to **number**.

If **number** is omitted, the function continues receiving characters until the last element of array is assigned or the carriage return character is received. The received carriage return is not stored in the array. The function return indicates the number of assigned array elements.

If **timeout** is specified, the function waits for input not more than the specified number of milliseconds. If **timeout** is omitted, the waiting time is not limited.

8.9.6 String Handling Commands

Text handling commands are used for sending text to the host for purposes of displaying on the monitor or for recording in a log.

8.9.6.1 DISP Command

Description

The **DISP** command builds an ASCII output string and sends it to a communication channel. Upon receipt, the host displays the message on the monitor.

Syntax

DISP *expression* | "*string*" [, *expression* | "*string*". . .]

Arguments

expression	ACSPL+ expression (can be a single variable)
string	A string, which must be enclosed with double quotation marks.

A string argument has the format of:

"[text] [escape-sequence] [format-specifier] . . ."

Where:

text	Any ASCII text characters
escape-sequence	The escape-sequence can be: \r - Carriage return 0x0d \n - New line 0x0a \t - Horizontal tabulation 0x09 \xHH - Any character. The two hexadecimal digits, HH, represent the character's ASCII code.
format-specifier	The format specification syntax adheres to a restricted version of the C language syntax: % [width] [.precision] type width - Optional number that specifies the minimum number of characters in the output. .precision - Optional number that specifies the maximum number of characters printed for all or part of the output field, or the minimum number of digits printed for integer values type - Required character that determines whether the associated argument is interpreted as a character, a string, or a number (see table below).

Table 8-8. String Format Type

Character	Output Format
d	Signed decimal integer.
l	Signed decimal integer.
o	Unsigned octal integer.
u	Unsigned decimal integer.
x	Unsigned hexadecimal integer, using "abcdef."
X	Unsigned hexadecimal integer, using "ABCDEF."

Character	Output Format
e	Signed value having the format: [-]D.DDDD E [SIGN]DDD Where: d is a single decimal digit dddd is one or more decimal digits ddd is exactly three decimal digits e indicates exponent sign is + or -.
E	Identical to the E format except that E rather than E indicates the exponent.
f	Signed value having the format: [-]DDDD.DDDD Where dddd is one or more decimal digits. The number of digits before the decimal point depends on the magnitude of the number, and the number of digits after the decimal point depends on the requested precision.
g	Signed value printed in F or E format, whichever is more compact for the given value and precision. The e format is used only when the exponent of the value is less than -4 or greater than or equal to the precision argument. Trailing zeros are truncated, and the decimal point appears only if one or more digits follow it.
G	Identical to the G format, except that E , rather than E , introduces the exponent (where appropriate).

If an input string argument contains n format specifiers, the specifiers apply to the n subsequent expression arguments.

The **DISP** command processes the arguments from left to right. The processing is as follows:

- > Expressions:
The expression is evaluated and the ASCII representation of the result is placed in the output string. The format of the result is determined by a formatting specification (if any) in the input string.
- > Input strings:
Text is sent as-is to the output string. Escape sequences are replaced by the ASCII codes that they represent. Formatting specifications are applied to the results of any expressions that follow the string.

Examples:

DISP "%15.10f",FPOS(0)	Display value of FPOS(0) in 15 positions with 10 digits after the decimal point
DISP "%1i",IN0.2	Display current state of IN0.2 as one digit 0 or 1.

DISP "FVEL(0)=%15.10f",FVEL(0)	Display value FVEL(0) with 10 decimal points, e.g., displayed output is: FVEL(0)= 997.2936183303
DISP "INO as hex: %04X",INO	Display value of INO in hex, e.g., displayed output is: INO as hex: 0A1D
disp "INO.0-3 as binary: %1U%1U%1U%1U", INO.0,INO.1,INO.2,INO.3	Display values of bits 0-3 of INO as binary, e.g., displayed output is: INO.0-3 as binary: 0110

The output string is sent to a communication channel. The channel is specified by the current value of standard **DISPCH** (default channel) variable. The following values are available:

- 1 – Serial communication channel COM1.
- 2 – Serial communication channel COM2.
- 6 – Ethernet network (TCP).
- 7 – Ethernet network (TCP).
- 8 – Ethernet network (TCP).
- 9 – Ethernet network (TCP).
- 10 – Ethernet Point-to-Point network. (UDP)
- 1 – No default channel is specified, the **DISP** command uses the last channel activated by the host.
- 2 – All channels.

In order to receive unsolicited messages by a host application, perform the following:

1. Set **DISPCH** to -2.
2. Set bit 4 of **COMMFL** to 1.
3. Send **SETCONF**(306,-1,1) from the same communication channel where unsolicited messages are expected to be received.

In order to stop the receipt of unsolicited messages by a host application:

Send **SETCONF**(306,-1,0) from the same communication channel where there is no need any more to receive unsolicited messages.

8.9.6.2 SEND Command

Description

The **SEND** command is the same as the **DISP** command but also specifies the communication channel for the output string. The communication channel is the first argument.

Syntax

SEND *channel-number, disp-arguments*

Where:

channel-number is an integer identifying the communication channel to which the message will be sent (see below).

Table 8-9. Channel Number Argument

Channel #	Description
1	Serial communication channel COM1
2	Serial communication channel COM2
6	Ethernet network (TCP)
7	Ethernet network (TCP)
8	Ethernet network (TCP)
9	Ethernet network (TCP)
10	Ethernet Point-to-Point network. (UDP)
-1	No default channel is specified, uses the last channel activated by the host
-2	All channels

disp-arguments are the same as those detailed for the **DISP** command (see [DISP Command](#)).

8.9.6.3 Differences between Query Commands and the DISP/SEND Commands

- > Query commands are executed immediately and cannot be stored in a program buffer. The **DISP** and **SEND** commands can be executed either immediately or can be stored in a buffer and executed as a part of program.
- > Query commands can address any variable: ACSPL+ standard, global or local. The **DISP** and **SEND** commands can address any ACSPL+ or global variable. Among the local variables, only the local variables defined in the same buffer where the command is located are accessible to the **DISP** command.
- > Query commands can address whole arrays or sub-arrays. The **DISP** and **SEND** commands must specify a calculable expression, i.e., only single elements of array may be involved.
- > Query commands cannot contain expressions. The **DISP** and **SEND** commands can contain expressions.
- > The controller sends the reply to a query command to the same channel from which the command was received. Results of the **DISP** command are sent to the communication channel defined by the **DISPCH** variable (see *SPiiPlus Command & Variable Reference Guide*). Results of the **SEND** command are sent to the communication channel defined by the communication channel argument (channel-number).

8.9.6.4 STR Function

Description

The **STR** function converts an integer array to a string.

Syntax

string STR (*variable* [,*start-index*] [,*number*])

Arguments

variable	Name of user-defined integer array.
start_index	Index of the array.
number	Specifies number of characters.

Return value

The function returns a string composed of the array elements interpreted as characters.

Comments

Each element of the **variable** array is interpreted as an ASCII character. If an element value is in the range from 0 to 255, it is directly converted to the corresponding ASCII character. Otherwise, the value's modulo 256 is converted.

If neither **start_index** nor **number** is specified, the conversion takes all elements of the array. If only **start_index** is specified, the conversion takes all characters from the specified index to the end of array. Specifying **number** limits the number of characters in the resulting string.

The function can be used within the **SEND** or the **DISP** GUI.

Example

The following example provides a mirror for channel 2, so that any received character is sent back:

```
int Char(1)
INP(2,Char,,1)
SEND 2,str(Char)
```

8.9.6.5 DSTR Function

Description

The **DSTR** function converts a string to an integer array.

Syntax

int DSTR(string, variable [,*start-index*] [,*number*])

Arguments

string	String of characters enclosed in double quotation marks
variable	Name of user-defined integer array.
start_index	Index of the array.
number	Specifies number of characters.

Return value

The function returns the number of actually assigned characters.

Comments

The function decomposes **string** into individual characters and assigns the characters to the sequential elements of the **variable** array.

Each ASCII character is represented as its numerical value and stored in a separate element of the array.

If **start_index** is specified, the first character is assigned to the array element with the specified index. If **start_index** is omitted, the assignment starts from the first element of the array.

If **number** is omitted, the function assigns all characters of the string. If **number** is specified, the function assigns the specified number of characters. In both cases the assignment stops when the last array element is reached.

8.10 TRIGGER

Description

TRIGGER specifies a triggering condition. Once the condition is satisfied, the controller issues an interrupt to the host computer, as follows:

1. Sets **AST<axis>.#TRIGGER** = 0
2. Examines the triggering condition every MPU cycle

Once the condition is satisfied, the controller performs the following:

1. Sets **AST<axis>.#TRIGGER** = 1
2. Produces an interrupt to the host application (software interrupt 10, enabled by **IENA.26**).

The controller continues calculating the **TRIGGER** expression until another **TRIGGER** command is executed in the same channel. Each time the expression changes its value from zero to non-zero, the controller sets **AST<axis>.#TRIGGER** = 1 and causes an interrupt.



Full application of the **TRIGGER** command to channels greater than 7 is not currently supported.

Syntax

TRIGGER *channel*, *expression*[, *timeout*]

Arguments

<i>channel</i>	An integer number from 0 to 7 that specifies the trigger's sequential number. The number defines the AST element where the triggering bit will be set and defines the bit in the interrupt tag that is automatically sent to the host application as an interrupt.
<i>expression</i>	Specifies the triggering condition. After TRIGGER is executed, the controller checks the expression condition each MPU cycle. Triggering occurs when the expression condition is satisfied. If the argument is omitted, triggering in the specified channel is disabled.

<i>timeout</i>	Specifies triggering timeout in milliseconds. A positive number specifies the time allowed for the triggering condition to be satisfied. If the timeout has elapsed and the triggering condition has not been satisfied, the controller unconditionally raises the trigger bit. After any triggering, the controller resets timeout counting to zero. If the argument is omitted, triggering works without timeout.
-----------------------	---

Table 8-10. Channel Designation for TRIGGER

Channel	Triggering Bit	Hexadecimal Interrupt Tag (Software Interrupt 10)
0	AST0.11	0x00000001
1	AST1.11	0x00000002
2	AST2.11	0x00000004
3	AST3.11	0x00000008
4	AST4.11	0x00000010
5	AST5.11	0x00000020
6	AST6.11	0x00000040
7	AST7.11	0x00000080

Related ACSPL+ Commands**INTERRUPT****Related ACSPL+ Variables****IENA, AST****COM Library Methods**

GetCallbackMask, SetCallbackMask

C Library Functions

acsc_GetCallbackMask, acsc_SetCallbackMask

Example

```

TRIGGER 1, MST(0).#MOVE=0, 3000      !1 - once the triggering condition is satisfied,
                                     !the triggering bit AST1.#TRIGGER will be set
                                     !to "1", and the interrupt tag to the host
                                     !application is 0x00000002.
                                     !MST(0).#MOVE=0 - the triggering condition.
                                     !Actuate trigger interrupt when the motor in
                                     !the 0 axis is in position (not moving).
                                     !3000 - check the triggering condition for
                                     !3000 msec. If the triggering condition is not
                                     !satisfied after 3000 msec, then set the
                                     !triggering bit AST(1).TRIGGER to "1".

```

8.11 Dynamic TCP/IP Addressing

8.11.1 TCP/IP Variable

The firmware supports both static and dynamic assignments of TCP/IP addresses.

The TCP/IP address is defined by the **TCPIP** variable. If **TCPIP** has a non-zero value, the controller uses the value as its TCP/IP address. In this case, other configuration parameters receive the following default values:

- > Subnet mask - 255.255.255.0
- > Gateway address - no gateway, i.e. no routing is supported

If **TCPIP** is zero, the controller uses the DHCP protocol to receive the network configuration from the DHCP server. The network configuration received from the DHCP server includes the following parameters:

- > Controller's TCP/IP address
- > Subnet mask
- > Gateway address

To retrieve the assigned address in an ACSPL+ program, use the **GETCONF** function (see *SPiiPlus Command & Variable Reference Guide*) with key 310.

To find all SPiiPlus controllers in the network segment, use the C Library function: [acsc_GetEthernetCards](#).

8.11.2 Using GETCONF/SETCONF to Access TCP/IP Address

The **GETCONF** and **SETCONF** functions with key 310 provide access to the controller TCP/IP address.

GETCONF(310, 0) returns an integer value that contains the TCP/IP address currently assigned to the controller. The index argument (second argument) of the function should be zero. If a TCP/IP protocol is not configured, or not supported, the return value is zero.

SETCONF(310, 0, address) configures the TCP/IP address for the controller. The index argument (second argument) of the function should be zero. If the address argument is zero, **SETCONF** activates a new execution of the DHCP protocol and obtains a new TCP/IP address from the host (the host may configure the same address as before). **SETCONF** does not change the **TCPIP** parameter. After power-up, the controller is initialized with the TCP/IP address set in the TCP/IP parameter.

The address value is a 32-bit integer that contains four bytes. Each byte represents one part of the TCP/IP dot address. The bytes follow in computer order. For example:

```
SETCONF(310, 0, 0x6400000a)    Assigns address 10.0.0.100
?x/GETCONF(310,0)             Executed from the terminal returns the
                               hexi-decimal value: 6400000a
```

There are several limitations when using **SETCONF(310)**:

- > If the **TCPIP** variable stored in the flash is zero, **SETCONF(310)** must be used only with zero address argument. In other words, if the controller is configured for dynamic addressing, assigning static address is not allowed.
- > If the **TCPIP** variable stored in the flash is not zero, **SETCONF(310)** must be used only with non-zero address arguments. In other words, if the controller is configured for static addressing, switching to a dynamic address is not allowed.
- > **SETCONF(310)** has a long execution time. During this time, communication with the controller is impossible using any communication channel. Use **SETCONF(310)** only within the controller initialization sequence. Avoid attempts to communicate with the controller and the motor **ENABLE** command or motion commands while **SETCONF(310)** is in progress.

8.11.3 Addressing Scenarios

There are four scenarios of assigning TCP/IP addresses to the controller:

1. Static Addressing

Configure the appropriate TCP/IP address in the **TCPIP** variable and store it in the flash.

After start-up, the controller adopts the address and answers to the corresponding telegrams.

For using static addressing in a local network, the system administrator needs to reserve this address to avoid identical addresses in the network segment.

2. Quasi-dynamic Addressing

Configure any non-zero TCP/IP address in the **TCPIP** variable and store it in the flash.

After start-up, use **SETCONF** with a non-zero address argument to assign an actual address to the controller.

This approach makes sense, if the controller is assigned with an address dependent on some condition available only at run time. For example, the controller may select its TCP/IP address on the basis of digital input states.

3. Dynamic Addressing

Configure a zero TCP/IP address in the **TCPIP** variable and store it in the flash.

After start-up, the controller initiates DHCP communication with the DHCP server and obtains the TCP/IP address and other network information.

To use this method, the DHCP server should be accessible at the time of controller start-up.

If the DHCP server is not accessible, the controller repeats the request several times. If all attempts fail, the controller disables Ethernet channel support and continues initialization.

4. Delayed Dynamic Addressing

Use this method, if the DHCP server is not available at the time of controller start-up. It may occur, for example, if the controller and the computer running DHCP server are activated at the same time, and nobody knows which is ready first.

Configure zero TCP/IP address in the **TCPIP** variable and store it in the flash.

Create an initialization routine in one of the buffers, and store it in the flash:

```
AUTOEXEC:
WHILE GETCONF(310,0) = 0
  SETCONF(310,0,0)
END
! continue initialization
```

As a result, the controller waits at the beginning of initialization until the DHCP server succeeds to in supplying a TCP/IP address.

8.12 Non-Default Connections

This section covers handling non-default connections.



It should be noted that many applications require switching between non-default or default connections within the process of operations. You can return to a default connection by setting the **MFLAGS.#DEFCON** (bit 17, Default Connection) to 1. This automatically resets the connect formula, updates dependence and equates corresponding **APOS** to **RPOS** (see [Connection to the Plant](#)).

8.12.1 ROFFS Variable

The **ROFFS** variable is an 8 element array, one element per axis, that reads the offset calculated by the controller in the connect formula. As long as the motor is in the default connection (**MFLAGS.#DEFCON** = 1), offset **ROFFS** is zero. However, once you have specified a **CONNECT** (see [CONNECT Command](#)) formula:

```
CONNECT RPOS(axis) = F(...)
```

the controller calculates offset **ROFFS(axis)** to prevent any immediate change of **RPOS(axis)** that may cause a jump of the motor. Then the controller calculates formula:

RPOS(axis) = F(...) + ROFFS(axis)

for each controller cycle.

The controller recalculates the offset to prevent motor jump when any of the following commands is executed:

- > CONNECT
- > SET
- > ENABLE
- > DISABLE

> KILL

ROFFS reads the current value of the offset. Watching the offset value facilitates development and debugging of application with complex kinematics.

8.12.2 DAPOS Variable

The **DAPOS** variable is an 8 element array, one element per axis, that reads the Axis Position synchronous with the Reference Position. The variable supplements the existing **APOS** variable. The problem of the **APOS** variable is that the axis position is not synchronous with the **RPOS** and **FPOS**. For this reason watching **APOS** against **RPOS** or **FPOS** in the SPiiPlus MMI Application Studio **Scope** is inconvenient.

DAPOS reads the same values of axis position, but synchronously with **RPOS** and **FPOS**. Using synchronous axis position facilitates analysis and debugging of the **CONNECT** formula.

Use only **DAPOS** for watching the axis position in the **Scope**.

Use only **APOS** in the **CONNECT** formula and in ACSPL+ program.

8.12.3 CONNECT Command

The **CONNECT** command defines the relation between motors and axes.



The **CONNECT** command cannot be executed as long as the default connection bit (**MFLAGS.#DEFCON**) is raised.

Syntax:

CONNECT RPOS (*axis*) = *expression*

Where **RPOS**(*axis*) is an axis **RPOS** variable (for example, **RPOS(0)**), which receives the value of the **expression**.

For more information about **RPOS** and other common motion variables refer to the *SPiiPlus Command & Variable Reference Guide*.

The **connect** command is not an **assignment** command (see [Assignment Command](#)). It does not simply assign the result of the formula on the right side to the axis **RPOS**. The formula is not evaluated when the **connect** command is executed (which would be the case for an **assignment** command); instead, the formula is stored and then evaluated by the controller every controller cycle to calculate the corresponding **RPOS**.

After power-up the controller always starts with the default connection. The default connection means the following for each axis:

- > Bit **MFLAGS.#DEFCON** is raised.
- > The default connect formula is defined as **CONNECT RPOS = APOS**.
- > **APOS** and **RPOS** are linked, i.e., explicit (through the **SET** command see [SET Command](#)) or implicit change of one of these variables causes the same change in the other one.

Once an application resets **MFLAGS.#DEFCON**, it can then execute a **CONNECT** and (typically) a **DEPENDS** command. At this point, the motor is considered to be in non-default connection.

Consider the following examples:

The commands

```
MFLAGS (1) .#DEFCON = 0
CONNECT RPOS (1) = APOS (0)
DEPENDS 1, 0
```

connect the 1 axis motor position to the 0 axis reference. If the 0 axis motor is also connected to the 0 axis reference, this provides gantry-like motion of two motors.

The command

```
PTP 0, 1000
```

will provide synchronous motion of both 0 and 1 axes motors.

The command:

```
connect RPOS (0) = APOS (0) + AIN (1)
```

connects the 0 axis motor position to the 0 axis reference plus analog input 1. In this case the 0 axis provides a motion and the analog input (for example, an external sensor) supplies a correction for the 0 axis motor.

The following commands

```
MFLAGS (2) .#DEFCON = 0
connect RPOS (2) = APOS (2) + APOS (3)
depends 2, (2,3)
```

connect the 2 axis motor to the sum of 2 and 3 axes. The axes can each execute an independent motion, with the 2 axis motor following the sum of the two motions. Or the axes can participate in a single multi-axis motion.

The following illustrate uses of the **CONNECT** command.



The **CONNECT** command is normally followed by a **DEPENDS** command (see [DEPENDS Command](#)).

Using a Non-Default Connection

Listed below are some of the tasks that can be resolved using the appropriate connect formula:

- > Introduce a gear ratio between a logical axis and a physical motor.
- > Compensate for encoder errors and backlash.
- > Compensate for non-orthogonality of machine slides.
- > Compensate for undesired mutual interference between machine coordinates.

- > Implement gantry axes
- > Define the physical motion as a sum of a logical motion and a compensating signal.
- > Define the physical motion as a sum of two or more logical motions.
- > Inverse kinematics, such as programming in Cartesian coordinates a machine that actually has polar kinematics.

Typically, the **CONNECT** command for a specific motor is executed only once in an ACSPL+ application.

A typical location for a **CONNECT** command is after the homing process in the code that follows an **AUTOEXEC** label (see [Names: Variable and Label](#)). The following pseudocode executes homing of X and Y axes and configures them as a gantry pair that follows the motion on the X axis:

<pre>AUTOEXEC: . . . MFLAGS1.#DEFCON=0 CONNECT RPOS(1)=APOS(0) DEPENDS 1,0 SET APOS(0)=0,RPOS(0)=0,RPOS(1)=0 . . .</pre>	The controller automatically starts the program from the AUTOEXEC label after power-up. Execute homing of 0 and 1 axes. Reset the #DEFCON bit. Set gantry-like connection (1 axis motor follows 0 axis). Specify dependence (1 axis motor depends on 0 axis). Set origin. Continue.
--	--

A more sophisticated application may require changing the connection in the middle of operations. The controller applies no limitations regarding when a connection can be changed. In a typical case, changing connection requires three commands:

<pre>CONNECT RPOS(0)=... DEPENDS 0,... SET APOS(0)=...,RPOS(0)=...</pre>	Specify connection of 0 axis motor. Specify dependence of 0 axis motor. Set origin of 0 axis and 0 axis motor.
--	---

To return to default connection use the following commands:

<pre>CONNECT RPOS(0)=APOS(0) MFLAGS(0).#DEFCON=1 SET RPOS(0)=...</pre>	The 0 axis motor will follow the 0 axis. Set the #DEFCON bit. Set origin of 0 axis motor (if #DEFCON=1, APOS is set to the same value).
--	---

Offset in Connect Formula

If a motor is in a non-default connection, the **APOS** and **RPOS** variables are not linked and may contain different values.

The **CONNECT** command specifies a basic formula that the controller uses to calculate the **RPOS**. However, in the process of **RPOS** calculation the controller also adds an implicit offset which is not specified in the connect formula.

The controller calculates the offset automatically and recalculates it in the following circumstances:

- > Execution of **CONNECT RPOS=...** command for the motor.
- > Execution of **SET RPOS=...** or **SET FPOS=...** command for the motor.
- > Execution of **SET APOS=...** command for any axis that the motor depends on.
- > Execution of **ENABLE** command for the motor.

When a **CONNECT** command is executed, the offset is adjusted so that the **RPOS** specified on the left side of the connect formula and any **APOS** specified on the right side retain their current values. For this reason the **CONNECT** command can be executed while the motor is enabled and does not cause a motor jump. Using this implicit offset the controller ignores any explicit offset specified in the **CONNECT** formula. For example, the following commands have exactly the same effect:

```
CONNECT RPOS(0) = 0.5*APOS(0) + 1000
```

and

```
CONNECT RPOS(0) = 0.5*APOS(0) + 2000
```

because the explicit offset is ignored.

When an **ENABLE** command is executed, the offset is adjusted so that the connection formula calculation yields the desired value (**RPOS** retains its value).

The **SET RPOS=...** or **SET FPOS=...** command immediately changes the values of **RPOS** and **FPOS**. The offset is recalculated so that the connection formula calculation yields the new desired value.

The **SET APOS=...** command immediately changes the value of axis position **APOS**. In order to retain the current values of all **RPOS** components, the controller recalculates the offsets of all motors that depend on the axis.

8.12.4 *DEPENDS Command*

Description

The **DEPENDS** command complements the connect command, specifying dependence between a motor and axes.



The **DEPENDS** command cannot be executed as long as the default connection bit (**MFLAGS.#DEFCON**) is raised.

Syntax

DEPENDS *dependent_axis, axes_specification*

Comments

The **dependent_axis** argument specifies an axis and the **axes_specification** argument specifies one or more axes on which the motor depends.

Typically, a **DEPENDS** command will follow a **CONNECT** command. Whereas a **CONNECT** command can define a mathematical correspondence between a motor's reference position (**RPOS**) and one or more axis positions (**APOS**), a **DEPENDS** command specifies the logical dependence between a motor and axes.

The **DEPENDS** command is necessary because generally the controller is not capable of deriving the dependence information from the **CONNECT** formula alone. For this reason, once a **CONNECT** command is executed, the controller resets the dependence information of the motor; the motor depends only on the corresponding axis.

Dependence information, as specified using a **DEPENDS** command, is required in the following cases. If the dependence information is not provided correctly, the controller may display strange behavior.

- > A motor/axis query (for example, **?0**) returns the non-default dependence for that motor.
- > When initiating a motion, the controller verifies if each motor dependent on the axes involved is enabled. If one or more motors are disabled, the motion does not start.
- > If in the process of motion a motor is disabled or killed due to a fault or due to a **DISABLE** or **KILL** command, the controller immediately terminates all motions involving the axes that the motor depends on.
- > Once a **SET APOS=...** command is executed, the controller adjusts offsets in the connection formula of the motors that depend on the specified axis.

8.12.5 MATCH Function

Description

The **MATCH** function calculates the axis position that matches current Reference Position of the same axis with zero offset.

Syntax

MATCH (*axis, from, to*)

Arguments

axis	Specifies the axis to be matched.
from/to	Restricts the range within which the matching value is searched.

Comments

The function is useful in the case of non-default connections if a motor depends only on the same axis, a typical example is error compensation. For example, the following connection:

```
CONNECT RPOS(1) = APOS(1) + MAPBY1(APOS(1), ErrorTable)
DEPENDS 1,1
```

defines 1-to-1 dependence. In this case, the command:

```
SET APOS(1) = match(1, -1000, 1000)
```

can be used to find matching **APOS(1)** in the range -1000 to +1000 and to set offset **ROFFS1** to zero. In mathematical terms, the function finds the root of equation:

$$RPOS = F(x)$$

where **RPOS** is the current value of the **RPOS** variable and **F(x)** is the connect formula specified by you with **APOS** substituted for x.

The function succeeds if the unique root exists in the specified range. If there are several roots in the range, the function returns one of them. If the root does not exist, the function result is incorrect. It is your responsibility to restrict the range so that the function provides proper result.

8.13 Input Shaping

Input Shaping is a feed-forward control technique for reducing vibrations in computer controlled machines. The SPiiPlus controller implements an Input Shaping algorithm patented by Convolve, Inc.

Input Shaping support is an optional feature in the SPiiPlus controller. To use the feature, specify the option in the controller order code, as described in the product data sheet.

8.13.1 INSHAPEON

Description

The **INSHAPEON** function starts Input Shape algorithm for specified axis. The result is a dynamic output signal equal to the convolution of the input signal and the convolution pulses.

Syntax

INSHAPEON Axis_Index, T_array, A_array

Arguments

Axis_Index	Designates the specific axis, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.
T_array	One-dimensional array specifying the times of each convolution in milliseconds.
A_array	One-dimensional array specifying the amplitudes of each convolution pulse

Return Value

None

Comments

Vectors T_array and A_array define characteristics of the convolution pulses. The array sizes should be identical.

Vector T_array contains real numbers, so fractional numbers may be specified. However, the position of each pulse is rounded to a multiple of the controller cycle. If the controller cycle is one millisecond, the numbers in T_array are rounded to integers. The elements of T_array must be arranged in ascending order.

The sum of A_array entries must equal 1.

See the Using the Convolve Web Site chapter in the *ACSPL+ Programmers Guide* to get the explanation how to calculate the T_array and A_array parameters.

This function is supported in version 3.00 and higher..

Examples

```

global real CnvT(5), CnvA(5), CnvB(420)
VEL(0) = 120
ACC(0) = VEL(0)*10
JERK(0) = ACC(0)*10

CnvT(0)=0; CnvT(1)=64 ; CnvT(2)=68 ;CnvT(3)=72; CnvT(4)=139
CnvA(0)=25345/1e5; CnvA(1)=160/1e5; CnvA(2)=30987/1e5; CnvA(3)=18949/1e5;
CnvA(4)=24559/1e5

enable 0
InShapeOn 0, CnvT, CnvA
ptp/e 0,0
ptp/e 0,50
till ^MST(0).#MOVE
InShapeOff 0
stop

```

```

!INPUTSHAPE EXAMPLE: IF CTIME < 1

!In this case we need to multiply CnvT array by CTIME. Here CTIME = 0.5
global real CnvT(5), CnvA(5), CnvB(420)
CnvT(0)=0*CTIME; CnvT(1)=1*CTIME ; CnvT(2)=214*CTIME ;CnvT(3)=253*CTIME;
CnvT(4)=501*CTIME
CnvA(0)=22960/1e5; CnvA(1)=10361/1e5; CnvA(2)=3186/1e5; CnvA
(3)=45767/1e5;
CnvA(4)=17726/1e5
enable 0
InShapeOn 0, CnvT, CnvA
ptp/e 0,0
ptp/e 0,30
till ^MST(0).#MOVE
!InShapeOff 0
stop

```

8.13.2 INSHAPEOFF

Description

The **INSHAPEOFF** function stops the Input Shape algorithm for the specified axis.

Syntax

INSHAPEOFF Axis_Index

Arguments

Axis_Index	Designates the specific axis, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1.
------------	---

Comments

This variable is supported in version 3.00 and higher.

Return Value

None

8.13.3 AST

The **#INSHAPE** bit is set to 1 after calling **INSHAPEON**. The bit is set to zero after calling **INSHAPEOFF**.

Description

AST is an integer array, with one element for each axis in the system, the elements of which contain a set of bits used for displaying the current axis state.

Syntax

[command] AST(axis_index).bit_designator

Arguments

<i>command</i>	Typical commands are DISP and the like.
<i>axis_index</i>	Designates the specific axis, valid numbers are: 0, 1, 2, ... up to the number of axes in the system minus 1. In the case of bit 2 (#PEGREADY) this parameter designates the PEG engine, not the axis.
<i>bit_designator</i>	A description of the AST bit designators is given in Table 8-11 .

Table 8-11. AST Bit Descriptions

Bit Name	No.	Description
#LEAD	0	1 = axis is leading in a group
#DC	3	1 = Axis data collection is in progress
#PEGREADY	4	1 = all values are loaded and the Incremental/Random PEG is ready to respond to movement When referring to this bit, <i>axis_index</i> designates the PEG engine rather than the axis.
#MOVE	5	1 = Axis is involved in a motion
#ACC	6	1 = Axis in accelerating motion state
#BUILDUP	7	1 = Segments build-up
#VELLOCK	8	1 = Slave is synchronized to master in velocity lock mode - slave velocity strictly follows the master velocity.

Bit Name	No.	Description
#POSLOCK	9	1 = Slave is synchronized to master in position lock mode - slave position strictly follows the master position.
#TRIGGER	11	1 = Produces an interrupt to the host application, enabled by IENA.26
#NEWSEGM	16	The controller sets the bit to inform that a new segment is required to be provided by the application. The bit is set starvation_margin ms before the starvation condition occurs. The starvation condition is indicated by #STARV bit.
#STARV	17	The controller sets the bit to indicate starvation condition. The starvation condition means that there are not enough further segments to continue the motion with required velocity. In this case, the controller starts decelerating the motion with $\frac{1}{2}$ JERK in order to prevent motion discontinuity and avoid mechanical jerks. Once the application begins supplying segments at a sufficient rate, the controller returns the motion back to normal condition. Note, that often the starvation condition causes inefficient velocity generation and increases the time required for completing the required motion path.
#ENCWARN	18	Indicates if there is an encoder warning. Cleared by the ACSPL+ FCLEAR command.
#ENC2WARN	19	Indicates if there is an secondary encoder warning. Cleared by the ACSPL+ FCLEAR command.
#INRANGE	20	Laser In Range
#LCTICKLE	21	0: tickle mode is off 1: tickle mode is active
#LCMODUL	22	0: modulation is off 1: modulation is active
#FOLLOWED	23	0: Axis in regular mode 1: Axis in slave mode and follows the profile generated by RTC6
#HOLD	24	0: hold is off 1: hold is in progress

Bit Name	No.	Description
#INHOMING	25	0: homing is not in process 1: homing is in process
#DECOMPON	26	0: dynamic error compensation is switched off 1: dynamic error compensation is switched on The bit is set to 0 in the following cases: > Calling ERRORMAPOFF function > Calling ERRORUNMAP function (in case there are no other active dynamic error compensation zones of the specified axis) > During homing process initiated by HOME command (switched ON back at the end of the process) > Encoder Error / Encoder Not Connected faults > Changing one of the encoder-related parameters (E_TYPE, E_FREQ, E_SCMUL, SLPBITS, E_AOFSS) > Changing encoder routing (SLPROUT)
#INSHAPE	27	0: Input Shaping not Active 1: Input Shaping Active
#ENCPROC	29	0: Encoder Initialization not in process 1: Encoder Initialization in process

Tag

7

Accessibility

Read-Only

Related ACSPL+ Commands**MASTER, SLAVE****Related ACSPL+ Variables****MST****COM Library Methods and .NET Library Methods**

ReadVariable, GetAxisState

C Library Functions

acsc_ReadInteger, acsc_GetAxisState

8.13.4 APOSFILT

Description

APOSFILT is real array with one element for each axis in the system. The array elements store the current desired motor position, including the filtering operation result, such as input shaping.



APOSFILT updates on every controller cycle according to the filtering algorithm. When the filtering algorithm is not configured, **APOSFILT** = **APOS**.

TAG

368

Comments

This variable is supported in version 3.00 and higher.

ACCESSIBILITY

Read-Only

RELATED ACSPL+ COMMANDS

All motion commands

RELATED ACSPL+ VARIABLES

FPOS, RPOS, APOS,

.NET LIBRARY METHODS

ReadVariable(), WriteVariable()

C LIBRARY FUNCTIONS

acsc_ReadReal(), acsc_WriteReal()

8.13.5 Using the Convolve Web Site

This section describes the procedures used for designing Input Shapers™ using the Convolve, Incorporated web site for designing Input Shapers™ located at: <http://inputshaping.convolve.com>.

Access to the web site requires a user name and password.



Use the name and password that you received from ACS Motion Control along with the controller.

The first time you log into the web site, you will be required to review the licensing agreement for use of the site. For each subsequent log-in you will be reminded of the licensing agreement and remaining number of trials until expiration of the account will be displayed.

8.13.6 Data Entry Dialog

The **Data Entry** dialog, is the primary means for entering the parameters to design an Input Shaper™. The **Data Entry** dialog displays the current **User ID**, **Name**, and **Date**. Use the form fields as follows:

- > **Constraint** - Set to **Digital Positive Mixed Constr.**. This gives you the maximum freedom for selecting other design parameters.
- > **Output Format** - Set to **Standard Format**.
- > **Axis** - Set the required axis
- > **Frequency or DeltaT** - Set **DeltaT** value according to the controller parameter **CTIME** (Cycle Time). **CTIME** defines the controller cycle in milliseconds, and **DeltaT** should be defined in seconds; therefore, if **CTIME**=1, set **DeltaT**=0.001. **Frequency** is automatically computed.
- > **Amplitude Sum** - Set = 100000
- > **Measured Frequency and Damping** entries - Enter from one up to six vibration frequencies and the associated damping for each frequency. The vibration frequencies are modeled as second order systems, which are normally characterized by a natural frequency and damping ratio (or zeta).

The damped natural frequency will be less than the natural frequency by the factor,
 $\sqrt{1 - \text{zeta}^2}$

The frequencies are referred to as "measured frequencies" because the program performs the calculations to convert them to "undamped natural frequencies".

- > **Hardware ID, Location, Tooling ID and Notes** - optional fields that can be used to document a configuration for a specific machine. Text entered into these fields is copied into the output file as a comment.
- > **Robust** check box - should normally be selected to design a "robust" Input Shaper™ for a range of ±15% of the nominal frequency. This box can be left unchecked when the range of frequency variation is less than ±5%. Non-robust Input Shapers™ are 50% shorter in duration than a robust Input Shapers™. Please refer to the discussion on Insensitivity Curves below for more information.

When all of the data has been entered, press the **Calculate** to start the calculation. It can take several seconds to complete the calculation.

Reset This Form

Constraint:
Digital Positive Mixed Constr.

Output Format:
Standard Format

Cancel
Calculate

Axis:
1

Frequency
1000

Or

DeltaT
0.001

Amplitude Sum:
100000

	Measured Frequency**	Damping	Robust?
1	100	0.05	☒
2	0	0	☐
3	0	0	☒
4	0	0	☒
5	0	0	☒
6	0	0	☒

NEW FEATURE: By deselecting "Robust" for a low frequency mode, you can solve for faster sequences. (Only use this when needed - Robustness is REDUCED)

**** Measured frequency is the damped frequency:** $f_d = f_u \cdot \sqrt{1 - \zeta^2}$
 f_d = damped natural frequency
 f_u = undamped natural frequency

Hardware ID:

Tooling ID:

Location:

Notes:

Figure 8-9. Data Entry Dialog

When the calculations are complete, the following screen appears.

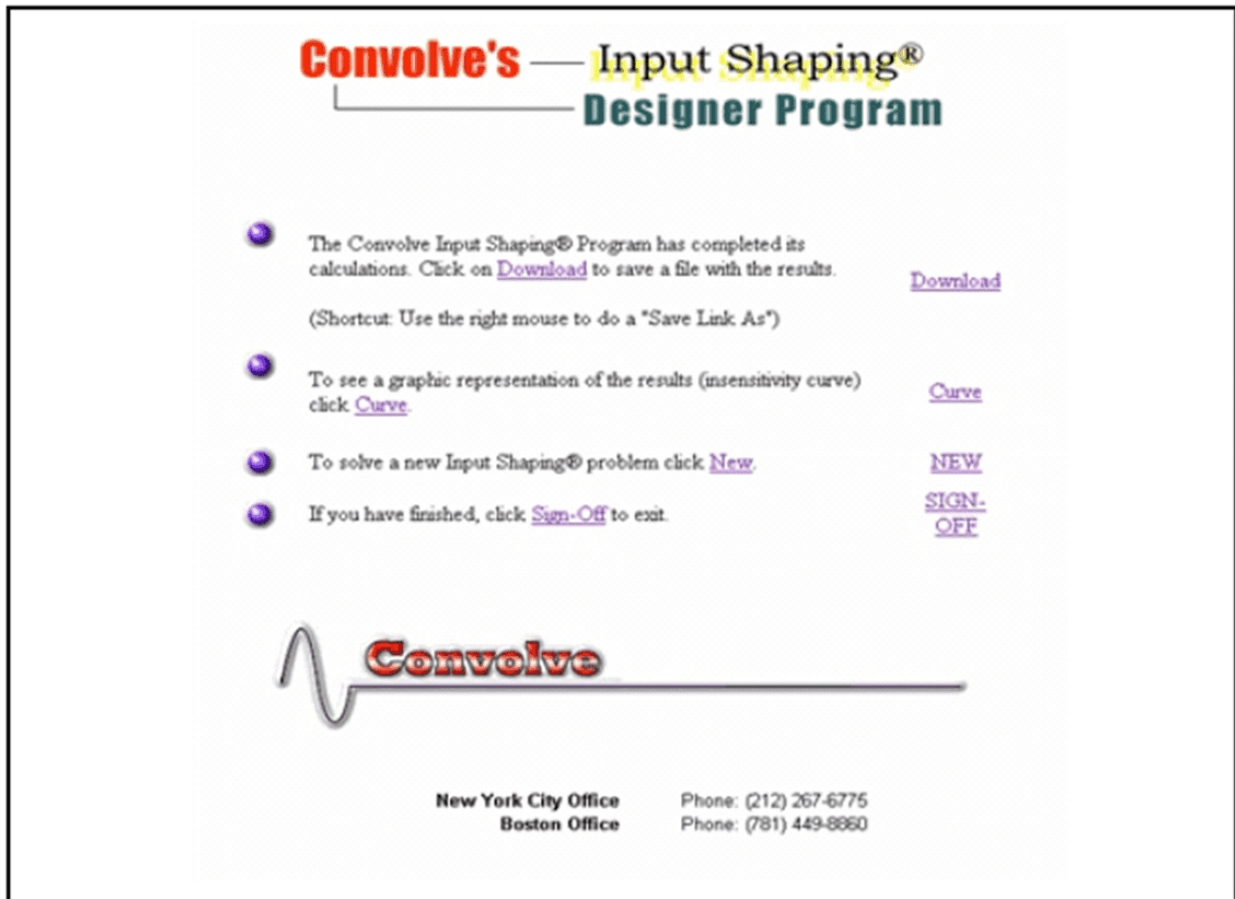


Figure 8-10. Screen at the Conclusion of Calculation

Click **Download** to access another window that contains the results of the calculation. A sample output file is shown below for the design of an Input Shaper with the following parameters:

- > Measured Vibration Frequency - 100 Hz
- > Damping - 0.05
- > Delta T for trajectory generation - 0.001 second
- > Amplitude sum - 100000
- > Robust was selected - (Constraint = 1)

This is an illustration of the output.



Figure 8-11. Window Accessed by Download

The output from the calculation is five impulses that characterize the Input Shaper™. The delays are specified in terms of digital time steps and for **Delta T** = .001 second, the delays would be at 0, 0.002, 0.005, 0.007 and 0.011 seconds.

The reported Time values should be used in the **T** argument of the **inshape** function. The reported **Amp** values are multiplied by 100000. The following example shows how the inshape arguments should be initialized to use the reported values:

```

global real CnvT(5), CnvA(5), CnvB(12)
                                !Definitions
CnvT(0)=0; CnvT(1)=2; CnvT(2)=5;CnvT(3)=7; CnvT(4)=11;
                                !Initialize CnvT array
CnvA(0)=0.18693; CnvA(1)=0.11701; CnvA(2)=0.29279;CnvA(3)=0.22495; CnvA(4)=0.17832;
                                !Initialize CnvA array
...                               !Program actions
stop
on MST(1).#ENABLED                !Execute auto routine once the 1 axis is
                                !enabled

MFLAGS(1).#DEFCON=0                !Reset default connection flag
connect RPOS(1)=inshape(APOS(1),5,CnvT,CnvA,CnvB)
                                !Define connection
depends 1,1                        !Define Dependence
ret

```

The results from the calculation can be saved directly by using the web browser file GUI **File → Save as ...**. Save the results as a text file (*.txt), so Input Shaper™ coefficients can be downloaded into a controller. The same file will sent by email to the email account specified when the account was created.

Click the **Curve** option (shown above) to display the useful frequency range for the Input Shaper™. A new window, opens to display the **Insensitivity Curve**. This plot is the **Insensitivity Curve** for the example:

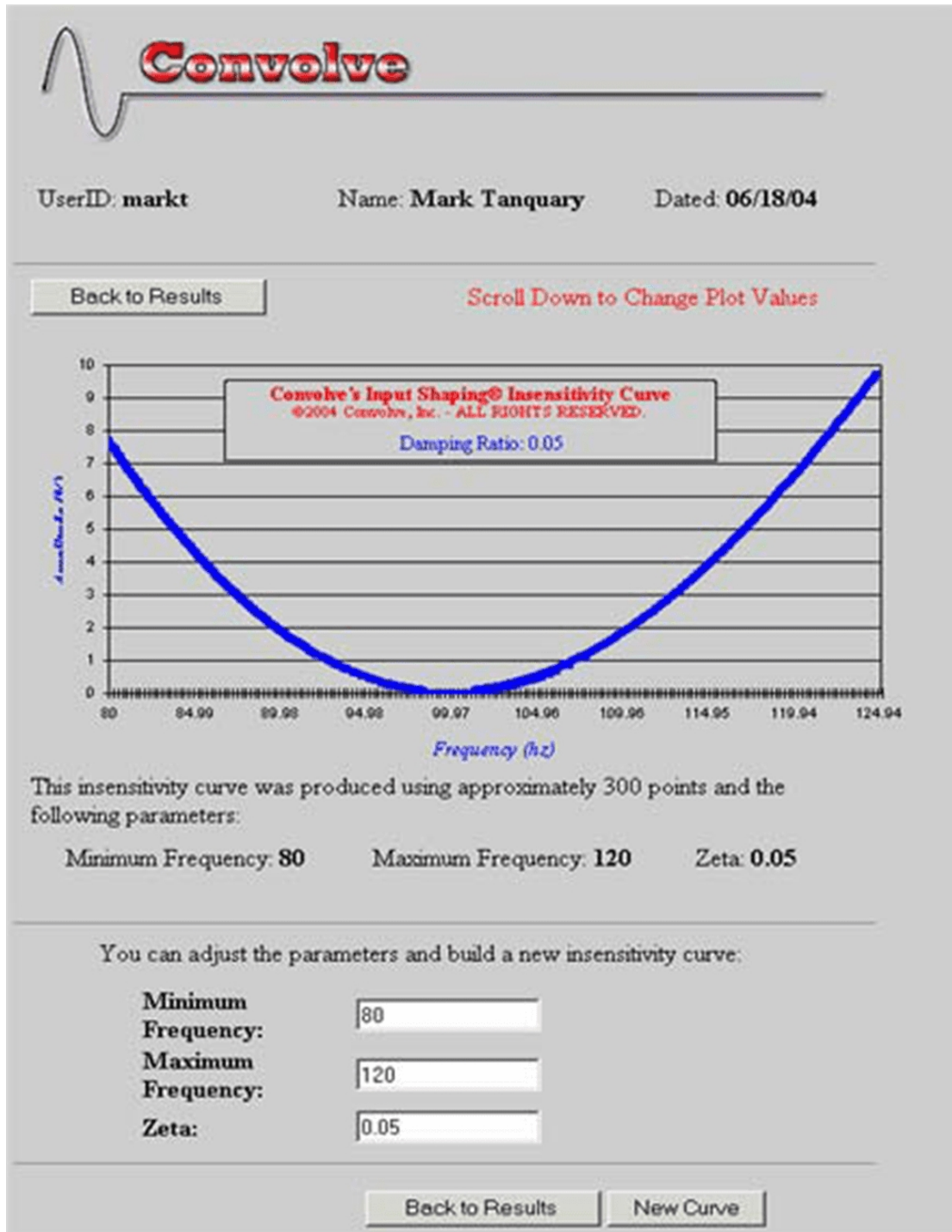


Figure 8-12. Insensitivity Curve Illustration

The **Insensitivity Curve** displays the theoretical amount of residual vibration that results after applying the Input Shaper™ to the system. The horizontal axis is the vibration frequency of the system and the vertical axis is the percentage of remaining or residual vibration. When the residual vibration = 0, for a perfect linear system, there should be no vibration after applying the Input

Shaper™. In the case shown above, the residual vibration should be 0 at a frequency of 100 Hz, which was the design frequency.

If the frequency of the actual system is not equal to 100 Hz, there will be residual vibration present. This level of vibration is expressed on a percentage basis. If the residual vibration equals 100%, then the no vibration reduction has occurred. For the Input Shaper™ shown above, the system vibration frequency could vary between 85 to 115 Hz and the residual vibration should be less than 5% (or when compared to the case of not using the Input Shaper™, the vibrations should be reduced by 95%).



Vibration reduction is expressed for a “perfect” linear system. There are some examples of perfect, linear systems in the real world that do achieve close to 100% cancellation. One good example of a “perfect” system is a simulation. Input Shaping usually performs very well in simulation. However for most physical systems, the vibration reduction will usually be in the range of 95 to 98% of the original vibrations.

The frequency range for the **Insensitivity Curve** can be changed by selecting new minimum and maximum frequency values and then clicking **New Curve**. The damping ratio can also be changed to determine the effect of changes in the estimated damping of the system. In general, changes in damping in the range from 0.005 to 0.1 have a limited effect on the **Insensitivity Curve**. When the system damping ratio is greater than 0.1, the effectiveness of the Input Shaper™ will be improved by specifying a damping ratio that is close to that of the actual system.

The **Insensitivity Curve** can also be used to examine the effect of selecting whether or not an Input Shaper™ should be designed using the **Robust** selection. If **Robust** is not selected for a particular frequency, the useful range of the Input Shaper™ will be reduced. For example the **Insensitivity Curve** for the same Input Shaper™ without **Robust** is:

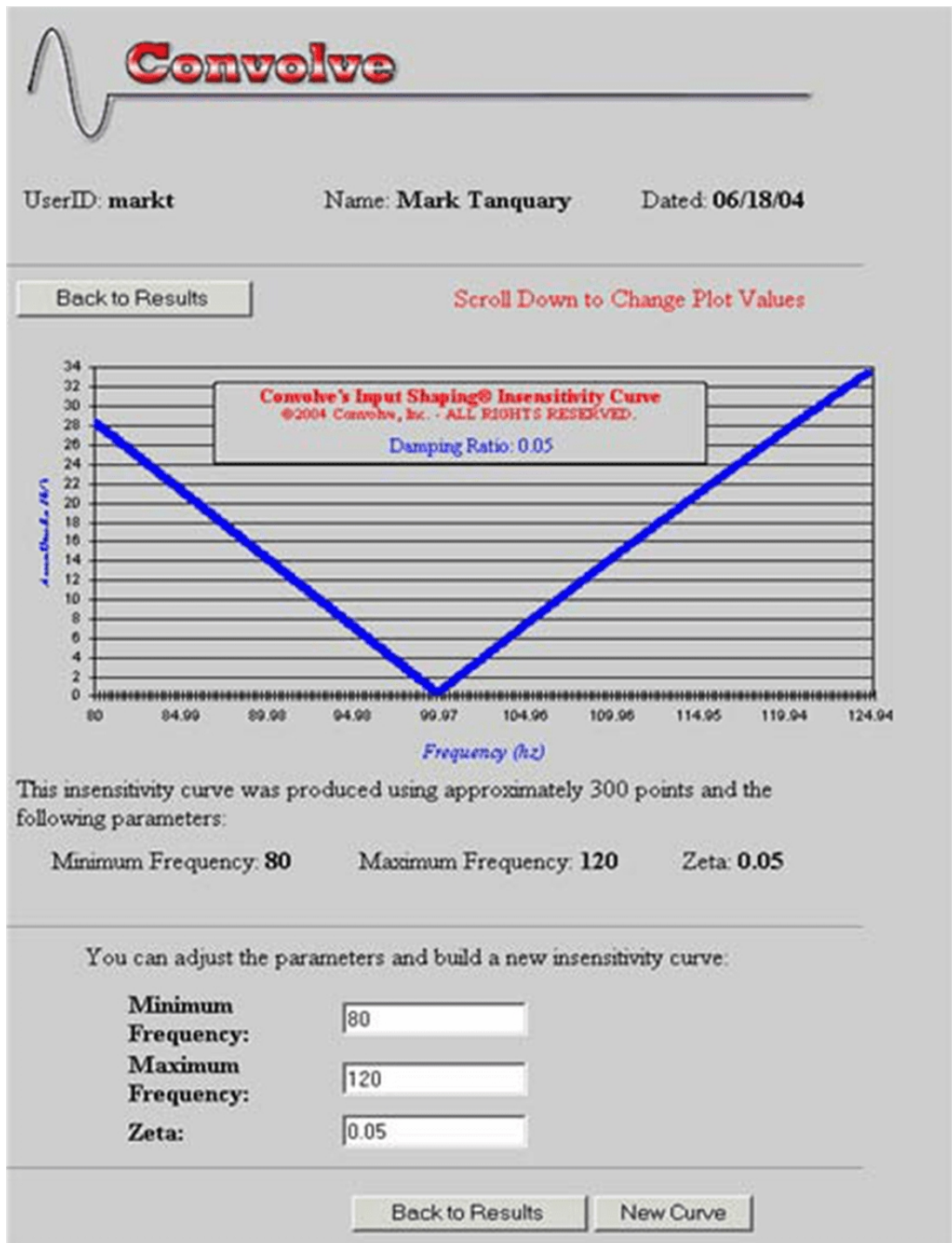


Figure 8-13. Insensitivity Curve without Robust

In this case, the frequency range for 95% cancellation of the vibration is much smaller, 95 to 105Hz. However the benefit is that the duration of the Input Shaper™ will be 50% less.

It is possible to use non-robust setting to design an Input Shaper™ that will be effective over a wider range of frequencies. Two frequencies can be specified to create a frequency band for vibration cancellation.

8.14 DRA Algorithm

The ACS proprietary Disturbance Rejection Algorithm (DRA) is used to improve the disturbance rejection response of the servo, and helps to minimize the position error during the settling phase and shorten the settling time.

The most common use of the algorithm is to improve the settling of systems mounted on passive isolation platforms. Passive isolation is typically used to isolate systems from disturbances transmitted from the floor. They employ a seismic mass supported on a soft spring made of rubber, metal, or air. The spring's damping action absorbs vibrations above the spring's resonance. For this reason, passive isolation manufacturers usually try to lower spring resonant frequency to increase the effective isolation range. When a servo force is applied to generate motion, it also acts on the isolated stationary base, causing it to vibrate. Because the frequency is low (usually below 1 Hz, to 10 Hz) and damping is very light, the isolation system continues vibrating long after the motion profile has ended. This vibration acts as disturbance to the servo system, introduces position error, and extends the settling time.

The Disturbance Rejection algorithm is used to minimize the latter effect and improve the position error during settling. The green graph shows the velocity command (in [mm/sec]) of a linear stage mounted on passive isolation, with a resonant frequency of approximately 5 Hz. The red graph shows the position error with a standard PIV algorithm. The 5Hz disturbance is clearly observed during settling. The disturbance is relatively small (less than 1 micron), yet it may be critical if the required settling window is very small (as an example, the resolution of semiconductor instruments is approaching and in some cases going below 1nm). This disturbance can be minimized by increasing the PIV gains (**SLVKP**, **SLVKI**, **SLPKP** - see *SPiiPlus ACSPL+ Command and Variable Reference Guide*), yet it cannot necessarily be eliminated and if the values of the PIV gains are too high this may lead to marginal stability. A better solution is using the DRA algorithm. As it can be seen in the blue graph the disturbance is fully eliminated.

If tuned properly the algorithm has minimal effect on the servo stability margins.

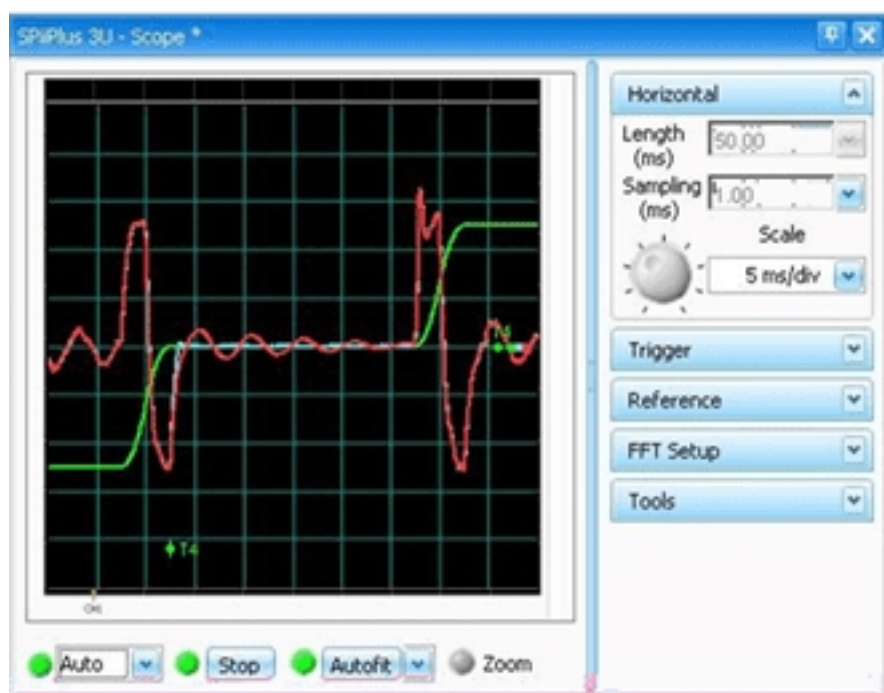


Figure 8-14. Example 1 of Using DRA

The meanings of the colors of the scope shot are:

- > Green - Reference velocity (200 mm/sec per division)
- > Red - Position error without DRA (1 division = 2 microns)
- > Blue - Position error with DRA (1 division = 2 microns)

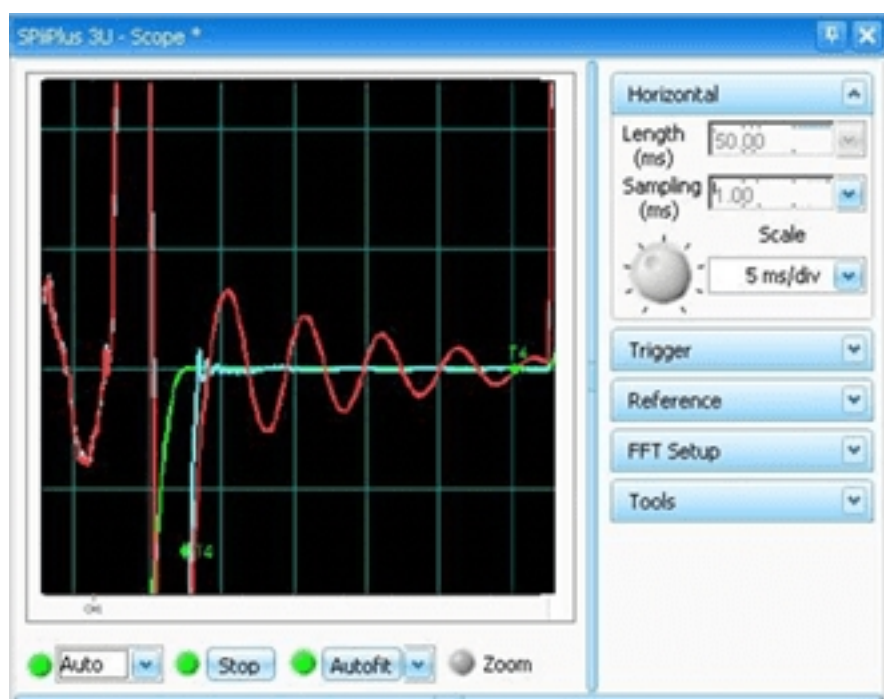


Figure 8-15. Example 2 of using DRA (zoomed)

The meanings of the colors of the scope shot are:

- > Green - Reference velocity (1 division = 100 mm/sec)
- > Red - Position error without DRA (1 division = 1 microns)
- > Blue - Position error with DRA (1 division = 1 microns)

DRA has two parameters that have to be tuned:

- > **SLDRA** - This is a frequency specified in [Hz]. It should typically be set to 1-2 times the crossover frequency of the open loop FRF. In the example shown below the open loop crossover frequency was about 100Hz, so SLDRA was set to the same value.
- > **SLDRX** - This parameter stands for maximum DRA correction and specified in units/sec.

As a rule-of-thumb, it should be set according to the maximal periodical velocity error during the settling process. This can be deduced by observing the feedback velocity (FVEL) variable in the SPiiPlus MMI Application Studio **SCOPE**, when SLDRX = 0. In the example shown above, it is about 0.01 mm/sec. Based on the time domain response the value should be further optimized to achieve the optimal response with minimal overshoot and minimal settling time.

In order to disable DRA both parameters are set to zero (default).



DRA is usually not very effective if the vibration frequency is relatively high (>10Hz), or the system bandwidth is very low.

Excessive values of SLRA, and SLDRX may cause servo instability, ringing and increased standstill jitter. In such cases the parameters should be significantly reduced. With good settings, you should usually be able to double the parameters without getting instability.

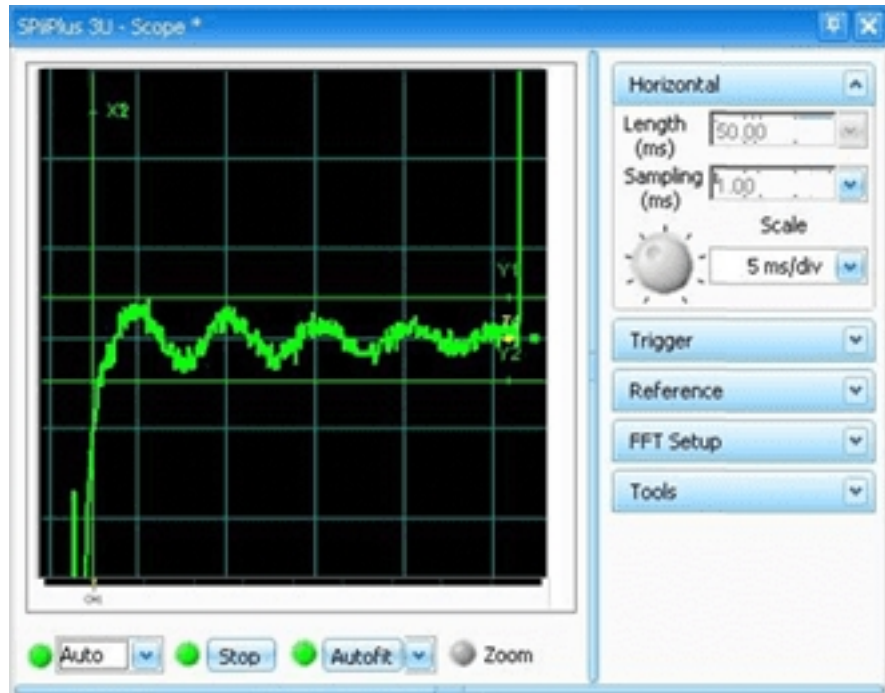


Figure 8-16. Example of Velocity Error

Velocity error (1 division = 0.02 mm/sec) during settling process of a linear axis. The maximal value of the periodical error is used to determine the SLDRX parameter.

8.15 Bi-Quad Filter

A Bi-Quad filter is added to the velocity loop control in addition to the existing 2nd order Low-pass and Notch filters.

The Bi-Quad filter is the most general 2nd order filter. It has two poles and two zeros. It can be thought of as a high-pass filter in series with a low-pass filter. The transfer function of the Bi-Quad filter is as follows:

$$H(s) = \frac{(s/\omega_N)^2 + 2\zeta_N(s/\omega_N) + 1}{(s/\omega_D)^2 + 2\zeta_D(s/\omega_D) + 1}$$

Where:

- > ω_N and ω_D are the numerator (high-pass filter zero) and denominator (low-pass filter pole) frequencies, respectively.
- > ζ_N and ζ_D are the numerator and denominator damping ratios, respectively.

The corresponding ACSPL+ parameters are:

- > **SLVBONF, SLVBODF** - numerator and denominator frequencies in [Hz]. Range: 0.1 - 4000 [Hz].
- > **SLVBOND, SLVBODD** - numerator and denominator damping ratios. Range: 0.01 - 1.
- > MFLAGS bit 16 is used to activate the filter (by default the filter is off).

The Bi-Quad filter can be used to compensate mechanical resonances, improve stability margins and system bandwidth.

The Bi-Quad filter can be configured as an additional Notch using the following formulas:

- > Set the numerator and denominator frequencies equal to the Notch frequency in[Hz]:

$$SLVB0NF = SLVB0DF = SLVNFRQ[Hz]$$
- > Set the numerator damping ratio equal to half the ratio between the Notch width and Notch frequency:

$$SLVB0ND = \frac{SLVNWID[Hz]}{2 \times SLVNFRQ[Hz]}$$

Maximal recommended ratio between width and frequency= 1/3.
- > Set the denominator damping ratio equal the numerator damping ratio times the Notch attenuation (in absolute value):

$$SLVB0DD = SLVB0ND \times SLVNATT$$

Below there are several examples that demonstrate the generality of the Bi-Quad filter.

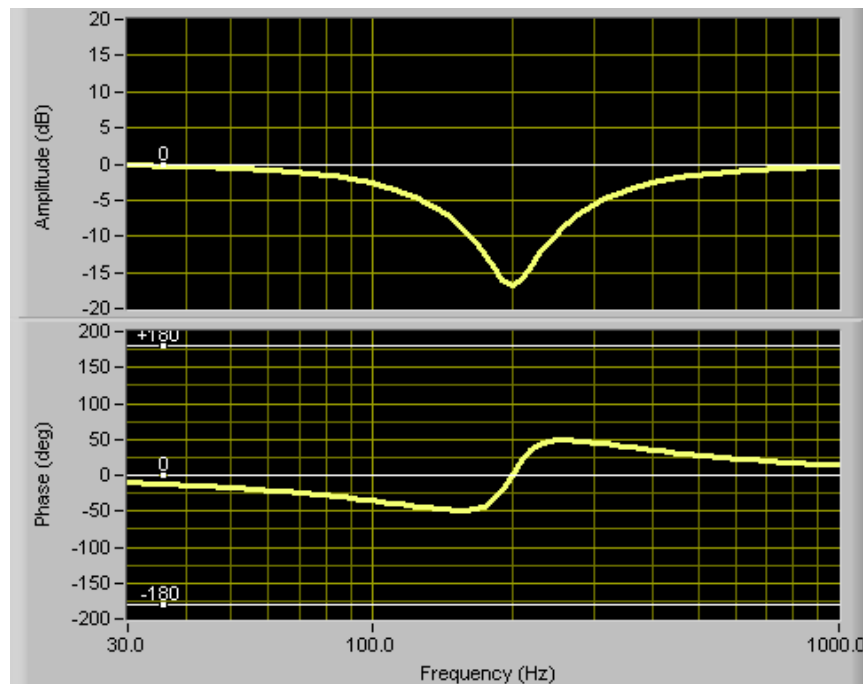


Figure 8-17. Bi-Quad Configured as a Notch Filter

$SLVB0NF = SLVB0DF$

$SLVB0ND < SLVB0DD$

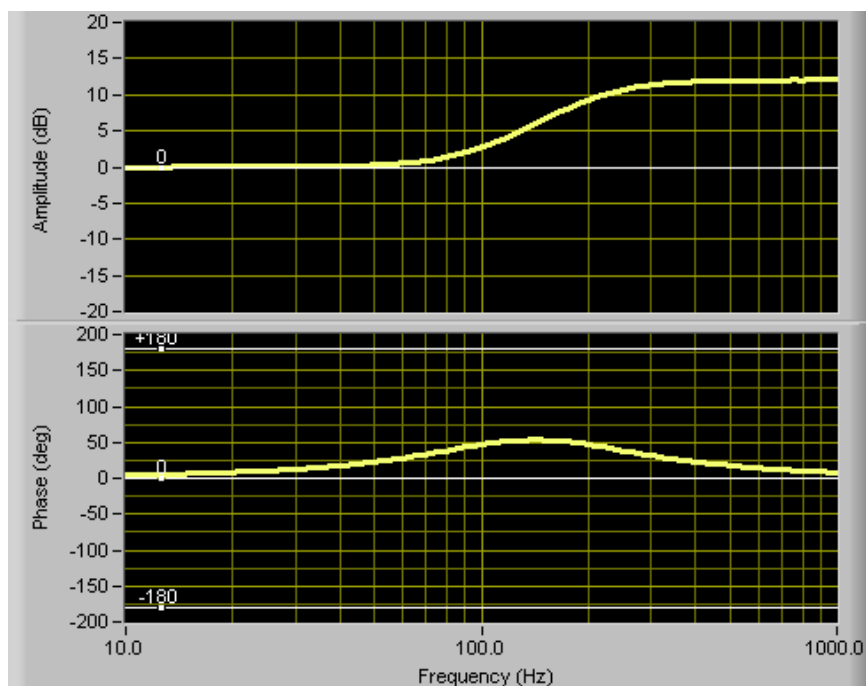


Figure 8-18. Bi-Quad Configured as a 2nd Order Lead Filter

This kind of filter is used to improve the phase margin

SLVBONF < SLVBODF

In the example the damping ratios are equal:

SLVBOND=SLVBODD =0.707

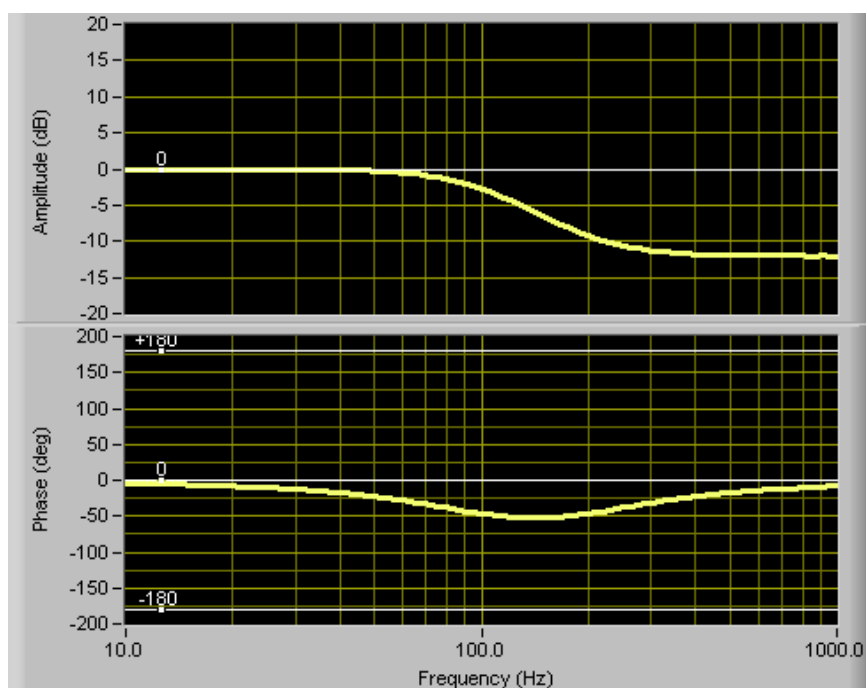


Figure 8-19. Bi-Quad Configured as a 2nd Order Lag Filter

This kind of filter is used to improve the gain margin

SLVBONF > SLVBODF

In the example the damping ratios are equal:

SLVBOND=SLVBODD =0.707

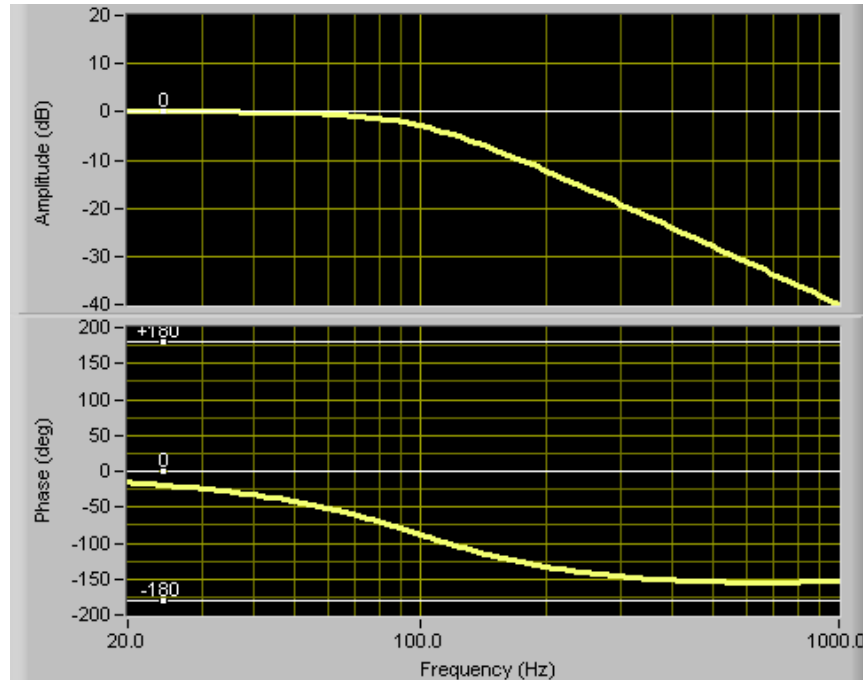


Figure 8-20. Bi-Quad Configured as a 2nd Order Low Pass Filter

SLVBONF > SLVBODF

8.16 Feedback Routing

The new routing implementation in NT allows any axis to use any encoder input available. Routing is effected through using the **SLPROUT**(position feedback), **SLVROUT** (velocity feedback) and **SLCROUT** (commutation position feedback) variables (for details of these variables see *SPiiPlus Command & Variable Reference Guide*).

Feedback is displayed using the **FPOS** (associated with SLPROUT), **FVEL** (associated with SLVROUT), and **FACC** (associated with SLCROUT) variables.

An encoder error in an encoder pointed to by **SLPROUT** will cause an encoder error in the axis. An encoder error in an encoder pointed to by either **SLVROUT** or **SLCROUT** will cause an encoder2 error to be reported.

Index and Mark inputs are also routed according to SLPROUT. Routing two axes to a single input will cause undefined index and mark behavior and should be avoided.

The firmware will clear all encoders associated (SLPROUT, SLCROUT, SLVROUT) with an axis following an encoder reset.

Encoder variables (**E_TYPE**, **E_SCMUL**, **SCCOFFS**, **SCSOFFS**, **SCGAIN**, and **SCPHASE**) are connected to the encoder feedback, not an axis; therefor they are unaffected by routing.

8.17 Voltage Mode

A mismatch of current ratings between a motor and a drive may result in poor motor operation due to low resolution of current feedback. Voltage mode is designed to overcome this issue by bypassing the current control.

Operation in voltage mode motor is still protected by the RMS protection (**XRMSM**), but no longer protected from the current spikes during motion or at idle (**XCURI** and **XCURV**). **XCURI** and **XCURV** represent the voltage limits instead of current limits.

Current limits may be calculated as follows:

$$\text{max_current_move} = \text{XCURV} / 100 * \text{bus_voltage} / \text{motor_resistance}$$
$$\text{max_current_idle} = \text{XCURI} / 100 * \text{bus_voltage} / \text{motor_resistance}$$

Voltage mode is available for the following products, starting from firmware version 3.13.01: UDMnt, UDMpc, UDMlc, UDMsd, UDMmc, NPMpm, IDM and ICM.

To enable voltage mode set `MFLAGSX(axis).3 (#VOLTMODE) = 1`.

Enabling or disabling the voltage mode is possible only while the motor is disabled.

If voltage mode is enabled, current loop tuning is unavailable in the Adjuster Wizard component of SPiiPlus MMI Application Studio.

9. Mechanical Brake

9.1 Enable/Disable/Brake

This window is used for defining the Enable operation and Mechanical Brake timing.



The variables you can define are:

- > **Protracted Enable** - This is optional, but if you select it, you must put a value in **Enable Delay** (ENTIME). The controller delays the Enable GUI by **ENTIME** mSec.
- > **Use Mechanical Brake** - You need to select this only if the application has a mechanical brake; otherwise leave it unselected. If you select it, you have to enter values for:
 - > **Brake Release Time (BOFFTIME)** - This value can be positive or negative and sets when the brake is released.
 - > **Brake Activation Time (BONTIME)** - This value can be positive or negative and sets when the brake is applied.



You are also informed of the Digital Output variable (**OUTx.x**) that will contain the state of brake. Make sure that you include it in your program to test the brake status.

The Digital Output variables for the SPiiPlus family of products are given in the following tables.

Table 8-12. Digital Outputs for Brake Control in SPiiPlus

Digital Output	Controls Mechanical Brake for Axis
OUT1.0	0
OUT1.1	1
OUT1.4	4
OUT1.5	5

Table 8-13. Digital Outputs for Brake Control in MC4U

Digital Output	Controls Mechanical Brake for Axis
OUT0.0	0
OUT0.1	1
OUT0.2	4
OUT0..	5
OUT0.4	2
OUT0.5	3
OUT0.6	6
OUT0.7	7

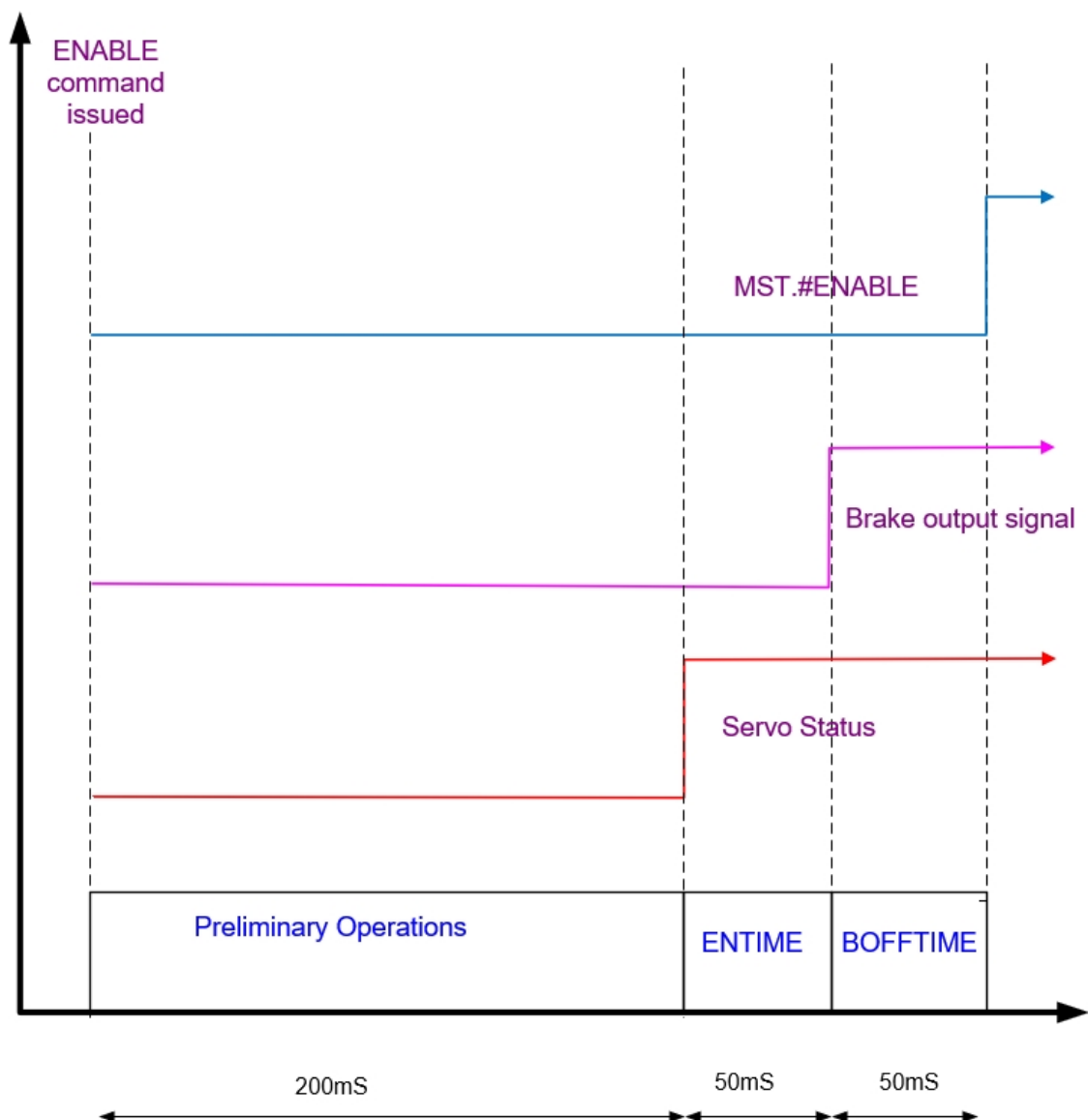
9.2 Mechanical Brake On Startup

Incorrect initialization sequences can lead to damage to devices or personal injuries may occur. The following diagrams illustrate some possible safe and unsafe initialization sequences.



The controller must be commutated before issuing the ENABLE command.

1. Start activity scenario: servo control starts before brake is released.



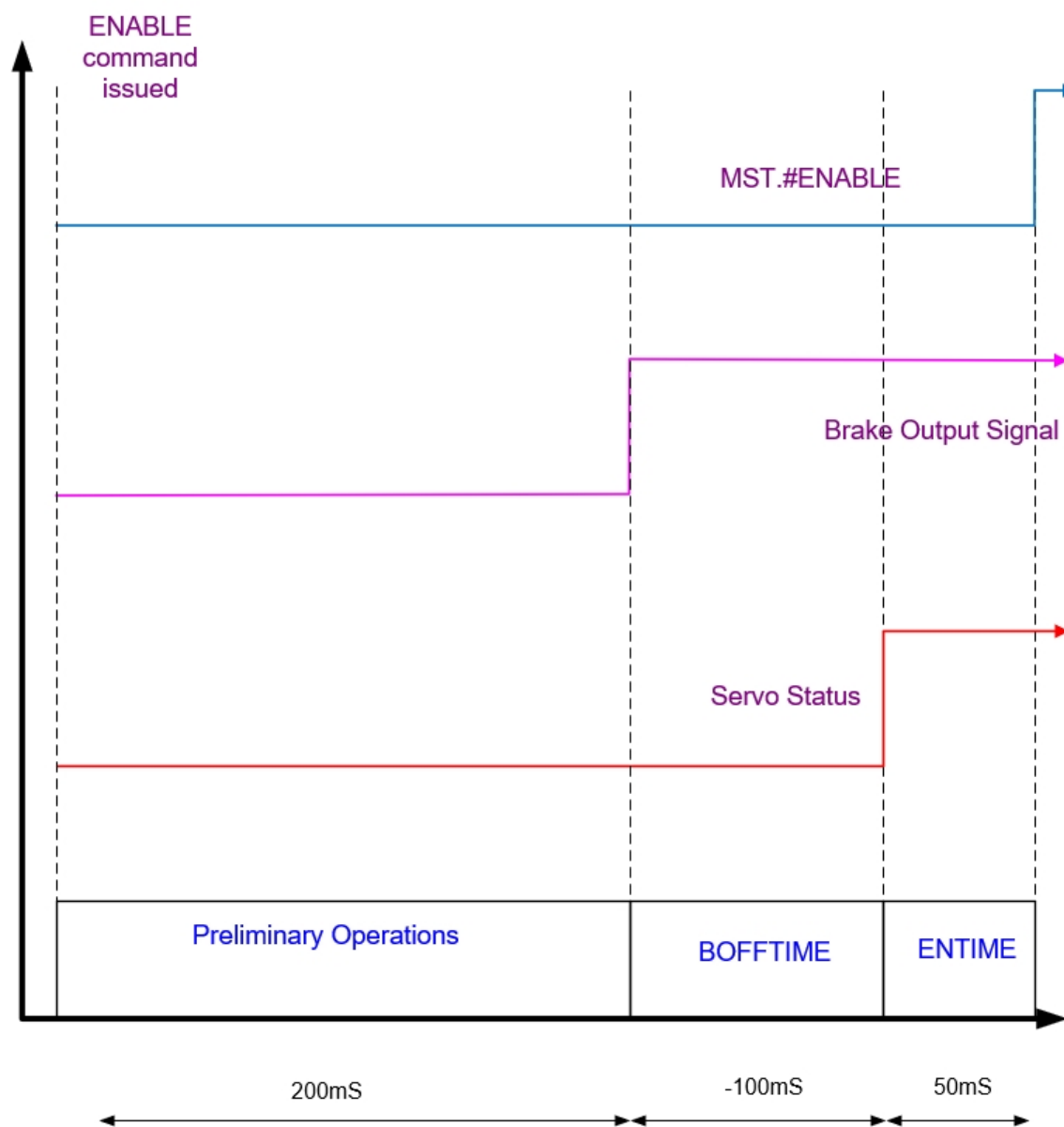
ENTIME=50; BOFFTIME=50

Enable output signal to drive (blue line)

Servo status (red line)

Brake output signal (violet line) – based on brake current measurement

2. Start activity scenario: Negative **BOFFTIME**, servo starts after brake signal rises.



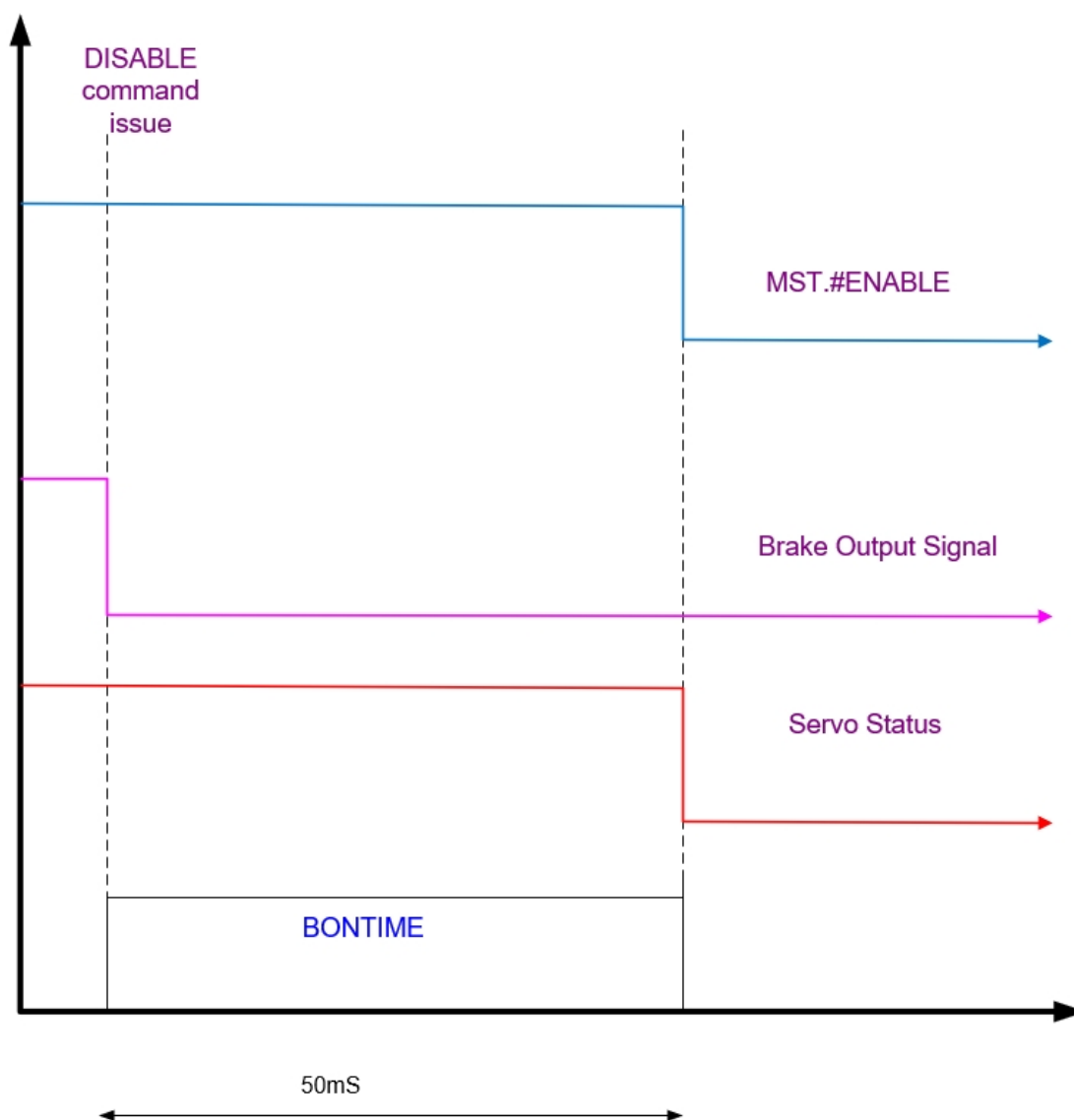
ENTIME=50; BOFFTIME=-100

Enable output signal to drive (blue line)

Servo status (red line)

Brake output signal (violet line) – based on brake current measurement

3. End Activity Scenario: Brake output off at start of BONTIME



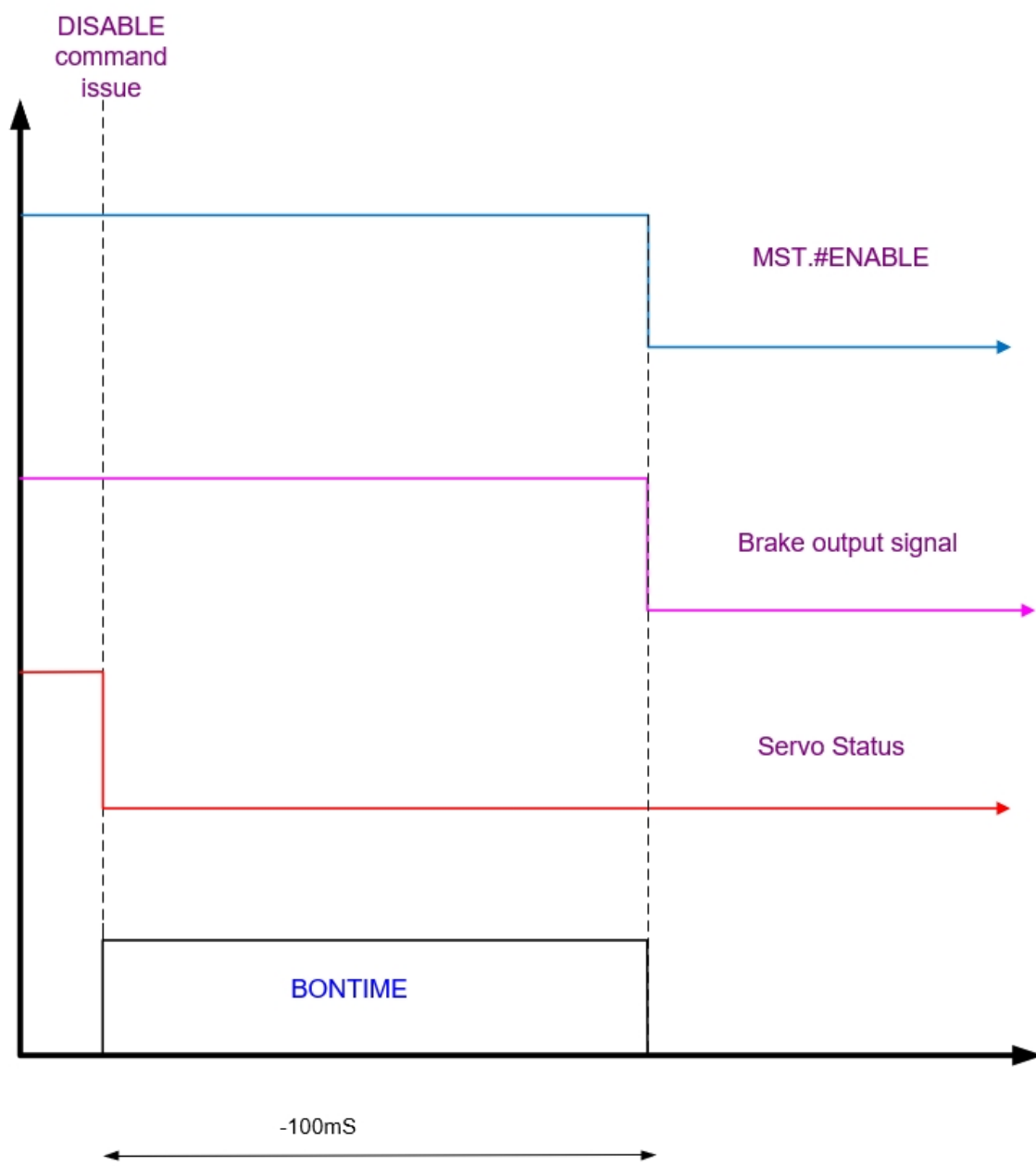
BONTIME=50

Enable output signal to drive (blue line)

Servo status (red line)

Brake output signal (violet line) – based on brake current measurement

4. End Activity Scenario: Brake status off after Negative BONTIME



BONTIME=-100 (Negative value)

Enable output signal to drive (blue line)

Servo status (red line)

Brake output signal (violet line) – based on brake current measurement

10. Generic EtherCAT Master

This chapter describes the generic interface of EtherCAT master functionality for the SpiiPlus family. This interface allows configuring and controlling any EtherCAT-compliant slave device via ACSPL+ commands and variables. Some devices, like qualified motion drives and I/O, will have a special interfaces in addition; however the generic one is the base and can be used for them too.

10.1 Stack Behavior

On the controller's start-up, the stack looks for the C:\ECAT.XML file. This file describes the network setup completely according to EtherCAT standard. The stack performs the following sequence of actions, if any of them is not successful, the sequence is stopped and the stack reports failure to go operational.

1. Scan the network and verify the scanned results via the XML file. In case of inconsistency stop
2. Initialize each slave
3. Initialize DC transmission
4. Set slaves to OP state
5. Initiate Master-Bus synchronization (if applicable) and wait for synchronization
6. Set Master state to OP

10.2 Interface Description

10.2.1 ACSPL+ Variables

10.2.1.1 ECST - EtherCAT State

The **ECST** is a one-byte variable the bits of which provide indications of the status of the EtherCAT. The status stored in the bits are:

Table 9-1. ECST Bits

Bit	Designator	Description
0	#SCAN	The scan process was performed successfully.
1	#CONFIG	There is no deviation between XML and actual setup.
2	#INITOK	All bus devices are successfully set to INIT state.
3	#CONNECTED	Indicates valid Ethernet cable connection to the master.
4	#INSYNC	If DCM is used, indicates synchronization between master and the bus.
5	#OP	The EtherCAT bus is operational.
6	#DCSYNC	Distributed clocks are synchronized.



All bits (except **#INSYNC** in some cases) should be true for proper bus functioning, for monitoring the bus state, checking the **#OP** bit is sufficient. Any bus error will reset the **#OP** bit.

10.2.2 ECERR

Any EtherCAT error sets **ECST.#OP** to false and the error code is latched in the **ECERR** variable. The only way to reset the error state and to clear the **ECERR** value is by calling **ECRESTART** function.

The EtherCAT error codes are:

Table 9-2. EtherCAT Error Codes

Error Code	Description
6000	General EtherCAT error.
6001	EtherCAT cable not connected.
6002	EtherCAT master is in incorrect state.
6003	Not all EtherCAT frames can be processed.
6004	EtherCAT Slave error.
6005	EtherCAT initialization failure.
6006	EtherCAT cannot complete the operation.
6007	EtherCAT work count error.
6008	Not all EtherCAT slaves are operational.
6009	EtherCAT protocol timeout.
6010	Slave initialization failed.
6011	Bus configuration mismatch

10.2.3 #ETHERCAT

The **#ETHERCAT** command is available for gaining complete information about the connected EtherCAT slaves: The command is entered through the SPiiPlus MMI Application Studio

Communication Terminal. The command provides the following:

- > Slave number
- > Vendor ID
- > Product ID
- > Revision

- > Serial number
- > EtherCAT physical address
- > DC support
- > Mailbox support
- > PdoIndex

Following the display of the above general data, a list of network variables is displayed. Each variable is described with:

- > Name (as in XML)
- > Offset inside the telegram (magic number that is used for mapping)
- > IN or OUT description
- > Data size

10.3 EtherCAT Functions

10.3.1 Mapping Functions

10.3.1.1 ECIN

Description:

The **ECIN** function is used for mapping input variables to the EtherCAT network.

Syntax:

ECIN[/b](*int offset, Varname*)

Arguments:

OFFSET	Internal EtherCAT offset (in bytes or bits) of network variable (which can be seen by running the #ETHERCAT command). > No suffix: EtherCAT offset in bytes > "/b" suffix: the offset is in bits
VARNAME	Valid name of ACSPL+ variable, global or standard.



It is recommended to use the **ECGETOFFSET** function to retrieve the offset, in bytes or bits.

Comments:

Once the function is called successfully, the Firmware copies the value of the network input variable at the corresponding EtherCAT offset into the ACSPL+ variable, every controller cycle.

There is no restriction on number of mapped network variables.

The mapping is allowed only when stack is operational, that is, **ECST.#OP** is true.

In the event of incorrect parameters or stack state, the function will produce the corresponding runtime error.

10.3.1.2 *ECOUT*

Description:

The **ECOUT** function is used for mapping output variables to the EtherCAT network.

Syntax:

ECOUT[/B](int offset, Varname)

Arguments:

OFFSET	Internal EtherCAT offset (in bits or bytes) of network variable (which can be seen by running the #ETHERCAT command). > No suffix: EtherCAT offset in bytes > "/b" suffix: the offset is in bits
VARNAME	Valid name of ACSPL+ variable, global or standard.



It's recommended to use the **ECGETOFFSET** function to retrieve the offset, in bytes or in bits.



It's possible to use combine the "/r suffix with the "/b" suffix. The syntax is as following:

```
Ecout/rb (int offset, Varname)
```

Comments:

The Firmware copies the value of ACSPL+ variable into the network output variable at the corresponding EtherCAT offset, every controller cycle.

There is no restriction on number of mapped network variables.

The mapping is allowed only when stack is operational, that is, **ECST.#OP** is true.

In the event of incorrect parameters or stack state, the function will produce the corresponding runtime error.

10.3.1.3 *ECUNMAP*

Description:

The **ECUNMAP** is used for resetting all previous mapping defined by **ECIN** and **ECOUT**.

Syntax:

ECUNMAP

Comments:

The function call is legal only when stack is operational, that is, **ECST.#OP** is true.

10.3.1.4 ECCLRREG**Description**

ESC Error Counters Registers Clear. The **ECCLRREG** function clears the contents of the error counters registers.

Syntax

void **ECCLRREG**(*index,offset*)

Arguments

Index	EtherCAT slave index.
Offset	Register offset in the Beckhoff memory.

Return Value

None

Comments

When the Offset value is -1, all error counters in all slaves are cleared. Otherwise, only the specific register at the specified Offset is cleared.

After executing the **ECCLRREG** function, we recommend to execute the **FCLEAR** function without parameters before running **ECGETREG**.

Example

Run the following code example in a Program Buffer.

```
ECCLRREG (0, 0x310)  
FCLEAR  
STOP
```

You can also enter this code in the SPiiPlus MMI Application Studio Connection Terminal:

```
ECCLRREG (0, -1) .
```

10.3.1.5 ECGETREG**Description**

ESC Error Counters Registers (Beckhoff Memory). The ESCs have numerous error counters that help you detect and locate errors. The **ECGETREG** function enables you to view the contents of these registers.

Syntax

int **ECGETREG**(*index,offset*)

Arguments

Index	EtherCAT slave index.
Offset	Register offset in the Beckhoff memory.

Return Value

None

Comments

The fsupported error counter registers are:

Table 9-3. Supported Error Counter Registers

Offset	Name	Description
0x300	Port Error Counter (CRC A)	Error Counted at the Auto-Forwarded (per port). Each register contains two counters: Invalid Frame Counter: 0x300/2/4/6 RX Error Counter: 0x301/3/5/7
0x302	Port Error Counter (CRC B)	
0x304	Port Error Counter (CRC C)	
0x306	Port Error Counter (CRC D)	
0x308	Forwarded RX Error Counter (CRC A/B)	Invalid frame with marking from previous ESC detected (per port).
0x309	Forwarded RX Error Counter	
0x30A	Forwarded RX Error Counter (CRC C/D)	
0x30B	Forwarded RX Error Counter	
0x30C	ECAT Processing Unit Error Counter	Invalid frame passing the EtherCAT Processing Unit (additional checks by processing unit).
0x30D	PDI Error Counter	Physical Errors detected by the PDI.
0x310	Lost Link Counter, Port A (IN)	Link Lost events (per port).

Offset	Name	Description
0x311	Lost Link Counter, Port B (OUT)	
0x312	Lost Link Counter, Port C	
0x313	Lost Link Counter, Port D	



If a cable is unplugged, we recommend using the **FCLEAR** command before using **ECGETREG**. The mapping is allowed only when stack is operational.

Example

Run the following code example in a Program Buffer.

```
I0=ECGETREG(0,0x310)
STOP
```

You can also enter this code in the SPiiPlus MMI Application Studio Connection Terminal:

```
?ECGETREG(0,0x310)
```

10.3.1.6 ECGETSLAVES

Description

This function is used to retrieve the number of slaves in an EtherCAT network.

Syntax

ECGETSLAVES

Arguments

None

Return Value

Number of EtherCAT slaves in the network.

Comments

If a slave was added or removed, the ECRESCAN command should be used before using **ECGETSLAVES** again.

10.3.1.7 ECUNMAPIN

Description

This function is used to reset all previous mapping defined by **ECIN** to a specific offset.

Syntax

ECUNMAPIN(*EOffset*)

Arguments

EOffset

An integer providing the offset to which a variable was mapped using **ECIN**.

Return Value

None

Comments

The mapping is allowed only when stack is operational.

Example

Given the previous execution of **ECIN**(48,I0), **ECUNMAPIN**(48) will unmap only I0.

10.3.1.8 ECUNMAPOUT

Description

This function is used to reset all previous mapping defined by **ECOUT** to a specific offset.

Syntax

ECUNMAPOUT(*EOffset*)

Arguments

EOffset

An integer providing the offset to which a variable has been mapped by **ECOUT**.

Return Value

None

Example

Assuming previous execution of **ECOUT**(48,I0) and **ECOUT**(50,I1), executing **ECUNMAPOUT**(48) will unmap only I0.

10.3.2 CoE Functions

CoE functions are required for SDO transfers in CoE. SDOs are part of the cyclic EtherCAT data transfer. It is impossible to define a generic function for any kind of mailbox transfer, such as protocols like EoE, FoE and VoE have their own definitions. So CoE is supported first.



The **#ETHERCAT** command can be used to check if a slave has Mailbox support.

10.3.2.1 COEWRITE

Description:

The **COEWRITE** function is used to write a value into a given slave.

Syntax:**COEWRITE***[/size] (int slave,int Index,int Subindex,real Value)***Arguments:**

size	1, 2 or 4 - the number of bytes in the OD or /f for floating.
slave	Slave number (can be obtained from the #ETHERCAT command)
index	Index in the OD.
subindex	Subindex in the OD.
Value	Value to be written.

Comments:

In case of wrong parameters, the corresponding runtime error will be generated. The function cannot be used in the SPiPlus MMI Application Studio **Communication Terminal**. The function delays the buffer execution on its line until it is successful or fails the whole buffer with timeout or other error.



This function with the "/d" suffix provides the capability of reading a double type (64 bit).

10.3.2.2 COEREAD**Description:**

The **COEREAD** function is used to read a value from a given slave.

Syntax:**real COEREAD***[/size] (int slave,int Index,int Subindex)***Arguments:**

size	1, 2 or 4 - the number of bytes in the OD or /f for floating.
slave	Slave number (can be obtained from the #ETHERCAT command).
index	Index in the OD.
subindex	Subindex in the OD.

Comments:

The function returns the received value or fails with runtime error.

In case of wrong parameters, the corresponding runtime error will be generated. The function cannot be used in the SPiPlus MMI Application Studio **Communication Terminal**. The function delays the buffer execution on its line until it is successful or fails the whole buffer with timeout or other error.



This function with the `"/d"` suffix provides the capability of reading a double type (64 bit).

11. Stepper Motor Servo Tuning

11.1 Stepper Closed-loop Position Correction

Stepper motors working under closed loop mode can now use the position correction mechanism to correct the actual position of the stepper. This may be implemented on the MPU using one of two firmware-based mechanisms or on a controller using the servo processor based method.

11.1.1 Stepper Closed Loop Firmware Based

The firmware-based position correction methods may be implemented on the PD1cl drive interface hardware. The Full Mode method performs position correction on every cycle during the motion. End Mode completes the correction at the end of the motion. In both cases, the correction only takes place if the position is outside the dead zone.

Position Correction is set to Full Mode by setting **MFLAGSX**=0.

Position Correction is set to End Mode by setting **MFLAGSX**=1.

The position correction is added to **RPOS**, as such **RPOS** turns into an accumulator for all corrections done for the axis and may inflate to large values – this does not reflect the position itself and further references to position should be done by **FPOS** and **APOS**.

These methods are controlled according to the values of **SLSKI**, **SLSKP**, **SLSRL**, **SLSMC**, **SLSDZ**, and **SLSOUT**.

The position correction is added to **RPOS**, so **RPOS** turns into an accumulator for all corrections done for the axis and may inflate to large values – this does not reflect the position itself and further references to position should be done by **FPOS** and **APOS**.

11.1.2 Stepper Closed Loop Servo Processor

Servo Processor based correction may be implemented on any PWM drive by setting **MFLAGSX**=2.

The mechanism is controlled according to the values of **SLPKP** and **SLSRL**.

12. Errors & Diagnostics

Error detection can occur in three circumstances:

- > An erroneous command is received through any communication channel
- > An error occurs while the controller executes an ACSPL+ program
- > The controller kills a motion or disables a motor because a fault was detected

This chapter provides a description of how error codes are generated. For the complete list of error codes see *SPiiPlus Command & Variable Reference Guide*.

12.1 Error Codes

Each error in the controller has a corresponding 4-digit error code. Regardless of how an error is detected and reported, you can request the error description from the controller. A request for error description consists of two question marks followed by the error code. For example, the following communication may occur when the controller is in protected mode:

VEL(0) = 10000	Attempt of assignment to a protected variable.
?2085	The controller reports an error.
??2085	Request for error description.
Protection violation	Error description

12.1.1 Error Code Ranges

Table 10-1. SPiiPlus Error Code Ranges

Range	Type	Description
0 - 999	Errors detected by SPiiPlus C Library	Errors are detected by C Library without communication with the controller. The controller itself never returns error codes in this range. A host application that calls a C Library function can receive this code if a function call failed. For explanation of the errors see the <i>SPiiPlus C Library Reference Guide</i> .
1000 - 1999	Errors in terminal commands	Errors in terminal commands are reported immediately in the prompt that is displayed in response to the command.

Range	Type	Description
2000 - 2999	ACSPL+ compilation errors	ACSPL+ compilation errors are reported either immediately when the erroneous line is inserted, or subsequently, when the controller attempts to compile an ACSPL+ program. If a program in a buffer undergoes compilation and an error is detected, the error code is stored in the corresponding element of the PERR array and the erroneous line number is stored in the corresponding element of the PERL array
3000 - 3999	ACSPL+ termination codes	Codes from 3000 to 3019 do not indicate an error. For example, code 3002 reports that the user has terminated the program. Codes from 3020 to 3999 indicate run-time errors. If an error occurs in immediate execution of ACSPL+ command, the error is indicated immediately in the prompt. If an error occurs when an ACSPL+ program is executed in a buffer, no immediate indication is provided. Instead, the error code and the line number are stored in the corresponding elements of the PERR and PERL arrays.
5000 - 5999	Motion termination codes and motor disable codes	Codes from 5000 to 5008 do not indicate an error. They report normal motion termination. Codes from 5009 and higher appear when a motion is terminated or a motor is disabled due to a fault detected by the controller. When a motion terminates abnormally, or a motor is disabled, the error code is stored in the MERR variable.
6000 - 6999	EtherCAT code	Stored in the ECERR variable, and when occurs, sets ECST.#OP to false.
7000 - 7999	EtherCAT codes based on AL Status codes	Stored in the ECALERR variable, and a value "0" indicates no error.
≥ 9000	User-defined codes	The user can execute commands KILL, KILLALL, DISABLE, DISABLEALL with an argument that specifies a user-defined error code. The specified error code is stored in the MERR variable.

12.2 Error Indication

12.2.1 Errors in Received Commands

If the controller receives a correct command, it responds with the normal prompt (colon).

If a command cannot be executed for any reason, the controller responds with the error prompt. The error prompt contains the "?" character and 4-digit error code. For example:

VEL(0) = 1000	
:	Normal prompt - the command was successful
FPOS(0) = 0	
?2020	Error - attempt of assignment to a read-only variable

12.2.2 Errors in ACSPL+ Programs

If the controller executes an ACSPL+ program and an error occurs due to the program, the controller does the following:

- > Aborts the erroneous program
- > Stores the error code in the corresponding element of the **PERR** array
- > Stores the line number that caused the error in the corresponding element of the **PERL** array
- > Activates the **#PROG** fault. The default controller response to the fault is to kill all executed motions.

No error prompt is issued. You must analyze the state of the buffer in order to detect program errors. For example:

?1	Query the state of buffer 1
Buffer 1: 78 lines, run-time error 3077 in line 37	
?PERR(1)	The state also can be monitored through variable PERR

12.2.3 Motion Termination Codes

A motion executed in the controller can terminate for different reasons:

- > The motion comes to its final point - normal termination
- > You interrupt the motion with a **HALT** or **KILL** command
- > The controller detects a fault that requires motion kill

In all cases the controller stores the termination reason in the corresponding element of the **AERR** (Axis Error) variable. No error prompt is issued. You have to analyze the state of the axis in order to detect motion termination. For example:

?\$0	Query of the state of axis 0.
Motor 0(X): enabled, idle, in positionAxis 0(X): motion was killed by user	
?AERR(0)	The state also can be monitored through the AERR variable.
5002	The termination code - motion was killed by user.
?\$13	Query of the state of axis 13.
Motor 13(Axis13): enabled, idle, in positionMotor 13(Axis13): motion failed, reason 5011	
?AERR(13)	The state also can be monitored through the AERR variable.
5011	The termination code - Left Limit fault

12.2.4 Motion Termination and Motor Disable Codes

A motion executed in the controller can terminate for the different reasons:

- > The motion comes to its final point - normal termination
- > You or the ACSPL+ program interrupts the motion with a **HALT**, **KILL**, or **KILLALL** command
- > A motor involved in the motion is disabled for any reason
- > The controller detects a fault that requires motion kill

The controller disables a motor for the following reasons:

- > You or the ACSPL+ program executes a **DISABLE** or **DISABLEALL** command
- > The controller detects a fault that requires motor disable

If a motor is disabled or a motion is killed due to a fault, the controller stores the reason in the corresponding element of the **MERR** (Motor Error) variable.

You can also specify a termination code for commands **KILL**, **KILLALL**, **DISABLE** or **DISABLEALL**. If one of the commands is used with a non-zero reason argument, the specified code is stored in one or more corresponding elements of **MERR** as a termination/disable code.

In a case where no error prompt is issued, you must analyze the state of the motor in order to detect whether the motor was disabled or a motion was killed abnormally. For example:

?2	Query of the state of motor 2
Motor 2(Z): disabled Disable reason: 5023 - Critical Position ErrorAxis 2(Z): motion was killed because a motor was disabled	
?MERR(2)	The state also can be monitored through variable MERR

The error code - motor was disabled because of critical position error fault

12.2.5 Getting Extended Drive Fault Status

Upon receipt of a Drive Alarm signal, the controller stores a general Drive Alarm code (5019) in the **MERR** variable. The extended Drive Fault status code can be obtained by executing the **GETCONF (246, Axis)** function (see *SPiiPlus Command & Variable Reference Guide*).

The following fault codes are returned:

- > 5061 – Short circuit
- > 5064 – Power supply too high
- > 5065 – Temperature too high
- > 5069 – Power down
- > 5071 – Drive not ready
- > 5072 – Over current

Using the **GETCONF** function, the faults 5064, 5065, 5069, 5071 can be read before **ENABLE** command is executed. In order to clear the Drive Fault status code, use the **SETCONF** function: **SETCONF(246, Axis, 0)**. This function clears the fault status on all axes that relate to the DDM3U Motor Drive that handles the specified axis.

13. Application Examples

This chapter contains practical examples for use in SPiiPlus applications.

For further examples, registered users may go to www.AcsMotionControl.com and download pertinent documents (selected from the **Categories** dropdown list on the page).

13.1 Encoder Error Compensation with Constant Step

Assume an axis was calibrated with a laser interferometer. The calibration process includes positioning to the points equally spaced according to the encoder feedback, and measuring the exact positions by the laser interferometer. The calibration points start from the coordinate 10000 and follow each 1000 counts. The last calibration point is 20000.

The calibration produced the following table:

Feedback	10000	11000	12000	13000	14000	15000	16000	17000	18000	19000	20000
Actual	10012	10998	11085	12081	13097	15007	16013	17023	18005	18993	19991
Error	+12	-2	-15	-19	-3	+7	+13	+23	+5	-7	-9

Only the third row is entered to a controller variable and is stored in the flash **X_ERROR** file. Details of the calibration routine, which can be implemented using ACSPL+, are not discussed here.

An application that uses the file may provide an initialization routine like this:

real X_ERR(11)	Declare real array X_ERR with 11 members.
AUTOEXEC:	ACSPL+ label initializing the routine on start-up.
WRITE X_ERRREAD X_ERR	Write to and read from the calibration table in the flash memory.
CONNECT RPOS(0)0 = APOS(0) - MAP (APOS(0), X_ERR, 10000, 1000)	
STOP	Finish initialization

The **CONNECT** function specifies that the reference position be calculated by subtracting the interpolated error from the desired position so that the actual value will be closer to the desired value.

13.2 Encoder Error Compensation with Arbitrary Step

Assume in the above example that the calibration routine does not calculate an error, but writes the first two lines from the above table to the **X_CALIBR** flash file. The table is stored as an array with

2 rows and 11 columns.

In this case the application can implement an initialization routine like this:

real X_CAL(2)(11)	Declare real 2x11 matrix X_CAL .
AUTOEXEC:	ACSPL+ label initializing the routine on start-up.
WRITE X_CALREAD X_CAL	Write to and read from calibration table from the flash memory.
CONNECT RPOS(0) = MAPBY2(APOS(0), X_CAL)	
STOP	Finish initialization

And the **X_CALIBR** file can also contain a table with non-uniform points.

13.3 Backlash Compensation

Assume that the 0 axis has a backlash of 20 counts. The following **CONNECT** command compensates for the backlash:

```
CONNECT RPOS(0) = APOS(0) + 10*dsign(RVEL(0), 0, 0)
```

By using this **CONNECT**, the value added to desired position changes immediately when the direction of the motion changes. In many cases such jumps in the desired position are harmful. In this case the third parameter in the **DSIGN** function (see *SPIIPlus Command & Variable Reference Guide*) can be used to gradually implement the backlash compensation. In the following example the backlash compensation is introduced by small steps, so that the compensation growth to the entire value by 20 milliseconds:

```
CONNECT RPOS(0) = APOS(0) + 10*dsign(RVEL(0), 0, 20)
```

If the 0 axis executes master-slave motion slaved to some physical value like encoder feedback, the **RVEL** value contains noise that can cause undesirable switching of the backlash sign. In this case the second parameter of the **DSIGN** function can be used to introduce anti-bouncing effect. In the following example the backlash compensation changes its sign only if **RVEL** holds its new sign for more than 10 milliseconds:

```
CONNECT RPOS(0) = APOS(0) + 10*DSIGN(RVEL(0), 10, 20)
```

13.4 Compensation of Encoder Error and Backlash

An arbitrary expression can be used as an argument in the design function and in the map-functions. It ensures combining different compensation function and other required transformations in one **CONNECT** command. The following example combines error and backlash compensations from the above examples:

real X_CAL(2)(11)	Declare real 2x11 matrix X_CAL .
AUTOEXEC:	ACSPL+ label initializing the routine on start-up.
WRITE X_CALREAD X_CAL	Write to and read from calibration table from the flash memory.

```
CONNECT RPOS(0) = MAPBY2(APOS(0)+10*DSIGN(RVEL(0), 10, 20), X_CAL)
```

```
STOP
```

```
Finish initialization
```

13.5 Cam Motion

Assume that the 1 axis must provide a cam motion following the 0 axis. The **CAMTABLE** matrix in the nonvolatile (flash) memory contains 2 rows and 1000 columns. Each column contains an X coordinate (for axis 0) in the first row and the corresponding Y coordinate (for axis 1) in the second row. The X coordinates in the first row can be spaced either equally or non-equally.

The following fragment initializes the cam motion:

global real CAMTABLE(2)(1000)	Declare 2x1000 matrix CAMTABLE .
AUTOEXEC:	
WRITE CAMTABLEREAD CAMTABLE,CAMTABLE	Write to and read from calibration table from the flash memory.
MASTER MPOS(1) = MAPBY1(FPOS(0), CAMTABLE)	Define master value via CAMTABLE
SLAVE(1)	Start master-slave motion

13.6 Joystick

Assume that a joystick that controls the motion of an X,Y table is connected to analog inputs **AIN(0)**, **AIN(1)**. The velocity of each coordinate must be proportional to the corresponding analog input. Analog input lower than 20 counts must be ignored to avoid motion due to analog drift or bias. The X (axis 0) motion is limited in the range from -500 to 100000 counts. The Y (axis 1) motion is limited in the range from -500 to 500000 counts.

The following program fragment initializes the joystick motion:

real JK	
JK = 10	Joystick factor
MASTER MPOS(0) = INTGR(AIN(0), 20, -500, 100000)	Define 0 master
MASTER MPOS(1) = INTGR(AIN(1), 20, -500, 500000)	Define 1 master
SLAVE(1)	Start master-slave motion

13.7 XYZ Rotation

It is sometimes necessary to rotate the reference plane which motion is commanded in. The basic rotation matrices can be expressed as:

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\phi) & -\sin(\phi) \\ 0 & \sin(\phi) & \cos(\phi) \end{bmatrix} \quad R_y = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{bmatrix} \quad R_z = \begin{bmatrix} \cos(\psi) & -\sin(\psi) & 0 \\ \sin(\psi) & \cos(\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The illustration below shows the rotation of the plane from the original (x, y, z) plane to a rotated (x''', y''', z''') plane.

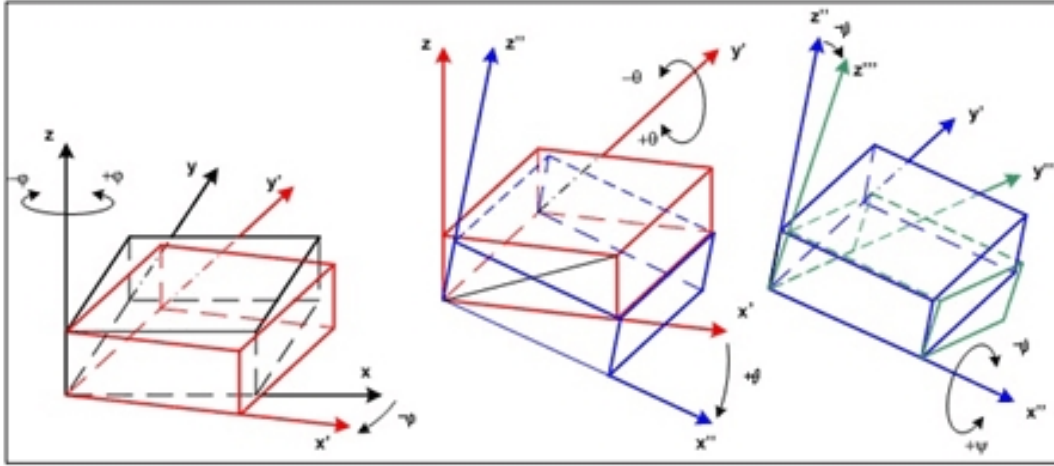


Figure 11-1. XYZ Rotation

Rotation of the plane can be expressed as:

$$\begin{bmatrix} X_F \\ Y_F \\ Z_F \end{bmatrix} = R \times \begin{bmatrix} X_1 \\ Y_1 \\ Z_1 \end{bmatrix}$$

Where **R** is the product of the rotation matrices used to shift the plane, $\vec{V}_F$ represents the vector coordinates in the rotated plane, and $\vec{V}_I$ represents the vector coordinates in the initial plane. For example, rotation about the Z axis can be expressed as: $\vec{V}_F = R_Z \times \vec{V}_I$. The three-dimensional rotation matrix is expressed as the product of all three individual matrices:

$$R_{xyz} = \begin{bmatrix} \cos(\theta) \cos(\psi) & \cos(\theta) \sin(\psi) + \sin(\phi) \sin(\theta) \cos(\psi) & \sin(\phi) \sin(\psi) - \cos(\phi) \sin(\theta) \cos(\psi) \\ -\cos(\theta) \sin(\psi) & \cos(\theta) \cos(\psi) + \sin(\phi) \sin(\theta) \sin(\psi) & \sin(\phi) \sin(\psi) + \cos(\phi) \sin(\theta) \sin(\psi) \\ \sin(\theta) & -\sin(\phi) \cos(\theta) & \cos(\phi) \cos(\theta) \end{bmatrix}$$

This allows the final plane coordinates to be solved for from the initial plane coordinates. To utilize this rotation on the ACS controller, it is also necessary to solve this equation such that the initial plane coordinates are solved for using the final plane coordinates, or $\vec{V}_I = R_{xyz}^{-1} \times \vec{V}_F$. In this case, R_{xyz}^{-1} can be solved for as:

$$R_{xyz}^{-1} = \begin{bmatrix} \cos(\theta) \cos(\psi) & -\cos(\theta) \sin(\psi) & \sin(\theta) \\ \cos(\phi) \sin(\psi) + \sin(\phi) \sin(\theta) \cos(\psi) & \cos(\phi) \cos(\psi) - \sin(\phi) \sin(\theta) \sin(\psi) & -\sin(\phi) \cos(\theta) \\ \sin(\phi) \sin(\psi) - \cos(\phi) \sin(\theta) \cos(\psi) & \sin(\phi) \sin(\psi) + \cos(\phi) \sin(\theta) \sin(\psi) & \cos(\phi) \cos(\theta) \end{bmatrix}$$

With these equations, it is now possible to utilize the SET and CONNECT equations to establish a rotated plane on the controller. Consider RPOS of the axes to be the axis positions in the initial plane, and APOS to be the commanded positions in the rotated plane. It is now possible to use the CONNECT equation to allow the controller to automatically calculate the actuator positions based on user commanded positions in the rotated plane. See below for an example of how this is done:

```
! 3D Rotation using CONNECT
GLOBAL REAL PHI, THETA, PSI
REAL PI
PI = ACOS(-1)
! Set rotation angle about X axis in radians
PHI = 0*(PI/180)
! Set rotation angle about Z axis in radians
THETA = 0*(PI/180)
! Set rotation angle about Z axis in radians
PSI = 0*(PI/180)
! Reset the default connection on each axis
MFLAGS(X).#DEFCON = 1; MFLAGS(Y).#DEFCON = 1; MFLAGS(Z).#DEFCON = 1;
MFLAGS(X).#DEFCON = 0; MFLAGS(Y).#DEFCON = 0; MFLAGS(Z).#DEFCON = 0;
! Forward kinematics
SET APOS(X) = (COS(THETA)*COS(PSI))*RPOS(X) + (COS(PHI)*SIN(PSI)+SIN
(PHI)*SIN(THETA)*COS(PSI))*RPOS(Y) + (SIN(PHI)*SIN(PSI)-COS(PHI)*SIN
(THETA)*COS(PHI))*RPOS(Z)
SET APOS(Y) = (-COS(THETA)*SIN(PSI))*RPOS(X) + (COS(PHI)*COS(PSI)-SIN
(PHI)*SIN(THETA)*SIN(PSI))*RPOS(Y) + (SIN(PHI)*COS(PSI)+COS(PHI)*SIN
(THETA)*SIN(PSI))*RPOS(Z)
SET APOS(Z) = (SIN(THETA))*RPOS(X) + (-SIN(PHI)*COS(THETA))*RPOS(Y) +
(COS(PHI)*COS(THETA))*RPOS(Z)
! Inverse kinematics
CONNECT RPOS(X) = (COS(THETA)*COS(PSI))*APOS(X) + (-COS(THETA)*SIN
(PSI))*APOS(Y) + (SIN(THETA))*APOS(Z)
CONNECT RPOS(Y) = (COS(PHI)*SIN(PSI)+SIN(PHI)*SIN(THETA)*COS(PSI))*APOS
(X) + (COS(PHI)*COS(PSI)-SIN(PHI)*SIN(THETA)*SIN(PHI))*APOS(Y) + (-SIN
(PHI)*COS(THETA))*APOS(Z)
CONNECT RPOS(Z) = (SIN(PHI)*SIN(PSI)-COS(PHI)*SIN(THETA)*COS(PSI))*APOS
(X) + (SIN(PHI)*COS(PSI)+COS(PHI)*SIN(THETA)*SIN(PSI))*APOS(Y) + (COS
(PHI)*COS(THETA))*APOS(Z)
DEPENDS X, (X,Y,Z)
DEPENDS Y, (X,Y,Z)
DEPENDS Z, (X,Y,Z)
STOP
```

Appendix A. PEG And MARK Mapping Tables

A.1 ASSIGNPEG Mapping

Mapping PEG engines to encoders

Table A-1. Mapping PEG Engines to Encoders (Servo Processor 0) for SPiiPlusNT/DC-LT/HP/LD

Bit Code	Encoder 0(X)	Encoder 1(Y)	Encoder 2(A)	Encoder 3(B)
000 (default)	PEG0	PEG1	PEG2	no
001	PEG0	PEG1	no	PEG2
010	PEG0 PEG2	PEG1	no	no
011	PEG0	PEG1 PEG2	no	no
100	PEG0 PEG1 PEG2	no	no	no
101	no	PEG0 PEG1 PEG2	no	no

Table A-2. Mapping PEG Engines to Encoders (Servo Processor 1) for SPiiPlusNT/DC-LT/HP/LD

Bit Code	Encoder 4(Z)	Encoder 5(T)	Encoder 6(C)	Encoder 7(D)
000 (default)	PEG4	PEG5	PEG6	no
001	PEG4	PEG5	no	PEG6
010	PEG4 PEG6	PEG5	no	no
011	PEG4	PEG5 PEG6	no	no
100	PEG4 PEG5 PEG6	no	no	no

Bit Code	Encoder 4(Z)	Encoder 5(T)	Encoder 6(C)	Encoder 7(D)
101	no	PEG4 PEG5 PEG6	no	no

**Table A-3. Mapping PEG Engines to Encoders (Servo Processor 0) for SPiiPlus
CMnt/CMhv/CMba/CMhp/
CMxa/UDMba/UDMhp/UDMxa/UDMhv/UDMnt/UDMpa/UDMpm/UDMpc/UDMcb**

Bit Code	Encoder 0(X)	Encoder 1(Y)	Encoder 2(A)	Encoder 3(B)
000 (default)	PEG0	PEG1	PEG2 ²	
001	PEG0	PEG1	no	PEG2 ^{1, 2}
010	PEG0 PEG2 ²	PEG1	no	
011	PEG0	PEG1 PEG2 ²	no	
100	PEG0 PEG1 PEG2 ²	no	no	
101	no	PEG0 PEG1 PEG2 ²	no	
110			PEG0 ^{1, 2} PEG1 ^{1, 2} PEG2 ^{1, 2}	
111			PEG0 ^{1, 2} PEG2 ^{1, 2}	PEG1 ^{1, 2}

¹ These combinations are **not** supported by UDMpc-x.

² These combinations are **not** supported by UDMnt-x.

Table A-4. Mapping PEG Engines to Encoders (Servo Processor 0) for UDMlc/UDlIt/UDlhp/UDMmc/PDlcl

Bit Code	Encoder 0	Encoder 1	Encoder 2	Encoder 3
000 (default)	PEG0	no	no	no
001	no	PEG0	no	no
010	no	no	PEG0	no
011	no	no	no	PEG0 ¹
100	no	no	no	no
101	no	no	no	no
110	no	no	no	no

¹ These combinations are **not** supported by LCM-x.

Table A-5. Mapping PEG Engines to Encoders (Servo Processor 0) for NPMpm/NPMpc-

Bit Code	Encoder 0	Encoder 1	Encoder 2	Encoder 3
000 (default)	PEG0	PEG1	no	no
001	PEG1	PEG0	no	no
010	PEG0 PEG1	no	no	no
011	no	PEG0 PEG1	no	no
100	no	PEG1	PEG0	no
101	no	PEG1	no	PEG0
110	PEG0	no	PEG1	no
111	PEG0	no	no	PEG1

Bit Code assignment example

Assume the system includes two Servo Processors:

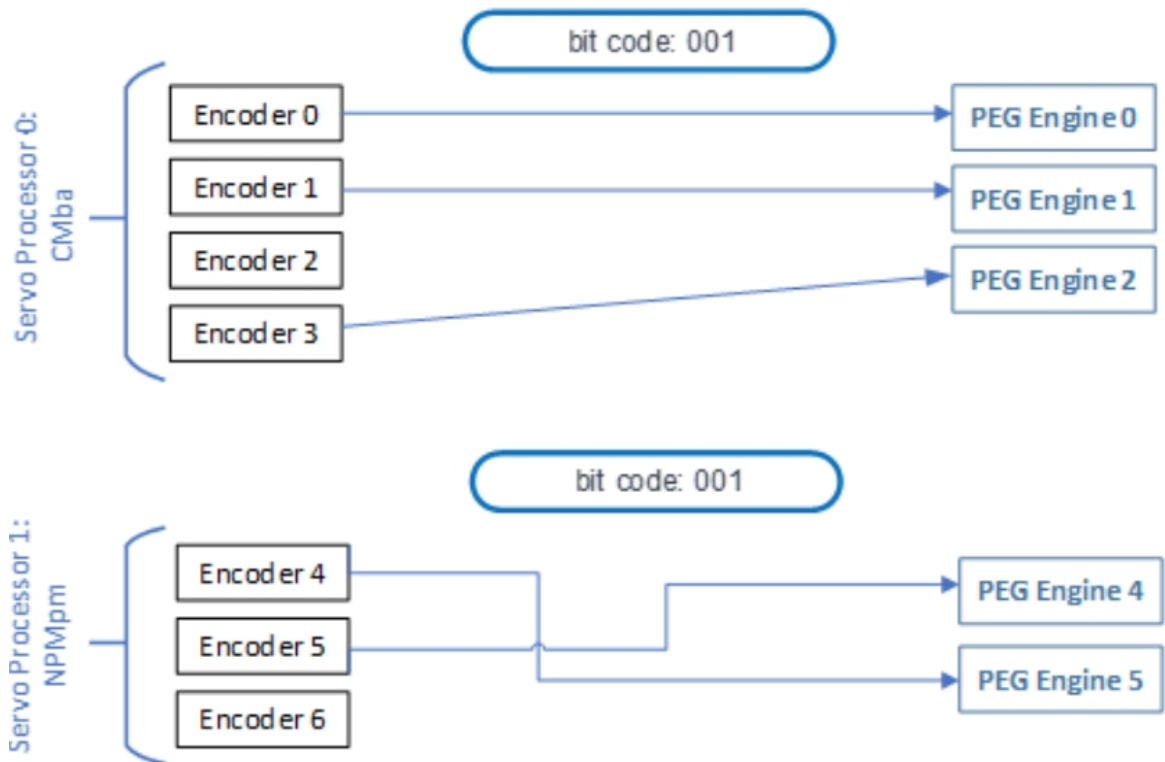
- > Servo Processor (node) 0: CMba with 4 encoders (0,1,2,3) and 3 PEG engines (0,1,2).
- > Servo Processor (node) 1: NPMpm with 4 encoders (4,5,6) and 2 PEG engines (4,5).

The bit code 001 for an axis associated with Servo Processor 0 (CMba) performs the following assignment:

- > PEG 0 engine is triggered by Encoder 0.
- > PEG 1 engine is triggered by Encoder 1.
- > PEG 2 engine is triggered by Encoder 3.

The bit code 001 for an axis associated with Servo Processor 1 (NPMpm) performs the following assignment:

- > PEG 4 engine is triggered by Encoder 5.
- > PEG 5 engine is triggered by Encoder 4.



General purpose outputs assignment to use as PEG pulse outputs

Table A-6. General Purpose Outputs Assignment for Use as PEG Pulse Outputs (Servo Processor 0) for SPiiPlusNT/DC-LT/HP/LD

Bit Code	GP Out 0	GP Out 1	GP Out 2	GP Out 3
0000 (default)	GP Out 0	GP Out 1	GP Out 2	GP Out 3
0001	PEGO_PULSE	GP Out 1	GP Out 2	GP Out 3
0010	GP Out 0	PEG2_PULSE	GP Out 2	GP Out 3

Bit Code	GP Out 0	GP Out 1	GP Out 2	GP Out 3
0011	GP Out 0	GP Out 1	PEG1_PULSE	GP Out 3
0100	GP Out 0	GP Out 1	GP Out 2	Reserved
0101	GP Out 0	PEG2_PULSE	GP Out 2	Reserved
0110	PEGO_PULSE	GP Out 1	PEG1_PULSE	GP Out 3
0111	PEGO_PULSE	PEG2_PULSE	PEG1_PULSE	Reserved
1000 - 1111	Reserved	Reserved	Reserved	Reserved

Table A-7. General Purpose Outputs Assignment for Use as PEG Pulse Outputs (Servo Processor 1) for SPiiPlusNT/DC-LT/HP/LD

Bit Code	GP Out 4	GP Out 5	GP Out 6	GP Out 7
0000 (default)	GP Out 4	GP Out 5	GP Out 6	GP Out 7
0001	PEG4_PULSE	GP Out 5	GP Out 6	GP Out 7
0010	GP Out 4	PEG6_PULSE	GP Out 6	GP Out 7
0011	GP Out 4	GP Out 5	PEG5_PULSE	GP Out 7
0100	GP Out 4	GP Out 5	GP Out 6	Reserved
0101	GP Out 4	PEG6_PULSE	GP Out 6	Reserved
0110	PEG4_PULSE	GP Out 5	PEG5_PULSE	GP Out 7
0111	PEG4_PULSE	PEG6_PULSE	PEG5_PULSE	Reserved
1000 - 1111	Reserved	Reserved	Reserved	Reserved

Table A-8. General Purpose Outputs Assignment for Use as PEG Pulse Outputs (Servo Processor 0) for SPiiPlus CMnt/UDMpm/CMhv/UDMhv-

Bit Code	GP Out 0	GP Out 1	GP Out 2	GP Out 3
0000 (default)	GP Out 0	GP Out 1	GP Out 2	GP Out 3
0001	PEGO_PULSE	GP Out 1	GP Out 2	GP Out 3
0010	GP Out 0	PEG1_PULSE	GP Out 2	GP Out 3
0011	GP Out 0	GP Out 1	PEG2_PULSE	GP Out 3

Bit Code	GP Out 0	GP Out 1	GP Out 2	GP Out 3
0100	GP Out 0	GP Out 1	GP Out 2	GP Out 3
0101	GP Out 0	PEG1_PULSE	GP Out 2	GP Out 3
0110	PEGO_PULSE	GP Out 1	PEG2_PULSE	GP Out 3
0111	PEGO_PULSE	PEG1_PULSE	PEG2_PULSE	GP Out 3
1000 - 1111	Reserved	Reserved	Reserved	Reserved

Table A-9. General Purpose Outputs Assignment for Use as PEG Pulse Outputs (Servo Processor 0) for UDMnt/UDMpa/UDMcb

Bit Code	GP Out 0	GP Out 1
0000 (default)	GP Out 0	GP Out 1
0001	PEGO_PULSE	GP Out 1
0010	GP Out 0	PEG1_PULSE
0011	PEG1_PULSE	GP Out 1
0100	GP Out 0	PEGO_PULSE
0101	PEGO_PULSE	PEG1_PULSE
0110 - 1111	Reserved	Reserved

Bit Code assignment example

For example, for an axis associated with Servo Processor 0 (SPiiPlusNT / DC-LT / HP / LD), **0110** switches **GP Out 0** to **PEGO_PULSE** and **GP Out 2** to **PEG1_PULSE**.

The same **Bit Code** applied to an axis associated with Servo Processor 1 switches **GP Out 4** to **PEG4_PULSE** and **GP Out 6** to **PEG5_PULSE**.

All other **GP Out** assignments are unchanged.

Table A-10. Engine to Encoder Assignment for IDMxx, ECMxx, and UDMxx

PEG Engine	Bits	HEX Code	Encoder 0	Encoder 1	Encoder 2	Encoder 3
0	0..7	00 (default)	PEG0			
		01		PEG0		
		02			PEG0	
		03				PEG0
1 ¹	8..15	00	PEG1			
		01 (default)		PEG1		
		02			PEG1	
		03				PEG1
2 ¹	16..23	00	PEG2			
		01		PEG2		
		02 (default)			PEG2	
		03				PEG2
3 ¹	24..31	00	PEG3			
		01		PEG3		
		02			PEG3	
		03				PEG3

¹ This row applies to XXMsm, XXMma, and XXMdx

² These rows apply only to XXMsm and XXMma

Instructions: the above table is used to build a hexadecimal value for the engines_to_encoders_ code argument. Byte x determines that PEG engine x will be triggered by a specific encoder.

Example for IDMsm/ECMsm

```
ASSIGNPEG 0, 0x03020100
```

This code configures the default mapping:

- > PEG engine 0 is triggered by Encoder 0.
- > PEG1 engine 1 is triggered by Encoder 1.
- > PEG engine 2 is triggered by Encoder 2.
- > PEG engine 3 is triggered by Encoder 3.

ASSIGNPEG 0x02010203

This code configures the following mapping:

- > PEG engine 0 is triggered by Encoder 3.
- > PEG engine 1 is triggered by Encoder 2.
- > PEG engine 2 is triggered by Encoder 1.
- > PEG engine 3 is triggered by Encoder 2.

A.2 ASSIGNPOUTS Mapping

Mapping of PEG engine outputs to physical outputs

Table A-11. Mapping of Engine Outputs to Physical Outputs (Servo Processor 0) for SPIiPlusNT/DC-LT/HP/LD

PEG OUTPUT PIN NAME Bit Code	0 X_PEG	1 Y_PEG	2 Z_PEG	3 T_PEG	4 H_DO_1 (HSSI)
000 (default)	PEG0_ PULSE	PEG1_ PULSE	PEG4_ PULSE	PEG5_ PULSE	HSSI1_DO
001	PEG4_ STATE0	PEG1_ STATE0	PEG2_ PULSE	PEG0_ STATE0	PEG0_ STATE1
010	Reserved	Reserved	PEG1_ STATE1	PEG2_ STATE0	PEG2_ STATE1
011	Reserved	Reserved	Reserved	Reserved	Reserved
100	Reserved	Reserved	Reserved	Reserved	Reserved
111	FGP_OUT0	FGP_OUT1	FGP_OUT2	FGP_OUT3	Reserved

Table A-12. SPiiPlusNT/DC-LT/HP/LD Mapping of Engine Outputs to Physical Outputs (Servo Processor 1)

PEG OUTPUT PIN NAME Bit Code	5 X_STATE0	6 X_STATE1	7 X_STATE2	8 H_DO_0 (HSSI)	9 H_CON_0 (HSSI)
000 (default)	PEG0_ STATE0	PEG0_ STATE1	PEG0_ STATE2	HSSIO_DO	HSSIO_ CON
001	PEG1_ STATE0	PEG1_ STATE1	PEG2_ STATE0	PEG0_ STATE2	PEG0_ STATE0
010	PEG4_ PULSE	PEG5_ PULSE	PEG6_ PULSE	PEG5_ STATE0	PEG2_ STATE1
011	Reserved	PEG4_ STATE0	PEG4_ STATE1	Reserved	PEG6_ STATE1
100	Reserved	Reserved	Reserved	Reserved	Reserved
111	FGP_OUT4	FGP_OUT5	FGP_OUT6	Reserved	Reserved

**Table A-13. Mapping of Engine Outputs to Physical Outputs (Servo Processor 0) for
CMnt/UDMpm/UDMpc/CMhv/UDMhv**

PEG OUTPUT PIN NAME Bit Code	0 PEGO	1 PEG1	5 STATE0	6 STATE1
000 (default)	PEGO_PULSE	PEG1_PULSE	PEGO_OUT0	PEGO_OUT1
001	Encoder X Phase A	Encoder X Phase B	PEG1_OUT0	PEG1_OUT1
010	Encoder Y Phase A	Encoder Y Phase B	PEG2_OUT0	PEG2_OUT1
011	Reserved	Reserved	Encoder X Phase A	Encoder X Phase B
100	Reserved	Reserved	Encoder Y Phase A	Encoder Y Phase B
101	Reserved	Reserved	Encoder X INDEX	Encoder X INDEX
110	Reserved	Reserved	Reserved	Encoder Y INDEX
111	FGP_OUT0	FGP_OUT1	Reserved	Reserved

Table A-14. Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, OUT 0-4) for CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa

PEG OUTPUT PIN NAME Bit Code	0 (0)_PEG_ PULSE	1 (1)_PEG_ PULSE	2 (1)_STATE0	3 (1)_STATE1	4 (1)_STATE2
000 (default)	PEG0_ PULSE	PEG1_ PULSE	PEG1_ STATE0	PEG1_ STATE1	PEG1_ STATE2
001	Encoder X Phase A	Encoder X Phase B	PEG0_ STATE0	PEG0_ STATE1	PEG0_ STATE2
010	Encoder Y Phase A	Encoder Y Phase B	PEG2_ STATE0	PEG2_ STATE1	PEG2_ STATE2
011	Encoder A Phase A	Encoder A Phase B	Encoder A Phase A	Encoder A Phase B	Encoder A INDEX
100	Reserved	PEG2_ PULSE	Reserved	Reserved	Reserved
101	PEG0_ PULSE or PEG2_ PULSE	PEG2_ PULSE or PEG1_ PULSE	Reserved	Reserved	Reserved
110	PEG0_ PULSE or PEG1_ PULSE or PEG2_ PULSE	PEG0_ PULSE or PEG1_ PULSE or PEG2_ PULSE	Reserved	Reserved	Reserved
111	FGP_OUT0	FGP_ OUT1	FGP_OUT2	Reserved	Reserved

Table A-15. Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, OUT_5-9) for CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa

PEG OUTPUT PIN NAME Bit Code	5 (0)_ STATE0	6 (0)_STATE1	7 (0)_ STATE2	8 (0)_ STATE3	9 (1)_STATE3
000 (default)	PEG0_ STATE0	PEG0_ STATE1	PEG0_ STATE2	PEG0_ STATE3	PEG1_ STATE3
001	PEG1_ STATE0	PEG1_ STATE1	PEG1_ STATE2	PEG1_ STATE3	PEG0_ STATE3
010	PEG2_ STATE0	PEG2_ STATE1	PEG2_ STATE2	PEG2_ STATE3	PEG2_ STATE3
011	Encoder X Phase A	Encoder X Phase B	Encoder Y Phase A	Encoder Y Phase B	PEG2_ STATE0
100	Encoder Y Phase A	EncoderY Phase B	Reserved	Encoder Y INDEX	PEG2_ STATE1
101	Encoder X INDEX	Encoder X INDEX	Reserved	Reserved	Reserved
110	PEG2_ PULSE	Encoder Y INDEX	PEG2_ PULSE	PEG2_ PULSE	Reserved
111	Reserved	Reserved	Reserved	Reserved	Reserved

Table A-16. Mapping of Engine Outputs to Physical Outputs (Servo Processor 0) for UDMnt/UDMpa/UDMcb

PEG OUTPUT PIN NAME Bit Code	0 PEG0	1 PEG1
000 (default)	PEG0_PULSE	PEG1_PULSE
001	Encoder X Phase A	Encoder X Phase B
010	Encoder Y Phase A	Encoder Y Phase B
011	PEG1_STATE0	PEG0_STATE0
100	PEG0_STATE0	PEG1_STATE0
101	Reserved	Reserved
110	Reserved	Reserved
111	FGP_OUT0	FGP_OUT1

Table A-17. Mapping of Engine Outputs to Physical Outputs (Servo Processor 0) for UDMlc/UDMmc/UDlIt/UDlhp/PDIcl

PEG OUTPUT PIN NAME Bit Code	0 PEGO
000 (default)	PEGO_PULSE
001	Reserved
010	Reserved
011	Reserved
100	Reserved
101	Reserved
110	Reserved
111	FGP_OUT0

Table A-18. NPMpm/NPMpc Mapping of Engine Outputs to Physical Outputs (Servo Processor 0)

PEG OUTPUT PIN NAME Bit Code	0 PEGO	1 PEG1
000 (default)	PEGO_PULSE	PEG1_PULSE
001	PEGO_STATE0	PEG1_STATE0
010	PEG1_STATE0	PEGO_STATE0
011	PEGO_STATE1	PEG1_STATE1
100	PEG1_STATE1	PEGO_STATE1
101	Reserved	Reserved
110	Reserved	Reserved
111	FGP_OUT0	FGP_OUT1

Bit Code: 111

The **Bit Code: 111**, both for Servo Processor 0 and Servo Processor 1, is used for switching the physical output pins to Fast General Purpose Outputs: **FGP_OUT0** to **FGP_OUT6**. The state of the Fast General

Purpose Outputs can be read or changed using the ACSPL+ **OUT(x)** variable. The Fast General Purpose Outputs are mapped as follows:

FGP_OUT0 is mapped to bit 16 of the ACSPL+ **OUT(x)** variable

FGP_OUT1 is mapped to bit 17 of the ACSPL+ **OUT(x)** variable

FGP_OUT2 is mapped to bit 18 of the ACSPL+ **OUT(x)** variable

FGP_OUT3 is mapped to bit 19 of the ACSPL+ **OUT(x)** variable

FGP_OUT4 is mapped to bit 20 of the ACSPL+ **OUT(x)** variable

FGP_OUT5 is mapped to bit 21 of the ACSPL+ **OUT(x)** variable

FGP_OUT6 is mapped to bit 22 of the ACSPL+ **OUT(x)** variable

Table A-19. IDMSM/ECMSM/UDMSM Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, Outputs 0-5)

PEG OUTPUT PIN NAME Bit Code	0 OUT_ CNFG_0	1 OUT_ CNFG_1	2 OUT_ CNFG_2	3 OUT_ CNFG_3	4 OUT_ CNFG_4	5 OUT_ CNFG_5
Connectors/pi nouts	J11-I/O: 36 (+),37(-)	J11-I/O: 38 (+),39(-)	J11-I/O: 40(+),41 (-)	J11-I/O: 25(+),10 (-)	J10- LIMITS: 9(+),22 (-)	J10- LIMITS: 10(+),23 (-)
0000	PEG0_ State0	PEG0_ State1	PEG0_ State2	PEG0_ State3	Reserv ed	Reserv ed
0001	PEG1_ State0	PEG1_ State1	PEG1_ State2	PEG1_ State3	Reserv ed	Reserv ed
0010	PEG2_ State0	PEG2_ State1	PEG2_ State2	PEG2_ State3	Reserv ed	Reserv ed
0011	PEG3_ State0	PEG3_ State1	PEG3_ State2	PEG3_ State3	Reserv ed	Reserv ed
0100	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed
0101	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed
0110	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed
0111	GP_ OUT0 (Defaul t)	GP_ OUT1 (Defaul t)	GP_ OUT2 (Defaul t)	GP_ OUT3 (Defaul t)	GP_ OUT4 (Defaul t)	GP_ OUT5 (Defaul t)
1000	PEG0_ Pulse	PEG0_ Pulse	PEG0_ Pulse	PEG0_ Pulse	Encode r 0 Phase A	Encoder 0 Phase A
1001	PEG1_ Pulse	PEG1_ Pulse	PEG1_ Pulse	PEG1_ Pulse	Encode r 1 Phase A	Encoder 1 Phase A

PEG OUTPUT PIN NAME Bit Code	0 OUT_ CNFG_0	1 OUT_ CNFG_1	2 OUT_ CNFG_2	3 OUT_ CNFG_3	4 OUT_ CNFG_4	5 OUT_ CNFG_5
Connectors/pinouts	J11-I/O: 36 (+),37(-)	J11-I/O: 38 (+),39(-)	J11-I/O: 40(+),41 (-)	J11-I/O: 25(+),10 (-)	J10- LIMITS: 9(+),22 (-)	J10- LIMITS: 10(+),23 (-)
1010	PEG2_ Pulse	PEG2_ Pulse	PEG2_ Pulse	PEG2_ Pulse	Encoder 2 Phase A	Encoder 2 Phase A
1011	PEG3_ Pulse	PEG3_ Pulse	PEG3_ Pulse	PEG3_ Pulse	Encoder 3 Phase A	Encoder 3 Phase A
1100	Reserved	Reserved	Reserved	Reserved	Encoder 0 Phase B	Encoder 0 Phase B
1101	Reserved	Reserved	Reserved	Reserved	Encoder 1 Phase B	Encoder 1 Phase B
1110	Reserved	Reserved	Reserved	Reserved	Encoder 2 Phase B	Encoder 2 Phase B
1111	Reserved	Reserved	Reserved	Reserved	Encoder 3 Phase B	Encoder 3 Phase B

Table A-20. IDMsM/ECMsM/UDMsM Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, Outputs 6-11)

PEG OUTPUT PIN NAME Bit Code	6 OUT_ CNFG_6	7 OUT_ CNFG_7	8 PEG0	9 PEG1	10 PEG2	11 PEG3
Connectors/pi nouts	J10- LIMITS: 11(+),24 (-)	J10- LIMITS: 12(+),25 (-)	J11-I/O: 21(+),6 (-)	J11-I/O: 22(+),7 (-)	J11-I/O: 23(+),8 (-)	J11-I/O: 24(+),9 (-)
0000	Reserv ed	Reserv ed	PEG0_ Pulse (Defaul t)	PEG1_ Pulse (Defaul t)	PEG2_ Pulse (Defaul t)	PEG3_ Pulse (Defaul t)
0001	Reserv ed	Reserv ed	PEG0_ State0	PEG0_ State1	PEG0_ State2	PEG0_ State3
0010	Reserv ed	Reserv ed	PEG1_ State0	PEG1_ State1	PEG1_ State2	PEG1_ State3
0011	Reserv ed	Reserv ed	PEG2_ State0	PEG2_ State1	PEG2_ State2	PEG2_ State3
0100	Reserv ed	Reserv ed	PEG3_ State0	PEG3_ State1	PEG3_ State2	PEG3_ State3
0101	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed
0110	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed
0111	GP_ OUT6 (Defaul t)	GP_ OUT7 (Defaul t)	GP_ OUT8	GP_ OUT9	GP_ OUT10	GP_ OUT11
1000	Encode r 0 Phase A	Encode r 0 Phase A	PEG0_ Pulse	PEG0_ Pulse	PEG0_ Pulse	PEG0_ Pulse
1001	Encode	Encode	PEG1_ Pulse	PEG1_ Pulse	PEG1_ Pulse	PEG1_ Pulse

PEG OUTPUT PIN NAME Bit Code	6 OUT_ CNFG_6	7 OUT_ CNFG_7	8 PEG0	9 PEG1	10 PEG2	11 PEG3
Connectors/pinouts	J10- LIMITS: 11(+),24 (-)	J10- LIMITS: 12(+),25 (-)	J11-I/O: 21(+),6 (-)	J11-I/O: 22(+),7 (-)	J11-I/O: 23(+),8 (-)	J11-I/O: 24(+),9 (-)
	r1 Phase A	r1 Phase A	Pulse	Pulse	Pulse	Pulse
1010	Encode r2 Phase A	Encode r2 Phase A	PEG2_ Pulse	PEG2_ Pulse	PEG2_ Pulse	PEG2_ Pulse
1011	Encode r3 Phase A	Encode r3 Phase A	PEG3_ Pulse	PEG3_ Pulse	PEG3_ Pulse	PEG3_ Pulse
1100	Encode r0 Phase B	Encode r0 Phase B	Reserv ed	Reserv ed	Reserv ed	Reserv ed
1101	Encode r1 Phase B	Encode r1 Phase B	Reserv ed	Reserv ed	Reserv ed	Reserv ed
1110	Encode r2 Phase B	Encode r2 Phase B	Reserv ed	Reserv ed	Reserv ed	Reserv ed
1111	Encode r3 Phase B	Encode r3 Phase B	Reserv ed	Reserv ed	Reserv ed	Reserv ed

Table A-21. ECMsa/IDMsa/UDMsa Mapping of Engine Outputs to Physical Outputs (Servo Processor 0)

PEG OUTPUT PIN NAME Bit Code	8 PEG0	9 PEG1
Connectors/pinouts	J7-I/O: 1(+), 9(-)	J7-I/O: 2(+),10(-)
0000	PEG0_Pulse(Default)	PEG1_Pulse(Default)
0001	PEG0_State0	PEG0_State1
0010	PEG1_State0	PEG1_State1
0011	PEG2_State0	PEG2_State1
0100	PEG3_State0	PEG3_State1
0101	Reserved	Reserved
0110	Reserved	Reserved
0111	GP_OUT8	GP_OUT9
1000	PEG0_Pulse	PEG0_Pulse
1001	PEG1_Pulse	PEG1_Pulse
1010	PEG2_Pulse	PEG2_Pulse
1011	PEG3_Pulse	PEG3_Pulse
1100	Reserved	Reserved
1101	Reserved	Reserved
1110	Reserved	Reserved
1111	Reserved	Reserved

Table A-22. ECMma / IDMma / UDMma Mapping of Engine Outputs to Physical Outputs (Servo Processor 0)

PEG OUTPUT PIN NAME	0 OUT0	1 OUT1	2 OUT2	3 OUT3	8 PEG0	9 PEG1	10 PEG2	11 PEG3
Bit Code								
Conne ctors/ pinout s	J11- I/O: 36	J11- I/O: 37	J11- I/O: 38	J11- I/O: 39	J9- I/O: 21	J9- I/O: 22 (+),7 (-)	J9- I/O: 23 (+),8 (-)	J9- I/O: 24 (+),9 (-)
0000	PEG 0_ State 0	PEG 0_ State 1	PEG 0_ State 2	PEG 0_ State 3	PEG 0_ Pulse (Defa ult)	PEG 1_ Pulse (Defa ult)	PEG 2_ Pulse (Defa ult)	PEG 3_ Pulse (Defa ult)
0001	PEG 1_ State 0	PEG 1_ State 1	PEG 1_ State 2	PEG 1_ State 3	PEG 0_ State 0	PEG 0_ State 1	PEG 0_ State 2	PEG 0_ State 3
0010	PEG 2_ State 0	PEG 2_ State 1	PEG 2_ State 2	PEG 2_ State 3	PEG 1_ State 0	PEG 1_ State 1	PEG 1_ State 2	PEG1_ State 3
0011	PEG 3_ State 0	PEG 3_ State 1	PEG 3_ State 2	PEG 3_ State 3	PEG 2_ State 0	PEG 2_ State 1	PEG 2_ State 2	PEG 2_ State 3
0100	Rese rved	Rese rved	Rese rved	Rese rved	PEG 3_ State 0	PEG 3_ State 1	PEG 3_ State 2	PEG 3_ State 3
0101	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Reser ved

PEG OUTPU T PIN NAME	0 OUT0	1 OUT1	2 OUT2	3 OUT3	8 PEG0	9 PEG1	10 PEG2	11 PEG3
Bit Code								
Conne ctors/ pinout s	J11- I/O: 36	J11- I/O: 37	J11- I/O: 38	J11- I/O: 39	J9- I/O: 21	J9- I/O: 22 (+),7 (-)	J9- I/O: 23 (+),8 (-)	J9- I/O: 24 (+),9 (-)
0110	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Reser ved
0111	GP_ OUT0 (Defa ult)	GP_ OUT1 (Defa ult)	GP_ OUT2 (Defa ult)	GP_ OUT3 (Defa ult)	GP_ OUT4	GP_ OUT5	GP_ OUT6	GP_ OUT7
1000	PEG 0_ Pulse	PEG 0_ Pulse	PEG 0_ Pulse	PEG 0_ Pulse	PEG 0_ Pulse	PEG 0_ Pulse	PEG 0_ Pulse	PEG 0_ Pulse
1001	PEG 1_ Pulse	PEG 1_ Pulse	PEG 1_ Pulse	PEG 1_ Pulse	PEG 1_ Pulse	PEG 1_ Pulse	PEG 1_ Pulse	PEG1_ Pulse
1010	PEG 2_ Pulse	PEG 2_ Pulse	PEG 2_ Pulse	PEG 2_ Pulse	PEG 2_ Pulse	PEG 2_ Pulse	PEG 2_ Pulse	PEG 2_ Pulse
1011	PEG 3_ Pulse	PEG 3_ Pulse	PEG 3_ Pulse	PEG 3_ Pulse	PEG 3_ Pulse	PEG 3_ Pulse	PEG 3_ Pulse	PEG 3_ Pulse
1100	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Reser ved
1101	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Reser ved

PEG OUTPU T PIN NAME	0 OUT0	1 OUT1	2 OUT2	3 OUT3	8 PEG0	9 PEG1	10 PEG2	11 PEG3
Bit Code								
Conne ctors/ pinout s	J11- I/O: 36	J11- I/O: 37	J11- I/O: 38	J11- I/O: 39	J9- I/O: 21	J9- I/O: 22 (+),7 (-)	J9- I/O: 23 (+),8 (-)	J9- I/O: 24 (+),9 (-)
1110	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Reser ved
1111	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Rese rved	Reser ved

Table A-23. ECMdx / IDMDx / UDMdx Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, Outputs 0-5)

PEG OUTPUT PIN NAME Bit Code	0 OUT0	1 OUT1	2 OUT2	3 OUT3	4 OUT4	5 OUT5
Connectors/pinouts	J9-I/O: 33	J9-I/O: 34	J9-I/O: 35	J9-I/O: 3	J9-I/O: 36	J9-I/O: 37
0000	PEG0_ State0	PEG0_ State1	PEG0_ State2	PEG0_ State3	Reserv ed	Reserv ed
0001	PEG1_ State0	PEG1_ State1	PEG1_ State2	PEG1_ State3	Reserv ed	Reserv ed
0010	PEG2_ State0	PEG2_ State1	PEG2_ State2	PEG2_ State3	Reserv ed	Reserv ed
0011	PEG3_ State0	PEG3_ State1	PEG3_ State2	PEG3_ State3	Reserv ed	Reserv ed
0100	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed
0101	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed
0110	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed	Reserv ed
0111	GP_ OUT0 (Defaul t)	GP_ OUT1 (Defaul t)	GP_ OUT2 (Defaul t)	GP_ OUT3 (Defaul t)	GP_ OUT4 (Defaul t)	GP_ OUT5 (Defaul t)
1000	PEG0_ Pulse	PEG0_ Pulse	PEG0_ Pulse	PEG0_ Pulse	Encode r 0 Phase A	Encoder 0 Phase A
1001	PEG1_ Pulse	PEG1_ Pulse	PEG1_ Pulse	PEG1_ Pulse	Encode r 1 Phase A	Encoder 1 Phase A

PEG OUTPUT PIN NAME Bit Code	0 OUT0	1 OUT1	2 OUT2	3 OUT3	4 OUT4	5 OUT5
Connectors/pinouts	J9-I/O: 33	J9-I/O: 34	J9-I/O: 35	J9-I/O: 3	J9-I/O: 36	J9-I/O: 37
1010	PEG2_ Pulse	PEG2_ Pulse	PEG2_ Pulse	PEG2_ Pulse	Encoder 2 Phase A	Encoder 2 Phase A
1011	PEG3_ Pulse	PEG3_ Pulse	PEG3_ Pulse	PEG3_ Pulse	Encoder 3 Phase A	Encoder 3 Phase A
1100	Reserved	Reserved	Reserved	Reserved	Encoder 0 Phase B	Encoder 0 Phase B
1101	Reserved	Reserved	Reserved	Reserved	Encoder 1 Phase B	Encoder 1 Phase B
1110	Reserved	Reserved	Reserved	Reserved	Encoder 2 Phase B	Encoder 2 Phase B
1111	Reserved	Reserved	Reserved	Reserved	Encoder 3 Phase B	Encoder 3 Phase B

Table A-24. ECMdx / IDMDx / UDMdx Mapping of Engine Outputs to Physical Outputs (Servo Processor 0, Outputs 6-11)

PEG OUTPUT PIN NAME Bit Code	6 OUT6	7 OUT7	8 PEG0	9 PEG1	10 PEG2	11 PEG3
Connectors/pinouts	J9-I/O: 38	J9-I/O: 4	J9-I/O: 21(+), 6 (-)	J9-I/O: 22(+), 7 (-)	J9-I/O: 23(+), 8 (-)	J9-I/O: 24(+), 9 (-)
0000	Reserved	Reserved	PEG0_ Pulse (Default)	PEG1_ Pulse (Default)	PEG2_ Pulse (Default)	PEG3_ Pulse (Default)
0001	Reserved	Reserved	PEG0_ State0	PEG0_ State1	PEG0_ State2	PEG0_ State3
0010	Reserved	Reserved	PEG1_ State0	PEG1_ State1	PEG1_ State2	PEG1_ State3
0011	Reserved	Reserved	PEG2_ State0	PEG2_ State1	PEG2_ State2	PEG2_ State3
0100	Reserved	Reserved	PEG3_ State0	PEG3_ State1	PEG3_ State2	PEG3_ State3
0101	Reserved	Reserved	Reserved	Reserved	Reserved	Reserved
0110	Reserved	Reserved	Reserved	Reserved	Reserved	Reserved
0111	GP_ OUT6	GP_ OUT7	GP_ OUT8	GP_ OUT9	GP_ OUT10	GP_ OUT11
1000	Encoder 0 Phase A	Encoder 0 Phase A	PEG0_ Pulse	PEG0_ Pulse	PEG0_ Pulse	PEG0_ Pulse
1001	Encoder 1 Phase A	Encoder 1 Phase A	PEG1_ Pulse	PEG1_ Pulse	PEG1_ Pulse	PEG1_ Pulse

PEG OUTPUT PIN NAME	6 OUT6	7 OUT7	8 PEG0	9 PEG1	10 PEG2	11 PEG3
Bit Code						
Connectors/pinouts	J9-I/O: 38	J9-I/O: 4	J9-I/O: 21(+), 6 (-)	J9-I/O: 22(+), 7 (-)	J9-I/O: 23(+), 8 (-)	J9-I/O: 24(+), 9 (-)
1010	Encoder 2 Phase A	Encoder 2 Phase A	PEG2_ Pulse	PEG2_ Pulse	PEG2_ Pulse	PEG2_ Pulse
1011	Encoder 3 Phase A	Encoder 3 Phase A	PEG3_ Pulse	PEG3_ Pulse	PEG3_ Pulse	PEG3_ Pulse
1100	Encoder 0 Phase B	Encoder 0 Phase B	Reserved	Reserved	Reserved	Reserved
1101	Encoder 1 Phase B	Encoder 1 Phase B	Reserved	Reserved	Reserved	Reserved
1110	Encoder 2 Phase B	Encoder 2 Phase B	Reserved	Reserved	Reserved	Reserved
1111	Encoder 3 Phase B	Encoder 3 Phase B	Reserved	Reserved	Reserved	Reserved

A.3 ASSIGNMARK Mapping

Mark Inputs to Encoders Mapping

Table A-25. Mark-1 Inputs to Encoders Mapping for SPiiPlusNT/DC-LT/HP/LD

Bit code	Latching of MARK Encoder 0 (X)	Latching of MARK Encoder 1 (Y)	Latching of MARK Encoder 4 (Z)	Latching of MARK Encoder 5 (T)	Latching of MARK Encoder 2 (A)	Latching of MARK Encoder 3 (B)	Latching of MARK Encoder 6 (C)	Latching of MARK Encoder 7 (D)
00000 (default)	X_ MARK1	Y_ MARK1	Z_ MARK1	T_ MARK1	-	-	-	-
00001	Y_ MARK1	Z_ MARK1	T_ MARK1	X_ MARK1	-	-	-	-
00010	Z_ MARK1	T_ MARK1	X_ MARK1	Y_ MARK1	-	-	-	-
00011	T_ MARK1	X_ MARK1	Y_ MARK1	Z_ MARK1	-	-	-	-
00100	-	Y_ MARK1	Z_ MARK1	T_ MARK1	X_ MARK1	-	-	-
00101	X_ MARK1	-	Z_ MARK1	T_ MARK1	-	Y_ MARK1	-	-
00110	X_ MARK1	Y_ MARK1	-	T_ MARK1	-	-	Z_ MARK1	-
00111	X_ MARK1	Y_ MARK1	Z_ MARK1	-	-	-	-	T_ MARK1
01000	-	-	Z_ MARK1	T_ MARK1	X_ MARK1	Y_ MARK1	-	-
01001	X_ MARK1	Y_ MARK1	-	-	-	-	Z_ MARK1	T_ MARK1

Bit code	Latching of MARK Encoder 0 (X)	Latching of MARK Encoder 1 (Y)	Latching of MARK Encoder 4 (Z)	Latching of MARK Encoder 5 (T)	Latching of MARK Encoder 2 (A)	Latching of MARK Encoder 3 (B)	Latching of MARK Encoder 6 (C)	Latching of MARK Encoder 7 (D)
0101 0	X_ MAR K1	-	-	-	-	Y_ MAR K1	Z_ MAR K1	T_ MAR K1
01011	-	Y_ MAR K1	-	-	X_ MAR K1	-	Z_ MAR K1	Z_ MAR K1
0110 0	-	-	Z_ MAR K1	-	X_ MAR K1	Y_ MAR K1	-	T_ MAR K1
01101	-	-	-	T_ MAR K1	X_ MAR K1	Y_ MAR K1	Z_ MAR K1 pin	-
01110	-	-	-	-	X_ MAR K1	Y_ MAR K1	Z_ MAR K1	T_ MAR K1
01111	-	-	-	-	Y_ MAR K1	Z_ MAR K1	T_ MAR K1	X_ MAR K1
1000 0	-	-	-	-	Z_ MAR K1	T_ MAR K1	X_ MAR K1	Y_ MAR K1
1000 1	-	-	-	-	T_ MAR K1	X_ MAR K1	Y_ MAR K1	Z_ MAR K1

Example

```
ASSIGNMARK 1, 1, 0x0b00010
```

By using SPiiPlusNT as the first node, entering the command performs the following assignments for these inputs:

- > Latching of Encoder 0(X) occurs once Z_MARK1 physical pin gets an input.
- > Latching of Encoder 1(Y) occurs once T_MARK1 physical pin gets an input.
- > Latching of Encoder 4(Z) occurs once X_MARK1 physical pin gets an input.

- > Latching of Encoder 5(T) occurs once Y_sMARK1 physical pin gets an input.

Table A-26. Mark-2 Inputs to Encoders Mapping for SPiiPlusNT/DC-LT/HP/LD

Bit code	Latching of M2ARK Encoder 0(X)	Latching of M2ARK Encoder 1(Y)	Latching of M2ARK Encoder 4(Z)	Latching of M2ARK Encoder 5(T)	Latching of M2ARK Encoder 2(A)	Latching of M2ARK Encoder 3(B)	Latching of M2ARK Encoder 6(C)	Latching of M2ARK Encoder 7(D)
00000 (default)	GP IN6	GP IN7	GP IN4	GP IN5	-	-	-	-
00001	GP IN7	GP IN4	GP IN5	GP IN6	-	-	-	-
00010	GP IN4	GP IN5	GP IN6	GP IN7	-	-	-	-
00011	GP IN5	GP IN6	GP IN7	GP IN4	-	-	-	-
00100	-	GP IN7	GP IN4	GP IN5	GP IN6	-	-	-
00101	GP IN6	-	GP IN4	GP IN5	-	GP IN7	-	-
00110	GP IN6	GP IN7	-	GP IN5	-	-	GP IN4	-
00111	GP IN6	GP IN7	GP IN4	-	-	-	-	GP IN5
01000	-	-	GP IN4	GP IN5	GP IN6	GP IN7	-	-
01001	GP IN6	GP IN7	-	-	-	-	GP IN4	GP IN5
01010	GP IN6	-	-	-	-	GP IN7	GP IN4	GP IN5
01011	-	GP IN7	-	-	GP IN6	-	GP IN4	GP IN5
01100	-	-	GP IN4	-	GP IN6	GP IN7	-	GP IN5

Bit code	Latching of M2ARK Encoder 0(X)	Latching of M2ARK Encoder 1(Y)	Latching of M2ARK Encoder 4(Z)	Latching of M2ARK Encoder 5(T)	Latching of M2ARK Encoder 2(A)	Latching of M2ARK Encoder 3(B)	Latching of M2ARK Encoder 6(C)	Latching of M2ARK Encoder 7(D)
01101	-	-	-	GP IN5	GP IN6	GP IN7	GP IN4	-
01110	-	-	-	-	GP IN6	GP IN7	GP IN4	GP IN5
01111	-	-	-	-	GP IN7	GP IN4	GP IN5	GP IN6
10000	-	-	-	-	GP IN4	GP IN5	GP IN6	GP IN7
10001	-	-	-	-	GP IN5	GP IN6	GP IN7	GP IN4

Example

```
ASSIGNMARK 1, 2, 0x0b00010
```

By using SPiiPlusNT as the first node, entering the command performs the following assignments for these inputs:

- > Latching of M2ARK of Encoder 0(X) occurs once GP IN4 physical pin gets an input.
- > Latching of M2ARK of Encoder 1(Y) occurs once GP IN5 physical pin gets an input.
- > Latching of M2ARK of Encoder 4(Z) occurs once GP IN6 physical pin gets an input.
- > Latching of M2ARK of Encoder 5(T) occurs once GP IN7 physical pin gets an input.

Table A-27. Mark-1 Inputs to Encoders Mapping for with SPiiPlus CMnt/UDMpm-x/UDMpc/CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa/CMhv/UDMhv

Bit code	Latching of Mark1 Encoder 0(X)	Latching of Mark1 Encoder 1(Y)	Latching of Mark1 Encoder 2(A)	Latching of Mark1 Encoder 3(B)
000 (default)	Mark1 of encoder 0(X) pin	Mark1 of encoder 1(Y) pin	Mark1 of encoder 0(X) pin	Mark1 of encoder 1(Y) pin
001	GP IN6	Mark1 of encoder 0(X) pin	Mark2 of encoder 0(X) pin	Mark1 of encoder 0(X) pin
010	-	GP IN4	GP IN6	GP IN6

Bit code	Latching of Mark1 Encoder 0(X)	Latching of Mark1 Encoder 1(Y)	Latching of Mark1 Encoder 2(A)	Latching of Mark1 Encoder 3(B)
011	-	GP IN6	-	-

Example

```
ASSIGNMARK 1, 1, 0x0b0001
```

By using CMhp as the first node, entering the command above performs the following assignments for these inputs:

- > Latching of Encoder 0 occurs once IN6 pin (pin 5, J9 connector at CMhp) gets an input.
- > Latching of Encoder 1 occurs once X(0)_MARK1+ physical pin (pin 12, J9 connector at CMhp) gets an input.
- > Latching of Encoder 2 occurs once X(0)_MARK2+ physical pin (pin 13, J9 connector at CMhp) gets an input.
- > Latching of Encoder 3 occurs once X(0)_MARK1+ physical pin (pin 12, J9 connector at CMhp) gets an input.

**Table A-28. Mark-2 Inputs to Encoders Mapping for with SPiiPlus
CMnt/UDMpm/UDMpc/CMba/CMhp/CMxa/UDMba/UDMhp/UDMxa/CMhv/UDMhv**

Bit code	Latching of Mark2 Encoder 0(X)	Latching of Mark2 Encoder 1(Y)	Latching of Mark2 Encoder 2(A)	Latching of Mark2 Encoder 3(B)
000 (default)	Mark2 of axis 0 (X) pin	Mark2 of axis 1 (Y) pin	GP IN6	GP IN7
001	Mark1 of axis 1 (Y) pin	Mark1 of axis 1 (Y) pin	Mark1 of axis 1 (Y) pin	Mark1 of axis 1 (Y) pin
010	Mark2 of axis 1 (Y) pin	GP IN5	Mark2 of axis 1 (Y) pin	-
011	GP IN7	GP IN7	GP IN7	-

Example

```
ASSIGNMARK 0x0b010
```

By using CMhp as the first node, entering the command above performs the following assignments for these inputs:

- > Latching of Encoder 0 occurs once Y(1)_MARK2+ physical pin (pin 15, J9 connector at CMhp) gets an input.
- > Latching of Encoder 1 occurs once IN5 pin (pin 23, J9 connector at CMhp) gets an input.

- > Latching of Encoder 2 occurs once Y(1)_MARK2+ physical pin (pin 15, J9 connector at CMhp) gets an input.

Table A-29. IDMxx/ECMxx/UDMsm/UDMsa/UDMma Encoder Mapping

Byte	3	2	1	0
	Latching of Encoder 3(B)	Latching of Encoder 2(A)	Latching of Encoder 1(Y)	Latching of Encoder 0(Y)
MARK0 input pin	0x00	0x00	0x00	0x00 (default)
MARK1 input pin	0x01	0x01	0x01(default)	0x01
MARK2 input pin	0x02	0x02(default)	0x02	0x02
MARK3 input pin	0x03(default)	0x03	0x03	0x03

The table above allows the user to build a hexadecimal value for the `inputs_to_encoder_bit_code` argument:

ASSIGNMARK axis, type, 0XAABBCCDD

Where AA is the code for encoder 3, BB is the code for encoder 2, CC is the code for encoder 1, and DD is the code for encoder 0.

Example 1 (default case)

```
ASSIGNMARK 1, 1, 0x03020100
```

By using UDMsm as the first node, entering the command above performs the following assignments for these inputs:

- > Latching of Encoder 0 occurs once MARK0 physical pin (pin 16, J11 connector) gets an input.
- > Latching of Encoder 1 occurs once MARK1 physical pin (pin 17, J11 connector) gets an input.
- > Latching of Encoder 2 occurs once MARK2 physical pin (pin 18, J11 connector) gets an input.
- > Latching of Encoder 3 occurs once MARK3 physical pin (pin 19, J11 connector) gets an input.

Example 2

```
ASSIGNMARK 1, 1, 0x03010102
```

By using UDMsm as the first node, entering the command above performs the following assignments for these inputs:

- > Latching of Encoder 0 occurs once MARK2 physical pin (pin 18, J11 connector) gets an input.
- > Latching of Encoder 1 occurs once MARK1 physical pin (pin 17, J11 connector) gets an input.

- > Latching of Encoder 2 occurs once MARK1 physical pin (pin 17, J11 connector) gets an input.
- > Latching of Encoder 3 occurs once MARK3 physical pin (pin 19, J11 connector) gets an input.

Smarter



Motion

5 HaTnufa St.
Yokneam Illit 2066717
Israel
Tel: (+972) (4) 654 6440 Fax: (+972) (4) 654 6443

Contact us: sales@acsmotioncontrol.com | www.acsmotioncontrol.com

